

MACHINE LEARNING

Without the Magic

From vectors and derivatives to a Mixture-of-Experts transformer, every idea built from scratch: intuition first, then the math, then working PyTorch, then a question to check yourself.

A 23-section course · by jiffies64 · text CC BY 4.0, code MIT

Contents

PART 1 · MATH FOUNDATIONS

- [1](#) Linear Algebra for ML
- [2](#) Calculus for ML
- [3](#) Probability & Statistics
- [4](#) Information Theory

PART 2 · CLASSICAL ML

- [5](#) What Learning Means
- [6](#) Logistic Regression
- [7](#) Overfitting & Regularization
- [8](#) A Tour of Classical Models

PART 3 · NEURAL NETS FROM SCRATCH

- [9](#) Neurons, Layers, Activations
- [10](#) Backpropagation by Hand
- [11](#) Build an MLP in NumPy
- [12](#) Training Dynamics & Optimizers

PART 4 · PYTORCH

- [13](#) Tensors & Autograd
- [14](#) nn.Module & the Training Loop
- [15](#) Debugging & Good Habits

PART 5 · DEEP LEARNING ARCHITECTURES

- [16](#) Convolutional Networks
- [17](#) Sequence Models: RNNs & LSTMs
- [18](#) Attention & Transformers
- [19](#) Language Modeling & GPT
- [20](#) Training at Scale

PART 6 · CAPSTONE: PAPER FROM SCRATCH

- [21](#) Reading a Paper Like an Engineer
- [22](#) Mixture of Experts from Scratch
- [23](#) Where to Go Next

PART 1

Math Foundations

Linear Algebra for ML

A neural network is a stack of vectors pushed through matrices. Learn to read that sentence, and you can read most of ML.

38 min read

YOU'LL LEARN

- vectors
- dot products
- matrices as transformations
- norms
- eigenvalues

Every piece of data you feed a model first becomes a **list of numbers**. This is true for an image, a sentence, or a row in a spreadsheet. Linear algebra is the set of rules for working with those lists efficiently. It is the grammar of machine learning. The good news: you already have the intuition. We only need to attach the notation to it.

◆ THE ONE IDEA TO HOLD ONTO

Data becomes **vectors**. Operations on data become **matrices**. Learning is the search for the right numbers to put inside those matrices. The rest of this lesson fills in the details.

1 Vectors: a point, and an arrow

◆ INTUITION FIRST

A **vector** is an ordered list of numbers. That is the plain definition. The more useful picture: a vector is an *arrow* in space. It points from the origin (the center, position zero) to some spot. The numbers tell you how far

to walk along each direction to reach that spot.

☂ EVERYDAY PARALLEL

Think of a grocery order: *2 apples, 1 loaf of bread, 3 coffees*. As a vector, that is $[2, 1, 3]$. Each slot always means the same thing (slot 1 is always apples), and the order never changes. A vector is a labeled shopping list where everyone agrees on the order, so the labels can be left out.

We write a vector as a column of numbers and give it a bold, lowercase name:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \in \mathbb{R}^3$$

In words: " $\mathbf{x}$ is a vector in $\mathbb{R}^3$." The symbol $\mathbb{R}^3$ means "a list of 3 real numbers." The small 3 is the *dimension*: how many numbers are in the list. An image with 784 pixels is a vector in $\mathbb{R}^{784}$.

◆ NOW THE MATH

You can do two things to vectors. You can **add** them (slot by slot), and you can **scale** them (multiply every slot by one number, called a *scalar*):

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \end{bmatrix}, \quad c\mathbf{a} = \begin{bmatrix} c a_1 \\ c a_2 \end{bmatrix}$$

In words: adding means walking along one arrow, then along the other, tip to tail. Scaling by c makes the arrow c times as long. If c is negative, the arrow also flips around. A **scalar** is a single number, as opposed to a list.

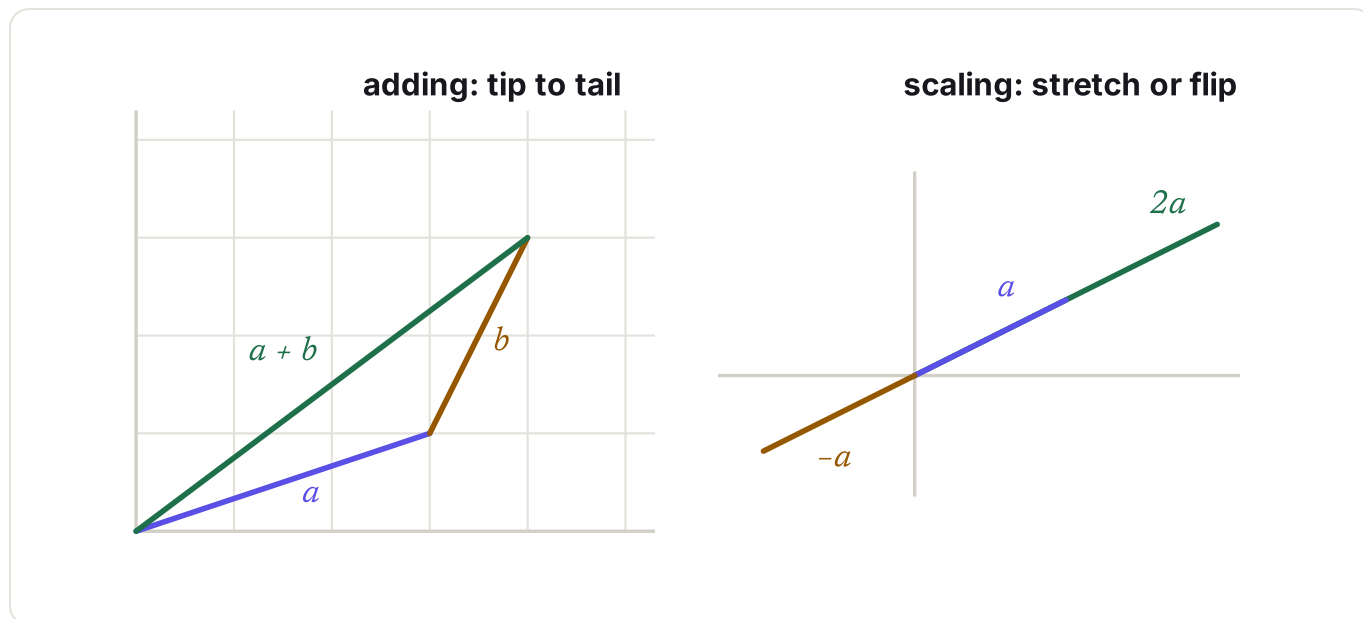


Figure 1.1 Left: to add $\mathbf{a} = [3, 1]$ and $\mathbf{b} = [1, 2]$, walk along $\mathbf{a}$, then along $\mathbf{b}$ from its tip. You land on $\mathbf{a} + \mathbf{b} = [4, 3]$. Right: $2\mathbf{a}$ points the same way and is twice as long; $-\mathbf{a}$ flips the direction.

CHECK YOURSELF 1

Take $\mathbf{a} = [3, 1]$ and $\mathbf{b} = [1, 2]$ from Figure 1.1. What is $\mathbf{a} - \mathbf{b}$, and where is that arrow in the picture?

2 The dot product: how aligned are two arrows?

◆ INTUITION FIRST

The **dot product** takes two vectors and returns one number. That number measures *how much the two arrows point the same way*. A large positive number means they point in similar directions. Zero means they are at right angles. A negative number means they point in opposite directions. This one operation appears inside almost every model you will build.

☂ EVERYDAY PARALLEL

A dating app lists five topics: hiking, jazz, cats, cooking, football. Each person is a vector with $+1$ for “likes” and -1 for “dislikes” in each slot. To compare two people, multiply their matching slots. The product is $+1$ where they agree and -1 where they disagree. Add the five products and you get one score. That score **is** the dot product. Agree on everything: $+5$. Disagree on everything: -5 . Agree on half: near 0.

◆ NOW THE MATH

Multiply the vectors slot by slot, then add up all the results:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

In words: the big Σ ("sigma") means "add up the following as i counts from 1 to n ." So $\mathbf{a} \cdot \mathbf{b}$ pairs up the slots, multiplies each pair, and sums the products. The output is always a single number, however long the vectors are.

A second, geometric formula gives the same number. It shows *why* the dot product measures alignment:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

In words: the dot product equals (length of $\mathbf{a}$) $\times$ (length of $\mathbf{b}$) $\times$ the cosine of the angle θ between them. The cosine is 1 when the angle is 0° and drops to 0 at 90° . So the dot product is largest when the arrows are parallel, and zero when they are perpendicular.

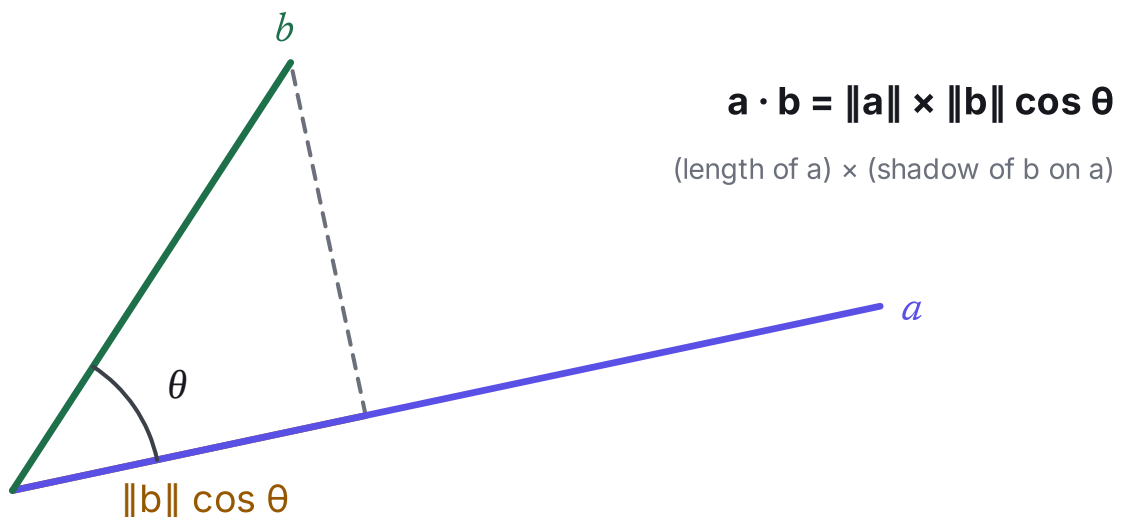


Figure 1.2 The dot product is the length of $\mathbf{a}$ times the length of the shadow that $\mathbf{b}$ casts on it (amber). Turn $\mathbf{b}$ toward $\mathbf{a}$ and the shadow grows. At 90° the shadow, and the dot product, is zero.

Why do the two formulas agree? Look at the arrow $\mathbf{a} - \mathbf{b}$, which joins the two tips. We can write its squared length in two ways. We define length properly in Section 4. For now, all we need is this: the squared length of a vector is its dot product with itself. So the first way is to expand that product:

$$\|\mathbf{a} - \mathbf{b}\|^2 = (\mathbf{a} - \mathbf{b}) \cdot (\mathbf{a} - \mathbf{b}) = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a} \cdot \mathbf{b}$$

Second, the three arrows $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{a} - \mathbf{b}$ form a triangle. The law of cosines from school geometry says:

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$$

In words: both lines describe the same length. Everything matches except the last term, so those last terms must be equal: $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$.

DEFINITION

Two vectors are **orthogonal** when their dot product is zero. “Orthogonal” is the mathematician’s word for *at a right angle*. It means the two arrows share no direction: moving along one does not move you along the other at all. (Careful: two data features whose vectors are orthogonal are not automatically “unrelated” in the statistical sense. That needs more conditions.)

◆ THE SAME THING IN PYTORCH

>_ DOT PRODUCT, TWO WAYS

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

# element-wise multiply, then sum: the definition
print((a * b).sum())          # tensor(32.)    = 1*4 + 2*5 + 3*6

# the built-in function most code uses
print(torch.dot(a, b))        # tensor(32.)
```

In words: `a * b` multiplies matching slots and gives a new vector [4, 10, 18]. `.sum()` adds those up. `torch.dot` does both steps in one call.

CHECK YOURSELF 2

Let $\mathbf{a} = [1, 2]$ and $\mathbf{b} = [2, -1]$. What is $\mathbf{a} \cdot \mathbf{b}$? Now double $\mathbf{b}$. What happens to the dot product, and what happens to the angle?

3 Matrices as transformations

◆ INTUITION FIRST

A **matrix** is a grid of numbers, like a stack of vectors. The key insight is this: **a matrix is a machine that takes a vector in and returns a different vector out.** It can rotate, stretch, squash and reflect space. When you multiply a matrix by a vector, you ask the machine: “where does this point end up after the transformation?”

☂ EVERYDAY PARALLEL

Think of the “rotate” and “stretch” tools in a photo editor. “Rotate 90°” sends every point of the picture to a new place, by one fixed rule. “Stretch horizontally by 2” doubles every point's distance from the left edge, again by one fixed rule. A matrix is such a rule. Give it a different picture and the same rule runs. (A slider like “brightness” is different: it *adds* the same amount to every pixel. A shift like that is not a matrix. It is the job of the bias **b** that we meet below.)

◆ NOW THE MATH

A matrix has rows and columns. We name it with a bold capital and write its shape as (*rows* × *columns*):

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \in \mathbb{R}^{2 \times 2}$$

To transform a vector, we use **matrix–vector multiplication**. The rule: each output slot is the dot product of one *row* of the matrix with the input vector.

$$\mathbf{W}\mathbf{x} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} w_{11}x_1 + w_{12}x_2 \\ w_{21}x_1 + w_{22}x_2 \end{bmatrix}$$

In words: the top output number is (row 1) · **x**. The bottom output number is (row 2) · **x**. So a matrix is *several dot products stacked up*, and you already understand dot products. Row 1 of **W** asks one question about **x**, row 2 asks another, and the answers stack into the output vector.

There is a second way to read the same product. Put **x** = [1, 0] in: the output is the first column of **W**. Put [0, 1] in: the output is the second column. So the columns of a matrix tell you where the two unit arrows land. Because the rule is linear, the whole grid follows them:

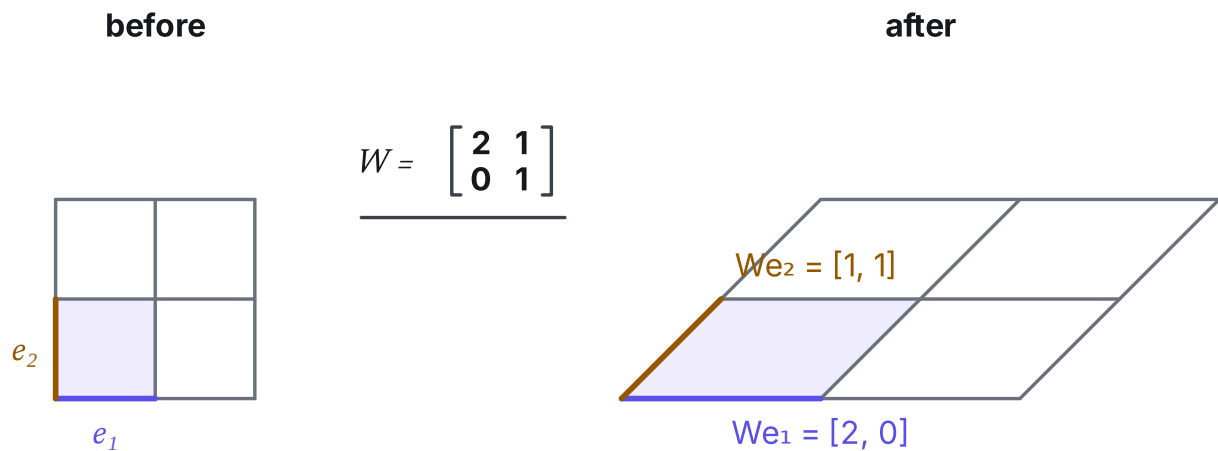


Figure 1.3 The matrix $\mathbf{W}$ moves the whole grid at once. Straight lines stay straight and evenly spaced; the unit square becomes a parallelogram. The columns of $\mathbf{W}$ tell you where the two unit arrows land: $\mathbf{e}_1 = [1, 0]$ goes to $[2, 0]$, and $\mathbf{e}_2 = [0, 1]$ goes to $[1, 1]$.

◆ WHY THIS MATTERS FOR DEEP LEARNING

A single layer of a neural network is $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. It transforms the input with a matrix $\mathbf{W}$, then shifts it by a vector $\mathbf{b}$ (the *bias*). Stack a few of these with a nonlinear step between them and you have a deep network. We build neurons, layers and nonlinearity properly in Section 9. When we say a model “learns,” we mean it searches for the numbers to put inside $\mathbf{W}$ and $\mathbf{b}$.

Matrix × matrix = do one transformation, then the next

Multiplying two matrices means *composing* their transformations: apply the right one first, then the left one. This is why order matters: $\mathbf{AB} \neq \mathbf{BA}$ in general. Rotating and then stretching is not the same as stretching and then rotating. The shape rule is the one thing you must memorize, because shape mismatches are among the most common bugs in ML code:

THE SHAPE RULE

To multiply $(m \times n) \times (n \times p)$, the **inner numbers must match** (both n). The result has the shape of the **outer numbers**: $(m \times p)$. Read it as: “the middle two cancel, the outer two survive.”

◆ THE SAME THING IN PYTORCH

> A MATRIX TRANSFORMING A VECTOR, AND THE SHAPE RULE

```

import torch

W = torch.tensor([[2.0, 0.0],      # stretch x by 2, leave y alone
                  [0.0, 1.0]])
x = torch.tensor([3.0, 4.0])

y = W @ x                      # '@' is matrix multiplication in Python
print(y)                       # tensor([6., 4.]) → x doubled, y unchanged

# shapes decide everything. Print them when confused:
A = torch.randn(2, 3)         # (2 x 3)
B = torch.randn(3, 5)         # (3 x 5)    inner 3s match
print((A @ B).shape)          # torch.Size([2, 5]) → outer numbers survive

```

In words: the first part applies a stretch to one vector. The second part multiplies two random matrices only to show the shapes. The values inside `A` and `B` do not matter here.

▲ WATCH OUT

`RuntimeError: mat1 and mat2 shapes cannot be multiplied (2x3 and 5x3)`. The inner numbers (3 and 5) do not match. Most of the time the fix is to *transpose* one matrix (swap its rows and columns with `.T`) so the inner numbers line up. When in doubt, `print(tensor.shape)` on every line.

CHECK YOURSELF 3

You have a batch of 64 images, each flattened to 784 numbers, stored as a matrix **X** of shape (64, 784). A layer has weight matrix **W** of shape (10, 784), one row per output, as in PyTorch's `nn.Linear`. Does `X @ W` work? What should you write instead, and what is the output shape?

4 Norms: measuring the size of a vector

◆ INTUITION FIRST

A **norm** answers a simple question: “how big is this vector?” It turns a whole list of numbers into one measure of length. We need this all the time. For example, we measure how far a prediction is from the truth, or how large a model's weights have grown.

☂ EVERYDAY PARALLEL

There are two ways to measure the distance from your home to a friend's apartment. The **L2** way is the straight-line distance, what a ruler on a map shows. The **L1** way is the “taxicab” distance: you can only drive along the street grid, so you add the blocks east and the blocks north. Same start, same end, two different and valid meanings of “how far.” ML uses both on purpose.

◆ NOW THE MATH

The **L2 norm** (the default, also called “Euclidean length”) is the Pythagorean theorem in any number of dimensions. Square each slot, add them, take the square root:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

Note that the sum under the root is $\mathbf{x} \cdot \mathbf{x}$. So $\|\mathbf{x}\|_2^2 = \mathbf{x} \cdot \mathbf{x}$, which is the fact we used in Section 2. The **L1 norm** adds up the sizes (absolute values) of the slots, with no squares and no root:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| = |x_1| + |x_2| + \cdots + |x_n|$$

In words: L2 punishes one big value hard, because it squares it. So it prefers many small values to one large one. L1 treats all values the same way. As you will see in Section 7, L1 tends to push some weights all the way to zero. This difference is the reason we have two kinds of *regularization* (a technique that keeps models simple). Remember it for Section 7.

◆ THE SAME THING IN PYTORCH

›_ L2 AND L1 NORMS OF ONE VECTOR

```
import torch
x = torch.tensor([3.0, 4.0])

print(torch.linalg.vector_norm(x))           # tensor(5.)      sqrt(3^2 + 4^2)
print(torch.linalg.vector_norm(x, ord=1))    # tensor(7.)      |3| + |4|
```

In words: `torch.linalg.vector_norm` computes the L2 norm by default. The argument `ord=1` asks for the L1 norm instead.

CHECK YOURSELF 4

The vectors $[3, 4]$ and $[5, 0]$ both have L2 norm 5. Predict their L1 norms. Which one does L1 consider “smaller,” and what does that tell you about the kind of vector L1 prefers?

5 Eigenvalues: the directions a matrix only stretches

◆ INTUITION FIRST

A matrix stretches and turns space. For most input directions, the arrow comes out pointing somewhere new. But many matrices have a few special directions that they do *not* turn. Along those directions the matrix only stretches or shrinks the arrow. It keeps pointing along the same line. These special directions are **eigenvectors**. The factor by which each one is stretched is its **eigenvalue**. (“Eigen” is German for “own”: these are the matrix's *own* directions.)

☂ EVERYDAY PARALLEL

Stretch a rubber sheet with a drawing on it: pull the left and right edges apart. Most arrows drawn on the sheet change direction as it stretches. But an arrow that points left–right stays left–right; it only gets longer. An arrow that points up–down also keeps its direction. Those two directions are the eigenvectors of the stretch, and “how much longer” is each eigenvalue.

◆ NOW THE MATH

The defining equation is short and says what we described in words:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

In words: apply the matrix $\mathbf{A}$ to its special vector $\mathbf{v}$. You get back the same vector $\mathbf{v}$, only scaled by the number λ (lambda). The matrix may turn other arrows, but on $\mathbf{v}$ it acts like plain multiplication by a number. No new direction, only a stretch by λ .

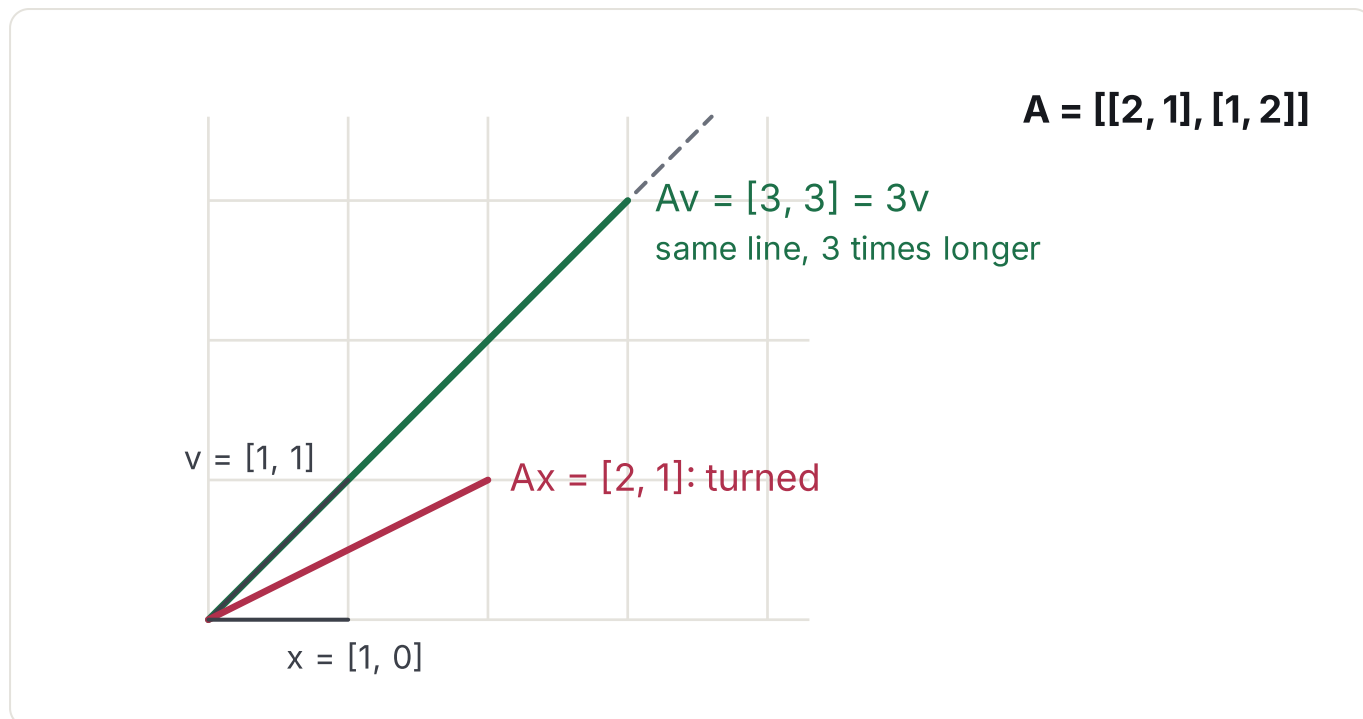


Figure 1.4 Thin arrows go in, thick arrows come out. A turns most arrows, like $[1, 0]$ (rose). The arrow $[1, 1]$ keeps its direction and only gets 3 times longer (green). So $[1, 1]$ is an eigenvector of A with eigenvalue 3.

Not every matrix has such directions among ordinary arrows. A 2-D rotation by 90° turns *every* arrow, so no real direction stays on its own line. Its eigenvalues are complex numbers, which you will see in the code below. Symmetric matrices (a matrix equal to its own transpose) always have real eigenvalues and eigenvectors.

Why care, before we have built a model? Because eigenvalues explain a lot of ML behavior you will meet later:

- **PCA** (Principal Component Analysis, a way to compress data) uses the eigenvectors of your data's covariance matrix. They are the directions along which the data varies most, so you keep those and drop the rest. We will not prove this here.
- **Exploding and vanishing gradients** (training failures we meet in Section 12 and Section 17) happen when the same matrix is applied many times. Along an eigenvector, each application multiplies the length by λ . If $|\lambda|$ is above 1, the length grows without limit; below 1, it fades to nothing.

◆ THE SAME THING IN PYTORCH

>_ EIGENVECTORS OF A SYMMETRIC MATRIX, AND A ROTATION THAT HAS NONE

```

import torch

A = torch.tensor([[2.0, 1.0],
                  [1.0, 2.0]])           # the matrix from Figure 1.4

x = torch.tensor([1.0, 0.0])
print(A @ x)                           # tensor([2., 1.]) turned: not an
v = torch.tensor([1.0, 1.0])
print(A @ v)                           # tensor([3., 3.]) = 3 * v → eig

vals, vecs = torch.linalg.eigh(A)      # eigh: for symmetric matrices
print(vals)                           # tensor([1., 3.]) the two stretch
print(vecs[:, 1])                     # tensor([0.7071, 0.7071]) v, sca

R = torch.tensor([[0.0, -1.0],
                  [1.0, 0.0]])         # rotation by 90 degrees
print(torch.linalg.eigvals(R))         # tensor([0.+1.j, 0.-1.j]) comple.

```

In words: `eigh` returns the eigenvalues in increasing order and the eigenvectors as the columns of `vecs`. Column 1 belongs to eigenvalue 3 and points along $[1, 1]$. The other eigenvector, $[1, -1]$ scaled to length 1, has eigenvalue 1: **A** leaves it unchanged. For the rotation, `eigvals` returns the imaginary numbers $\pm i$ (the `j` in the output means i).

CHECK YOURSELF 5

A matrix has an eigenvalue of 0.5 along some direction. You apply the matrix 20 times to a vector along that direction. What happens to the vector's length? What if the eigenvalue were 2?

6 What to carry forward

- **Vectors** are lists of numbers that you can picture as arrows. Data becomes vectors.
- The **dot product** multiplies matching slots and sums them. It returns one number that measures alignment: $\|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$.
- A **matrix** is a reusable transformation. Matrix–vector multiplication is several dot products stacked, and the columns show where the unit arrows land. One neural layer is $\mathbf{W}\mathbf{x} + \mathbf{b}$.

- The **shape rule**: inner dimensions must match, outer dimensions survive. Print shapes to find bugs.
- **Norms** measure vector size (L2 = straight line, L1 = taxicab). **Eigenvalues** are the stretch factors along the directions a matrix does not turn.

Next we learn how to *change* those numbers on purpose. That needs calculus, and the central tool of training: the gradient.

Answers to “Check yourself”

1. Take $\mathbf{a} = [3, 1]$ and $\mathbf{b} = [1, 2]$ from Figure 1.1. What is $\mathbf{a} - \mathbf{b}$, and where is that arrow in the picture?

Subtract slot by slot: $\mathbf{a} - \mathbf{b} = [3 - 1, 1 - 2] = [2, -1]$. In the picture, it is the arrow that goes from the tip of $\mathbf{b}$ to the tip of $\mathbf{a}$. You can check this: $\mathbf{b} + (\mathbf{a} - \mathbf{b}) = \mathbf{a}$. So a difference of two vectors is “how to get from one point to the other.” We use this later to measure how far a prediction is from the truth.

2. Let $\mathbf{a} = [1, 2]$ and $\mathbf{b} = [2, -1]$. What is $\mathbf{a} \cdot \mathbf{b}$? Now double $\mathbf{b}$. What happens to the dot product, and what happens to the angle?

$\mathbf{a} \cdot \mathbf{b} = (1)(2) + (2)(-1) = 0$, so the arrows are orthogonal. Doubling $\mathbf{b}$ gives $[4, -2]$, and the dot product is still 0: the angle stays 90° . For arrows that are *not* at right angles, doubling $\mathbf{b}$ doubles the dot product, but the angle does not change. So the dot product mixes two things: length and alignment. To measure alignment alone, divide by both lengths. The result, $\cos \theta$, is called *cosine similarity*.

3. You have a batch of 64 images, each flattened to 784 numbers, stored as a matrix $\mathbf{X}$ of shape (64, 784). A layer has weight matrix $\mathbf{W}$ of shape (10, 784), one row per output, as in PyTorch's `nn.Linear`. Does `X @ W` work? What should you write instead, and what is the output shape?

`X @ W` fails: $(64, 784) \times (10, 784)$ has inner numbers 784 and 10. Transpose $\mathbf{W}$ first: $\mathbf{W}^\top$ has shape (784, 10). Now $(64, 784) \times (784, 10)$ works, and the outer numbers survive: the output `X @ W.T` has shape (64, 10). That is 10 scores for each of the 64 images. This is the last layer of a digit classifier, and it is how the book writes a batched layer: $\mathbf{Y} = \mathbf{XW}^\top + \mathbf{b}$.

4. The vectors $[3, 4]$ and $[5, 0]$ both have L2 norm 5. Predict their L1 norms. Which one does L1 consider “smaller,” and what does that tell you about the kind of vector L1 prefers?

$\|[3, 4]\|_1 = 7$ and $\|[5, 0]\|_1 = 5$. L2 sees them as equal, but L1 calls $[5, 0]$ smaller. The vector with a zero in it wins. Suppose you penalize the L1 norm of the weights. Then the cheapest way to lower the penalty is often to set some weights to zero. That is the effect Section 7 builds on.

5. A matrix has an eigenvalue of 0.5 along some direction. You apply the matrix 20 times to a vector along that direction. What happens to the vector's length? What if the eigenvalue were 2?

Each application multiplies the length by 0.5. After 20 applications the length is $0.5^{20} \approx 0.00000095$ times the start, which is almost zero. The vector *vanishes*. With eigenvalue 2 you get $2^{20} \approx 1,000,000$ times the start: the vector *explodes*. This is the vanishing and exploding gradient problem that makes deep and recurrent networks hard to train. It is why training methods try to keep these factors close to 1.

Calculus for ML

The chain rule is not a small side topic. It is backpropagation: the rule that lets a network of millions of numbers work out how to improve itself.

46 min read

YOU'LL LEARN

- derivatives
- partial derivatives
- gradients
- the chain rule
- the Jacobian

Before you start: Section 1: Linear Algebra for ML.

In the last section we learned to *represent* data as vectors and to push it through matrices. But a model that never changes is useless. Learning means changing the numbers inside those matrices, again and again, in the direction that makes the model a little less wrong. Calculus is the mathematics of **change**. It tells us which way to change each number, and by how much. The ideas here are central to the whole course, so we go slowly and derive everything.

◆ THE ONE IDEA TO HOLD ONTO

A **derivative** tells you which way to push a number to make an output go up or down. A neural network learns by asking each of its numbers: “if I change you a tiny bit, does the error get better or worse?” Then it changes each number accordingly. That question is a derivative. Training is the job of answering it efficiently for millions of numbers at once.

1 Derivatives: the slope at a single point

◆ INTUITION FIRST

A **derivative** measures how fast something changes *right now*. Suppose a quantity y depends on an input x . The derivative answers: “if I increase x by a tiny amount, how much does y move, and in which direction?” In a graph, it is the **slope** of the curve at one point: how steeply the curve rises or falls there. A positive slope means “going up.” A negative slope means “going down.” A slope of zero means “flat, at this point.”

☂ EVERYDAY PARALLEL

Look at a car's **speedometer**. The odometer shows the total distance traveled. The speedometer shows how fast that distance changes *at this moment*, and that is a derivative. 60 km/h means “at this rate, you would cover 60 km in the next hour.” You do not have to wait an hour to know your speed now. Speed is the derivative of distance with respect to time.

◆ NOW THE MATH

Here is the problem. Slope is “rise over run”: change in output divided by change in input. But at a *single point* there is no run to divide by. The fix: measure the slope over a small step of width h , then shrink h toward zero and see which number the slope settles on. This settling process is a *limit* (the value an expression approaches as a variable approaches some value). It gives the definition of the derivative:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In words: take two points on the curve, one at x and one at $x+h$. The fraction is (change in output) $\div$ (change in input). It is the slope of the short line that joins the two points, called a *secant*. As h shrinks to zero ($h \rightarrow 0$), that line fits the curve more and more closely. Its slope becomes the slope *at the single point* x . The result $f'(x)$ (read “f-prime of x”) is the derivative.

Let's compute one from scratch, so the limit stops being abstract. Take $f(x) = x^2$ and apply the definition step by step:

$$\begin{aligned}
 f'(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\
 &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\
 &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\
 &= \lim_{h \rightarrow 0} (2x + h) \\
 &= 2x
 \end{aligned}$$

In words: we expanded $(x + h)^2$, and the two lone x^2 terms canceled. Then we divided every remaining term by h , which left $2x + h$. Now let h go to zero: the leftover h disappears, and $2x$ remains. So the slope of x^2 at any point is $2x$. At $x = 3$ the curve rises with slope 6. At $x = 0$ the slope is 0 (the bottom of the bowl, where the curve is flat). We did not guess; we derived it.

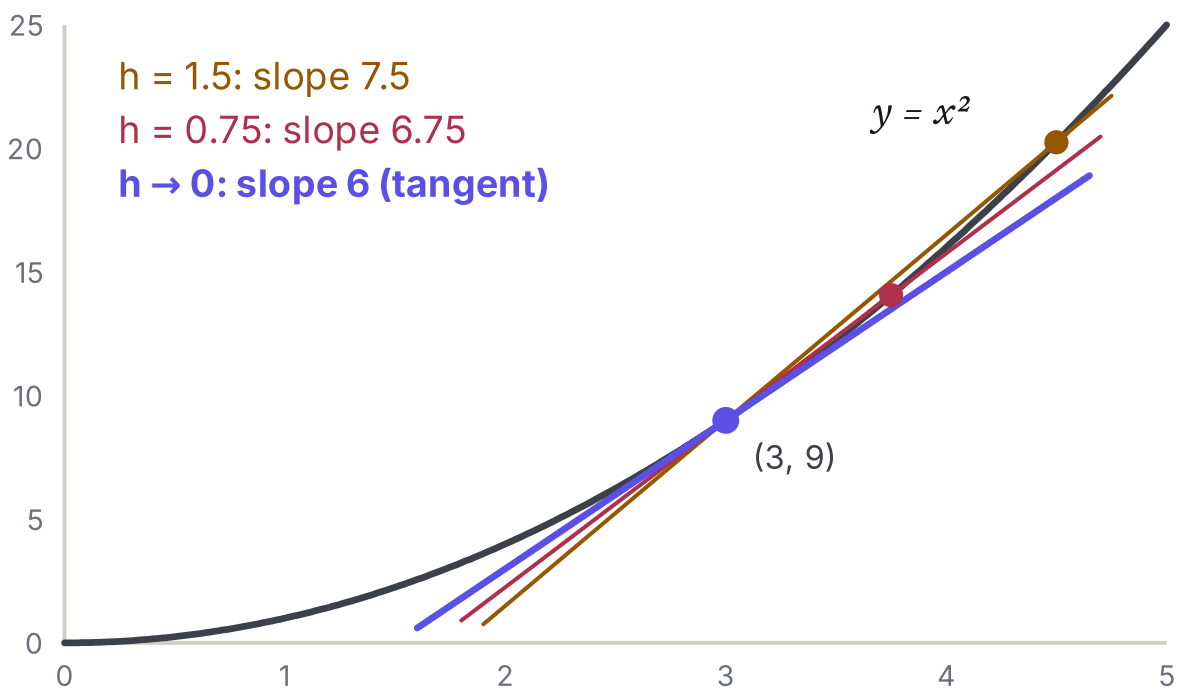


Figure 2.1 Secant lines from the point $(3, 9)$ on $y = x^2$. Each one joins two points a distance h apart, and its slope is $6 + h$. As h shrinks, the secant turns into the tangent line, whose slope is the derivative $f'(3) = 6$.

The definition also gives a rule we will use many times. For a small step h , the fraction is already close to $f'(x)$. Multiply both sides by h :

$$f(x + h) \approx f(x) + f'(x)h \quad (\text{for small } h)$$

In words: near a point, a smooth curve looks like a straight line with slope $f'(x)$. Move the input by h , and the output moves by about $f'(x)$ times h . For x^2 at $x = 3$: $f(3.01) = 9.0601$, and the rule predicts $9 + 6 \times 0.01 = 9.06$.

DEFINITION

The **derivative** of a function is another function that gives the original's slope at every point. We write it $f'(x)$ or $\frac{df}{dx}$ (read “dee-f dee-x”). The second form means “a tiny change in f per tiny change in x .” Both notations mean the same thing.

◆ THE SAME THING IN PYTORCH

PyTorch never computes the limit numerically. It applies known rules, like our $2x$, for you. You mark an input as something to track, compute an output, and call `.backward()` to get the derivative:

>_ THE DERIVATIVE OF x^2 AT $x = 3$

```
import torch

x = torch.tensor(3.0, requires_grad=True)    # "track this number"
y = x ** 2                                    #  $f(x) = x^2$ 

y.backward()                                  # compute  $df/dx$  at  $x = 3$ 
print(x.grad)                                # tensor(6.)  $\rightarrow 2*x = 2*3 =$ 
```

In words: `requires_grad=True` tells PyTorch to record how y was built from x . `backward()` walks back through that record and stores the derivative in `x.grad`. It is the $2x = 6$ we derived. You will rarely differentiate by hand again, but you must know what the machine computes.

CHECK YOURSELF 1

Use the limit definition: what is the derivative of the constant function $f(x) = 5$? And for $y = wx + b$, what is dy/dx , and why does b not appear in it?

2 Partial derivatives: change one input, freeze the rest

◆ INTUITION FIRST

Real models depend on thousands of numbers, not one. A function like $f(x, y)$ has two inputs, so “the slope” is unclear: the slope as we change x , or as we change y ? A **partial derivative** answers one at a time. It measures how the output changes as you change *one* input while you hold all the others still. It is the ordinary derivative from Section 1. Every variable except the chosen one is treated as a fixed constant.

☂ EVERYDAY PARALLEL

A shower has two taps: **hot** and **cold**. The water temperature depends on both. To learn what the hot tap alone does, leave the cold tap where it is. Turn only the hot one, and feel how the temperature responds. That is a partial derivative: the effect of one input while the other is held fixed. Turn only the cold tap instead, and you measure the other partial derivative.

◆ NOW THE MATH

We replace the straight d with a curly ∂ (“partial”). It signals: “there are other inputs, but I am holding them still.” To compute $\frac{\partial f}{\partial x}$, treat every other variable as a plain number. Take $f(x, y) = x^2y + 3y$:

$$\frac{\partial f}{\partial x} = 2xy, \quad \frac{\partial f}{\partial y} = x^2 + 3$$

In words: for $\partial f / \partial x$ we treat y as a constant. Then x^2y behaves like (constant) $\times x^2$, which gives $2xy$. The term $3y$ has no x in it, so it is a constant, and its derivative is zero. For $\partial f / \partial y$ we freeze x instead. Now x^2y is (constant) $\times y$, whose slope in y is the constant x^2 . The term $3y$ adds 3. The rules are the same as before; only one variable moves at a time.

◆ THE SAME THING IN PYTORCH

>_ TWO PARTIAL DERIVATIVES AT ONCE

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

f = x**2 * y + 3*y          #  $f(x, y) = x^2 * y + 3y$ 
f.backward()                 # fills in BOTH partial derivatives

print(x.grad)               # tensor(20.) →  $2*x*y = 2*2*5 = 20$ 
print(y.grad)               # tensor(7.) →  $x^2 + 3 = 4 + 3 = 7$ 
```

In words: one call to `backward()` computes the partial derivative for every tracked input in a single pass. This is what makes training possible: the machine gets the effect of *all* the inputs together, not one slow experiment per input.

CHECK YOURSELF 2

For $f(x, y) = 4x + y^3$, what is $\partial f / \partial x$? At the point $(1, 2)$, which input changes f faster, x or y ?

3 The gradient: the direction of steepest climb

◆ INTUITION FIRST

If a function has many inputs, collect *all* its partial derivatives into one vector. That vector is the **gradient**, written ∇f (the symbol ∇ is called “nabla” or “del”). The gradient points in the direction in which the function **increases fastest**. Its length says how steep that increase is. Flip its sign, and you get the direction of steepest *decrease*. When the function is our error, that is the way we want to move.

☂ EVERYDAY PARALLEL

You are on a mountainside in **thick fog**, and you want to get down. You cannot see the valley, but you can feel the ground under your feet. You feel which way slopes down most steeply, take a step that way, then feel again and repeat. The steepest downhill direction under your feet is the negative gradient. Like the model, you never see the whole map. You only feel the local slope, and that is enough to walk downhill.

◆ NOW THE MATH

For a function of n inputs, the gradient is the column of all partial derivatives, one per input:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

In words: slot i of the gradient is “how much the output changes when I change input i .”

Together, these slots form an arrow in input space. Large values mark inputs that matter a lot at this point. Values near zero mark inputs that barely move the output here.

Why does this arrow point uphill fastest? Take a small step of length ε in some direction $\mathbf{u}$ (a vector of length 1). The straight-line rule from Section 1, applied to each input, says the output changes by about the sum of (partial derivative $\times$ step) over all inputs. That sum is a dot product:

$$\begin{aligned} f(\mathbf{x} + \varepsilon \mathbf{u}) - f(\mathbf{x}) &\approx \varepsilon \sum_i \frac{\partial f}{\partial x_i} u_i = \varepsilon \nabla f \cdot \mathbf{u} \\ &= \varepsilon \|\nabla f\| \|\mathbf{u}\| \cos \theta = \varepsilon \|\nabla f\| \cos \theta \end{aligned}$$

In words: θ is the angle between your step and the gradient. We used the geometric form of the dot product from Section 1. The change is largest when $\cos \theta = 1$, that is, when you step along ∇f . It is most negative when $\cos \theta = -1$: step along $-\nabla f$. It is zero when you step at a right angle to the gradient. That is the direction along a contour line, where the height stays the same.

DEFINITION

Gradient descent is a method for minimizing a function. From where you stand, compute the gradient, then take a small step in the *opposite* direction (downhill). Repeat. If the step size is small enough, each step lowers the value, and enough steps carry you toward a minimum. In ML the function we minimize is the **loss** L (a number that measures how wrong the model is; smaller is better). So going down the loss means the model gets better.

The step is one short update rule, applied again and again:

$$\theta \leftarrow \theta - \alpha \nabla L$$

In words: θ ("theta") stands for the model's parameters, all the numbers inside the weight matrices from Section 1. ∇L is the gradient of the loss L with respect to those parameters: the direction toward more error. We subtract it, so we move toward *less* error. α ("alpha") is the *learning rate*, a small positive number that sets the size of each step. The arrow $\leftarrow$ means "compute the right side, then replace θ with it."

Why "small enough"? Use the dot-product result above with the step $-\alpha \nabla L$:

$$L(\theta - \alpha \nabla L) \approx L(\theta) - \alpha \nabla L \cdot \nabla L = L(\theta) - \alpha \|\nabla L\|^2$$

In words: the straight-line estimate says the loss drops by $\alpha \|\nabla L\|^2$, which is never negative. But the estimate holds only for small steps. With a large α , the curve bends away from the straight line before you land, and the loss can go up instead.

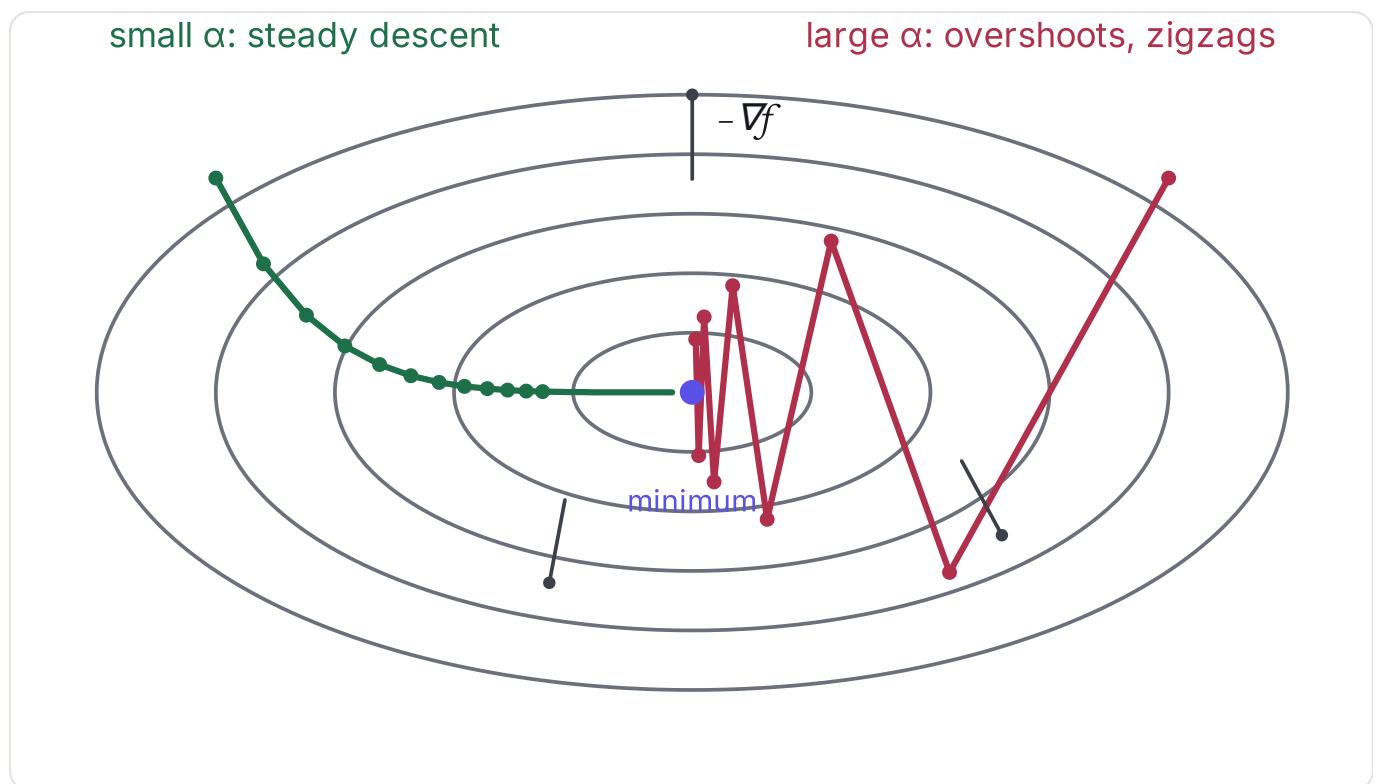


Figure 2.2 Contour lines of $f(x, y) = x^2 + 4y^2$ (each ellipse joins points of equal height). The short dark arrows are $-\nabla f$: they cross the contours at right angles and point downhill. Green: gradient descent with $\alpha = 0.05$. Rose: $\alpha = 0.23$, which overshoots the narrow direction on every step.

▲ WATCH OUT

The learning rate α is the step size, and it needs care. Too **small**, and training is very slow: thousands of tiny steps to get anywhere. Too **large**, and you overshoot the valley on every step. You bounce from wall to wall, or the values grow without limit until the loss becomes `NaN` (“not a number”). In the code below, `lr = 1.1` instead of `0.1` sends x to $-12, 14.4, -17.28$, and further out on every step. Choosing α is the first setting you will tune on almost every model.

◆ THE SAME THING IN PYTORCH

>_ GRADIENT DESCENT ON $F(X) = X^2$, WALKING TO THE BOTTOM

```
import torch

x = torch.tensor(10.0, requires_grad=True)  # start far from the minimum
lr = 0.1                                     # learning rate (alpha)

for step in range(5):
    loss = x ** 2                            # we want to minimize this
    loss.backward()                          # grad = df/dx = 2x
    with torch.no_grad():                    # don't track the update itself
        x -= lr * x.grad                    # theta ← theta - alpha * grad
    x.grad.zero_()                           # clear grad before the next step
    print(f"x = {x.item():.4f}")

# x = 8.0000
# x = 6.4000
# x = 5.1200
# x = 4.0960
# x = 3.2768 → heading toward 0, the minimum of x^2
```

In words: each pass reads the slope at the current point and steps against it by `lr * grad`. Here each step multiplies x by $1 - 2\alpha = 0.8$, so the value slides down the bowl toward zero. Clearing `x.grad` matters because PyTorch *adds* new gradients to old ones. Forget it, and the previous step's slope mixes into this one.

CHECK YOURSELF 3

You minimize a loss, and the gradient with respect to one weight is $+4$. With learning rate 0.1 , does the weight go up or down, and by how much? In the code above, which learning rate would reach $x = 0$ in one step?

4 The chain rule: this is backpropagation

◆ INTUITION FIRST

Neural networks are **compositions**: functions inside functions inside functions. The input feeds layer 1. Its output feeds layer 2, whose output feeds layer 3, and so on to the loss at the end. To train, we need the derivative of that final loss with respect to a weight deep inside, many layers away from the output. The **chain rule** reaches back through the whole stack. Its idea is simple: the derivative of a chain of functions is the *product* of the small derivatives along the chain. Each link contributes its own local rate, and you multiply them.

☂ EVERYDAY PARALLEL

You change money: **US dollars** → **euros** → **pounds**. The rates change every day. Today 1 dollar buys 0.9 euros, and 1 euro buys 0.85 pounds. How many pounds does 1 extra dollar get you today? You *multiply* today's rates: $0.9 \times 0.85 = 0.765$ pounds per dollar. Tomorrow the rates are different, and so is the product. In the chain rule, too, each local rate depends on where you are (the slope of u^2 is $2u$). You multiply the rates at the current point.

◆ NOW THE MATH

Suppose y depends on u , and u depends on x : a two-link chain $x \rightarrow u \rightarrow y$. The chain rule says:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

In words: "how y changes as x moves" equals "how y changes as u moves" times "how u changes as x moves." The middle variable u seems to cancel, like in a fraction, which helps you remember the rule. Each factor is a *local* derivative, easy on its own. The chain rule joins them by multiplication.

Where does the rule come from? Use the straight-line rule from Section 1 twice. A small change Δx changes u by about $\frac{du}{dx} \Delta x$. That change in u changes y by about $\frac{dy}{du}$ times itself:

$$\Delta u \approx \frac{du}{dx} \Delta x, \quad \Delta y \approx \frac{dy}{du} \Delta u \approx \frac{dy}{du} \frac{du}{dx} \Delta x$$

In words: divide by Δx and let it shrink to zero. The ratio $\Delta y / \Delta x$ becomes dy/dx , and the right side becomes the product of the two local derivatives. (A fully careful proof handles the limits more precisely, but this is the idea.)

Let's work a full example. Take $y = (3x + 1)^2$ and split it into an inner and an outer piece:

$$u = 3x + 1, \quad y = u^2$$

Now the two local derivatives, each easy on its own:

$$\frac{dy}{du} = 2u, \quad \frac{du}{dx} = 3$$

Multiply them, then put $u = 3x + 1$ back in:

$$\begin{aligned} \frac{dy}{dx} &= \frac{dy}{du} \cdot \frac{du}{dx} \\ &= 2u \cdot 3 \\ &= 6u \\ &= 6(3x + 1) \\ &= 18x + 6 \end{aligned}$$

In words: first we found the outer function's slope ($2u$, the derivative of a square). Then we found the inner function's slope (3 , the derivative of $3x + 1$). We *multiplied* them. Then we put $u = 3x + 1$ back, so the answer is in terms of x . Check it: expand $(3x + 1)^2 = 9x^2 + 6x + 1$, whose derivative is $18x + 6$. It matches, and the chain rule never needed the expansion.

The multivariable chain rule: sum over every path

In a network, a variable usually affects the output through *several* routes at once. Then the chain rule gains one addition: differentiate along each path, then **add the paths up**. Suppose L depends on a and b , and both of those depend on x :

$$\frac{dL}{dx} = \frac{\partial L}{\partial a} \cdot \frac{da}{dx} + \frac{\partial L}{\partial b} \cdot \frac{db}{dx}$$

In words: x reaches L in two ways, through a and through b . Each route is a chain-rule product (multiply the rates along it). Both routes carry an effect, so we *add* them. More paths mean more terms in the sum. The reason is the same straight-line rule, now with two inputs:

$\Delta L \approx \frac{\partial L}{\partial a} \Delta a + \frac{\partial L}{\partial b} \Delta b$. Replace Δa and Δb with their own straight-line estimates, and divide by Δx .

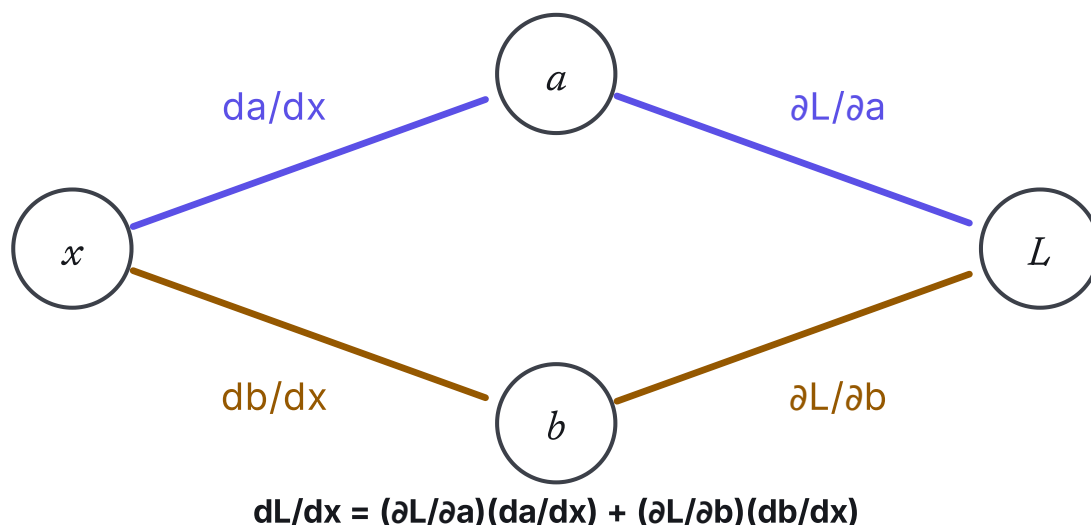


Figure 2.3 Two paths lead from x to L . Multiply the local derivatives along each path (violet, amber), then add the two products.

◆ WHY THIS MATTERS FOR ML

Backpropagation is the chain rule applied to a neural network. The network is a long composition of layers, and the loss is at the far end. To learn, we need the gradient of the loss with respect to every weight, however deep. Backpropagation starts at the loss and multiplies local derivatives layer by layer as it walks *backward*. It adds the products wherever paths meet. When PyTorch's `.backward()` ran in the code above, this is what it did. We build it by hand, with every multiplication visible, in Section 10.

◆ THE SAME THING IN PYTORCH

>_ THE CHAIN RULE, COMPUTED FOR US

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = (3*x + 1) ** 2          # the composed function from above

y.backward()                # the chain rule runs automatically
print(x.grad)               # tensor(42.) → 18*x + 6 = 18*2 + 6 = 42
```

In words: PyTorch recorded that y is the square of $3x + 1$. `backward()` walked that chain from the outside in. It multiplied $2u$ by 3, as we did by hand, and returned 42. That matches $18x + 6$ at $x = 2$. The rule you learned is the rule PyTorch runs.

CHECK YOURSELF 4

Let $y = (x^2 + 1)^3$. Use the chain rule with inner $u = x^2 + 1$ to find dy/dx . Then imagine a chain of 30 links, each with local derivative 0.5. What is the derivative of the whole chain?

5 The Jacobian: a gradient for every output at once

◆ INTUITION FIRST

So far the output was a single number. But a layer usually maps a vector of inputs to a vector of outputs. Each output has its own slope with respect to each input. Put all of these slopes in a grid, with one row per output and one column per input. That grid is the **Jacobian** matrix. It extends the gradient idea. A function with one output has one gradient *vector*. A function with many outputs has a *matrix* of partial derivatives.

☂ EVERYDAY PARALLEL

A mixing desk in a music studio has several input sliders (drums, bass, voice) and several output speakers. Each speaker's volume responds to each slider by some amount. Write those amounts in a table: one row per speaker, one column per slider. That table is a Jacobian. Row i tells you how speaker i responds to every slider.

◆ NOW THE MATH

For a function $\mathbf{f}$ with n inputs and m outputs, the Jacobian $\mathbf{J}$ has shape $(m \times n)$. Its entry in row i , column j is

$$J_{ij} = \frac{\partial f_i}{\partial x_j}$$

In words: row i is the gradient of output i , laid on its side. Column j shows how all outputs respond to input j .

An example with two inputs and two outputs: $\mathbf{f}(x, y) = [x^2y, 5x + \sin y]$.

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} = \begin{bmatrix} 2xy & x^2 \\ 5 & \cos y \end{bmatrix} \xrightarrow{x=1, y=2} \begin{bmatrix} 4 & 1 \\ 5 & -0.416 \end{bmatrix}$$

Now chain two such functions: $\mathbf{u} = \mathbf{f}(\mathbf{x})$, then $\mathbf{y} = \mathbf{g}(\mathbf{u})$. Each input x_j reaches each output y_i through every middle variable u_k . The multivariable chain rule sums over those paths:

$$\frac{\partial y_i}{\partial x_j} = \sum_k \frac{\partial y_i}{\partial u_k} \frac{\partial u_k}{\partial x_j} \quad \implies \quad \mathbf{J}_{\mathbf{g} \circ \mathbf{f}} = \mathbf{J}_{\mathbf{g}} \mathbf{J}_{\mathbf{f}}$$

In words: the sum on the left is “row i of $\mathbf{J}_{\mathbf{g}}$ dotted with column j of $\mathbf{J}_{\mathbf{f}}$.” That is the rule for matrix multiplication from Section 1. So the chain rule for vector functions is a matrix product of Jacobians. This is where Section 1's shape rules and this section's chain rule meet. Backpropagation is a sequence of such products (in practice, it never builds the full matrices, as Section 10 shows).

◆ THE SAME THING IN PYTORCH

>_ THE JACOBIAN OF A TWO-OUTPUT FUNCTION

```
import torch
from torch.autograd.functional import jacobian

def f(v):                                #  $f(x, y) = [x^2 * y, 5x + \sin y]$ 
    x, y = v[0], v[1]
    return torch.stack([x**2 * y, 5*x + torch.sin(y)])

J = jacobian(f, torch.tensor([1.0, 2.0])) # at the point  $x = 1, y = 2$ 
print(J)
# tensor([[ 4.0000,  1.0000],
#         [ 5.0000, -0.4161]])
```

In words: `jacobian` computes every partial derivative of every output. Row 0 is the gradient of x^2y , and row 1 is the gradient of $5x + \sin y$. The numbers match the matrix we derived by hand.

CHECK YOURSELF 5

$\mathbf{f}$ maps 3 numbers to 2 numbers, and $\mathbf{g}$ maps 2 numbers to 4 numbers. What are the shapes of $\mathbf{J}_f$, $\mathbf{J}_g$, and the Jacobian of $\mathbf{g} \circ \mathbf{f}$? Why must the product be $\mathbf{J}_g \mathbf{J}_f$ and not $\mathbf{J}_f \mathbf{J}_g$?

6 What to carry forward

- A **derivative** is a rate of change at one point: the slope, defined by shrinking a step h to zero. The slope of x^2 is $2x$. Near a point, $f(x + h) \approx f(x) + f'(x)h$.
- A **partial derivative** is the ordinary derivative with respect to one input while all others are held fixed: the hot tap while the cold tap stays put.
- The **gradient** ∇f stacks all partial derivatives into a vector that points uphill fastest. **Gradient descent** steps the other way to shrink a loss: $\theta \leftarrow \theta - \alpha \nabla L$, with a learning rate α that is small enough.
- The **chain rule** multiplies local derivatives along a composition, and adds over paths when they meet. This *is* backpropagation, which we build in Section 10.
- The **Jacobian** extends the gradient to many outputs, $J_{ij} = \partial f_i / \partial x_j$. The chain rule for vector functions becomes a matrix product of Jacobians.

You now have two of the three foundations of ML: linear algebra to move data, and calculus to improve the model. Next comes the third, probability, so that a model can give not only an answer but also how sure it is.

Answers to “Check yourself”

1. Use the limit definition: what is the derivative of the constant function $f(x) = 5$? And for $y = wx + b$, what is dy/dx , and why does b not appear in it?

For $f(x) = 5$: $f(x + h) - f(x) = 5 - 5 = 0$. The fraction is $0/h = 0$ for every non-zero h , so the limit is **0**. A flat line has slope zero everywhere; a constant never changes. For $y = wx + b$: the wx part has slope w , and the constant b adds 0. So $dy/dx = w$. The bias moves the line up or down but does not tilt it. (This is true for this straight line. Once a curved function is applied after $wx + b$, the slope can depend on b .)

2. For $f(x, y) = 4x + y^3$, what is $\partial f / \partial x$? At the point $(1, 2)$, which input changes f faster, x or y ?

$\partial f / \partial x = 4$. When we differentiate with respect to x , we freeze y . So y^3 is a fixed number, and its derivative is zero. The other partial is $\partial f / \partial y = 3y^2$, which is **12** at $y = 2$. So at $(1, 2)$, a small change in y moves f three times as much as the same change in x . At $y = 0.5$ the answer would flip: $3y^2 = 0.75 < 4$. Partial derivatives depend on the point.

3. You minimize a loss, and the gradient with respect to one weight is +4. With learning rate 0.1, does the weight go up or down, and by how much? In the code above, which learning rate would reach $x = 0$ in one step?

The update is $\theta - 0.1 \times 4 = \theta - 0.4$. The weight goes **down by 0.4**. A positive gradient means “increasing this weight increases the loss,” so we push the weight the other way. For x^2 each step multiplies x by $1 - 2\alpha$. That is zero when $\alpha = 0.5$, so one step lands exactly on the minimum. Above $\alpha = 1$, the factor $|1 - 2\alpha|$ is bigger than 1, and x grows: the overshoot in the Watch-out box.

4. Let $y = (x^2 + 1)^3$. Use the chain rule with inner $u = x^2 + 1$ to find dy/dx . Then imagine a chain of 30 links, each with local derivative 0.5. What is the derivative of the whole chain?

Outer: $y = u^3$ gives $dy/du = 3u^2$. Inner: $u = x^2 + 1$ gives $du/dx = 2x$. Multiply: $dy/dx = 3u^2 \cdot 2x = 6x(x^2 + 1)^2$. The chain rule saved us from expanding a cube. For 30 links: $0.5^{30} \approx 9 \times 10^{-10}$, almost zero. This is why gradients in deep networks can be computed at all (each layer adds one factor to a product). It is also why they can *vanish* or *explode* when many factors are below or above 1. We return to this in Section 12.

5. $\mathbf{f}$ maps 3 numbers to 2 numbers, and $\mathbf{g}$ maps 2 numbers to 4 numbers. What are the shapes of $\mathbf{J}_\mathbf{f}$, $\mathbf{J}_\mathbf{g}$, and the Jacobian of $\mathbf{g} \circ \mathbf{f}$? Why must the product be $\mathbf{J}_\mathbf{g}\mathbf{J}_\mathbf{f}$ and not $\mathbf{J}_\mathbf{f}\mathbf{J}_\mathbf{g}$?

$\mathbf{J}_\mathbf{f}$ is (2×3) : 2 outputs, 3 inputs. $\mathbf{J}_\mathbf{g}$ is (4×2) . The chain $\mathbf{g} \circ \mathbf{f}$ maps 3 numbers to 4, so its Jacobian is (4×3) . The shape rule confirms the order: $(4 \times 2)(2 \times 3) = (4 \times 3)$, with the inner 2s matching. The other order, $(2 \times 3)(4 \times 2)$, has inner numbers 3 and 4, so it is not even defined.

Probability & Statistics

Models do not output answers; they output beliefs. Probability is the language for reasoning carefully about things we are not sure of. It is also where the common loss functions come from.

48 min read

YOU'LL LEARN

- distributions
- expectation
- variance
- Bayes' rule
- maximum likelihood

Before you start: Section 2: Calculus for ML.

Machine learning lives in a world of uncertainty. Is this email spam? Which digit is in this blurry photo? A good model does not give a single answer as if it were certain. It gives a **degree of belief**: “90% spam,” “probably a 7, maybe a 1.” Probability is the mathematics of those graded beliefs. Statistics is how we adjust them to fit data. This section builds that toolkit from the start.

◆ THE ONE IDEA TO HOLD ONTO

A model outputs a **distribution**: belief spread across the possible answers, not one answer. Everything in this section does one of three things. It *describes* a distribution (expectation, variance). It names a common *shape* (Bernoulli, Gaussian). Or it *updates or fits* a distribution from evidence (Bayes, maximum likelihood). Keep that map in mind and the symbols will fall into place.

1 Random variables and distributions

◆ INTUITION FIRST

A **random variable** is a number whose value is uncertain until you look. It is the outcome of some process you cannot predict exactly. Roll a die and call the result X . Before the roll, X could be any of 1 to 6. A **distribution** is the full description of that uncertainty. It tells you, for every possible value, how likely that value is. If the random variable is the question (“what will come up?”), the distribution is the answer sheet, with the chance written next to each option.

Random variables come in two kinds, and the difference matters for every formula below:

- **Discrete:** the values are separate and countable, like 1, 2, 3. You can list them. Examples: a die, a coin, the number of emails in your inbox.
- **Continuous:** the values can be any real number in some range, with no gaps. Examples: a temperature, a height, a waiting time.

☂ EVERYDAY PARALLEL

Compare a **die roll** with **tomorrow's temperature**. The die is discrete: six outcomes, each with chance $1/6$. It makes sense to ask “what is the chance of a 4?” The temperature is continuous: it could be 21.0° , 21.3° , 21.3007° , and so on. Asking “what is the chance it is *exactly* $21.30000\dots^\circ$?” is the wrong question. There are infinitely many values, so any single one has zero chance. Instead you ask “what is the chance it lands *between* 21° and 22° ?” This shift from single values to ranges is the main difference between the two kinds.

◆ NOW THE MATH

For a **discrete** variable we use a *probability mass function* (PMF), written $P(X = x)$. It is the amount of probability that sits *on* each value. Two rules always hold. Every mass is between 0 and 1. And all the masses add up to 1 (something must happen).

$$0 \leq P(X = x) \leq 1, \quad \sum_x P(X = x) = 1$$

In words: each outcome gets a share of belief between “impossible” (0) and “certain” (1). The shares of all outcomes together make one whole. For a fair die, $P(X = x) = 1/6$ for each of the six faces, and $6 \times \frac{1}{6} = 1$. It is called “mass” because you can picture a lump of probability placed on each value.

For a **continuous** variable, single values have zero probability. So we use a *probability density function* (PDF) $p(x)$ instead. Density is not probability. It is probability *per unit of x* , like grams per centimeter. To get a probability, you add up the density over a range. For a smooth curve, that means taking an **integral** (the area under the curve):

$$P(a \leq X \leq b) = \int_a^b p(x) dx, \quad \int_{-\infty}^{\infty} p(x) dx = 1$$

In words: the chance that X falls between a and b is the *area* under the density curve over that range. The $\int$ sign is the continuous version of the $\sum$ sign: “add up over a smooth range” instead of “add up over a list.” The total area under the whole curve is 1, the continuous version of “the masses add up to 1.”

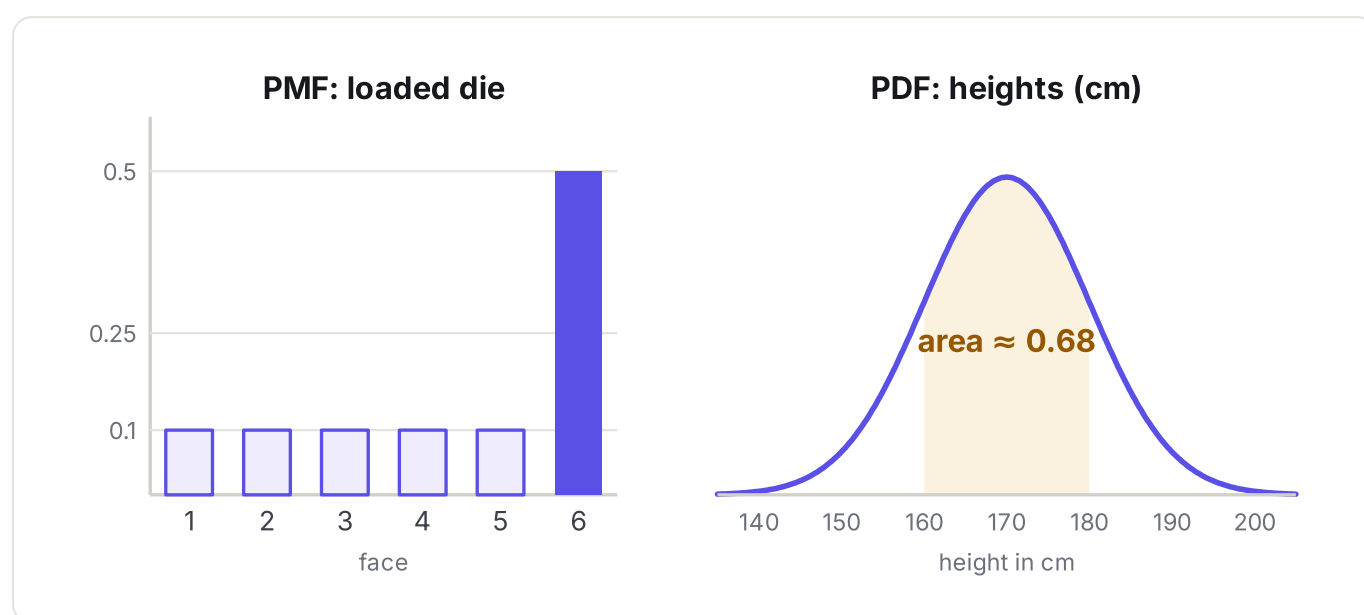


Figure 3.1 Left: a PMF puts a lump of probability on each value; the bar heights add up to 1. Right: a PDF is a curve, and probability is area under it. The shaded area between 160 cm and 180 cm is about 0.68.

DEFINITION

A **distribution** is the complete description of how likely each value of a random variable is. For discrete variables it is a **PMF** (a mass on each value). For continuous variables it is a **PDF** (a density curve whose *area* gives probability). The masses add up to 1, and the total area is 1.

◆ THE SAME THING IN PYTORCH

>_ A DISCRETE DISTRIBUTION (A LOADED DIE) AND SAMPLING FROM IT

```
import torch
torch.manual_seed(0)

# a PMF over 6 faces: six numbers that must add up to 1
probs = torch.tensor([0.10, 0.10, 0.10, 0.10, 0.10, 0.50]) # face 6 is loaded
print(probs.sum()) # tensor(1.) → a valid distribution

die = torch.distributions.Categorical(probs=probs)

# draw 10,000 rolls and count how often the loaded face appears
samples = die.sample((10_000,))
print((samples == 5).float().mean()) # tensor(0.5046) (index 5 is the loaded face)
```

In words: `Categorical` is PyTorch's name for a distribution over a list of outcomes. It numbers the faces from 0, so the face “6” is index 5. The share of 6s in 10,000 rolls is close to 0.5, as the PMF says. A different seed gives a slightly different number, near 0.50.

CHECK YOURSELF 1

A PDF has the value $p(3.0) = 2$ at one point. Is that a mistake, since probabilities cannot be above 1? What is $P(X = 3.0)$, and what should you ask instead?

2 Expectation and variance

◆ INTUITION FIRST

Once you have a distribution, you first want a one-number summary: “on average, what value should I expect?” That is the **expectation** (also called the *mean* or *expected value*), written $\mathbb{E}[X]$. It is the long-run average you would reach if you repeated the random process many, many times. It is a *weighted* average: outcomes that are more likely pull the average toward them more strongly.

☂ EVERYDAY PARALLEL

A slot machine pays *0 dollars on most spins, but 100 dollars one time in fifty*. What do you win **per spin, on average**? Not 100 dollars (that is rare), and not 0 dollars (you do win sometimes). You weight each payout by how often it happens: $\frac{49}{50} \times 0 + \frac{1}{50} \times 100 = 2$ dollars per spin. Those 2 dollars are the expectation. The casino sets the machine so that your expected win is less than the price of a spin. The casino's profit is an expectation calculation in its favor.

◆ NOW THE MATH

For a discrete variable, the expectation is the probability-weighted sum of the values:

$$\mathbb{E}[X] = \sum_x x P(X = x)$$

In words: go through every possible value x , multiply it by its probability, and add all the products. Likely values contribute more; rare ones barely change the total. The result is the balance point of the distribution: the place where it would balance on a seesaw.

Let's compute one. A **fair die** has values 1 to 6, each with probability $1/6$:

$$\begin{aligned}\mathbb{E}[X] &= 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} \\ &= \frac{1}{6} (1 + 2 + 3 + 4 + 5 + 6) \\ &= \frac{1}{6} \cdot 21 = 3.5\end{aligned}$$

In words: the expected roll is 3.5. You can never *roll* a 3.5. The expectation does not have to be a possible outcome; it is the long-run average. Roll ten thousand times, and the running average will be very close to 3.5.

The mean tells you *where* a distribution sits. The next question is *how spread out* it is. Do outcomes stay close to the mean, or scatter widely? That is the **variance**, written $\text{Var}[X]$. We measure each outcome's distance from the mean $\mu = \mathbb{E}[X]$ and *square* it. Then we take the expected value of that:

$$\text{Var}[X] = \mathbb{E}[(X - \mu)^2]$$

In words: variance is the average squared distance from the mean. Large variance means outcomes often land far from the average (an unpredictable variable). Small variance means they stay close to it (a steady one). We square the distances so that a swing of +3 and a swing of -3 both count as spread, instead of canceling. The square root of the variance, σ (sigma), is the *standard deviation*. It is the same idea, back in the original units.

A second form of the variance is often easier to compute. Expand the square and use the fact that an expectation of a sum is the sum of expectations:

$$\begin{aligned}\text{Var}[X] &= \mathbb{E}[X^2 - 2\mu X + \mu^2] \\ &= \mathbb{E}[X^2] - 2\mu \mathbb{E}[X] + \mu^2 \\ &= \mathbb{E}[X^2] - \mu^2\end{aligned}$$

In words: the mean of the squares, minus the square of the mean. (The last step uses $\mathbb{E}[X] = \mu$.) For the fair die, $\mathbb{E}[X^2] = \frac{1}{6}(1 + 4 + 9 + 16 + 25 + 36) = \frac{91}{6}$. So $\text{Var}[X] = \frac{91}{6} - 3.5^2 = \frac{35}{12} \approx 2.92$.

EVERYDAY PARALLEL

Two basketball players both average 20 points a game: same expectation. One scores 19, 21, 20, 20, 20 (a **reliable** player, tiny variance). The other scores 2, 40, 5, 38, 15 (a **streaky** player, huge variance). The mean cannot tell them apart; the variance can. A coach who chooses a player for the last seconds of a close game cares about variance as much as about the average.

◆ THE SAME THING IN PYTORCH

>_ MEAN AND VARIANCE, FROM SAMPLES AND FROM LISTS

```
import torch
torch.manual_seed(0)

# simulate the fair die and check both summaries
rolls = torch.randint(1, 7, (100_000,)).float() # 100k rolls of 1..6

print(rolls.mean()) # tensor(3.4991) close to E[X] = 3.5
print(rolls.var(correction=0)) # tensor(2.9190) close to 35/12 = 2.9167

# reliable vs streaky: same mean, very different variance
reliable = torch.tensor([19., 21., 20., 20., 20.])
streaky = torch.tensor([ 2., 40.,  5., 38., 15.])
print(reliable.mean(), streaky.mean()) # tensor(20.) tensor(20.)
print(reliable.var(correction=0), streaky.var(correction=0)) # tensor(0.) tensor(120.)
```

In words: `correction=0` divides by N , the number of values. That is the average squared distance, as in our formula. PyTorch's default is `correction=1`, which divides by $N - 1$. This “sample variance” is slightly larger (0.5 and 324.5 here). It corrects for the fact that the mean was estimated from the same few values. With 100,000 rolls, the two versions are almost equal.

CHECK YOURSELF 2

A bet pays +10 dollars with probability 0.2 and −3 dollars with probability 0.8. What is the expected payout? Now add 5 dollars to every payout. How do the mean and the variance change?

3 Two distributions you will meet everywhere

Most of ML relies on a small number of standard distribution shapes. Two of them appear so often that they are worth knowing well.

The Bernoulli: one yes-or-no event

◆ INTUITION FIRST

The **Bernoulli distribution** is the simplest one there is. It describes a single event with two outcomes, 1 (“success”) or 0 (“failure”). One number p controls it: the probability of a 1. Examples: a coin flip, a click or no click, spam or not spam. Whenever a model answers a single yes/no question, a Bernoulli is behind it.

☂ EVERYDAY PARALLEL

Flip a bent coin that comes up heads 70% of the time. Then $p = 0.7$: heads (1) has probability 0.7, and tails (0) has probability 0.3. That one number p is the entire distribution. There is nothing else to know.

◆ NOW THE MATH

Its PMF can be written as one line that works for both outcomes:

$$P(X = x) = p^x (1 - p)^{1-x}, \quad x \in \{0, 1\}$$

In words: put in $x = 1$ and the formula becomes p (the chance of a 1). Put in $x = 0$ and it becomes $1 - p$ (the chance of a 0). The exponents work as a switch: for whichever outcome you ask about, the other factor gets exponent 0 and becomes 1.

Its mean and variance follow from Section 2. Because X is only ever 0 or 1, $X^2 = X$:

$$\mathbb{E}[X] = 0 \cdot (1 - p) + 1 \cdot p = p$$

$$\mathbb{E}[X^2] = 0^2 \cdot (1 - p) + 1^2 \cdot p = p$$

$$\text{Var}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = p - p^2 = p(1 - p)$$

In words: the mean of a 0/1 variable is the chance of a 1. The variance $p(1 - p)$ is 0 when p is 0 or 1 (no randomness at all). It is largest at $p = 0.5$, where it equals 0.25.

◆ THE SAME THING IN PYTORCH

›_ A BERNOULLI COIN: SAMPLED AND EXACT

```
import torch
torch.manual_seed(0)

coin = torch.distributions.Bernoulli(probs=0.7)
flips = coin.sample((100_000,))    # 100k flips, each 0.0 or 1.0

print(flips.mean())                # tensor(0.7007) close to p = 0.7
print(flips.var(correction=0))     # tensor(0.2097) close to p(1-p) = 0.21
print(coin.mean, coin.variance)    # tensor(0.7000) tensor(0.2100) the exact values
```

In words: the sample mean and variance of many flips come close to the formulas we derived. The distribution object also knows the exact values, p and $p(1 - p)$.

The Gaussian: the bell curve

◆ INTUITION FIRST

The **Gaussian** (also called the **normal** distribution) is the well-known bell curve. It is a smooth hump, tall in the middle, and it falls off equally on both sides. It is the default shape for a continuous quantity that stays near a typical value with random variation on either side. Examples: heights, measurement errors, exam scores, sensor noise.

☂ EVERYDAY PARALLEL

Think of the heights of adults in a city. Most people are near the average, say 170 cm. Fewer are 185 cm, and even fewer 200 cm. The same thinning happens toward the short end. Plot how many people have each height and you get a bell: crowded in the middle, sparse at the edges, roughly symmetric. That is a Gaussian in real life.

◆ NOW THE MATH

Two numbers control the Gaussian PDF: its mean μ and its standard deviation σ :

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

In words: μ (mu) is the *center*: where the peak is, the typical value. σ (sigma) is the *width*: how far values spread around the center. The main part of the formula is $-(x - \mu)^2$. The farther x is from the center, the more negative this gets, and $\exp$ of a large negative number is tiny. So the density falls quickly as you move away from μ . The factor $\frac{1}{\sigma\sqrt{2\pi}}$ in front makes the total area equal 1. (We will not prove that area here; it needs a calculus trick with a double integral.)

◆ WHY THE GAUSSIAN IS EVERYWHERE

The main reason is the **central limit theorem**. When you add up (or average) many small, independent random effects, the total tends toward a Gaussian. This happens almost regardless of the shape of each single effect. We will not prove it here. A person's height is the sum of many small genetic and environmental effects. A measurement error is the sum of many tiny disturbances. Many quantities in the world are “many small random pushes added together,” so the bell curve appears again and again. That is why ML often assumes that noise is Gaussian.

◆ THE SAME THING IN PYTORCH

>_ SAMPLING HEIGHTS FROM A GAUSSIAN

```
import torch
torch.manual_seed(0)

# a Gaussian centered at mu = 170 with spread sigma = 10 (heights in cm)
normal = torch.distributions.Normal(loc=170.0, scale=10.0)

samples = normal.sample((100_000,))
print(samples.mean(), samples.std())    # tensor(169.9756) tensor(10.0124)

# about 68% of a Gaussian lies within one sigma of the mean:
within_1sigma = ((samples > 160) & (samples < 180)).float().mean()
print(within_1sigma)                    # tensor(0.6811) the 68% rule
```

In words: the samples recover the two numbers we chose. The share of samples between 160 cm and 180 cm is the shaded area in Figure 3.1. For any Gaussian, the area within one σ of the mean is about 0.683.

CHECK YOURSELF 3

One Bernoulli has $p = 0.5$ and another has $p = 0.9$. If you always guess the more likely outcome, how often are you wrong for each? How does the variance $p(1 - p)$ match that?

4 Conditional probability and Bayes' rule

◆ INTUITION FIRST

Conditional probability, written $P(A \mid B)$ and read “the probability of A *given* B ,” is how likely A is *when you already know that B happened*. New information makes the set of possibilities smaller, and the chances change. If you learn that the sky is cloudy, the chance of rain goes up. The plain chance of rain and the chance of rain given clouds are two different numbers.

☂ EVERYDAY PARALLEL

The chance that a stranger is over 190 cm tall is small. But if you learn that they play professional basketball, that chance rises sharply. The fact did not change the person. It changed *your information*, and so it changed the probability you should give. That is all a conditional probability is: a belief updated with a known fact.

◆ NOW THE MATH

Start from the *joint probability*: the chance that *both* A and B happen, written $P(A, B)$. There are two equally valid ways to build it from a condition, depending on which event you condition on:

$$P(A, B) = P(A | B) P(B) = P(B | A) P(A)$$

In words: the chance of both A and B is the chance of B, times the chance of A once B is known: "B happens, and then A happens given B." You can also tell the story the other way round (A first, then B given A). Both ways reach the same joint probability, so the two products are equal.

Now set those two expressions equal and divide by $P(B)$. That one step of algebra is **Bayes' rule**:

$$\begin{aligned} P(A | B) P(B) &= P(B | A) P(A) \\ P(A | B) &= \frac{P(B | A) P(A)}{P(B)} \end{aligned}$$

In words: Bayes' rule lets you *reverse* a conditional. Suppose you know how likely the evidence B is when A is true, and how common A is to begin with. Then you can compute how likely A is once you have seen B. It is the formula for updating a belief with evidence.

THE THREE WORDS OF BAYES

The **prior** $P(A)$ is your belief in A *before* the evidence. The **likelihood** $P(B | A)$ is how probable the evidence B would be if A were true. The **posterior** $P(A | B)$ is your updated belief in A *after* seeing the evidence. Bayes' rule turns prior and likelihood into posterior.

EVERYDAY PARALLEL

Think of a **medical test** for a rare disease. You take it, and it comes back *positive*. Your first thought is “I am sick.” But Bayes says: first consider how rare the disease is (the prior). Then consider how often the test gives a false alarm for healthy people (false positives). The number that feels right and the true number can be very far apart. Let's put real numbers on it.

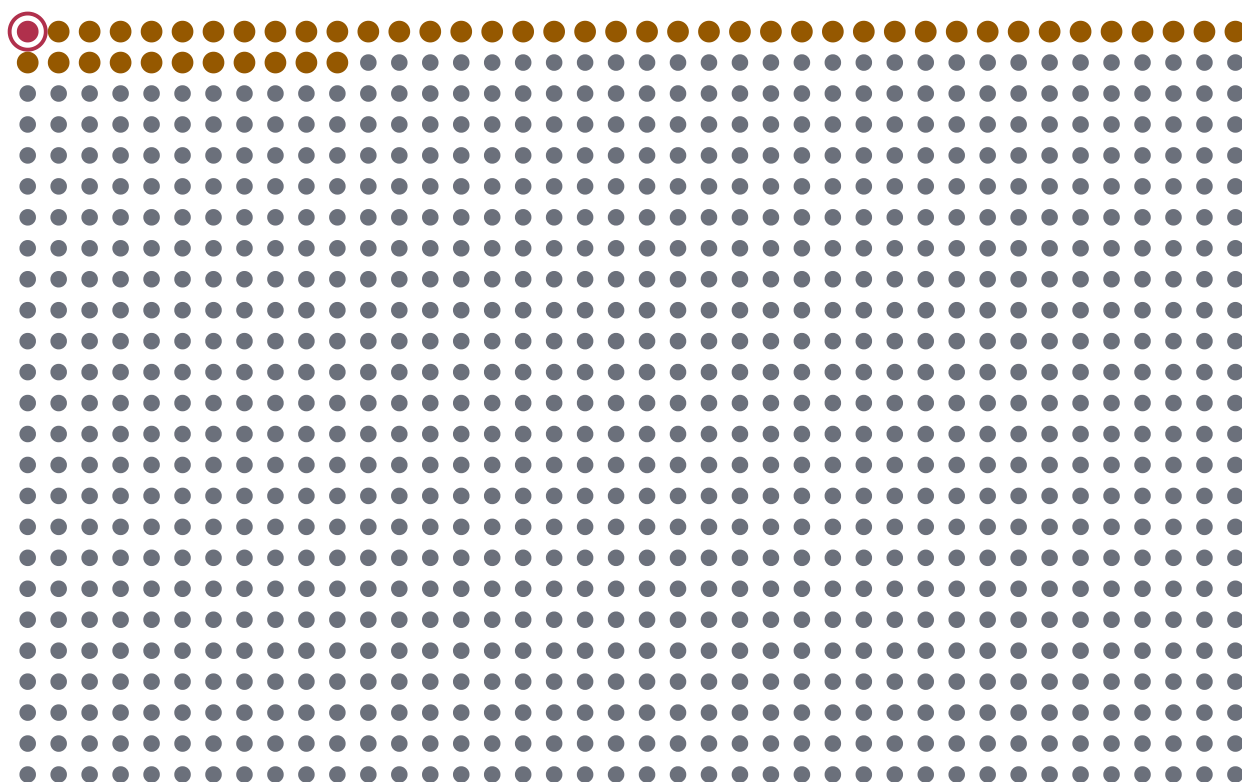
A concrete number: the base-rate effect

Suppose a disease affects **1 in 1000** people. The test is quite good. If you *have* the disease, it is positive 99% of the time (the likelihood). If you are *healthy*, it is wrongly positive only 5% of the time (the false-positive rate). You test positive. What is the real chance that you are sick?

Let D = “has the disease” and $+$ = “tests positive.” We want the posterior $P(D \mid +)$. Bayes' rule needs the total chance of a positive result, $P(+)$. That is the positives from sick people *plus* the positives from healthy people:

$$\begin{aligned} P(+) &= \underbrace{P(+ \mid D) P(D)}_{\text{true positives}} + \underbrace{P(+ \mid \neg D) P(\neg D)}_{\text{false positives}} \\ &= (0.99)(0.001) + (0.05)(0.999) \\ &= 0.00099 + 0.04995 = 0.05094 \\ P(D \mid +) &= \frac{P(+ \mid D) P(D)}{P(+)} = \frac{(0.99)(0.001)}{0.05094} \approx 0.0194 \end{aligned}$$

In words: the test catches 99% of real cases. Even so, after a positive result your chance of being sick is only about **2%**. The reason is that the disease is rare: the prior $P(D) = 0.001$ is tiny. Most positive results come from the large healthy group and its 5% false-positive rate. These far outnumber the few true positives. This is the **base-rate effect**. If you ignore how rare something is, you will overreact to evidence. Bayes' rule keeps the prior in the calculation.



- 1 sick person, tests positive
- ≈ 50 healthy people, test positive by mistake
- ≈ 949 healthy people, test negative

Figure 3.2 1000 people, one dot each. About 51 of them test positive, and only 1 of those 51 is sick. So a positive result means a chance of about 1 in 51, or 2%, that you are sick.

◆ THE SAME THING IN PYTORCH

>_ BAYES' RULE FOR THE MEDICAL TEST

```
import torch

p_disease = torch.tensor(0.001)    # prior: 1 in 1000
p_pos_sick = torch.tensor(0.99)    # likelihood: true-positive rate
p_pos_well = torch.tensor(0.05)    # false-positive rate

# total probability of a positive test (sick + healthy sources)
p_pos = p_pos_sick * p_disease + p_pos_well * (1 - p_disease)

posterior = (p_pos_sick * p_disease) / p_pos
print(posterior)    # tensor(0.0194) → about 2%, not 99%
```

In words: the code follows the calculation line by line. First the total chance of a positive result, then the share of it that comes from sick people.

CHECK YOURSELF 4

Suppose the disease is much more common: 1 in 10 people (prior 0.1) instead of 1 in 1000. Predict: does a positive test now make you more or less sure that you are sick? Then compute the posterior.

5 Maximum likelihood: fitting a distribution to data

◆ INTUITION FIRST

So far the distributions came with their numbers already known. In real ML you have *data*, and you must find the numbers yourself. **Maximum likelihood estimation** (MLE) is the standard method for that. Its idea is simple: **choose the parameters that make the observed data as probable as possible**. Of all the coins that *could* have produced your flips, MLE chooses the one under which those flips were most likely.

🌂 EVERYDAY PARALLEL

Someone rolled one of three dice: a fair die, a die loaded toward 6 (a 6 half the time), or a die loaded toward 1. You see the rolls 6, 6, 3, 6, 5, 6. Which die best explains them? For each die, compute the probability of these exact rolls. The fair die gives $(1/6)^6 \approx 0.00002$. The 6-loaded die gives $0.5^4 \times 0.1^2 = 0.000625$, about 29 times more. The 1-loaded die gives 0.1^6 , far less. MLE picks the 6-loaded die. With a coin, the “dice” are all the values of p between 0 and 1.

◆ NOW THE MATH

Let's derive it fully for a coin. You flip a coin with unknown heads probability p and observe H heads in N flips. Each flip is a Bernoulli. The flips are independent, so the probability of the whole sequence is the product of the single probabilities. That product, viewed as a function of the unknown p , is the *likelihood*. We write it $\mathcal{L}(p)$, with a curly L, because plain L means the loss in this book:

$$\mathcal{L}(p) = p^H (1 - p)^{N-H}$$

In words: to get exactly this data, you need H heads (each contributes a factor p) and $N - H$ tails (each contributes a factor $1 - p$). Multiply them all, and you get the chance of the observed sequence for a given p . We want the p that makes this as large as possible.

A product of many probabilities is a tiny number, and products are hard to differentiate. So we take the **logarithm** first. This helps in two ways. First, it turns products into sums, which makes the calculus easier. Second, log is an increasing function. So *the p that maximizes $\log \mathcal{L}$ is the same p that maximizes $\mathcal{L}$* . We lose nothing by working with the log. (In this book, log means the natural log.)

$$\ell(p) = \log \mathcal{L}(p) = H \log p + (N - H) \log(1 - p)$$

In words: the log brought the exponents down as multipliers and turned the product into a sum. This $\ell(p)$ is the *log-likelihood*, and it is what we maximize.

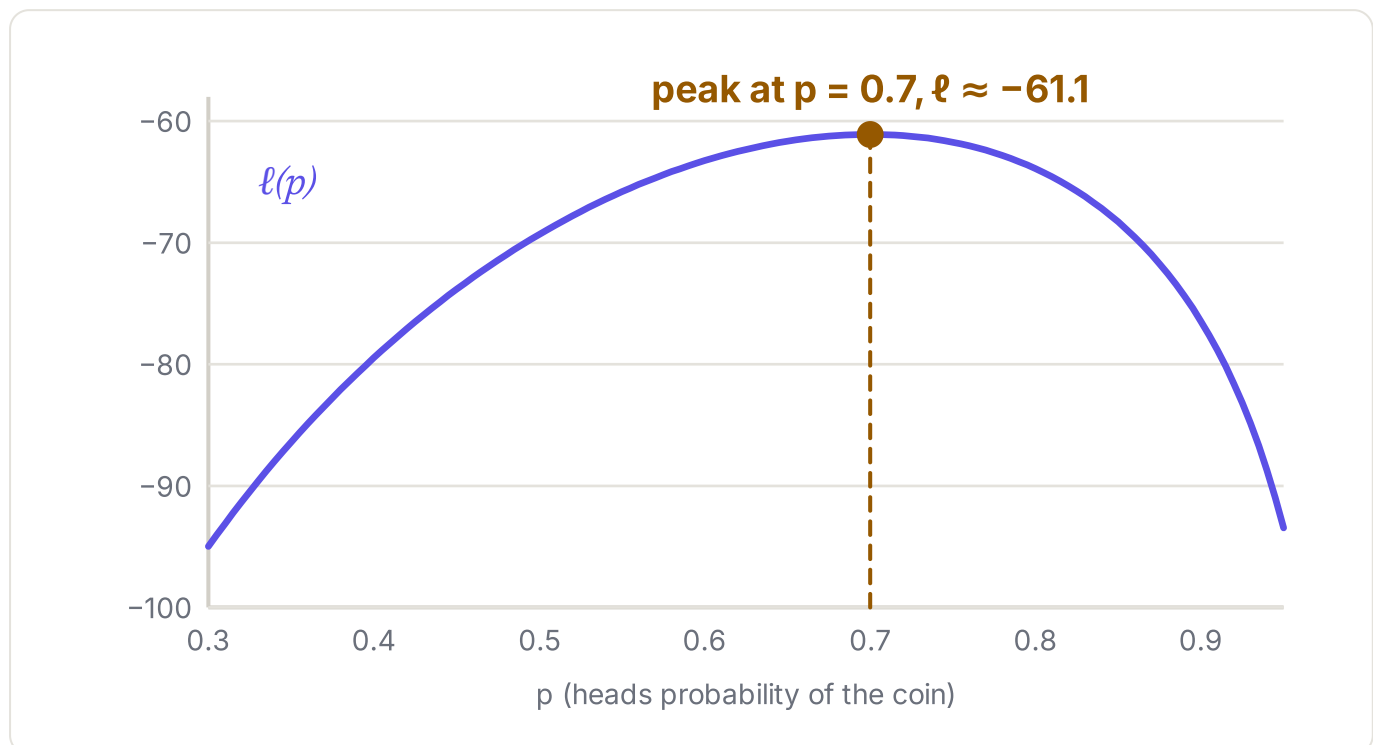


Figure 3.3 The log-likelihood $\ell(p) = 70 \ln p + 30 \ln(1 - p)$ for 70 heads in 100 flips. It has one peak, at $p = 0.7$. There the slope is zero, which is the condition we solve below.

To find the maximum, use calculus from Section 2. Take the derivative with respect to p , set it to zero (the slope is flat at a peak), and solve. Recall that $\frac{d}{dp} \log p = \frac{1}{p}$:

$$\begin{aligned}\frac{d\ell}{dp} &= \frac{H}{p} - \frac{N-H}{1-p} = 0 \\ \frac{H}{p} &= \frac{N-H}{1-p} \\ H(1-p) &= (N-H)p \\ H - Hp &= Np - Hp \\ H &= Np \\ \hat{p} &= \frac{H}{N}\end{aligned}$$

In words: the best estimate is $\hat{p} = H/N$, the fraction of flips that came up heads. Flip 100 times, see 70 heads, and MLE says the most likely coin has $p = 0.7$. The hat on $\hat{p}$ marks it as an *estimate from data*, not the true hidden value. Common sense says “count the heads,” and the maximum-likelihood principle gives the same answer. (The slope $H/p - (N-H)/(1-p)$ is positive left of H/N and negative right of it, so this flat point is a peak, as Figure 3.3 shows.)

◆ THE SAME THING IN PYTORCH

>_ FINDING P BY GRADIENT ASCENT ON THE LOG-LIKELIHOOD

```
import torch

# 100 flips, 70 heads: recover p by maximizing the log-likelihood with gradient ascent
H, N = 70.0, 100.0
p = torch.tensor(0.5, requires_grad=True)  # start with a fair-coin guess

for _ in range(500):
    log_lik = H * torch.log(p) + (N - H) * torch.log(1 - p)
    log_lik.backward()  # d(log_lik)/dp, from Section 2
    with torch.no_grad():
        p += 0.001 * p.grad  # step UPHILL (maximize)
        p.grad.zero_()

print(p)  # tensor(0.7000, requires_grad=True) → matches p_hat
```

In words: this is gradient descent from Section 2 with the sign flipped. We *add* the gradient because we want to go up the log-likelihood, not down. After 500 small steps, p reaches the peak at 0.7.

◆ WHERE LOSS FUNCTIONS COME FROM

Look at what we maximized: a log-likelihood. Now flip the sign. Maximizing $\log \mathcal{L}$ is the same as *minimizing* $-\log \mathcal{L}$. That quantity, the **negative log-likelihood**, is the loss function most classifiers are trained on. You will meet it as *cross-entropy* in Section 4. Do the same derivation for a Gaussian instead of a Bernoulli, and you get the *mean-squared-error* loss behind Section 5's linear regression. So training a model is maximum likelihood in another form. “Find the parameters that make the data most probable” means the same as “find the parameters that minimize the loss.”

CHECK YOURSELF 5

You flip a coin 10 times and get 10 heads. What does MLE give for $\hat{p}$? Why is that suspicious, and what does it tell you about the limits of pure MLE?

6 What to carry forward

- A model outputs a **distribution**: belief spread over outcomes. Discrete variables use a **PMF** (a mass on each value). Continuous variables use a **PDF** (area under a curve gives probability).
- **Expectation** $\mathbb{E}[X]$ is the probability-weighted average, the balance point. **Variance** $\mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2$ is the spread: reliable versus streaky.
- The **Bernoulli** models one yes/no event with a single p (mean p , variance $p(1 - p)$). The **Gaussian** is the bell curve with mean μ and standard deviation σ . It appears everywhere because sums of many small random effects tend toward it.
- **Bayes' rule** $P(A | B) = \frac{P(B|A)P(A)}{P(B)}$ updates a *prior* belief with a *likelihood* into a *posterior*. Never ignore the base rate.
- **MLE** chooses the parameters that make the observed data most probable. For a coin it gives the plain count $\hat{p} = H / N$. Minimizing the negative log-likelihood *is* where loss functions come from.

Next, Section 4 looks at the same probabilities from another angle: how much *surprise* each outcome carries. That view leads straight to the cross-entropy loss.

Answers to “Check yourself”

1. A PDF has the value $p(3.0) = 2$ at one point. Is that a mistake, since probabilities cannot be above 1? What is $P(X = 3.0)$, and what should you ask instead?

It is not a mistake. A density is probability *per unit*, so it can be larger than 1 on a narrow range, as long as the total area is 1. $P(X = 3.0)$ is 0: the area over a slice of zero width is zero. The right question is about a *range*, for example $P(2.9 \leq X \leq 3.1)$. If the density stays near 2 over that range, the answer is about $2 \times 0.2 = 0.4$.

2. A bet pays +10 dollars with probability 0.2 and −3 dollars with probability 0.8. What is the expected payout? Now add 5 dollars to every payout. How do the mean and the variance change?

$\mathbb{E} = (0.2)(10) + (0.8)(-3) = 2.0 - 2.4 = -0.40$. On average you *lose* 40 cents per play. The win is larger than the loss, but the loss happens four times as often. If you add 5 dollars to every payout, the mean rises by 5, to 4.60. The variance does not change: every outcome and the mean move together, so all distances from the mean stay the same.

3. One Bernoulli has $p = 0.5$ and another has $p = 0.9$. If you always guess the more likely outcome, how often are you wrong for each? How does the variance $p(1 - p)$ match that?

For $p = 0.5$ there is no better guess, so you are wrong half the time. For $p = 0.9$ you guess “1” and are wrong only 10% of the time. The variance agrees: $0.5 \times 0.5 = 0.25$ versus $0.9 \times 0.1 = 0.09$. The variance $p(1 - p)$ is largest at $p = 0.5$ and shrinks toward the ends. So the harder a single outcome is to predict, the larger its variance. (Section 4 measures this unpredictability in a different way, with entropy.)

4. Suppose the disease is much more common: 1 in 10 people (prior 0.1) instead of 1 in 1000. Predict: does a positive test now make you more or less sure that you are sick? Then compute the posterior.

Much more sure. With $P(D) = 0.1$: $P(+) = (0.99)(0.1) + (0.05)(0.9) = 0.099 + 0.045 = 0.144$. So $P(D | +) = 0.099/0.144 \approx 0.69$, about 69%, compared with 2% before. The test did not change; only the prior did. When the base rate is high, the false positives no longer outnumber the true positives. The same evidence means very different things depending on how common the condition was to begin with.

5. You flip a coin 10 times and get 10 heads. What does MLE give for $\hat{p}$? Why is that suspicious, and what does it tell you about the limits of pure MLE?

$\hat{p} = 10/10 = 1.0$: a coin that *never* lands tails. MLE uses only the data in front of it, and a small sample can mislead. Ten heads in a row from a fair coin is unlikely (about 1 in 1000), but possible. Yet MLE now gives probability *zero* to ever seeing a tail, after only ten flips. The fix is to include a **prior** belief (“coins are usually close to fair”) and use Bayes’ rule. That pulls the estimate back toward 0.5 until there is enough data to outweigh the prior. This balance between the data and the prior belief leads from maximum likelihood to Bayesian learning.

Information Theory

By the end of this page you will have derived, not been told, why cross-entropy is the loss function for classification. Every step is shown, and where we skip a proof, we say so.

46 min read

YOU'LL LEARN

- information & surprise
- entropy
- cross-entropy
- KL divergence
- why cross-entropy is THE classification loss

Before you start: Section 3: Probability & Statistics.

Every time a model makes a prediction, we need one number that says **how wrong it was**. That number is the loss. For classification, the loss is almost always *cross-entropy*. Many courses give you the formula and move on. We will build it from nothing. We start with a question a child could ask: “*why is some news surprising and other news boring?*” Step by step, we arrive at the loss function that trains most image classifiers and language models today.

◆ THE ONE IDEA TO HOLD ONTO

Information is **surprise**, and surprise can be measured as **the length of the message you need to report an event**. Entropy is your average surprise. Cross-entropy is your average surprise when you hold the wrong beliefs. The gap between them, the wasted message length, is what a classifier learns to shrink.

1 Information: surprise, measured

◆ INTUITION FIRST

Start with a plain observation about the news. A headline is big news when it tells you something you did *not* expect. “Dog bites man” is not a story: it happens often, so you learn almost nothing. “Man bites dog” is a story. It is rare, and because it is rare, it carries a lot of **information**. So information is not about how important an event feels. It is about how *unlikely* the event was. The rarer the event, the more you learn when it happens.

☂ EVERYDAY PARALLEL

A friend who says “the sun rose this morning” has told you nothing: probability close to 1, no surprise. A friend who says “it snowed in the desert” has told you a lot: tiny probability, large surprise. Both used a few words, but they delivered very different amounts of information. Information is the *gap between what you expected and what happened*.

◆ NOW THE MATH

We want a number, called **information content** or *surprise*. It takes the probability $p(x)$ of an event x and grows as that probability shrinks. Instead of guessing a formula, let's list the properties any sensible measure of surprise *must* have. Then we see which function they lead to.

1. **Certain events are not surprising.** If $p(x) = 1$, the surprise must be 0. You learn nothing from something that was guaranteed.
2. **Rarer means more surprising.** As $p(x)$ goes down, surprise must go up.
3. **Surprises from independent events add up.** Suppose a coin lands heads *and* a die rolls a six. The total surprise should be the surprise of the first plus the surprise of the second. (“Independent” means that the outcome of one tells you nothing about the other.)

The third rule is the strict one. For independent events, probabilities *multiply*: $p(\text{both}) = p(x) p(y)$. But we want the surprises to *add*. So we need a function that turns multiplication inside into addition outside. The **logarithm** does this, because $\log(ab) = \log a + \log b$. In fact, it is the only choice: every continuous function with this property is a constant times a logarithm. (We will not prove that here.) So surprise must be a log of the probability. To make it *decrease* as probability rises (rule 2), we put a minus sign in front:

$$I(x) = -\log p(x) = \log \frac{1}{p(x)}$$

In words: the information in an event is “minus the log of how likely it was.” Probabilities are between 0 and 1, so their log is negative or zero, and the minus sign makes $I(x)$ positive or

zero. A sure thing ($p = 1$) gives $-\log 1 = 0$: no surprise. A one-in-a-million event gives a large positive number. The second form reads the same idea another way. An event with probability $1/8$ is as surprising as picking one box out of 8 equally likely boxes. Surprise is the log of that number of equally likely options.

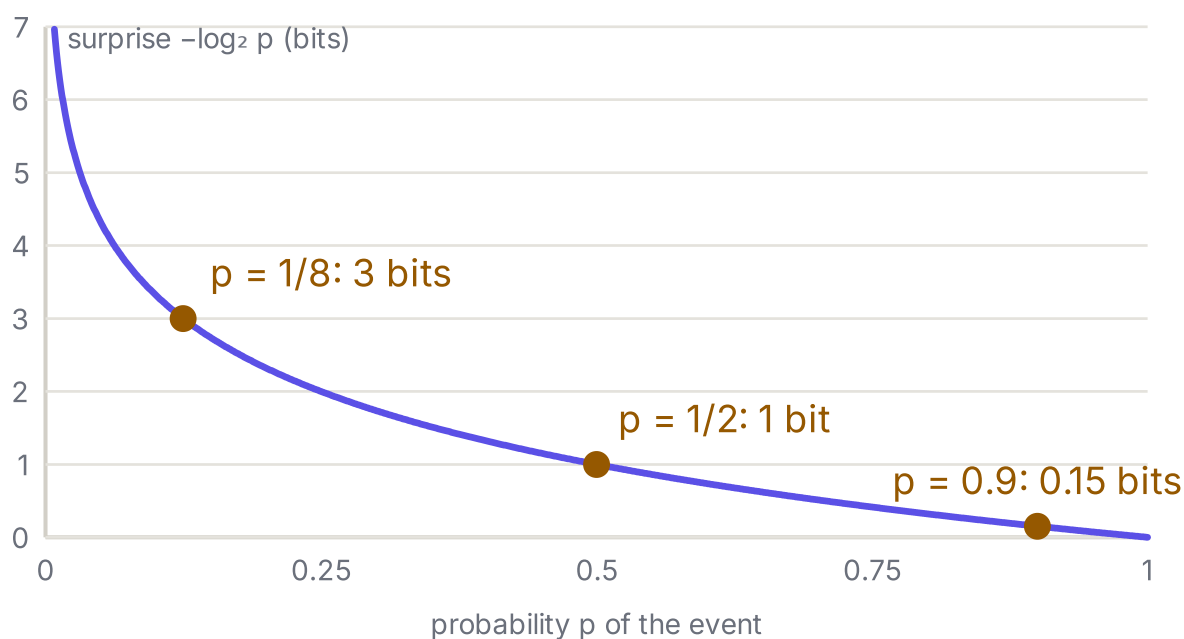


Figure 4.1 Surprise $-\log_2 p$ in bits. A sure event ($p = 1$) carries 0 bits. Surprise grows without limit as p goes to 0.

DEFINITION

The **information content** (also called **surprise** or **surprisal**) of an outcome x is $I(x) = -\log p(x)$.

When the log has base 2, the unit is **bits**. When it is the natural log $\ln$ (base e), the unit is **nats**. One bit is the surprise of one fair coin flip.

Why is one fair coin flip worth one bit? Put in $p = \frac{1}{2}$ with a base-2 log: $-\log_2 \frac{1}{2} = \log_2 2 = 1$. One yes/no answer, one bit. Eight independent coin flips carry $8 \times 1 = 8$ bits. That is why a byte (8 bits) can name $2^8 = 256$ equally likely things.

◆ THE SAME THING IN PYTORCH

> SURPRISE IN BITS AND NATS

```
import torch

# a rare event vs a common one
p_rare = torch.tensor(1/1000)    # one in a thousand
p_common = torch.tensor(0.9)      # happens most of the time

# base-2 log → answer in BITS
print(-torch.log2(p_rare))        # tensor(9.9658)  about 10 bits of surprise
print(-torch.log2(p_common))      # tensor(0.1520)  about a seventh of a bit

# natural log → answer in NATS (what PyTorch losses use)
print(-torch.log(p_rare))         # tensor(6.9078)  same event, different uni
```

In words: the rare event carries about 66 times more surprise than the common one. The two units differ by a constant factor: $9.9658/6.9078 \approx 1.44$. We explain that factor in Section 2.

CHECK YOURSELF 1

A weather app gives rain tomorrow a probability of $1/8$. How many bits of surprise is rain? And how many bits is “rain on both of the next two days,” if the days are independent?

2 Entropy: your average surprise

◆ INTUITION FIRST

Surprise is defined for a *single* outcome. But usually we care about a whole **distribution**. Think of a source that keeps producing outcomes, like a coin you keep flipping or a language that keeps producing words. **Entropy** answers the question “*on average, how surprised will I be by each outcome from this source?*” It is the mean surprise, weighted by how often each outcome happens.

☂ EVERYDAY PARALLEL

Think of the game **20 Questions**. Entropy is the smallest average number of yes/no questions you need to guess the answer, if you choose your questions well. For a fair coin, one question settles it: entropy 1. For one of 1,024 equally likely names, you need 10 questions: entropy 10. For a source that *always* says “heads,” you need no questions, because you already know: entropy 0. **Entropy is uncertainty measured in questions.**

◆ NOW THE MATH

To average the surprise, we weight each outcome's surprise $-\log p(x)$ by its probability $p(x)$ and add them all. That weighted average is written with the expectation symbol $\mathbb{E}$ from Section 3:

$$H(p) = \mathbb{E}_{x \sim p}[-\log p(x)] = -\sum_x p(x) \log p(x)$$

In words: go through every possible outcome x . Take its surprise $-\log p(x)$, multiply by how often it happens, $p(x)$, and add over all outcomes. Common outcomes are not very surprising, but they happen often. Rare outcomes are very surprising, but they happen seldom. Entropy combines the two into one number: *the surprise you should expect per draw*.

Let's compute it for a coin that lands heads with probability p and tails with $1 - p$. There are only two outcomes, so the sum has two terms:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

Now look at some values of p :

Coin	p (heads)	Entropy H (bits)	What it means
Two-headed (rigged)	1	0	No uncertainty: you always know the result.
Heavily biased	0.1	0.47	Usually tails; little uncertainty.
Fair	0.5	1	Maximum uncertainty: a full bit per flip.
Two-tailed (rigged)	0	0	No uncertainty again: always tails.

(For $p = 0$ or $p = 1$, one term is $0 \log 0$. We define it as 0, because $p \log p$ goes to 0 as p goes to 0.) Notice the shape. Entropy is **zero at both ends**: a rigged coin holds no surprise. It is **largest in the middle**, at $p = 0.5$. Uncertainty is highest when the outcomes are equally likely, so you have the least idea what comes next.

We can *prove* that the peak is at $p = 0.5$, instead of reading it from the table. Use the natural log for the calculus, and differentiate $H(p) = -p \ln p - (1 - p) \ln(1 - p)$ with respect to p :

$$\begin{aligned}\frac{dH}{dp} &= -(\ln p + 1) - (-\ln(1 - p) - 1) \\ &= -\ln p - 1 + \ln(1 - p) + 1 \\ &= \ln \frac{1 - p}{p}.\end{aligned}$$

In words: the slope of the entropy curve is the log of “tails probability over heads probability.” For $p < 0.5$ the fraction is above 1, so its log is positive: the curve rises. For $p > 0.5$ the fraction is below 1, so the log is negative: the curve falls. The slope is zero only where $\frac{1-p}{p} = 1$, that is, at $p = 0.5$. A curve that rises and then falls has its top at the flat point. So the fair coin is the most uncertain coin.

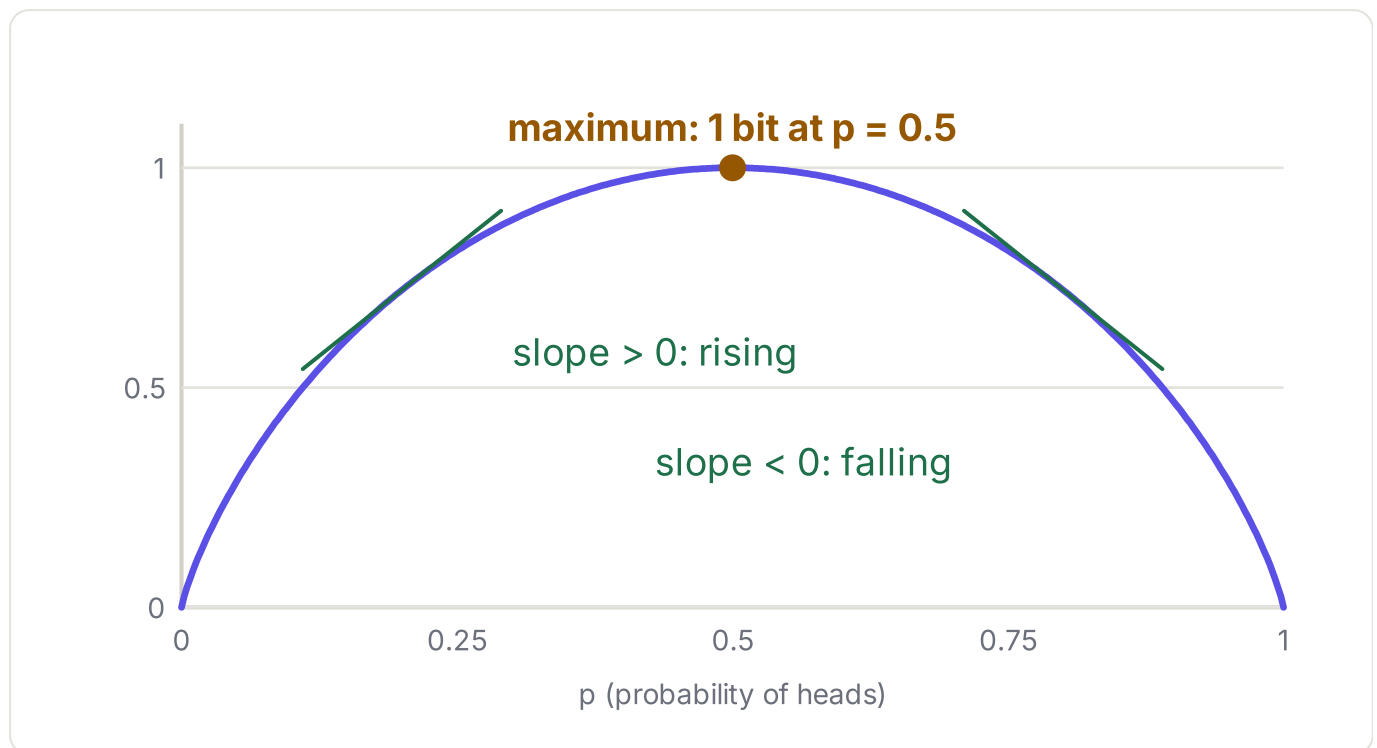


Figure 4.2 Entropy of a coin, $H(p)$ in bits. The slope is positive left of $p = 0.5$ and negative right of it, so the one flat point is the top of the hill.

DEFINITION

Bits vs nats. Entropy is the same quantity whichever log you use; only the unit changes. Base-2 log gives **bits** (natural for yes/no questions and file sizes). The natural log gives **nats**. Since $\log_2 x = \ln x / \ln 2$, a value in nats becomes bits when you divide by $\ln 2 \approx 0.693$. So $1 \text{ nat} = 1 / \ln 2 \approx 1.44 \text{ bits}$. Machine learning code uses nats, because derivatives of $\ln$ are simplest.

◆ THE SAME THING IN PYTORCH

> _ ENTROPY OF A COIN FOR SEVERAL BIASES

```
import math
import torch

def coin_entropy_bits(p):
    # entr(t) = -t * ln(t), with entr(0) = 0 built in; divide by ln 2 for
    return (torch.special.entr(p) + torch.special.entr(1 - p)) / math.log(2)

for p in [0.0, 0.1, 0.5, 0.9, 1.0]:
    print(f"p={p:.1f} H={coin_entropy_bits(torch.tensor(p)):.3f} bits")
# p=0.0 H=0.000 bits
# p=0.1 H=0.469 bits
# p=0.5 H=1.000 bits
# p=0.9 H=0.469 bits
# p=1.0 H=0.000 bits → peak in the middle
```

In words: `torch.special.entr` computes $-t \ln t$ and returns 0 at $t = 0$, which follows our convention. A naive version, `-(p * torch.log(p))`, gives `nan` at $p = 0$, because $0 \times (-\infty)$ is undefined in floating point.

CHECK YOURSELF 2

A fair six-sided die has entropy $\log_2 6 \approx 2.585$ bits. Can a yes/no strategy find the face with 2.585 questions on average? What would make the die's entropy lower?

3 Cross-entropy: surprise under the wrong beliefs

◆ INTUITION FIRST

Entropy assumed that you *know* the true probabilities p . A model does not know them. It only has a *guess*, a predicted distribution we call q . **Cross-entropy** asks: *if the world follows p , but I plan for it as if it follows q , how surprised will I be on average?* It is your average surprise when your beliefs are q but reality is p .

📌 EVERYDAY PARALLEL

Think of a code like Morse code, built for English. Common English letters, like E and T, get short signals; rare ones, like Q and Z, get long signals. In the ideal code, a letter with frequency q gets a signal about $-\log_2 q$ bits long. Now use this English code to send *Polish* text. The letters arrive at Polish frequencies p , but each one costs the length the English code gave it, $-\log_2 q$. Your average signal length is the cross-entropy. Polish uses Z often, and each Z costs a long English signal, so the average grows.

◆ NOW THE MATH

The formula is the same as for entropy (average the surprise, weighted by how often each outcome happens) with **one change**. The outcomes still happen at the true rate $p(x)$, because reality decides how often things occur. But the surprise you *feel* comes from your belief $q(x)$, because that is the probability you gave. So we weight by p but take the log of q :

$$H(p, q) = \mathbb{E}_{x \sim p} [-\log q(x)] = - \sum_x p(x) \log q(x)$$

In words: outcomes arrive at their real rate $p(x)$, but each one surprises you by $-\log q(x)$, the surprise *your* model gave it. If your guess q matched reality p perfectly, this would be the ordinary entropy $H(p)$. Any mismatch makes it larger. Cross-entropy is never smaller than entropy; we prove this in the next section.

◆ READ THE TWO ARGUMENTS CAREFULLY

The order matters. $H(p, q)$ weights by the **first** argument p (what is true). It takes the log of the **second** argument q (what you believe). Swap them and you get a different number. A short way to remember it: “*weighted by truth, surprised by belief.*”

◆ THE SAME THING IN PYTORCH

>_ CROSS-ENTROPY BETWEEN A TRUE AND A PREDICTED DISTRIBUTION

```
import torch

# true distribution p over 3 classes, and two guesses q
p      = torch.tensor([0.7, 0.2, 0.1])  # reality
q_ok   = torch.tensor([0.6, 0.3, 0.1])  # a decent guess
q_bad  = torch.tensor([0.1, 0.2, 0.7])  # confidently wrong

def cross_entropy(p, q):
    return -(p * torch.log(q)).sum()    # nats

print(cross_entropy(p, p))              # tensor(0.8018) = H(p), the lowest poss.
print(cross_entropy(p, q_ok))           # tensor(0.8286) a little above it
print(cross_entropy(p, q_bad))          # tensor(1.9694) far above → bad model
```

In words: the function multiplies each true probability by the log of the guessed one and adds the results. The better the guess, the closer the result is to $H(p)$.

CHECK YOURSELF 3

In `q_bad`, the model gives the first class only 0.1, though it really happens 70% of the time. Which single term of the sum contributes most to the cross-entropy? Estimate it.

4 KL divergence: the extra cost of being wrong

◆ INTUITION FIRST

Cross-entropy $H(p, q)$ is your total surprise with a wrong belief. Entropy $H(p)$ is the surprise you cannot avoid, even with a perfect belief. The *difference* is the part you could have avoided: the **waste** caused by believing q instead of p . That waste is the **Kullback–Leibler divergence**, written $D_{\text{KL}}(p||q)$.

☂ EVERYDAY PARALLEL

Back to the English code used for Polish. A code built for Polish would send Polish text with an average length of $H(p)$. That is the shortest possible. The English code needs $H(p, q)$. The KL divergence is the **extra** length per letter that you pay only because the code was built for the wrong language. The asymmetry is also clear here. Sending Polish with an English code wastes a different amount than sending English with a Polish code.

◆ NOW THE MATH

By definition, it is the gap between cross-entropy and entropy. Subtract, and the two sums combine term by term into one sum:

$$\begin{aligned} D_{\text{KL}}(p||q) &= H(p, q) - H(p) \\ &= \left(- \sum_x p(x) \log q(x) \right) - \left(- \sum_x p(x) \log p(x) \right) \\ &= \sum_x p(x) (\log p(x) - \log q(x)) \\ &= \sum_x p(x) \log \frac{p(x)}{q(x)}. \end{aligned}$$

In words: for each outcome, look at the ratio of what is true to what you believed, $p(x)/q(x)$. Take its log, weight it by how often the outcome happens, $p(x)$, and add. Where your belief matched reality, the ratio is 1 and its log is 0: no penalty.

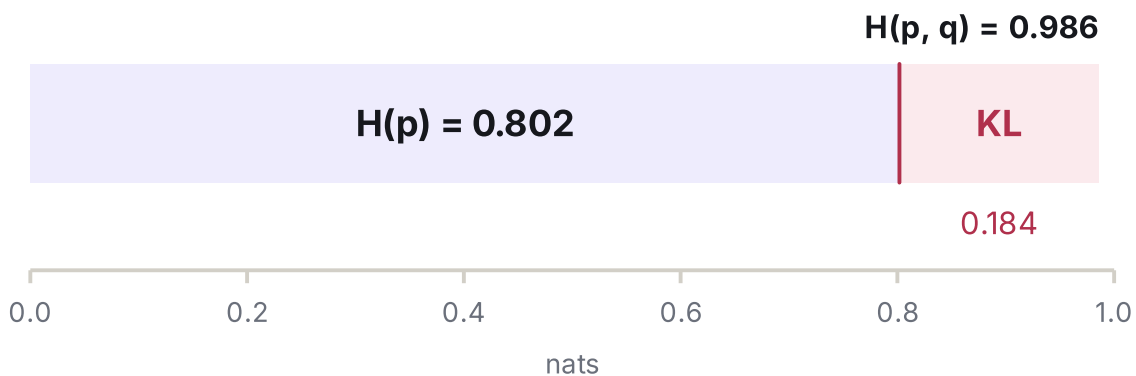


Figure 4.3 Cross-entropy splits into two parts. The entropy $H(p)$ is the part no model can remove. The KL divergence is the extra cost of believing q . Numbers from the code below, with $p = [0.7, 0.2, 0.1]$ and $q = [0.4, 0.4, 0.2]$.

Why KL divergence is never negative (Gibbs' inequality)

We keep saying “cross-entropy $\geq$ entropy” and “the extra cost is never negative.” Let's prove it, because the whole story depends on it. The claim is $D_{\text{KL}}(p\|q) \geq 0$, known as **Gibbs' inequality**. We work in nats, with the natural log $\ln$. (In bits, every log is divided by the positive constant $\ln 2$, which does not change the sign.)

We need one fact about $\ln$: $\ln t \leq t - 1$ for all $t > 0$. The line $t - 1$ is the tangent to $\ln t$ at $t = 1$. The curve $\ln t$ is *concave* (it bends downward), so it never rises above its tangent line. This fact is true only for the natural log. For example, $\log_2 1.5 \approx 0.585$ is larger than $1.5 - 1 = 0.5$.

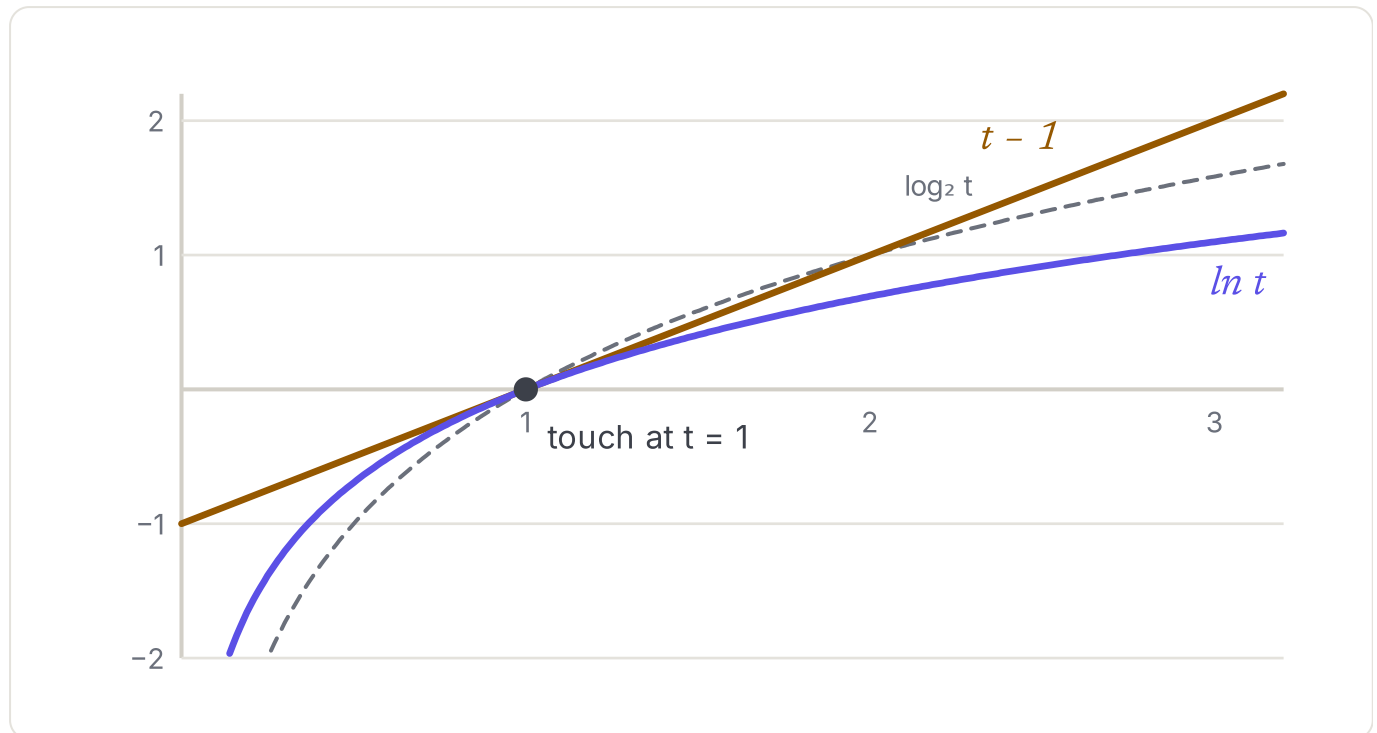


Figure 4.4 The natural log $\ln t$ (violet) never rises above its tangent line $t - 1$ (amber); they touch only at $t = 1$. The dashed curve $\log_2 t$ does go above the line between 1 and 2, so the bound needs the natural log.

Now apply the fact to each term:

$$\begin{aligned}
 -D_{\text{KL}}(p\|q) &= \sum_x p(x) \ln \frac{q(x)}{p(x)} && \text{(flip the ratio; the minus sign flips the log)} \\
 &\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) && \text{(use } \ln t \leq t - 1 \text{ with } t = \frac{q(x)}{p(x)}) \\
 &= \sum_x (q(x) - p(x)) \\
 &= \sum_x q(x) - \sum_x p(x) = 1 - 1 = 0.
 \end{aligned}$$

In words: we showed $-D_{\text{KL}} \leq 0$, which is the same as $D_{\text{KL}} \geq 0$. The last line uses the one thing we know for sure about probability distributions: each one adds up to 1. (The sums run

over outcomes with $p(x) > 0$. Skipping the others can only make $\sum q$ smaller than 1, so the result still holds.) So the extra cost of a wrong belief is *always* zero or positive, never a bonus. Equality ($D_{\text{KL}} = 0$) needs $\ln t = t - 1$ in every term. That happens only at $t = 1$, so only when $q(x) = p(x)$ everywhere.

◆ WHY THIS MATTERS FOR TRAINING

Gibbs' inequality is *why* minimizing cross-entropy is a sensible goal. We have $H(p, q) = H(p) + D_{\text{KL}}(p||q)$, and $H(p)$ is a constant that the model cannot change. So pushing cross-entropy down is the same as pushing D_{KL} down. That moves your predicted distribution q toward the truth p . The lowest value is zero, reached only when $q = p$.

▲ WATCH OUT: KL IS NOT A DISTANCE

KL divergence is **asymmetric**: in general, $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$. The cost of using the wrong code depends on which language is really being sent. So do not call KL a “distance.” A true distance is symmetric, and KL is not. It measures “extra surprise if the truth is p and you planned for q ”

◆ THE SAME THING IN PYTORCH

>_ KL IS THE GAP, AND IT IS ASYMMETRIC

```
import torch

p = torch.tensor([0.7, 0.2, 0.1])
q = torch.tensor([0.4, 0.4, 0.2])

def H(p, q=None):                                # entropy if q is None, else cross-entropy
    q = p if q is None else q
    return -(p * torch.log(q)).sum()

kl_pq = H(p, q) - H(p)                           # extra cost if truth is p, believed q
kl_qp = H(q, p) - H(q)                           # the reversed question

print(kl_pq)   # tensor(0.1838)
print(kl_qp)   # tensor(0.1920) → different number: KL is asymmetric
print(kl_pq ≥ 0, kl_qp ≥ 0) # tensor(True) tensor(True) → Gibbs holds
```

In words: the code computes KL as “cross-entropy minus entropy,” in both directions. Here $H(p) = 0.8018$ and $H(p, q) = 0.9856$, the numbers in Figure 4.3.

CHECK YOURSELF 4

A model's prediction q equals the truth p . What are $D_{\text{KL}}(p||q)$ and $H(p, q)$? Can the cross-entropy loss ever reach zero when the source is truly uncertain?

5 Why cross-entropy is the classification loss

◆ INTUITION FIRST

Everything above was general. Now apply it to **classification**: putting an input into one of several categories (is this image a cat, a dog, or a bird?). Here the truth is not uncertain: each training example belongs to *exactly one* class. This one fact makes cross-entropy very simple. It shows that “minimize cross-entropy” and “make the correct answer as likely as possible” are *the same instruction*.

☂ EVERYDAY PARALLEL

Imagine a multiple-choice test where students must give a probability for each answer, and only one answer is correct. The grader looks only at *the probability the student put on the right answer*. “100% sure, and correct” costs nothing. “5% on the correct answer” costs a lot. “0% on the correct answer” costs without limit: the penalty $-\log q$ has no upper bound. Cross-entropy grades this way.

◆ NOW THE MATH

In classification, the true distribution p for one example is **one-hot**. It puts probability 1 on the correct class c and 0 on every other class: $p(c) = 1$, and $p(x) = 0$ for $x \neq c$. (“One-hot” means a vector of zeros with a single 1 that marks the right answer.) Put that into the cross-entropy sum, and every term but one disappears:

$$\begin{aligned} H(p, q) &= - \sum_x p(x) \log q(x) \\ &= - \left[p(c) \log q(c) + \sum_{x \neq c} p(x) \log q(x) \right] \\ &= - \left[1 \cdot \log q(c) + \sum_{x \neq c} 0 \cdot \log q(x) \right] \\ &= - \log q(c). \end{aligned}$$

In words: the truth is 1 for the correct class and 0 everywhere else. So every wrong-class term is multiplied by 0 and disappears. One term remains: *minus the log of the probability your model gave the correct class*. The whole cross-entropy of a labeled example is $-\log q(\text{correct class})$: the surprise of the right answer under your model.

◆ CROSS-ENTROPY = NEGATIVE LOG-LIKELIHOOD

$-\log q(c)$ is the **negative log-likelihood** of the correct label. The *likelihood* is how probable the model thinks the true answer is, and the loss is minus its log. So minimizing cross-entropy is **maximizing the likelihood** of the correct answers. That is the principle of **maximum likelihood estimation** from Section 3. Two ideas that looked unrelated, “shortest code” from information theory and “most probable model” from statistics, lead to the same loss.

Take a dataset of N independent examples. Their likelihoods multiply, so their log-likelihoods add. The training loss is the *average* negative log-likelihood, one cross-entropy term per example:

$$L = \frac{1}{N} \sum_{i=1}^N -\log q_i(c_i)$$

In words: for each training example i , take the model's probability on that example's true class c_i . Take its negative log (the surprise), and average over all N examples. When you lower this number, the model becomes less surprised by the right answers, example by example.

Where do the probabilities q come from? The softmax

A neural network's last layer does not output probabilities. It outputs a vector of raw real numbers called **logits**: one score per class, which can be negative, large, anything. To turn logits $z_1, \dots, z_K$ (for K classes) into a valid probability distribution q , we use the **softmax** function. It exponentiates each score (making it positive) and divides by the total (making the set add up to 1):

$$q_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

In words: e^{z_k} makes every score positive and makes the gaps larger, so the largest logit dominates. Dividing by the sum turns the scores into probabilities that add up to 1. The class with the largest logit gets the largest probability. Softmax connects the raw network scores to a distribution q that cross-entropy can use.

Put softmax into the loss, and the log cancels the exponential:

$$-\log q_c = -z_c + \log \sum_{j=1}^K e^{z_j}$$

In words: the loss needs only the logits, never the probabilities themselves. The second term can be computed safely even for very large logits, by subtracting the largest z_j before exponentiating. This is why PyTorch computes the two steps together.

◆ SOFTMAX + CROSS-ENTROPY: THE STANDARD PAIR

Feed the network's logits through softmax to get q , then score q with cross-entropy $-\log q(c)$. This pair is the standard output of classifiers. PyTorch combines both steps in one function,

`nn.CrossEntropyLoss` (or `F.cross_entropy`), which takes the *raw logits* directly. You almost never apply softmax yourself before the loss; the loss does it for you, safely.

◆ THE SAME THING IN PYTORCH

> FROM RAW LOGITS TO THE CLASSIFICATION LOSS, TWO EQUIVALENT WAYS

```
import torch
import torch.nn.functional as F

# a batch of 2 examples, 3 classes; these are raw LOGITS from a network
logits = torch.tensor([[2.0, 0.5, -1.0],      # example 0
                      [0.1, 0.2, 3.0]])      # example 1
targets = torch.tensor([0, 2])                # true classes 0 and 2 (given)

# --- by hand: softmax → pick correct class → -log → average ---
q = F.softmax(logits, dim=1)                  # logits → probabilities per
p_correct = q[range(2), targets]              # q given to each true class
manual = (-torch.log(p_correct)).mean()        # mean negative log-likelihood
print(manual)                                 # tensor(0.1755)

# --- the usual way: one combined, numerically stable call on RAW logits
builtin = F.cross_entropy(logits, targets)     # softmax + negative log-likelihood
print(builtin)                                # tensor(0.1755) → identical
```

In words: the first half follows our derivation step by step. The second half gets the same number from one call. In real code, use the second.

▲ **WATCH OUT: THE DOUBLE-SOFTMAX BUG**

`nn.CrossEntropyLoss` and `F.cross_entropy` apply softmax *internally*. If you also call `softmax` on the model's output first, you apply softmax twice. That squashes the probabilities toward uniform, so the model trains slowly and badly. Rule: give the loss **raw logits**, never probabilities.

CHECK YOURSELF 5

Your classifier puts probability 0.9 on the correct class for one image and 0.1 for another. Predict which image adds more to the loss, and by how much (in nats). What happens as the second probability goes toward 0?

6 What to carry forward

- **Information (surprise)** is $-\log p(x)$. Rare events carry more, and the log makes surprises from independent events add up.
- **Entropy** $H(p) = -\sum p \log p$ is your average surprise. It is the lower limit on the average number of yes/no questions, and it is largest when outcomes are equally likely.
- **Cross-entropy** $H(p, q) = -\sum p \log q$ is your average surprise when reality is p but you believe q : weighted by truth, surprised by belief.
- **KL divergence** $D_{\text{KL}}(p\|q) = H(p, q) - H(p) \geq 0$ is the extra cost of a wrong belief (Gibbs' inequality, proved with $\ln t \leq t - 1$). It is asymmetric, so it is not a distance.
- **The main result:** for one-hot labels, cross-entropy becomes $-\log q(\text{correct class})$, the negative log-likelihood. So minimizing it *is* maximum likelihood (Section 3). Softmax turns logits into q . Cross-entropy returns in binary form in Section 6, and as the softmax + cross-entropy pair in Section 11 and Section 19.

You now have the loss function that trains most classifiers. And you *derived* it, from “why is snow in the desert surprising?” all the way to `F.cross_entropy`. Next we build our first full model, linear regression, and use the gradient from Section 2 to lower a real loss from start to finish.

Answers to “Check yourself”

1. A weather app gives rain tomorrow a probability of $1/8$. How many bits of surprise is rain? And how many bits is “rain on both of the next two days,” if the days are independent?

$-\log_2(1/8) = \log_2 8 = 3$ bits. One option in 8 is like “which of 8 boxes?”, and 3 yes/no questions find one of 8 boxes ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). Two independent rainy days have probability $1/8 \times 1/8 = 1/64$, and $\log_2 64 = 6$ bits. That is $3 + 3$: the surprises add, as rule 3 demands.

2. A fair six-sided die has entropy $\log_2 6 \approx 2.585$ bits. Can a yes/no strategy find the face with 2.585 questions on average? What would make the die's entropy lower?

Not quite. The best strategy splits the six faces into groups of 3 and 3, then 1 and 2. Two faces need 2 questions and four faces need 3, so the average is $(2 + 2 + 3 + 3 + 3 + 3)/6 = 8/3 \approx 2.67$. Entropy is a lower limit on the average; a question-by-question strategy reaches it exactly only when the probabilities are powers of $\frac{1}{2}$. For k equally likely options the entropy is $\log_2 k$, the largest possible. To *lower* it, make the die unfair, for example a 6 half the time. Then “is it a 6?” is often the only question you need. Predictability always lowers entropy.

3. In `q_bad`, the model gives the first class only 0.1, though it really happens 70% of the time. Which single term of the sum contributes most to the cross-entropy? Estimate it.

The first term: $-0.7 \ln 0.1 = 0.7 \times 2.303 \approx 1.61$ nats. The other two terms add only $-0.2 \ln 0.2 - 0.1 \ln 0.7 \approx 0.32 + 0.04 = 0.36$. Total: about 1.97, as the code prints. Most of the cost comes from a common outcome that the model thought was rare. Cross-entropy punishes that kind of mistake most.

4. A model's prediction q equals the truth p . What are $D_{\text{KL}}(p\|q)$ and $H(p, q)$? Can the cross-entropy loss ever reach zero when the source is truly uncertain?

When $q = p$: $D_{\text{KL}} = 0$ (no waste), so $H(p, q) = H(p) + 0 = H(p)$. The cross-entropy can go no lower than the source's own entropy. So its lowest value is $H(p)$, not zero. For a truly uncertain source ($H(p) > 0$), the loss never reaches zero, however good the model is. Some surprise cannot be removed. The loss can reach zero only when the truth itself is certain (one-hot). That is the classification case we turn to now.

5. Your classifier puts probability 0.9 on the correct class for one image and 0.1 for another. Predict which image adds more to the loss, and by how much (in nats). What happens as the second probability goes toward 0?

The loss per example is $-\ln q_{\text{correct}}$. First image: $-\ln 0.9 \approx 0.105$ nats. Second image: $-\ln 0.1 \approx 2.303$ nats, about 22 times more. As the probability on the correct class goes toward 0, $-\ln q$ grows without limit. So cross-entropy punishes confident mistakes very strongly, and this pushes the model to stop being confidently wrong. The same loss, applied to “predict the next word,” trains language models; see Section 19.

PART 2

Classical ML

What Learning Means

The models in this course share three ingredients: a hypothesis, a loss, and an optimizer. Linear regression is the clearest place to see all three, and we can solve it by hand.

40 min read

YOU'LL LEARN

- hypothesis classes
- loss functions
- optimization
- linear regression (closed form & gradient descent)

Before you start: Section 2: Calculus for ML and Section 3: Probability & Statistics.

“**T**raining a model” sounds mysterious. It becomes clear when you split it into three questions. What kinds of guesses do we allow? How do we score a guess? How do we search for the best guess? When these three questions feel familiar, you can read the training loop of almost any model in this course, including the large ones later on.

◆ THE ONE IDEA TO HOLD ONTO

Learning is **search**. You fix a family of candidate functions. You fix a number that measures how wrong each candidate is. Then you search the family for the candidate with the lowest number. Every algorithm in this course is a different way to do that search.

1 The learning triad: hypothesis, loss, optimization

◆ INTUITION FIRST

Every learning algorithm in this course has three separate parts.

- A **hypothesis class**: the family of functions you allow as “the answer”. It can be only straight lines, or anything a neural network can express.
- A **loss**: one number that says how wrong one function’s guesses are. You compute it by comparing its predictions with the real answers.
- **Optimization**: the search procedure. It tries functions from the hypothesis class and moves toward the one with the lowest loss.

Learning, from start to end, is these three parts working together.

EVERYDAY PARALLEL

You adjust a shower with one knob. The **hypothesis class** is every position the knob can take. The **loss** is how far the water is from the temperature you like. **Optimization** is what you do next: feel the water, turn the knob a little in the direction that helped, and feel again. Nobody tells you the correct knob position. You find it by repeated small corrections.

DEFINITION

A **hypothesis class** is the set of candidate functions a model may choose from. You choose it before training starts. For example, “only straight lines” is a much smaller class than “any curve a deep network can draw”.

DEFINITION

A **loss function** takes one hypothesis’s predictions and the true answers and returns a single number. Lower means better. Every training run minimizes some loss function. (Many losses, such as the squared error below, are never negative and reach 0 for a perfect fit. This is not true of every loss. Section 4 showed that cross-entropy cannot go below the entropy of the data.)

DEFINITION

An **optimization** procedure is the algorithm that searches the hypothesis class for the member with the lowest loss. It does not know the answer in advance. It only knows how to turn the current guess into a slightly better one.

◆ NOW THE MATH

Call the adjustable numbers of a model its *parameters* θ (theta). The model with parameters θ is a function f_θ . We have N training examples (x_i, y_i) . The three parts become three lines:

$$\begin{aligned} f_\theta &\in \mathcal{H} \\ L(\theta) &= \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i) \\ \theta^* &= \arg \min_{\theta} L(\theta) \end{aligned}$$

In words: the three lines are the hypothesis, the loss, and the optimization. $\mathcal{H}$ is the hypothesis class. ℓ scores one prediction against one true answer, and L averages that score over all N examples. $\arg \min_{\theta}$ means “the value of θ that makes L smallest”. The optimizer is the method that finds (or approaches) θ^* .

◆ THE SAME THING IN PYTORCH

>_ THE THREE PARTS AS THREE SMALL FUNCTIONS

```
import torch

def hypothesis(theta, x):          # which functions we allow: straight l.
    return theta[0] * x + theta[1]

def loss(theta, x, y):             # how we score a guess: mean squared e.
    return ((hypothesis(theta, x) - y) ** 2).mean()

def optimize_step(theta, x, y, lr): # how we search: one small downhill
    theta = theta.detach().requires_grad_(True)
    loss(theta, x, y).backward()
    return (theta - lr * theta.grad).detach()
```

The rest of this lesson fills in each of these functions for linear regression.

CHECK YOURSELF 1

Your shower knob only turns between “cold” and “warm”, but you like hot water. Can a better optimization strategy fix this?

2 The hypothesis class: linear regression

◆ INTUITION FIRST

The simplest useful hypothesis class says: the prediction is a **straight-line function** of the input. Linear regression allows only this family. No curves, no bends.

Our running example is predicting a house's price from its floor area. Bigger houses tend to cost more, roughly in proportion. Each extra square meter adds a roughly fixed amount of money. That is the shape of a straight line: a starting price plus a rate per square meter.

☂ EVERYDAY PARALLEL

A taxi fare works like a straight line. There is a fixed starting fee, and then a fixed price for each kilometer. The starting fee is the bias b , and the price per kilometer is the weight w . If you know both numbers, you can predict any fare from the distance alone.

◆ NOW THE MATH

With a single input x (the size in m²), the hypothesis is:

$$\hat{y} = wx + b$$

In words: $\hat{y}$ ("y-hat") is the model's predicted price. x is the house's size. w (the *weight*) is the money added per extra square meter: the slope. b (the *bias*) is the price the line gives at size zero: the intercept. Learning this hypothesis means finding good numbers for w and b .

Real houses have more than one useful feature: size, number of bedrooms, age, distance to the city center. Put these d numbers in a vector $\mathbf{x}$. Give each one its own weight in a vector $\mathbf{w}$. The model is the same idea with more inputs:

$$\begin{aligned}\hat{y} &= \mathbf{w}^\top \mathbf{x} + b \\ &= w_1x_1 + w_2x_2 + \cdots + w_dx_d + b\end{aligned}$$

In words: $\mathbf{w}^\top \mathbf{x}$ is a dot product (Section 1): multiply each feature by its own weight and add the results. Then add the single bias number b . There is one weight per feature and one shared bias.

◆ THE SAME THING IN PYTORCH

```
import torch

w = torch.tensor([1800.0])    # dollars per extra m2
b = torch.tensor([48000.0])   # baseline price
x = torch.tensor([100.0])     # house size, m2

y_hat = x * w + b
print(y_hat)                  # tensor([228000.])
```

CHECK YOURSELF 2

You fit $\hat{y} = wx + b$ to some houses. Then every price in the data doubles (sizes stay the same). What are the new best w and b ?

3 The loss: Mean Squared Error

◆ INTUITION FIRST

We need one number that answers: “across every house in our data, how wrong were this line’s guesses?”

Mean Squared Error (MSE) does this. For each house, subtract the true price from the predicted price. Square that difference. Then average the squares over all houses.

☂ EVERYDAY PARALLEL

Imagine each data point is tied to the line by a vertical spring. A stretched spring stores energy proportional to the *square* of its stretch. Let the line move freely, and it settles where the total spring energy is lowest. That resting place is the line with the smallest sum of squared errors.

◆ NOW THE MATH

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

In words: here $\hat{y}_i = wx_i + b$ is the prediction for house i . For each of the N houses, take (predicted price – true price) and square it. Then average all N squared errors into one number. Squaring does two jobs. It makes every error positive, so an over-guess and an under-guess of the same size do not cancel. It also punishes large mistakes much more than small ones.

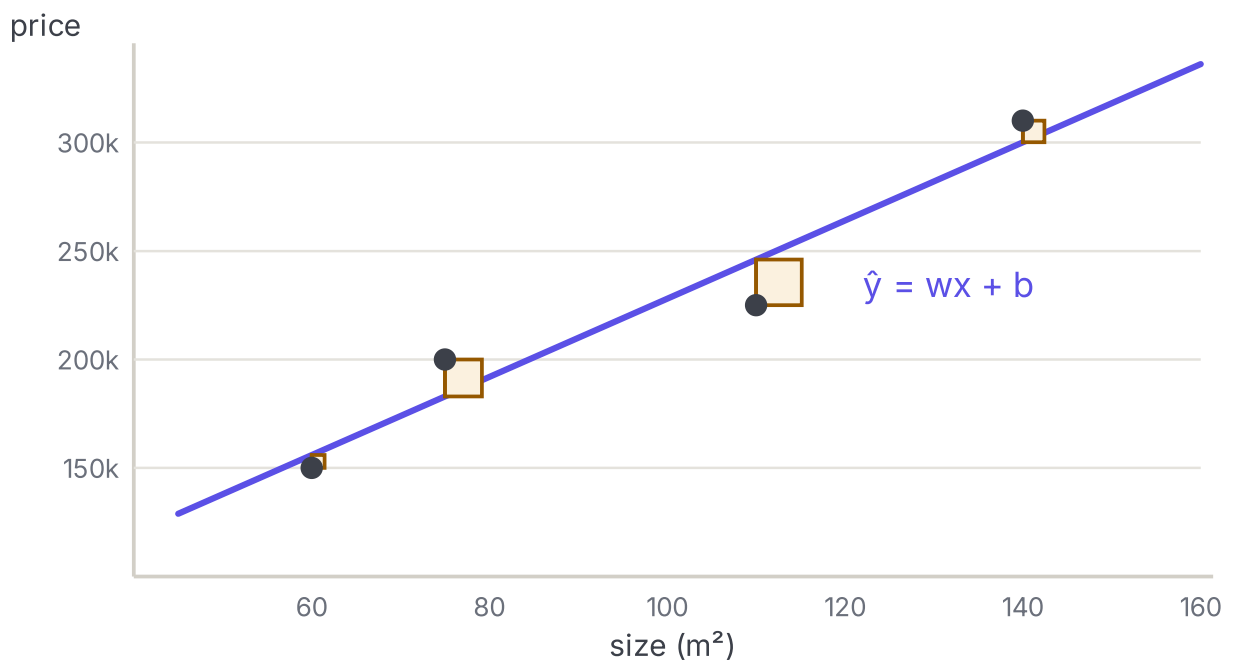


Figure 5.1 Four houses (dots) and the best-fit line. Each amber square has a side equal to one error, so its area is that error squared. MSE is the average area. The best line makes the total area as small as possible.

WHY SQUARE, AND NOT ABSOLUTE VALUE?

The absolute value $|\hat{y} - y|$ also makes errors positive. But it is not smooth: at zero its slope jumps from -1 to $+1$. That is awkward for gradient-based optimization (Section 2). The square is smooth everywhere. Also, as the next box shows, it follows from a simple assumption about noisy data. It is not an arbitrary choice.

◆ WHERE MSE COMES FROM

Suppose the true price is a straight line plus random noise: $y_i = wx_i + b + \varepsilon_i$. Each noise value ε_i comes from a Gaussian (bell-curve) distribution with mean 0. Its standard deviation σ is the same for every house (see Section 3). Then the probability *density* of price y_i , given x_i , is a Gaussian centered at $\hat{y}_i = wx_i + b$:

$$\begin{aligned} p(y_i | x_i; w, b) \\ = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right) \end{aligned}$$

(As Section 3 explained, a density is not a probability. It is a height, and higher means “more likely near this value”. That is all we need here.) If the houses are independent, the likelihood of the *whole* dataset is the product of these densities. The log turns the product into a sum, and it turns the exponential into a plain term. For one house:

$$\begin{aligned} \log p(y_i | x_i) = & -\frac{1}{2} \log(2\pi\sigma^2) \\ & - \frac{(y_i - \hat{y}_i)^2}{2\sigma^2} \end{aligned}$$

Adding this over all N houses gives the log-likelihood, which we write as ℓ :

$$\begin{aligned} \ell(w, b) = & -\frac{N}{2} \log(2\pi\sigma^2) \\ & - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \end{aligned}$$

In words: the first term does not contain w or b . The factor $\frac{1}{2\sigma^2}$ is a positive constant. So *maximizing* this log-likelihood over w, b is the same job as *minimizing* $\sum_i (y_i - \hat{y}_i)^2$. That is MSE, up to the constant $1/N$. Squared error is what you get when you assume Gaussian noise.

CHECK YOURSELF 3

One house in your data has a typo: its price is ten times too high. Which loss pulls the fitted line more toward that house, squared error or absolute error? Why?

4 Optimization, take one: solving directly

◆ INTUITION FIRST

For a straight line, the MSE loss is a smooth bowl. When no feature is a copy of another (see the check below), the bowl has a single lowest point and no other dips. So we do not need to search by trial and error. We can use calculus to go straight to the bottom: set the slope to zero and solve, as you would for a simple parabola.

☂ EVERYDAY PARALLEL

A ball thrown upward follows a parabola. You do not need to film the whole flight to find its highest point. Because you know the shape, a formula gives the top directly. The normal equation below does the same for the MSE bowl: we know its shape, so a formula gives the bottom.

◆ NOW THE MATH

Put all the houses in one table. The *data matrix* $\mathbf{X}$ has one row per house and one column per feature, so its shape is $N \times d$. The true prices form a vector $\mathbf{y}$ of length N .

A trick handles the bias b without carrying it separately. Add a column of 1s to the front of $\mathbf{X}$, and add b to the front of the weight vector. Call the new versions $\tilde{\mathbf{X}}$ (shape $N \times (d + 1)$) and $\tilde{\mathbf{w}}$ (length $d + 1$, with $\tilde{w}_0 = b$). Now $\tilde{\mathbf{X}}\tilde{\mathbf{w}}$ gives every prediction $\hat{y}_i = \mathbf{w}^\top \mathbf{x}_i + b$ at once, and the loss is:

$$L(\tilde{\mathbf{w}}) = \frac{1}{N} \sum_{i=1}^N (\tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}} - y_i)^2$$

In words: $\tilde{\mathbf{x}}_i$ is row i of $\tilde{\mathbf{X}}$, one house's features with a 1 in front. This is the same MSE as before, with the bias folded in.

Differentiate with respect to one weight $\tilde{w}_k$. Use the chain rule: the derivative of a square, times the derivative of the inside. The inside $\tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}}$ changes by $\tilde{x}_{i,k}$ when $\tilde{w}_k$ grows by 1:

$$\frac{\partial L}{\partial \tilde{w}_k} = \frac{1}{N} \sum_{i=1}^N 2 (\tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}} - y_i) \tilde{x}_{i,k}$$

In words: for every house, take its residual (predicted – actual) and multiply it by that house's k -th feature value. Then average and double. Stack this over every k to get a vector. That is the full gradient in matrix form:

$$\nabla_{\tilde{\mathbf{w}}} L = \frac{2}{N} \tilde{\mathbf{X}}^\top (\tilde{\mathbf{X}} \tilde{\mathbf{w}} - \mathbf{y})$$

In words: $\tilde{\mathbf{X}} \tilde{\mathbf{w}} - \mathbf{y}$ is every residual at once, a vector of length N . Multiplying by $\tilde{\mathbf{X}}^\top$ does the step above (“multiply by each feature column and sum over houses”) for every weight at the same time.

Now find the minimum: set the gradient to the zero vector and solve.

$$\begin{aligned} \frac{2}{N} \tilde{\mathbf{X}}^\top (\tilde{\mathbf{X}} \tilde{\mathbf{w}} - \mathbf{y}) &= \mathbf{0} \\ \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - \tilde{\mathbf{X}}^\top \mathbf{y} &= \mathbf{0} \\ \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} &= \tilde{\mathbf{X}}^\top \mathbf{y} \\ \tilde{\mathbf{w}} &= (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y} \end{aligned}$$

In words: multiply both sides by $N/2$, so the constant goes away. Move the $\mathbf{y}$ term to the other side. Then multiply both sides by the inverse of $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ to get $\tilde{\mathbf{w}}$ alone. This is the **normal equation**: one formula that gives the exact best-fit weights, with no search.

▲ WATCH OUT

The inverse $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1}$ exists only when $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ is non-singular. Roughly, this means no feature is an exact linear combination of the others (for example, one column exactly twice another). Real code does not call a matrix-inverse function. It calls a linear-solve routine, which gives the same answer faster and with smaller rounding errors.

◆ THE SAME THING IN NUMPY

>_ FIT THE LINE WITH THE NORMAL EQUATION

```

import numpy as np

def fit_normal_equation(X, y):
    N = X.shape[0]
    X_aug = np.hstack([np.ones((N, 1)), X])      # bias trick
    w_aug = np.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)  # solve; do it once
    b, w = w_aug[0], w_aug[1:]
    return w, b

X = np.array([[60.0], [75.0], [110.0], [140.0]])  # house size, m²
y = np.array([150_000.0, 200_000.0, 225_000.0, 310_000.0])

w, b = fit_normal_equation(X, y)
print(w, b)  # [1801.29240711] 47875.60581583208

```

So the best line is about $\hat{y} = 1801.29 x + 47876$: roughly 1,800 dollars per square meter on top of a base of about 48,000 dollars. Figure 5.1 shows this line.

CHECK YOURSELF 4

Two of your features are “size in m²” and “size in cm²”. One is the other times 10,000. What goes wrong in the normal equation, and why?

5 Optimization, take two: gradient descent

◆ INTUITION FIRST

The normal equation is exact, but it must solve a system with a $(d + 1) \times (d + 1)$ matrix. That becomes expensive when the number of features d is very large. **Gradient descent** works differently. It starts from any guess and takes many small steps downhill on the loss surface. The gradient (Section 2) points toward the steepest increase, so each step goes in the opposite direction.

☂ EVERYDAY PARALLEL

Think of tuning a guitar string by ear. You hear that the note is too high, so you turn the peg down a little. You listen again and correct again. When the note is far off, you turn more. When it is close, you turn less. The size of the error sets the size of the correction, and that is also true of the gradient. Turn the peg too far in one go, and you overshoot to the other side. A learning rate that is too large does the same thing.

◆ NOW THE MATH

We already derived the gradient above. Gradient descent does not set it to zero. Instead it takes a small step against it:

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \alpha \nabla_{\tilde{\mathbf{w}}} L$$

In words: the arrow means “replace the old value with this new one”. α (alpha) is the *learning rate*, a small positive number that sets the step size. We move a little in the direction *opposite* the gradient, because the gradient points toward higher loss. Repeat the update many times, and $\tilde{\mathbf{w}}$ slides down the bowl toward its bottom.

◆ NORMALIZE THE FEATURES FIRST

Gradient descent is sensitive to the scale of the features. The fix is to *normalize* each feature: subtract its mean μ and divide by its standard deviation s . The new feature $x' = (x - \mu)/s$ has mean 0 and standard deviation 1. Figure 5.2 shows why this helps. With a raw feature, the loss bowl is a long, narrow valley. The gradient points mostly across the valley, not along it, so progress is slow. After normalizing, the bowl is nearly round, and the gradient points almost straight at the bottom.

After training on x' , you can convert the weights back to the original units. Put $x' = (x - \mu)/s$ into the model and expand:

$$\begin{aligned}\hat{y} &= w' \frac{x - \mu}{s} + b' \\ &= \frac{w'}{s} x + \left(b' - \frac{w' \mu}{s} \right)\end{aligned}$$

In words: the weight in original units is w'/s . The bias in original units is b' minus the weight times the mean. The model is the same line; only the units of its numbers change.

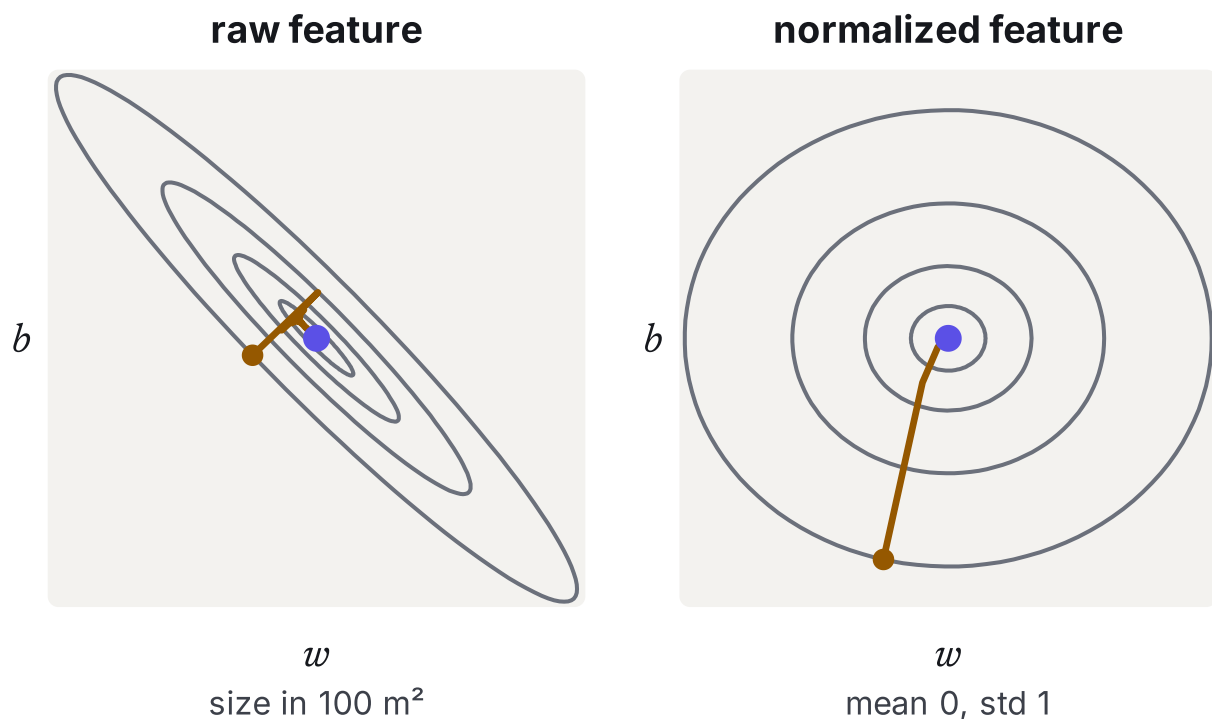


Figure 5.2 MSE contours for our four houses, with 30 gradient-descent steps (amber) from $w = b = 0$ with the same learning rate. Left: size in hundreds of m^2 , not centered. The valley is long and thin, and the path stops far from the minimum (violet). Right: normalized size. The contours are nearly circles, and the path reaches the minimum.

◆ WHEN GRADIENT DESCENT BEATS THE CLOSED FORM

Solving the normal equation takes about d^3 arithmetic steps for d features (written $O(d^3)$, “order d cubed”). Some datasets have a million features. An example is a *bag-of-words* text representation, with one feature per word in the vocabulary that counts how often the word appears. Then $d^3 = 10^{18}$, which is far too much. Such data is also *sparse*: almost all entries are zero. Storing only the nonzero entries saves memory, but $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ is often dense and huge.

Gradient descent needs only matrix–vector products at each step, and these are cheap for sparse data. It can also process the data in small batches, so the whole dataset need not fit in memory. The cost: it gives an approximate answer that improves over many steps, not an exact answer from one formula.

◆ THE SAME THING IN PYTORCH

>_ GRADIENT DESCENT ON NORMALIZED SIZE, THEN BACK TO DOLLARS PER M^2

```

import torch

X = torch.tensor([[60.], [75.], [110.], [140.]]) # house size, m2
y = torch.tensor([150_000., 200_000., 225_000., 310_000.])

# normalize the feature first (see the box above)
X_mean, X_std = X.mean(), X.std()
Xn = (X - X_mean) / X_std

w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
lr = 0.1

for step in range(500):
    y_hat = Xn.squeeze() * w + b # forward pass (the hypothesis)
    loss = ((y_hat - y) ** 2).mean() # MSE

    loss.backward() # autograd fills w.grad and b.grad

    with torch.no_grad(): # the update is not part of the graph
        w -= lr * w.grad
        b -= lr * b.grad
        w.grad.zero_()
        b.grad.zero_()

print(w.item(), b.item(), loss.item())
# 64685.78125 221249.96875 216498320.0

# undo the normalization: w = w'/s, b = b' - w'*mean/s
w_orig = w.item() / X_std.item()
b_orig = b.item() - w.item() * X_mean.item() / X_std.item()
print(round(w_orig, 2), round(b_orig))
# 1801.29 47876

```

The first line prints the weights in normalized units, so they look very different from the normal-equation answer. After we convert them back, they match it: about 1,801 dollars per m² and a base of about 47,876 dollars. Two different optimizers found the same bottom of the same bowl.

CHECK YOURSELF 5

In the code above, delete the normalization (use `Xn = X`) and keep `lr = 0.1`. Predict what happens.

6 What to carry forward

- Learning = **hypothesis class** (which functions we allow) + **loss** (how we score wrongness) + **optimization** (how we search). The algorithms in this course use these three parts in different forms.
- Linear regression's hypothesis is $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$: a straight line (or plane, or hyperplane) fit to the data.
- **MSE** squares and averages the errors. It is the maximum-likelihood loss when the noise is Gaussian.
- The **normal equation** $\tilde{\mathbf{w}} = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$ finds the exact minimum by setting the gradient to zero.
- **Gradient descent** walks downhill step by step. It works best on normalized features. Iterative methods like it are the usual choice when the data is too large or too sparse for the normal equation.

Next we keep the same triad but change the hypothesis and the loss. We will predict a category instead of a number, in Logistic Regression.

Answers to “Check yourself”

1. Your shower knob only turns between “cold” and “warm”, but you like hot water. Can a better optimization strategy fix this?

No. The best position the knob allows is “warm”, so the loss can never reach the value you want. The problem is the **hypothesis class**: it does not contain a good answer. A better search only finds the best member of the class faster. Models behave the same way. If a straight line cannot describe the data, no optimizer will make a straight line fit it well.

2. You fit $\hat{y} = wx + b$ to some houses. Then every price in the data doubles (sizes stay the same). What are the new best w and b ?

Both double. Take any line (w', b') and compare it with the doubled prices. Its errors are exactly twice the errors of the line $(w'/2, b'/2)$ on the old prices. So its loss is four times that line’s old loss. The new loss is smallest when $(w'/2, b'/2)$ is the old best line, that is, when $w' = 2w$ and $b' = 2b$. (Running the normal-equation code below with doubled `y` gives `[3602.58] 95751.21`, twice the original.)

3. One house in your data has a typo: its price is ten times too high. Which loss pulls the fitted line more toward that house, squared error or absolute error? Why?

Squared error. If that house’s error is 1,000,000, its squared error is 10^{12} , far larger than all the other houses together. The line bends toward it to shrink that one term. Absolute error only counts 1,000,000, so the other houses still matter. This matches the Gaussian story: a Gaussian says huge errors almost never happen, so one huge error must be “explained away” at great cost.

4. Two of your features are “size in m²” and “size in cm²”. One is the other times 10,000. What goes wrong in the normal equation, and why?

$\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ becomes singular (it has no inverse). The two columns carry the same information. You can raise one weight and lower the other by the matching amount, and the predictions do not change. So infinitely many weight pairs give the same lowest loss, and there is no single best answer. In the picture of the loss, the bowl has a flat valley floor along that direction instead of a single bottom. This is why the intuition above says “a single bottom when no feature is a copy of another”. The usual fix is *ridge regularization* (Section 7). It adds a small penalty that makes the matrix invertible again.

5. In the code above, delete the normalization (use `Xn = X`) and keep `lr = 0.1`. Predict what happens.

The weights blow up, and the printed values become `nan`. The size of one gradient step on w grows with x^2 , which is about 10,000 here. With $\alpha = 0.1$, each step overshoots the bottom by a huge amount, lands farther away, and the next step is even larger. You would need a learning rate thousands of times smaller, and then b would move extremely slowly. Normalizing removes this conflict.

Logistic Regression

One S-shaped curve turns a raw number into a probability. One loss function turns wrong probabilities into a clear, learnable signal.

36 min read

YOU'LL LEARN

- sigmoid
- probabilities
- decision boundaries
- binary cross-entropy

Before you start: Section 4: Information Theory and Section 5: What Learning Means.

Linear regression predicts a number: a price, a temperature, a score. Many real questions want a category instead. Is this email spam? Will this patient get sick again? Does this photo contain a cat? Logistic regression uses almost the same machinery as linear regression. It adds one extra part that turns the output into a probability.

◆ THE ONE IDEA TO HOLD ONTO

Compute a plain linear score, as in linear regression. Then pass that score through a curve whose output is always between 0 and 1, so you can read it as “probability of class 1”. The rest of this section builds that curve and the loss that trains it.

1 Classification vs. regression

◆ INTUITION FIRST

Regression predicts a continuous number, anywhere on a scale. **Classification** predicts which category, from a fixed set, an example belongs to. The output is not “how much” but “which one”.

☂ EVERYDAY PARALLEL

A weather forecast does both jobs. “Tomorrow’s high will be 22 °C” is regression: a number on a continuous scale, and missing by half a degree is a small mistake. “Will it rain tomorrow, yes or no?” is classification. There is no “half a rain”. There are two categories, and the model says which one is more likely.

CHECK YOURSELF 1

You want to predict how many stars (1 to 5) a customer will give a product. Is this regression or classification?

2 The sigmoid: turning any number into a probability

◆ INTUITION FIRST

A linear score $wx + b$ can be any real number: huge, tiny, or negative. A probability must be between 0 and 1. The **sigmoid function** is an S-shaped curve. It takes any real number and returns a number between 0 and 1. So we can give it a raw linear score and get back something we may call a probability.

☂ EVERYDAY PARALLEL

Think about a student’s chance of passing an exam as their study hours grow. With almost no study, the chance is near 0%. In the middle range, each extra hour helps a lot. After many hours, the chance is already close to 100%, so a few more hours change it very little. The chance rises in a smooth S-shape and never quite reaches 0% or 100%. Hours play the role of the score z , and the chance plays the role of $\sigma(z)$.

◆ NOW THE MATH

For any real number z , the sigmoid is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In words: e^{-z} shrinks toward 0 as z grows. It grows toward infinity as z becomes very negative. So the fraction moves toward $1/(1+0) = 1$ for large positive z , and toward 0 for very negative z . That is the squashing we wanted.

The input z is called the *logit*. The name comes from the inverse of the sigmoid. Solve $p = \sigma(z)$ for z :

$$\begin{aligned} p(1 + e^{-z}) &= 1 \\ e^{-z} &= \frac{1-p}{p} \\ z &= \log \frac{p}{1-p} \end{aligned}$$

In words: $p/(1-p)$ is the *odds*: the probability that something happens divided by the probability that it does not. So the logit is the *log-odds*. A logit of 0 means odds of 1 to 1. Each step of +1 in the logit multiplies the odds by $e \approx 2.7$.

We will need two more properties later. First, the midpoint:

$$\sigma(0) = \frac{1}{1+e^0} = \frac{1}{1+1} = \frac{1}{2}$$

In words: a logit of zero gives a probability of one-half. Both classes are equally likely.

Second, the sigmoid's derivative. We need it for the gradient of the loss. Write $\sigma(z) = (1 + e^{-z})^{-1}$ and apply the chain rule (Section 2):

$$\begin{aligned} \frac{d\sigma}{dz} &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\ &= \sigma(z)(1 - \sigma(z)) \end{aligned}$$

In words: the last line uses $1 - \sigma(z) = \frac{e^{-z}}{1+e^{-z}}$ (subtract the fraction from 1 and simplify). So the slope of the sigmoid is $\sigma(z)$ times $1 - \sigma(z)$. Both factors are positive, so the slope is always greater than 0. The slope is largest at $z = 0$, where it equals 0.25. It shrinks toward 0 at both ends, where the curve is almost flat.

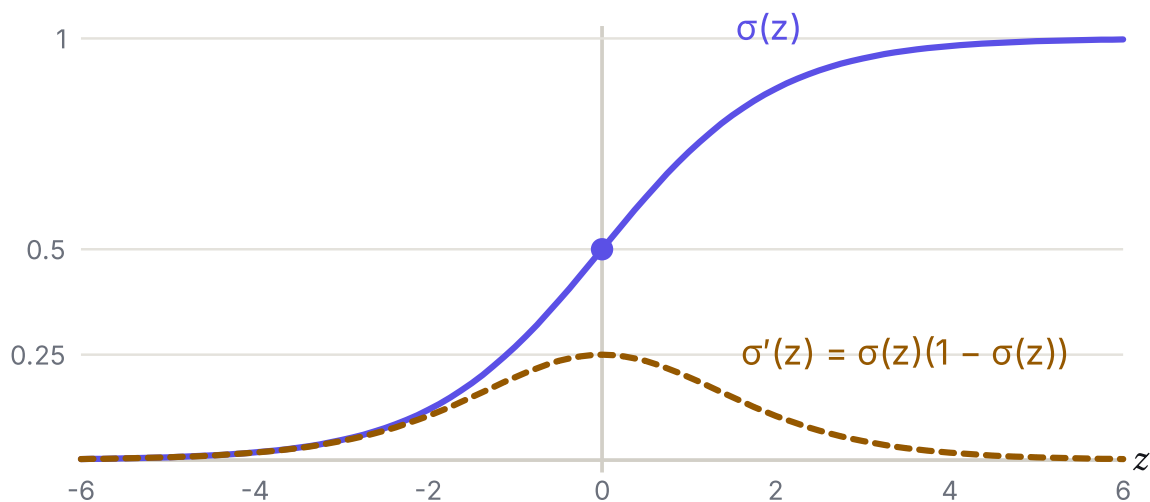


Figure 6.1 The sigmoid (violet) passes through 0.5 at $z = 0$. Its slope (amber, dashed) is largest there, at 0.25, and is almost zero when $|z|$ is larger than about 5. Remember these flat regions: they cause the problem in the loss section below.

◆ THE SAME THING IN PYTORCH

>_ THE SIGMOID AT THREE LOGITS

```
import torch

z = torch.tensor([-5.0, 0.0, 5.0])
print(torch.sigmoid(z))  # tensor([0.0067, 0.5000, 0.9933])
```

CHECK YOURSELF 2 $\sigma(3) \approx 0.953$. Without a calculator, what is $\sigma(-3)$?

3 From logit to probability to decision boundary

◆ INTUITION FIRST

Logistic regression computes the same linear score as linear regression, then passes it through the sigmoid. At some places the model changes from predicting class 0 to predicting class 1. Together these places form the

decision boundary. For logistic regression, the boundary is a straight line (or a flat plane in higher dimensions). Nothing about it is curved.

◆ NOW THE MATH

The logit is the linear-regression formula:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

Pass it through the sigmoid to get the probability of class 1:

$$p = \sigma(z) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

In words: z can be any real number, depending on $\mathbf{x}$. p is that number squashed into a probability between 0 and 1.

The prediction rule is: predict class 1 when $p \geq 0.5$. We showed that $\sigma'(z) = \sigma(z)(1 - \sigma(z)) > 0$ everywhere. So the sigmoid is *strictly increasing*: a larger z always gives a larger p . We also showed $\sigma(0) = \frac{1}{2}$. Together:

$$p \geq \frac{1}{2} \iff \sigma(z) \geq \sigma(0) \iff z \geq 0$$

In words: the sigmoid always goes up, and it passes one-half at $z = 0$. So these two questions always have the same answer: “is the probability at least one-half?” and “is the logit at least zero?” The curved probability test becomes a straight-line test on the score.

The boundary is where $p = 0.5$ exactly, so it is the set of points where $\mathbf{w}^\top \mathbf{x} + b = 0$. With two features x_1, x_2 , rearrange this into the equation of a line:

$$\begin{aligned} w_1 x_1 + w_2 x_2 + b &= 0 \\ x_2 &= -\frac{w_1}{w_2} x_1 - \frac{b}{w_2} \end{aligned}$$

In words: this is a line in the (x_1, x_2) plane with slope $-w_1/w_2$ and intercept $-b/w_2$ (assuming $w_2 \neq 0$). With three features the boundary is a flat plane. With more, it is a flat *hyperplane*. It is always flat, even though the probability changes along a curve as you cross it.

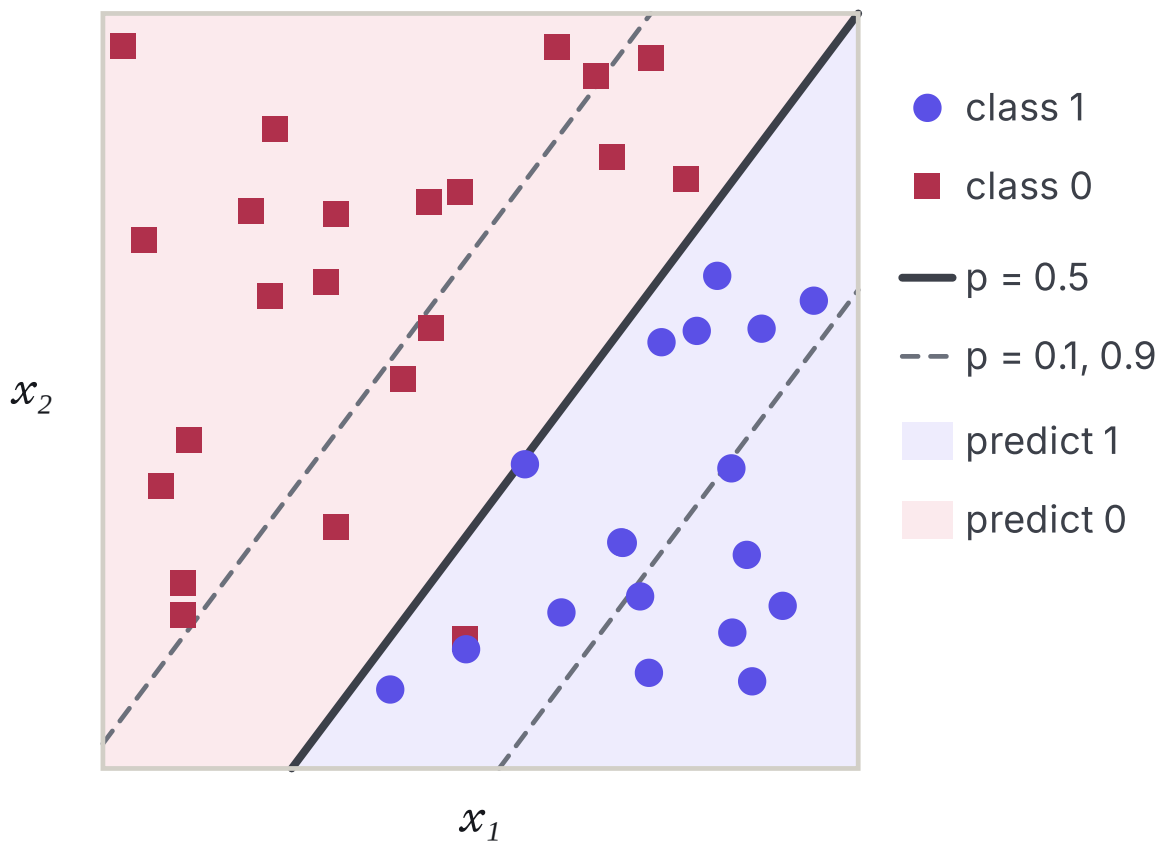


Figure 6.2 A model with $\mathbf{w} = [2, -1]$ and $b = -3$. The solid line is the boundary $2x_1 - x_2 - 3 = 0$. The dashed lines, where $p = 0.1$ and $p = 0.9$, are parallel to it: every level of probability is a straight line. Points near the boundary are the uncertain ones, and some of them fall on the “wrong” side.

CHECK YOURSELF 3

With $\mathbf{w} = [2, -1]$ and $b = -3$, where is the point $\mathbf{x} = [2, 1]$, and what does the model predict for it? What if you double both $\mathbf{w}$ and b ?

4 The loss: binary cross-entropy

◆ INTUITION FIRST

You might reuse Mean Squared Error on the probability: predicted probability minus true label, squared. That choice makes training slow and unreliable. Classification uses **binary cross-entropy** instead, a loss built for comparing probabilities.

☂ EVERYDAY PARALLEL

Imagine scoring a weather forecaster. Each day, look at the probability they gave to what really happened, and charge them $-\log$ of it. If they said 90% rain and it rained, the charge is small ($-\log 0.9 \approx 0.1$). If they said 1% rain and it rained, the charge is large ($-\log 0.01 \approx 4.6$). A forecaster who says “0%” and is wrong pays an unlimited price. This is binary cross-entropy.

DEFINITION

A function is **convex** when its graph is bowl-shaped: the straight segment between any two points on the graph never goes below the graph. A convex function has no false valleys. Every local minimum is a global minimum, so gradient descent (with a small enough learning rate) cannot get stuck in a bad dip. A **strictly convex** function has at most one lowest point. A convex function can still have no minimum at all (think of e^{-z} , which keeps going down toward 0), and this matters below.

◆ NOW THE MATH

First, where binary cross-entropy comes from. The label y is 0 or 1, so it is a Bernoulli variable (Section 3). The model says “ $y = 1$ with probability $\hat{y}$ ”. The probability the model gives to the true label can be written in one formula:

$$P(y \mid \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y}$$

In words: if $y = 1$, the formula gives $\hat{y}$. If $y = 0$, it gives $1 - \hat{y}$. The exponents only select the right factor.

Maximum likelihood says: make this probability large. That is the same as making its negative log small:

$$\begin{aligned} L &= -\log P(y \mid \mathbf{x}) \\ &= -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \end{aligned}$$

In words: for each example, only one term is active. If $y = 1$, the loss is $-\log \hat{y}$. It is small when $\hat{y}$ is near 1 and grows without limit as $\hat{y} \rightarrow 0$. If $y = 0$, the loss is $-\log(1 - \hat{y})$, with the roles swapped. This is also the cross-entropy $H(p, q)$ from Section 4, with true distribution $p = (y, 1 - y)$ and model distribution $q = (\hat{y}, 1 - \hat{y})$.

Now, why does squared error behave badly on top of a sigmoid? Let $L_{sq} = (\hat{y} - y)^2$ with $\hat{y} = \sigma(z)$. By the chain rule, with $\sigma'(z) = \hat{y}(1 - \hat{y})$:

$$\begin{aligned}\frac{\partial L_{sq}}{\partial z} &= 2(\hat{y} - y) \sigma'(z) \\ &= 2(\hat{y} - y) \hat{y}(1 - \hat{y})\end{aligned}$$

In words: the gradient contains the sigmoid's slope $\hat{y}(1 - \hat{y})$. Suppose the true label is $y = 1$, but the model is confident and wrong: $\hat{y} \approx 0$. Then $\hat{y}(1 - \hat{y}) \approx 0$, so the whole gradient is almost zero. The model gets almost no correction exactly when it needs a large one. Training slows down, sometimes for a very long time.

There is a second, separate problem: squared error on a sigmoid is not convex. For $y = 1$ the loss is $f(z) = (1 - \sigma(z))^2$. A convex function has a slope that never decreases as z grows. Here $f'(0) = -2 \cdot \frac{1}{2} \cdot \frac{1}{4} = -\frac{1}{4}$. But as $z \rightarrow -\infty$, the curve flattens out, so $f'(z) \rightarrow 0$. The slope therefore *decreases* from about 0 far to the left down to $-\frac{1}{4}$ at $z = 0$. That is impossible for a convex function. Figure 6.3 shows the flat shelf on the left.

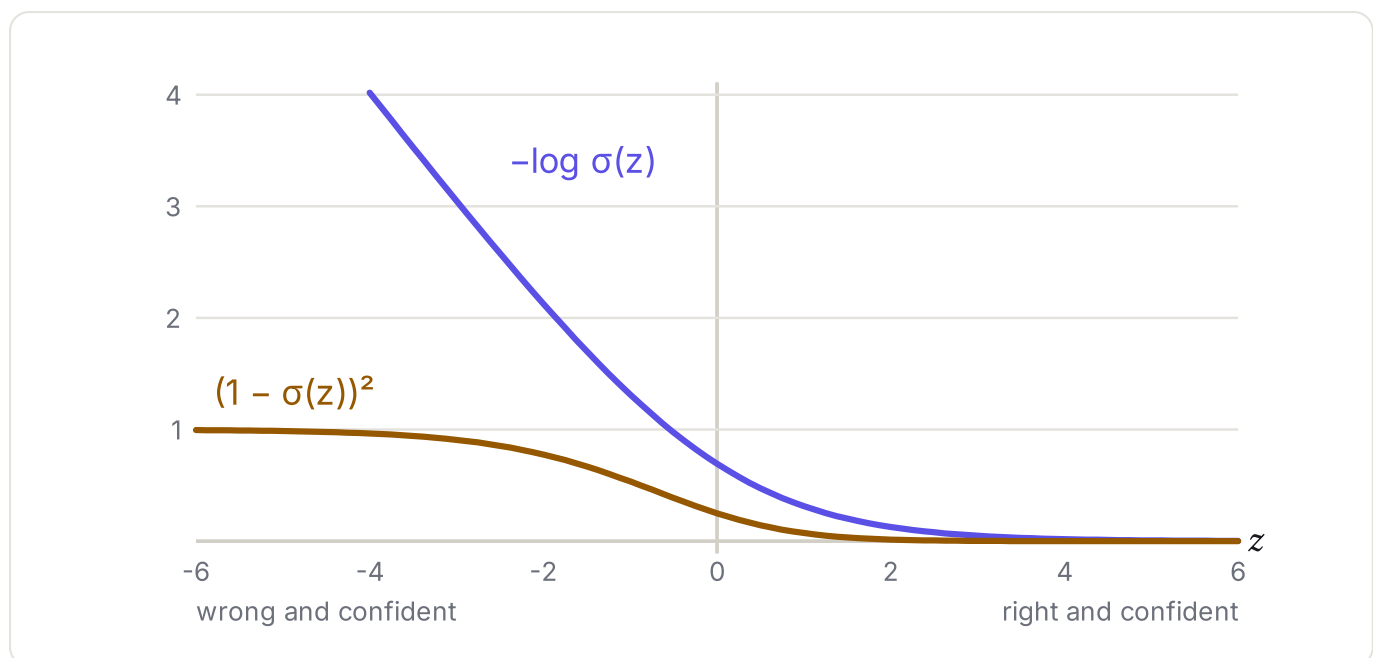


Figure 6.3 Loss for one example with true label $y = 1$, as a function of the logit z . Cross-entropy (violet) keeps rising in a nearly straight line when the model is confidently wrong, so its slope stays near -1 . Squared error (amber) flattens out at 1, so its slope goes to 0 there. It also bends the wrong way for a convex function.

Now derive the gradient of binary cross-entropy with respect to the logit z . First differentiate with respect to $\hat{y}$:

$$\begin{aligned}
\frac{\partial L}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \\
&= \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})} \\
&= \frac{-y + y\hat{y} + \hat{y} - y\hat{y}}{\hat{y}(1-\hat{y})} \\
&= \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}
\end{aligned}$$

Then apply the chain rule through $\hat{y} = \sigma(z)$, using $\partial \hat{y} / \partial z = \hat{y}(1 - \hat{y})$:

$$\begin{aligned}
\frac{\partial L}{\partial z} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \\
&= \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y
\end{aligned}$$

In words: the $\hat{y}(1 - \hat{y})$ factors cancel. What is left is the simplest possible gradient: predicted probability minus true label. It does not vanish when the model is confidently wrong. Its size is proportional to how wrong the prediction is.

Finally, $z = \mathbf{w}^\top \mathbf{x} + b$, so $\partial z / \partial w_j = x_j$ and $\partial z / \partial b = 1$. One more chain rule gives:

$$\frac{\partial L}{\partial \mathbf{w}} = (\hat{y} - y) \mathbf{x}, \quad \frac{\partial L}{\partial b} = \hat{y} - y$$

In words: the gradient for each weight is (error) $\times$ (that feature's value). This has the same shape as linear regression's gradient, although the loss is different. That is why the training loop for logistic regression looks almost the same as the one for linear regression.

Is binary cross-entropy convex in $\mathbf{w}$ and b ? Yes. As a function of z , its second derivative is $\partial(\hat{y} - y) / \partial z = \hat{y}(1 - \hat{y}) > 0$, so it curves upward everywhere. A convex function of a linear function of the weights is convex in the weights, and a sum of convex functions is convex. So there are no false valleys. But convex does not guarantee that a minimum exists, as the next section shows.

CHECK YOURSELF 4

True label $y = 1$. Compare a prediction of $\hat{y} = 0.2$ with $\hat{y} = 0.01$. For each, what is $\partial L / \partial z$, and which way does gradient descent push the logit?

5 Training logistic regression in PyTorch

◆ THE SAME THING IN PYTORCH

>_ TRAIN ON FOUR STUDENTS: HOURS STUDIED, PASSED OR FAILED

```
import torch
import torch.nn as nn

X = torch.tensor([[2.0], [4.0], [6.0], [8.0]]) # hours studied
y = torch.tensor([0., 0., 1., 1.]) # failed / passed

w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
lr = 0.1

# BCEWithLogitsLoss = sigmoid + binary cross-entropy in one stable function
# It takes the raw logit z, not a probability.
loss_fn = nn.BCEWithLogitsLoss()

for step in range(1000):
    z = X.squeeze() * w + b # the logit: no sigmoid before this loss
    loss = loss_fn(z, y)

    loss.backward()
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad
        w.grad.zero_()
        b.grad.zero_()

probs = torch.sigmoid(X.squeeze() * w + b) # logits to probabilities, for printing
print(probs, loss.item())
# tensor([0.0301, 0.2764, 0.8246, 0.9830], grad_fn=<SigmoidBackward0>) 0.125
print(w.item(), b.item())
# 1.2552133798599243 -5.98319673538208
```

The model predicts “pass” for 6 and 8 hours and “fail” for 2 and 4 hours. Its boundary is at $-b/w \approx 4.77$ hours.

▲ WATCH OUT

Give `nn.BCEWithLogitsLoss` the raw logit. Do not apply `torch.sigmoid` yourself and then use `nn.BCELoss`. The combined function computes $\log \sigma(z)$ in a numerically stable way (with the log-sum-exp trick), so large logits cause no problems. The two-step version fails when the sigmoid saturates. In float32, $\sigma(-200)$ is exactly 0. `nn.BCELoss` then clamps $\log 0$ to -100 , so the loss stays finite, but the gradient is 0 and learning stops.

›_ THE SAME CONFIDENT MISTAKE, TWO WAYS

```
import torch
import torch.nn as nn

target = torch.tensor([1.])

z = torch.tensor([-200.], requires_grad=True)
loss = nn.BCELoss()(torch.sigmoid(z), target)
loss.backward()
print(loss.item(), z.grad)    # 100.0 tensor([-0.])

z = torch.tensor([-200.], requires_grad=True)
loss = nn.BCEWithLogitsLoss()(z, target)
loss.backward()
print(loss.item(), z.grad)    # 200.0 tensor([-1.])
```

The stable version reports the true loss (200) and the correct gradient $\hat{y} - y = -1$. The two-step version reports a clamped loss and no gradient at all.

◆ WHEN THE CLASSES CAN BE SEPARATED PERFECTLY

Our four students can be split perfectly: everyone below 5 hours failed, everyone above passed. For such *separable* data, the loss has no minimum. Making w and b larger (while keeping $-b/w$ near 5) always makes every probability more confident and the loss smaller. So training never stops growing the weights:

›_ W, B AND LOSS AS TRAINING CONTINUES

steps	w	b	loss
1000	1.26	-5.98	0.1411
10000	3.00	-14.81	0.0248
100000	5.27	-26.15	0.0026

The model becomes more and more certain, even though four examples cannot justify that certainty. The usual fixes are regularization (a penalty on large weights, Section 7) or stopping training early.

CHECK YOURSELF 5

Add a fifth student who studied 7 hours but failed. Does training still grow w and b forever?

6 What to carry forward

- **Classification** predicts a category, not a number. Logistic regression handles the two-category case.
- The **sigmoid** $\sigma(z) = 1/(1 + e^{-z})$ squashes any logit into a valid probability. $\sigma(0) = 0.5$, and the slope $\sigma(z)(1 - \sigma(z))$ is always positive. The logit is the log-odds.
- The **decision boundary** ($p = 0.5$) is where the logit $\mathbf{w}^\top \mathbf{x} + b$ crosses zero. It is always flat: a line or hyperplane.
- **Binary cross-entropy** is the negative log-likelihood of a Bernoulli label. It replaces squared error, whose gradient vanishes on confidently wrong predictions. Its gradient is $(\hat{y} - y)\mathbf{x}$.
- The loss is convex, but on separable data it has no minimum. Regularization or early stopping keeps the weights finite.

In Section 5 we solved a regression problem in two ways. Here we reused the same hypothesis–loss–optimization triad for classification, with a squashing function and a different loss.

Answers to “Check yourself”

1. You want to predict how many stars (1 to 5) a customer will give a product. Is this regression or classification?

It can be either, and the choice matters. As classification, the model treats the five ratings as unrelated categories, so predicting 4 when the truth is 5 is as wrong as predicting 1. As regression, the model knows that 4 is close to 5, but it may predict values like 3.7 that no customer can give. Many people choose regression here and round the result. This shows that the problem does not decide the type alone. How you score mistakes decides it.

2. $\sigma(3) \approx 0.953$. Without a calculator, what is $\sigma(-3)$?

About 0.047, because $\sigma(-z) = 1 - \sigma(z)$ for every z . To see why, multiply the top and bottom of $\sigma(-z) = \frac{1}{1+e^z}$ by e^{-z} . You get $\frac{e^{-z}}{e^{-z}+1}$, which is the expression for $1 - \sigma(z)$ from the derivation above. The output above shows the same symmetry: 0.0067 and 0.9933 add up to 1.

3. With $\mathbf{w} = [2, -1]$ and $b = -3$, where is the point $\mathbf{x} = [2, 1]$, and what does the model predict for it? What if you double both $\mathbf{w}$ and b ?

$z = 2 \cdot 2 + (-1) \cdot 1 - 3 = 0$, so the point is on the boundary and $p = \sigma(0) = 0.5$. The model is equally unsure about both classes. The rule “predict 1 when $p \geq 0.5$ ” still gives an answer: class 1. Doubling $\mathbf{w}$ and b does not move the boundary, because $2z = 0$ exactly where $z = 0$. It does make the probabilities change faster: the dashed lines in Figure 6.2 move closer to the boundary.

4. True label $y = 1$. Compare a prediction of $\hat{y} = 0.2$ with $\hat{y} = 0.01$. For each, what is $\partial L / \partial z$, and which way does gradient descent push the logit?

For $\hat{y} = 0.2$: $\partial L / \partial z = 0.2 - 1 = -0.8$. For $\hat{y} = 0.01$: -0.99 . Gradient descent subtracts a positive multiple of the gradient, so in both cases the logit moves *up* and $\hat{y}$ moves toward 1. The more wrong prediction gets the larger push. With squared error, the pushes would be $2(0.2 - 1)(0.2)(0.8) = -0.256$ and $2(0.01 - 1)(0.01)(0.99) \approx -0.0196$: the more wrong prediction would get the *smaller* push.

5. Add a fifth student who studied 7 hours but failed. Does training still grow w and b forever?

No. Now no single threshold separates the classes: the student at 7 hours failed, but the one at 6 hours passed. If the weights grew without limit, the model would become certain that the 7-hour student passed. That student’s loss $-\log(1 - \hat{y})$ would grow without limit too. So at some finite w and b the total loss is smallest, and gradient descent settles there. Overlapping classes give the loss a real minimum.

Overfitting & Regularization

A model that scores 100% on its training data has not yet shown you anything. This section is about the number that counts, measured on data the model has never seen, and how to protect it.

38 min read

YOU'LL LEARN

- generalization
- bias–variance
- L1/L2 regularization
- train/val/test
- cross-validation

Before you start: Section 5: What Learning Means.

You can make almost any model fit its training data very closely. That part is easy, and it tells you little. What we care about is whether the model works on **new** data: tomorrow's customer, the next photo, a sentence it has not read. This section is about the gap between those two, why it opens, and the tools that close it.

◆ THE ONE IDEA TO HOLD ONTO

The score that matters is performance on data the model never saw during training. Every idea below (splitting your data, cross-validation, regularization) exists to keep that one number trustworthy.

1 Generalization: doing well on data you haven't seen

◆ INTUITION FIRST

Generalization is a model's ability to perform well on new examples, not only on the ones it trained on. A model that generalizes has learned the real pattern in the data. A model that does not has memorized the specific examples, including their noise and accidents.

☂ EVERYDAY PARALLEL

Two students prepare for a history exam. One **memorizes** last year's exam paper word for word. The other **understands** the period: causes, results, and how events connect. Give both students last year's paper, and the memorizer wins easily. Give them this year's paper, which asks about the same material in new ways, and the memorizer fails. The memorizer *overfit* to one paper. The other student *generalized* to the subject.

DEFINITION

Overfitting is when a model fits the training data too closely, including its noise. Training error keeps going down while error on new data gets worse. **Underfitting** is the opposite. The model is too simple to capture even the real pattern, so it does poorly on both training data and new data. **Model capacity** is a rough measure of how flexible a model is: how complicated a pattern it can represent. A straight line has low capacity. A deep decision tree with a thousand splits has very high capacity.

Capacity controls both failure modes. If it is too low, the model cannot represent the real pattern, so it underfits. If it is too high, the model has so much freedom that it fits the random noise in your particular training set. Then it overfits. The best capacity is somewhere in between. Most of this section is about finding it.

◆ THE SAME THING IN NUMPY

>_ FITTING THE SAME NOISY DATA WITH THREE CAPACITIES

```

import numpy as np

rng = np.random.default_rng(0)
x = np.linspace(-3, 3, 20)
y = 0.5 * x**2 + rng.normal(0, 1, size=x.shape)      # true pattern: a p

x_test = rng.uniform(-3, 3, size=1000)              # new points from
y_test = 0.5 * x_test**2 + rng.normal(0, 1, size=x_test.shape)

for degree in (1, 2, 15):
    coeffs = np.polyfit(x, y, degree)
    train_mse = np.mean((np.polyval(coeffs, x) - y) ** 2)
    test_mse = np.mean((np.polyval(coeffs, x_test) - y_test) ** 2)
    print(f"degree {degree:2d}: train MSE {train_mse:.3f}    test MSE {tes

```

>_ OUTPUT

```

degree  1: train MSE 3.846    test MSE 2.935
degree  2: train MSE 0.592    test MSE 1.182
degree 15: train MSE 0.139    test MSE 5.758

```

Read the three rows one at a time.

- **Degree 1** (a line) cannot bend to follow a parabola. Both errors are high: it underfits. (Its test error is a little lower than its training error only by chance: 20 training points are a small sample.)
- **Degree 2** (a parabola) matches the true pattern. Its test error, 1.18, is close to 1.0, the variance of the noise. No model can predict the noise, so no model can do much better on average.
- **Degree 15** has the lowest training error but by far the highest test error. It learned the noise, not the pattern: it overfits.

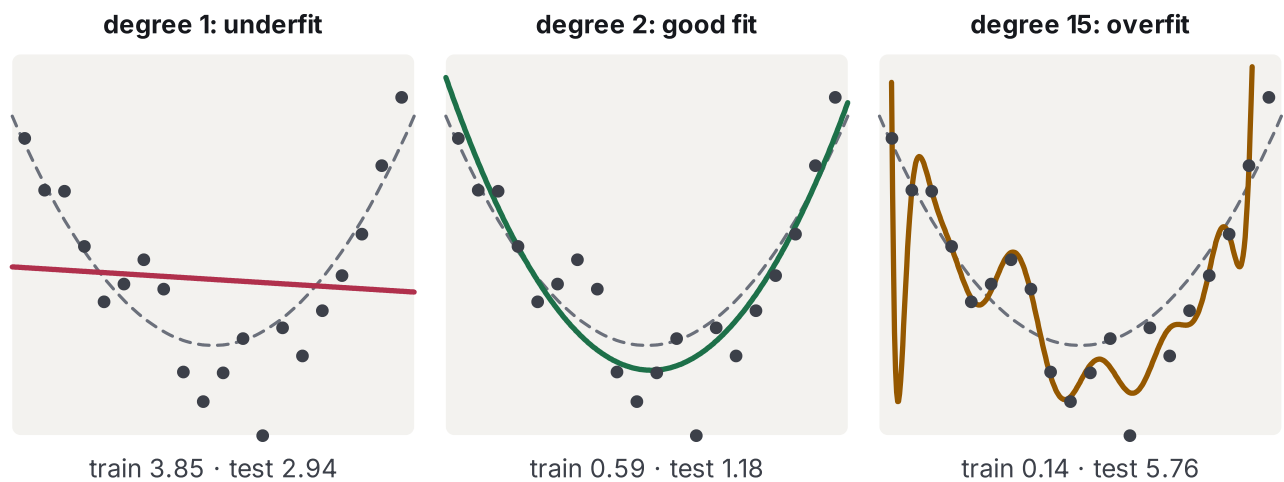


Figure 7.1 The 20 training points from the code, with the true parabola (dashed). The line misses the curve. The parabola follows it. The degree-15 curve passes close to the points but swings far away between them, and those swings are what the test points punish.

CHECK YOURSELF 1

A model reaches 99% accuracy on its training set but only 61% on its validation set. Is this overfitting or underfitting? What is a good first fix?

2 Train / validation / test — and cross-validation

◆ INTUITION FIRST

If you measure your model only on the data it trained on, you cannot detect overfitting. The memorizer always looks excellent on the paper it memorized. So we split the data into three separate parts. Each part has a different job, and we never let them mix.

☂ EVERYDAY PARALLEL

Preparing for a real exam has three phases. You do **practice questions** to learn the material (the **training set**). You take a **practice exam** to see how you are doing and to decide what to study again (the **validation set**). Finally, you take the **real exam** once (the **test set**). Its value comes from the fact that you never changed your studying because of it. Suppose you saw the real exam early and kept “improving” based on it. Then your final grade would no longer tell you how well you know the material.

DEFINITION

The **training set** is what the model learns its numbers from. The **validation set** is held out during training and used to make decisions: which model, which settings. These settings are called **hyperparameters**: choices you make before training, such as capacity or the regularization strength below. The **test set** is used once, at the end, to report a final, unbiased number. If you tune anything based on the test set, it becomes a second validation set, and your reported score is no longer trustworthy.

◆ NOW THE MATH

With only one validation split, your estimate of “how good is this model” depends on which examples happened to land in that split. A small, unlucky validation set can make a good model look bad, or the other way around. **k-fold cross-validation** fixes this. It rotates which part is held out, so every example is used for validation exactly once. Let s_i be the validation score of a model trained on all folds except fold i and evaluated on fold i . Then:

$$\text{CV score} = \frac{1}{k} \sum_{i=1}^k s_i$$

In words: split the training data into k folds of equal size (5 and 10 are common choices). Train k separate times, each time leaving out a different fold to validate on. Then average the k validation scores. Every example is validated on exactly once, so the average depends much less on one unlucky split. The spread of the k scores also tells you how stable the estimate is.

◆ THE SAME THING IN NUMPY

>_ SPLIT 10 ROWS INTO 5 FOLDS

```
import numpy as np

X = np.arange(20).reshape(10, 2)           # 10 training examples, 2 features
k = 5
rng = np.random.default_rng(0)
folds = np.array_split(rng.permutation(len(X)), k)  # shuffle, then cut

for i in range(k):
    val_idx = folds[i]
    train_idx = np.concatenate([folds[j] for j in range(k) if j != i])
    print(f"fold {i}: train on {len(train_idx)} rows, validate on rows {sorted(val_idx)}
```

>_ OUTPUT

```
fold 0: train on 8 rows, validate on rows [4, 6]
fold 1: train on 8 rows, validate on rows [2, 7]
fold 2: train on 8 rows, validate on rows [3, 5]
fold 3: train on 8 rows, validate on rows [0, 9]
fold 4: train on 8 rows, validate on rows [1, 8]
```

Each of the 10 rows appears in exactly one validation fold. In practice you would use a library for this, for example scikit-learn's `KFold(n_splits=5, shuffle=True)`, which does the same job.

CHECK YOURSELF 2

While tuning a model, you check test-set accuracy after every change and keep the version with the best test score. You never train on the test set. What is wrong?

3 The bias–variance tradeoff

◆ INTUITION FIRST

Overfitting and underfitting are two sides of a deeper split. A model's test error comes from two separate sources, **bias** and **variance**. As capacity changes, one goes down while the other goes up.

☂ EVERYDAY PARALLEL

Picture a dartboard. Each dart is one model, trained on a new random sample of data. The bullseye is the true answer. **High bias** means the darts land in a tight group, but the group is far from the bullseye: consistent, but consistently wrong. **High variance** means the darts scatter over the board: each new sample gives a very different model. A good model is both consistent *and* centered on the bullseye. A bad one can fail in either way, or both.

DEFINITION

Bias is systematic error. The model's assumptions are too simple for the real pattern, so its predictions are wrong in the same direction whatever training data it sees. A straight line forced to fit a curve has high bias. **Variance** is sensitivity to the particular training set. Retrain on a slightly different sample, and the predictions change a lot. A deep, unconstrained tree that bends around every training point has high variance.

◆ NOW THE MATH

Suppose the true relationship is $y = f(x) + \epsilon$. The noise ϵ has mean 0 and variance σ^2 , and it is independent of everything else. No model can predict it, so we call it irreducible. We train a model on a random training set D and get the prediction $\hat{f}_D(x)$ at a test point x . Let $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$ be the *average* prediction over many random training sets.

We want the expected squared error at x , averaged over both the training set D and the noise ϵ in the test label. Add and subtract $\bar{f}(x)$, and write the error as a sum of three parts:

$$y - \hat{f}_D(x) = a + b_D + \epsilon$$

In words: $a = f(x) - \bar{f}(x)$ is a fixed number (it does not depend on D). $b_D = \bar{f}(x) - \hat{f}_D(x)$ changes with the training set. ϵ is the test-label noise. The three parts add up to $y - \hat{f}_D(x)$ because $y = f(x) + \epsilon$.

Square the sum and take the expectation of each term:

$$\begin{aligned}\mathbb{E}[(a + b_D + \epsilon)^2] \\ &= a^2 + \mathbb{E}[b_D^2] + \mathbb{E}[\epsilon^2] \\ &\quad + 2a\mathbb{E}[b_D] + 2a\mathbb{E}[\epsilon] \\ &\quad + 2\mathbb{E}[b_D\epsilon]\end{aligned}$$

In words: the last three terms (the cross terms) are all zero. Also, $\mathbb{E}[\epsilon^2] = \sigma^2$.

$\mathbb{E}[b_D] = \bar{f}(x) - \mathbb{E}_D[\hat{f}_D(x)] = 0$ by the definition of $\bar{f}$. $\mathbb{E}[\epsilon] = 0$ because the noise has mean zero.

$\mathbb{E}[b_D \epsilon] = \mathbb{E}[b_D] \mathbb{E}[\epsilon] = 0$ because the test noise is independent of the training set.

What remains is the bias–variance decomposition:

$$\begin{aligned} \mathbb{E}[(y - \hat{f}_D(x))^2] &= \underbrace{(f(x) - \bar{f}(x))^2}_{\text{bias}^2} \\ &\quad + \underbrace{\mathbb{E}_D[(\bar{f}(x) - \hat{f}_D(x))^2]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}} \end{aligned}$$

In words: three parts add up to the test error. **Bias²** is how far the average prediction is from the truth. **Variance** is how much predictions change between training sets. **Noise** σ^2 is the part no model can predict. All three are non-negative, so the test error can never go below σ^2 .

◆ THE TRADEOFF

More capacity (a deeper tree, more parameters, a higher polynomial degree) usually **lowers bias**, because the model can follow the true pattern more closely. It usually also **raises variance**, because the model has more freedom to follow the noise in its training set. So total test error, plotted against capacity, is often U-shaped (Figure 7.2). Regularization, next, is a way to reduce variance without giving up all the capacity.

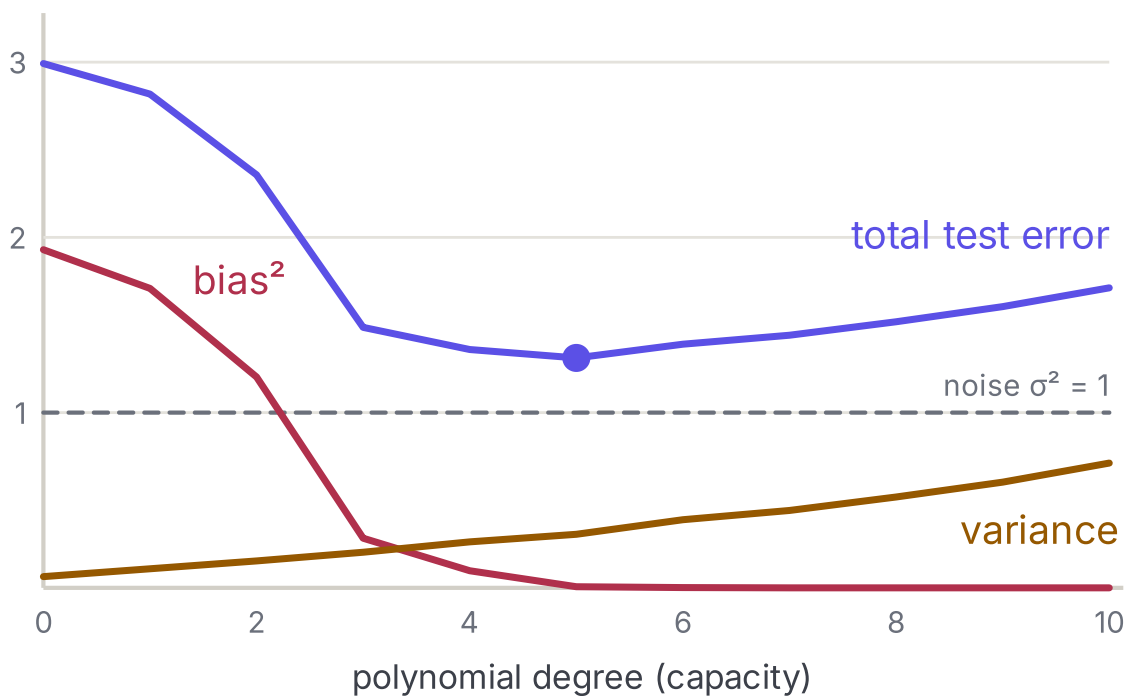


Figure 7.2 Bias² (rose) and variance (amber) measured by simulation. We drew 2,000 training sets of 15 noisy points from a wavy curve and fit polynomials of each degree. Total test error (violet) is bias² + variance + noise (dashed line at 1). It is lowest at degree 5, where bias² is already near zero and variance has not yet grown much.

CHECK YOURSELF 3

You keep the model (degree 15) but collect ten times more training data from the same source. Which of bias², variance and noise changes, and in which direction?

4 Regularization: penalizing the size of the weights

◆ INTUITION FIRST

Regularization fights overfitting directly. It adds an extra term to the loss that penalizes *large* weights, on top of the usual term that penalizes wrong predictions. The model can still use every weight, but it pays a price for large ones. This pushes it toward simpler, smoother solutions that are less likely to follow noise. The penalty uses the norms from Section 1: L2 or L1, applied to the weights.

☂ EVERYDAY PARALLEL

Imagine two ways to charge you for shopping. **L1** is a flat fee on every dollar you spend. Per dollar, a small purchase pays the same fee as a big one. A purchase that is only a little useful is not worth that fee, so you drop it completely. **L2** is a tax on the *square* of each purchase. Big purchases are taxed heavily, but small ones are taxed almost nothing, so you keep them, only a little smaller. L1 drops weights to zero. L2 shrinks all weights but rarely drops one.

◆ NOW THE MATH

L2 regularization (also called *ridge*, or *weight decay* with gradient descent) adds the squared L2 norm of the weights θ to the loss:

$$\begin{aligned} L_{\text{ridge}}(\theta) &= L_{\text{data}}(\theta) + \lambda \|\theta\|_2^2 \\ &= L_{\text{data}}(\theta) + \lambda \sum_i \theta_i^2 \end{aligned}$$

In words: L_{data} is the ordinary loss for wrong predictions. λ (lambda) is a hyperparameter you choose. It sets how strongly we punish large weights.

The penalty $\lambda\theta_i^2$ has derivative $2\lambda\theta_i$. Put the full gradient into a gradient descent step with learning rate α :

$$\begin{aligned} \theta_i &\leftarrow \theta_i - \alpha \left(\frac{\partial L_{\text{data}}}{\partial \theta_i} + 2\lambda\theta_i \right) \\ &= \underbrace{(1 - 2\alpha\lambda)}_{\text{shrink factor}} \theta_i - \alpha \frac{\partial L_{\text{data}}}{\partial \theta_i} \end{aligned}$$

In words: each update multiplies the weight by a factor a little below 1, then applies the usual data-driven step. That is why L2 is called “weight decay”: the weight decays toward zero every step, on top of whatever the data pushes it toward. The shrink is proportional to the weight’s own size, so it gets weaker as the weight gets small. It does not by itself drive a weight to exactly zero.

L1 regularization (also called *lasso*) adds the L1 norm instead:

$$\begin{aligned} L_{\text{lasso}}(\theta) &= L_{\text{data}}(\theta) + \lambda \|\theta\|_1 \\ &= L_{\text{data}}(\theta) + \lambda \sum_i |\theta_i| \end{aligned}$$

In words: for $\theta_i \neq 0$, the penalty’s gradient is $\lambda \text{sign}(\theta_i)$. This is a *constant* push toward zero, the same size for a huge weight and a tiny one. L2 shrinks by a fraction. L1 shrinks by a fixed

amount. Suppose a weight sits at zero and its data gradient there is smaller than λ in size. Then the data cannot overcome the push, so the best value for that weight is exactly zero. That is L1 doing *feature selection*: it decides that some inputs are not worth using.

There is a catch. The best value is exactly zero, but plain gradient descent does not reach it. Near zero, the constant push overshoots: the weight jumps from +0.001 to -0.001 and back, and it rarely lands on 0.0 exactly. Solvers that produce exact zeros use a different step. After the ordinary data step, they apply *soft-thresholding*: move each weight toward zero by $\alpha\lambda$, and stop at zero instead of crossing it.

$$\theta_i \leftarrow \text{sign}(\theta_i) \max(|\theta_i| - \alpha\lambda, 0)$$

In words: a weight smaller than $\alpha\lambda$ becomes exactly 0. A larger weight shrinks by $\alpha\lambda$. This is called a *proximal* gradient step. scikit-learn's `Lasso` uses coordinate descent, which applies the same soft-thresholding to one weight at a time.

◆ THE SAME THING IN PYTORCH

>_ L1 WITH PLAIN GRADIENT STEPS VS. WITH SOFT-THRESHOLDING

```

import torch

torch.manual_seed(0)
X = torch.randn(200, 6)
true_w = torch.tensor([3.0, -2.0, 0.0, 0.0, 0.0, 0.0]) # only 2 features
y = X @ true_w + 0.5 * torch.randn(200)

lam, lr = 0.1, 0.05

def data_grad(w): # gradient of the MSE part only
    return 2 * X.T @ (X @ w - y) / len(y)

# (a) plain gradient descent on MSE + lam * |w| (sign(w) as the gradient)
w = torch.zeros(6)
for step in range(2000):
    w = w - lr * (data_grad(w) + lam * torch.sign(w))
print("plain GD:", [round(v, 4) for v in w.tolist()])

# (b) proximal step: a normal step on the data loss, then soft-thresholding
w = torch.zeros(6)
for step in range(2000):
    w = w - lr * data_grad(w)
    w = torch.sign(w) * torch.clamp(w.abs() - lr * lam, min=0) # shrink
print("proximal:", [round(v, 4) for v in w.tolist()])

# plain GD: [2.9864, -1.9335, 0.0072, 0.0029, 0.0008, 0.0035]
# proximal: [2.9871, -1.934, 0.0, 0.0, 0.0, 0.0]

```

Both runs find the two useful weights. Only the proximal version sets the four useless weights to exactly zero. Plain gradient descent leaves them moving around near zero.

◆ WHY L1 ZEROES WEIGHTS AND L2 DOESN'T: THE GEOMETRIC REASON

You can view each penalty as a budget: minimize the data loss while keeping the penalty below some limit t . The L2 budget $\sum \theta_i^2 \leq t$ is a round disk (a ball in higher dimensions). The L1 budget $\sum |\theta_i| \leq t$ is a **diamond**. Its sharp corners lie on the axes, where one weight is zero.

Now let the loss contours grow outward from the unconstrained best point. The solution is where they first touch the budget region. A round disk has no special points, so the contact is almost never on an axis. A diamond's corners stick out, so the contours often touch a corner first. Touching at a corner means a weight is exactly zero (Figure 7.3).

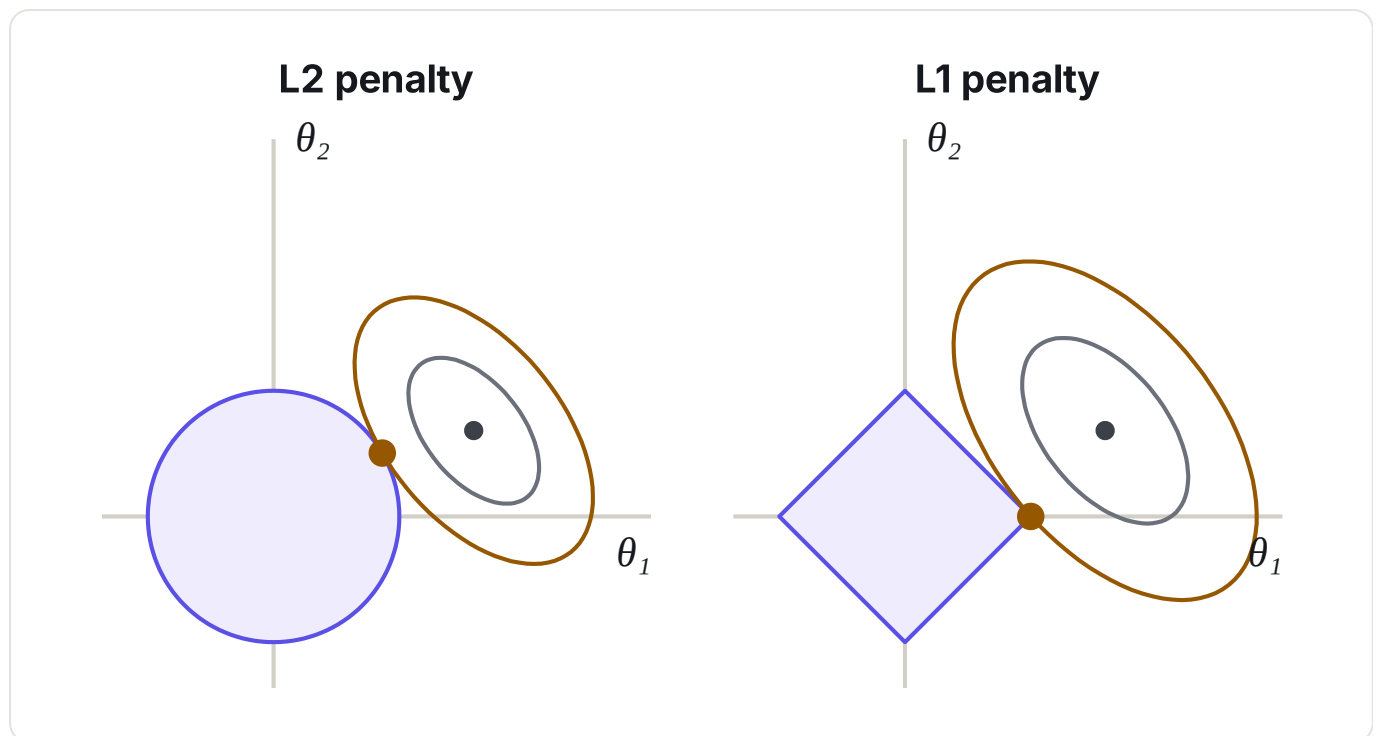


Figure 7.3 The same loss (ellipses; the small dot is its unconstrained best point) with two budgets of the same size. The amber ellipse is the first contour to touch each budget, and the amber dot is the solution. With L2 (left), both weights are nonzero. With L1 (right), the contact is at the corner, so $\theta_2 = 0$ exactly.

◆ THE SAME THING IN PYTORCH

>_ L2 VIA `WEIGHT_DECAY` , L1 ADDED TO THE LOSS BY HAND

```

import torch
import torch.nn as nn

torch.manual_seed(0)
model = nn.Linear(10, 1)
criterion = nn.MSELoss()
x = torch.randn(32, 10)
y = torch.randn(32, 1)

# L2 / ridge / "weight decay": built into the optimizer.
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, weight_decay=1e-4)
loss = criterion(model(x), y)
loss.backward()
optimizer.step()

# L1 / lasso: no optimizer flag, so add the penalty to the loss by hand.
lam = 1e-4
optimizer.zero_grad()
loss = criterion(model(x), y) + lam * model.weight.abs().sum()
loss.backward()
optimizer.step()

```

Two details connect this code to the math. First, PyTorch's `weight_decay` adds `weight_decay * w` to each gradient, so each step multiplies the weight by `(1 - lr * weight_decay)`. Our formula has $(1 - 2\alpha\lambda)$, so `weight_decay` = 2λ . Second, `weight_decay` here applies to *all* parameters, including the bias. The bias only shifts the output, so there is usually no reason to penalize it. To exclude it, pass the weights and biases as separate parameter groups with different `weight_decay` values. The L1 line above already penalizes only `model.weight`.

CHECK YOURSELF 4

You fit a linear model with L1 using scikit-learn's `Lasso`, and again with a hand-written PyTorch loop (penalty added to the loss, plain SGD). Lasso shows 12 weights at exactly 0.0. PyTorch shows none, only values like 0.0003. Is your PyTorch code wrong?

57 What to carry forward

- **Generalization** means doing well on unseen data. **Overfitting** fits the training data's noise. **Underfitting** is too simple for the real pattern. **Capacity** moves a model between them.
- **Train / validation / test** have separate jobs. Training learns the weights, validation guides decisions, and the test set is used once for a final number. **Cross-validation** with k folds rotates the held-out part so every example is validated once.
- Test error = **bias**² + **variance** + **irreducible noise**. More capacity usually lowers bias and raises variance. The goal is the capacity with the smallest sum.
- **L2 (ridge)** shrinks every weight by a fraction each step ("weight decay"). **L1 (lasso)** pushes by a constant amount. Its best solution often has weights at exactly zero, because its budget is a diamond with corners on the axes. You need soft-thresholding (or coordinate descent) to get those exact zeros.

Next, we look beyond a single algorithm. We tour the classical models (kNN, decision trees, ensembles, and SVMs) and see when each one is the right tool, in Section 8.

Answers to “Check yourself”

1. A model reaches 99% accuracy on its training set but only 61% on its validation set. Is this overfitting or underfitting? What is a good first fix?

This is **overfitting**. Near-perfect training performance next to much worse performance on unseen data is its typical sign. The model has enough capacity to memorize the training set instead of learning the general pattern. Good first fixes: reduce capacity (a smaller model, fewer tree splits), or add regularization (later in this section). You can also collect more training data, because a bigger set is harder to memorize. Or stop training earlier, before the model starts fitting noise.

2. While tuning a model, you check test-set accuracy after every change and keep the version with the best test score. You never train on the test set. What is wrong?

You did not train on the test set, but you **selected** based on it, and selecting is a form of fitting. After enough attempts, some version will score well on the test set by luck, because it happens to match that set’s particular noise. Picking “the best on the test set” then has the same effect as tuning on it. The reported number no longer estimates real-world performance. The fix is to do all tuning with the validation set (or cross-validation) and use the test set once, at the end.

3. You keep the model (degree 15) but collect ten times more training data from the same source. Which of bias², variance and noise changes, and in which direction?

Variance goes down. With more points, each random training set pins the curve down more firmly, so different training sets give more similar models. Bias² stays about the same, because it depends on what the model *can* represent, and that did not change. (Here it is already near zero.) Noise does not change at all: it belongs to the data, not the model. This is why more data is the classic cure for overfitting.

4. You fit a linear model with L1 using scikit-learn’s `Lasso`, and again with a hand-written PyTorch loop (penalty added to the loss, plain SGD). Lasso shows 12 weights at exactly 0.0. PyTorch shows none, only values like 0.0003. Is your PyTorch code wrong?

Probably not. Both minimize the same loss, and its minimum has those 12 weights at zero. The difference is the solver. Plain gradient descent with the $\lambda \text{sign}(\theta)$ push overshoots zero on every step, so those weights move back and forth near zero. Lasso’s coordinate descent uses soft-thresholding, which stops exactly at zero. To get exact zeros from PyTorch, add a soft-thresholding step after each update. Or set weights below a small threshold to zero at the end.

A Tour of Classical Models

Machine learning worked well before neural networks. Meet four ideas that are still the best choice for many everyday table-shaped datasets.

30 min read

YOU'LL LEARN

- k-nearest neighbors
- decision trees
- ensembles
- SVM intuition

Before you start: Section 7: Overfitting & Regularization.

This section is a fast tour, on purpose. For each model you get one clear intuition, one everyday parallel, and at most one or two small formulas. That is enough to know what each model does and when to use it. The rest of the course goes deep on neural networks. Still, these classical models remain the right choice for many problems with *tabular* data. Tabular data fits in a table: one row per example, one column per feature.

◆ THE ONE IDEA TO HOLD ONTO

Each model below makes a different assumption about how to turn “examples I have seen” into “a prediction for something new”. If you know the assumption, you know when the model will work well.

1 k-Nearest Neighbors: ask the closest examples

◆ INTUITION FIRST

k-Nearest Neighbors (kNN) has almost no training step. It stores every labeled example. To predict for a new point, it finds the k stored examples closest to it and takes a majority vote of their labels. “Closest” usually means ordinary distance: the L2 norm from Section 1.

☂ EVERYDAY PARALLEL

You want to know whether you will like a new film. You find the 5 people whose past film ratings are most similar to yours. If most of them liked the new film, you predict that you will like it too. That is kNN. Each person is a stored example, “similar ratings” is the distance, and the majority of the 5 is the prediction.

◆ NOW THE MATH

For a new point $\mathbf{x}$, compute its distance to each stored point $\mathbf{x}_i$:

$$d_i = \|\mathbf{x} - \mathbf{x}_i\|_2$$

Let $\mathcal{N}_k(\mathbf{x})$ be the set of the k stored points with the smallest d_i . The prediction is the most common label among them:

$$\hat{y} = \arg \max_c |\{i \in \mathcal{N}_k(\mathbf{x}) : y_i = c\}|$$

In words: for each class c , count how many of the k nearest neighbors have label c . Predict the class with the largest count. The vertical bars $|\cdot|$ mean “the number of elements in”.

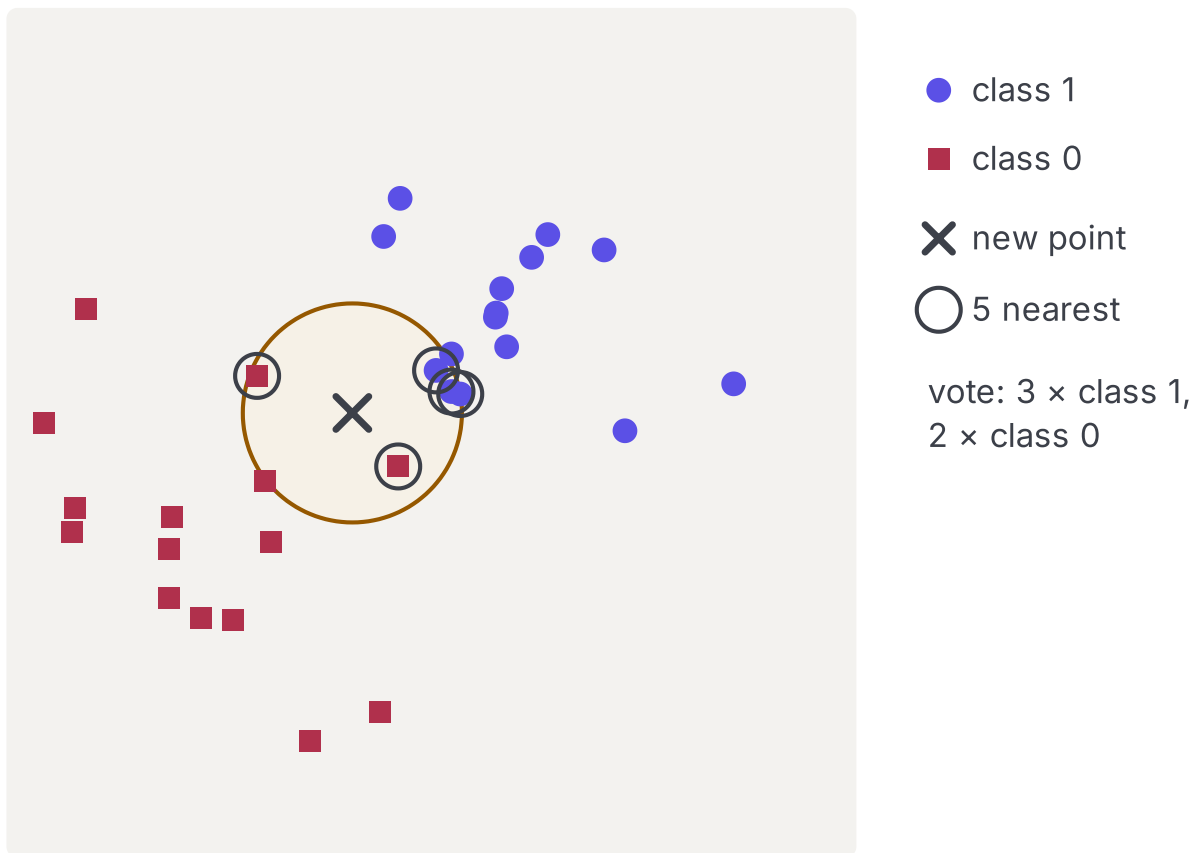


Figure 8.1 kNN with $k = 5$ on the data from the code below. The circle is centered on the new point and reaches its fifth-nearest neighbor. Inside are three class-1 points and two class-0 points, so the prediction is class 1.

▲ WATCH OUT: FEATURE SCALE AND THE CURSE OF DIMENSIONALITY

Distance adds up all features, so a feature measured in large units (income in dollars) outweighs one in small units (age in years). Normalize the features first (Section 5). Also, with very many features, almost all pairs of points are about the same distance apart. Then “closest” means little, and kNN gets worse as the number of dimensions grows.

◆ THE SAME THING IN NUMPY

>_ KNN PREDICTION FROM SCRATCH

```

import numpy as np

rng = np.random.default_rng(4)
X_train = np.vstack([rng.normal([2, 2], 1.0, size=(15, 2)),      # class 0
                    rng.normal([5, 4], 1.0, size=(15, 2))])    # class 1
y_train = np.array([0] * 15 + [1] * 15)

def knn_predict(X_train, y_train, x_new, k=5):
    dists = np.linalg.norm(X_train - x_new, axis=1)    # distance to every
    nearest = np.argsort(dists)[:k]                  # indices of the
    votes = np.bincount(y_train[nearest], minlength=2) # count the labels
    return votes.argmax(), votes

label, votes = knn_predict(X_train, y_train, np.array([3.2, 3.2]))
print(label, votes)    # 1 [2 3]

```

With scikit-learn, the same model is `KNeighborsClassifier(n_neighbors=5)`. Its `fit` method only stores the data (and may build a search structure, see the check below).

CHECK YOURSELF 1

The code above computes the distance to every stored point. What happens to prediction time when the training set grows 100 times larger? How can libraries reduce this?

2 Decision trees: a flowchart of yes/no questions

◆ INTUITION FIRST

A **decision tree** predicts by asking a series of simple yes/no questions about the features (“is age > 30?”, “is income > 50,000?”). It follows the answers down the branches until it reaches a leaf, which holds the prediction. Training a tree means choosing, at each branch, the question that best separates the classes.

☂ EVERYDAY PARALLEL

It is the game of **20 Questions**. “Is it alive?” “Is it larger than a chair?” Each answer removes many possibilities. A few good questions narrow a huge range of options down to one guess. A decision tree plays this game automatically. It chooses its questions from the training data.

◆ NOW THE MATH

How does a tree measure “best separates”? A common choice is *Gini impurity*. If a group of examples has a fraction p_c of class c , its impurity is:

$$G = 1 - \sum_c p_c^2$$

In words: G is 0 when the group is pure (one class only). For two classes it is largest, 0.5, when they are mixed half and half. A candidate question splits a group into two smaller groups. The tree picks the question whose two groups have the lowest impurity on average, weighted by group size. (Another common measure is the entropy from Section 4.)

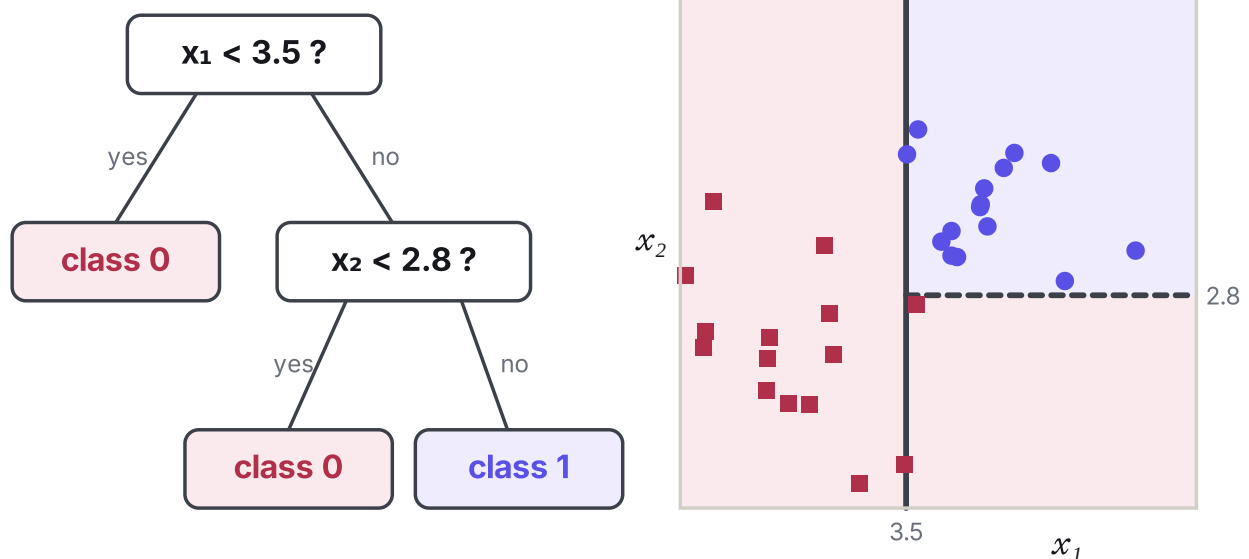


Figure 8.2 A depth-2 tree chosen by Gini impurity on the kNN data. The root question $x_1 < 3.5$ lowers the weighted impurity from 0.5 to about 0.06. Each question is a vertical or horizontal cut, so a tree always divides the plane into rectangles.

Trees are very **interpretable**: you can print the whole flowchart and read why it made a prediction. But on their own they **overfit easily**. With enough questions, a tree can make a tiny leaf for every training example. It then memorizes the data instead of learning the pattern: the failure from Section 7.

In practice you train these models with a library such as scikit-learn, not by hand. The remaining snippets use it and assume `X_train` and `y_train` hold your data.

>_ A DEPTH-LIMITED TREE IN SCIKIT-LEARN

```
from sklearn.tree import DecisionTreeClassifier

# X_train, y_train: your data, e.g. from the kNN example above
model = DecisionTreeClassifier(max_depth=4) # cap the depth: at most 4
model.fit(X_train, y_train)
```

CHECK YOURSELF 2

A group has 8 examples of class A and 2 of class B. What is its Gini impurity? Would splitting it into (4 A, 0 B) and (4 A, 2 B) help?

3 Ensembles: combine many weak models

◆ INTUITION FIRST

A single tree is unstable and overfits easily. **Ensembles** fix this by combining many models instead of trusting one. There are two main ways to do it.

Bagging (short for “bootstrap aggregating”) trains many trees. Each tree sees a *bootstrap sample* of the data: a random sample of the same size, drawn with replacement. Then it averages their votes. A **random forest** is bagging with one more trick: at each split, each tree may only look at a random subset of the features.

Boosting trains models one after another. Each new model is trained to correct the errors that the ensemble so far still makes. The models are added together, not averaged.

☂ EVERYDAY PARALLEL

Bagging is like asking many people to guess the number of beans in a jar and averaging the guesses. One guess can be far off, but averaging many guesses cancels part of the error. It cancels more when the people guess independently. If they all talked to each other first, they would make similar mistakes, and averaging would help less. **Boosting** is like a series of tutors. Each new tutor looks at the questions the student still gets wrong after all earlier tutoring, and works on those.

This is why random forests sample features. Trees trained on bootstrap samples of the same data are still similar, so their errors are *correlated*, and averaging cancels only part of them. Restricting each split to random features makes the trees more different from each other, so averaging cancels more.

>_ BAGGING AND BOOSTING IN SCIKIT-LEARN

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# X_train, y_train: your data
forest = RandomForestClassifier(n_estimators=200)           # bagging: 200 trees
forest.fit(X_train, y_train)

boosted = GradientBoostingClassifier(n_estimators=200)    # boosting: each tree
boosted.fit(X_train, y_train)
```

CHECK YOURSELF 3

You build a “forest” of 200 trees, but every tree is trained on the full dataset with all features and no randomness. How much better is it than one tree?

4 SVM intuition: the widest street between classes

◆ INTUITION FIRST

A **support vector machine (SVM)** for classification looks for the line that separates the two classes with the **widest possible margin**. In higher dimensions the line becomes a flat boundary called a *hyperplane*. The margin is the empty gap on either side of the line before you reach a training point.

☂ EVERYDAY PARALLEL

Imagine building the widest possible straight road between two neighborhoods, without the road touching any house. Many roads would separate the neighborhoods. The widest one leaves the most space if a new house is built near the edge later. An SVM finds that widest road. The houses touching its edges are the *support vectors*. They are the only houses that decide where the road goes.

◆ NOW THE MATH

Write the boundary as $\mathbf{w}^\top \mathbf{x} + b = 0$. The distance from a point $\mathbf{x}$ to this boundary is $|\mathbf{w}^\top \mathbf{x} + b| / \|\mathbf{w}\|$. We can multiply $\mathbf{w}$ and b by any positive number without moving the boundary. So choose the scale where the closest points on each side have $\mathbf{w}^\top \mathbf{x} + b = \pm 1$. Then each edge of the street is at distance $1 / \|\mathbf{w}\|$ from the boundary, and:

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

In words: a wide street means a small $\|\mathbf{w}\|$. So the SVM minimizes $\|\mathbf{w}\|^2$. At the same time, it requires every training point to be on the correct side, with $|\mathbf{w}^\top \mathbf{x} + b| \geq 1$.

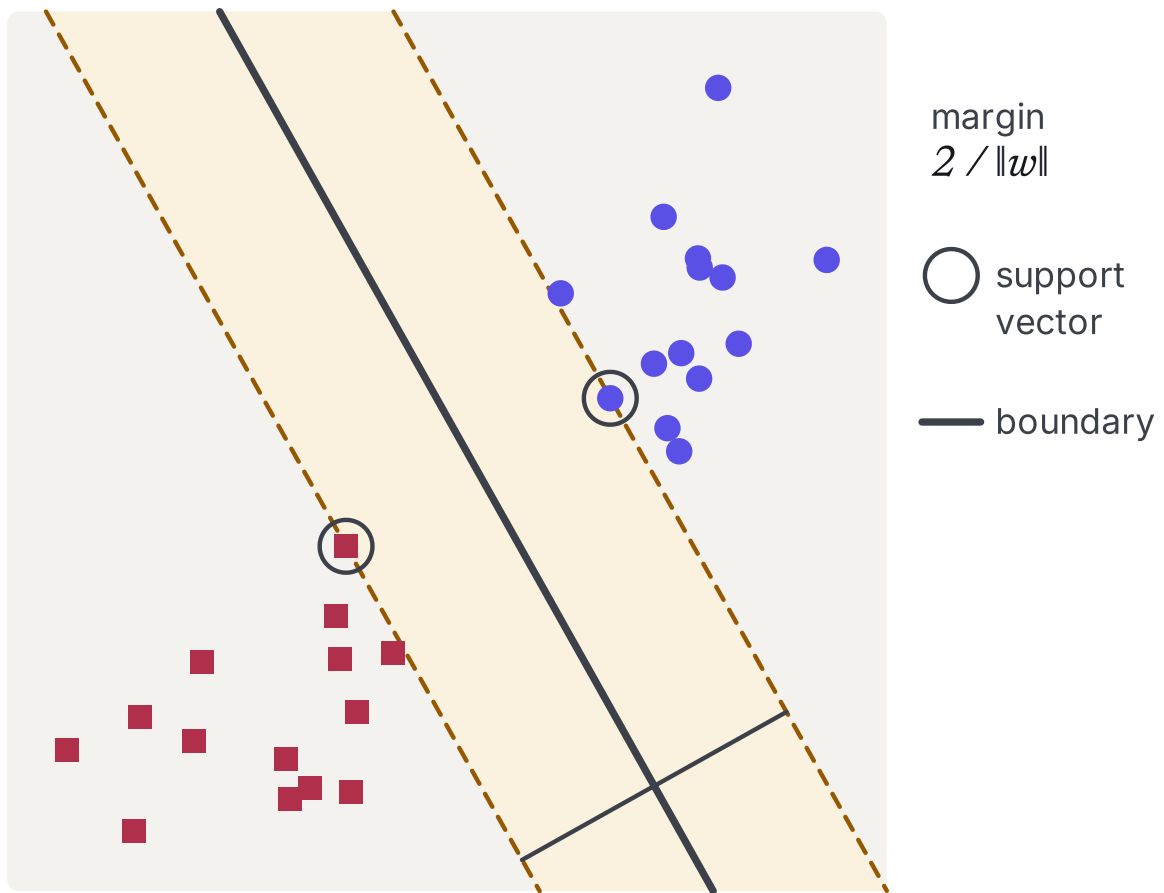


Figure 8.3 The widest street between two separable classes, found by searching over all directions. The solid line is the boundary. The dashed lines are the street edges. Only the two circled points touch the edges. They are the support vectors: moving any other point (without entering the street) does not change the answer.

Real data is rarely separable by a straight line. The **kernel trick** helps. First, imagine lifting the data into a space with more dimensions, for example by adding products of features as new features. In that space, a straight boundary may separate classes that were mixed in the original space. Seen back in the original space, that boundary is curved.

The trick is that the SVM only needs dot products between points, never the points themselves. For some lifts, you can compute the dot product in the big space directly from the original points. Here is a small example in 2-D. Define the lift $\phi(\mathbf{x}) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$. Then:

$$\begin{aligned}\phi(\mathbf{x}) \cdot \phi(\mathbf{z}) &= x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 \\ &= (x_1 z_1 + x_2 z_2)^2 = (\mathbf{x} \cdot \mathbf{z})^2\end{aligned}$$

In words: the dot product in the 3-D space equals the square of the ordinary 2-D dot product. For $\mathbf{x} = (1, 2)$ and $\mathbf{z} = (3, 1)$: $\mathbf{x} \cdot \mathbf{z} = 5$, so both sides equal 25. We never had to build the 3-D points. The “rbf” kernel below does the same for a lift with infinitely many dimensions.


```
from sklearn.svm import SVC
```

```
# X_train, y_train: your data (normalize the features first)
```

```
model = SVC(kernel="rbf")      # "rbf" kernel: a curved boundary through t
```

```
model.fit(X_train, y_train)
```

CHECK YOURSELF 4

After training an SVM, you delete one training point that is far from the street. Does the boundary change? What if you delete a support vector?

5 When to use what

Model	Strength	Weakness	When to use it
k-Nearest Neighbors	Simple, no training step, can follow any boundary shape	Slow predictions on large data; needs feature scaling; weak in high dimensions	Quick baseline on small, low-dimensional datasets
Decision tree	Interpretable; no feature scaling needed; handles numeric and categorical features (some libraries, such as scikit-learn, need categories encoded as numbers first)	Overfits easily; unstable (small data changes → different tree)	When you (or a regulator) need to read the exact rule
Random forest (bagging)	Much more stable and accurate than a single tree; resists overfitting	Less interpretable; slower and larger than one tree	Strong, low-effort default for tabular data
Gradient boosting	Often the most accurate classical model on tabular data	More hyperparameters to tune; trains sequentially (slower to fit); can overfit if trained too long	Getting the best possible accuracy on structured data
SVM	Strong with a clear margin; kernel trick handles curved boundaries; works with few samples relative to features	Training does not scale well to huge datasets; needs feature scaling	Small-to-medium datasets, especially with a clear margin (possibly after a kernel)

CHECK YOURSELF 5

You have 50,000 rows of customer data with 30 mixed numeric features, and you want good accuracy with little tuning. Which model from the table do you try first, and why not kNN?

6 What to carry forward

- **kNN** predicts by majority vote among the closest stored examples. It has no real training step and needs scaled features.
- **Decision trees** are readable yes/no flowcharts, chosen with a measure such as Gini impurity. On their own they overfit easily.
- **Ensembles** fix that. Bagging and random forests average many trees, and randomness keeps the trees different so their errors partly cancel. Boosting adds models that each correct the ensemble's current errors.
- **SVMs** find the boundary with the widest margin, $2/\|\mathbf{w}\|$. The kernel trick makes that boundary curved by working with dot products in a lifted space.
- A few intuitions (closeness, splitting, voting, margins) take you a long way in practice.

Next we leave the classical toolkit and build the **neuron**, the unit that every deep network is made of, in Section 9.

Answers to “Check yourself”

1. The code above computes the distance to every stored point. What happens to prediction time when the training set grows 100 times larger? How can libraries reduce this?

With this brute-force search, each prediction becomes about 100 times slower, because kNN does its work at prediction time, not at training time. Libraries reduce this with search structures such as KD-trees and ball trees. They split space into regions, so the search can skip regions that are far away. This works well with few features. With many features (the curse of dimensionality again), the structures help less, and approximate nearest-neighbor methods are used instead.

2. A group has 8 examples of class A and 2 of class B. What is its Gini impurity? Would splitting it into (4 A, 0 B) and (4 A, 2 B) help?

$G = 1 - 0.8^2 - 0.2^2 = 0.32$. After the split, the first group is pure ($G = 0$). The second has $G = 1 - (4/6)^2 - (2/6)^2 \approx 0.444$. The weighted average is $\frac{4}{10} \cdot 0 + \frac{6}{10} \cdot 0.444 \approx 0.267$. That is lower than 0.32, so the split helps a little. A split that put both B examples alone in one group would reach 0.

3. You build a “forest” of 200 trees, but every tree is trained on the full dataset with all features and no randomness. How much better is it than one tree?

Not better at all. With the same data, the same features, and no randomness, the tree algorithm builds the same tree 200 times. All their errors are identical, so averaging cancels nothing. Averaging helps only when the models make *different* errors. That is why random forests add two sources of randomness: bootstrap samples and random feature subsets. (In the terms of Section 7: bagging reduces variance, and only when the trees vary.)

4. After training an SVM, you delete one training point that is far from the street. Does the boundary change? What if you delete a support vector?

Deleting a far point changes nothing. It never limited the width of the street, so the widest street stays the same. Deleting a support vector usually does change the boundary. That point was holding an edge of the street in place. Without it, the street can become wider and may shift or turn. This is why a trained SVM only needs to keep its support vectors. It is also why its prediction cost grows with the number of support vectors, not with the size of the whole training set.

5. You have 50,000 rows of customer data with 30 mixed numeric features, and you want good accuracy with little tuning. Which model from the table do you try first, and why not kNN?

A random forest is a good first try. It is a strong default for tabular data, needs no feature scaling, and has few settings that matter. Gradient boosting may then add some accuracy with more tuning. kNN would need careful feature scaling. With 30 features and 50,000 rows, its predictions are also slow and its distances less meaningful.

PART 3

Neural Nets from Scratch

Neurons, Layers, Activations

A neuron is a weighted vote passed through a switch. Without that switch, extra layers add nothing.

42 min read

YOU'LL LEARN

- the artificial neuron
- layers as matrices
- activation functions
- why nonlinearity is essential

Before you start: Section 1: Linear Algebra for ML and Section 6: Logistic Regression.

Everything you have built so far (vectors, matrices, dot products) prepares you for one small object: the **neuron**. A neuron takes a few numbers, weighs them, adds them up, and passes the result through a small switch. Connect a few thousand of these and you have a neural network. In this section we build the neuron and stack neurons into layers. Then we prove a key fact about them: the switch is necessary. Remove it, and a hundred layers collapse into one.

◆ THE ONE IDEA TO HOLD ONTO

One neuron computes $\phi(\mathbf{w}^\top \mathbf{x} + b)$: a weighted sum passed through a nonlinear switch ϕ . A layer runs many neurons at once, and that is one matrix multiply. The switch (the **activation**) is the only thing that makes extra layers add power. The rest of this section explains why.

1 The artificial neuron: a weighted vote through a switch

◆ INTUITION FIRST

A **neuron** makes one small decision. It looks at several input numbers and decides how much each one matters. It adds up this weighted evidence. Then it passes the total through a switch that decides how strongly to “fire.” A big total gives a strong output. A small or negative total gives a weak or zero output. No single neuron is clever. The power of a network comes from connecting many simple neurons.

🔑 EVERYDAY PARALLEL

Think about how you decide to go for a walk. The weather matters a lot to you (a big weight). Your plans for the day matter somewhat (a medium weight). Your energy level matters a little (a small weight). You add up the weighted evidence. The more the total favors a walk, the more likely you are to go. A neuron does the same thing: it weighs the factors, sums them, and turns the total into an output. The activation decides *how* the total becomes a decision: gradually (sigmoid) or with a hard floor at zero (ReLU).

◆ NOW THE MATH

Write the inputs as a vector $\mathbf{x} \in \mathbb{R}^n$ (a list of n numbers). Give the neuron a matching list of **weights** $\mathbf{w} \in \mathbb{R}^n$ and one extra number b , the **bias**. Call the activation function ϕ (the Greek letter phi). The neuron computes:

$$y = \phi(\mathbf{w}^\top \mathbf{x} + b) = \phi\left(\sum_{i=1}^n w_i x_i + b\right)$$

In words: the piece $\mathbf{w}^\top \mathbf{x}$ is the *dot product* from Section 1. Multiply each input by its weight and add the results. This gives one number (“how much do I care, in total?”). Add the bias b to shift that number up or down. Finally, pass the result through the activation ϕ to get the neuron's output y .

DEFINITION

A **weight** w_i says how much input x_i matters, and in which direction. A large positive weight means “this input pushes the output up.” A negative weight means “this input pushes it down.” The **bias** b is one constant, added once to the weighted sum. It sets the neuron's default lean: how big the weighted sum must be before the neuron starts to fire. The **activation** ϕ is a fixed nonlinear function applied to that sum.

The number *before* the activation, $z = \mathbf{w}^\top \mathbf{x} + b$, appears so often that it has a name: the **pre-activation**. (Some texts call it the *logit* when it is the final score before a probability.) Keep the two apart: z is the weighted vote, and $y = \phi(z)$ is the vote after the switch.

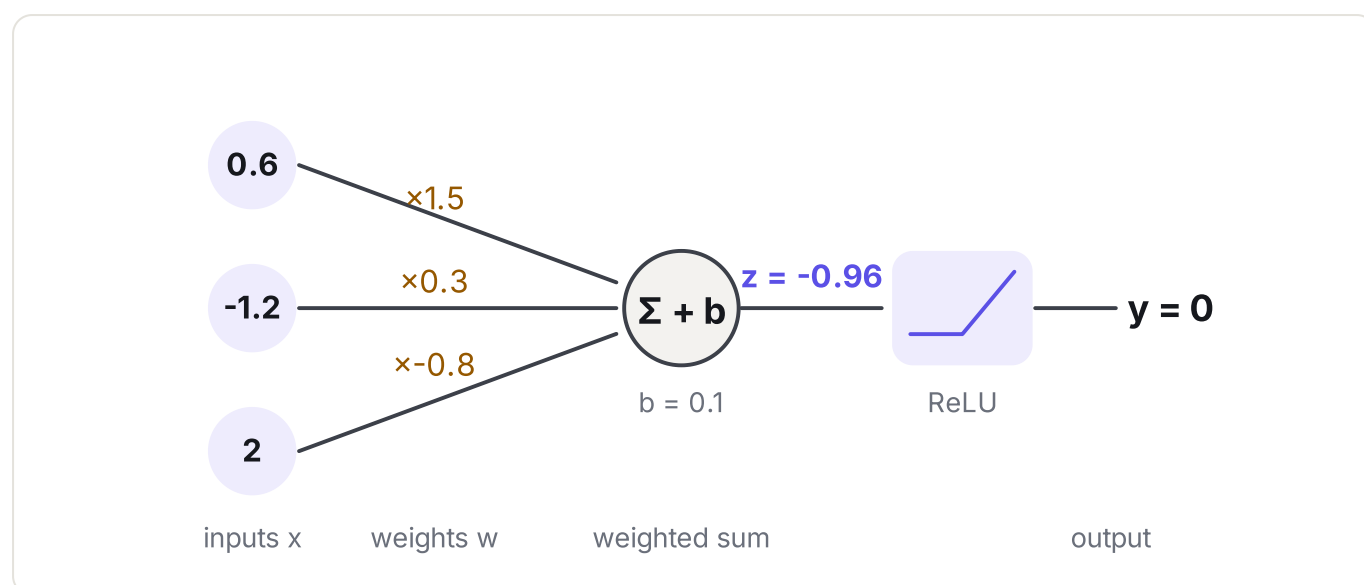


Figure 9.1 The neuron from the code below. Each input is multiplied by its weight. The products are summed and the bias is added, which gives $z = -0.96$. ReLU turns the negative z into 0, so the neuron stays silent.

◆ THE SAME THING IN PYTORCH

>_ ONE NEURON BY HAND


```
import torch

x = torch.tensor([0.6, -1.2, 2.0])    # three inputs
w = torch.tensor([1.5, 0.3, -0.8])    # a weight per input
b = torch.tensor(0.1)                # one bias

z = torch.dot(w, x) + b              # pre-activation: the weighted vote
y = torch.relu(z)                    # pass it through the switch (ReLU)

print(z)    # tensor(-0.9600)    0.6*1.5 + (-1.2)*0.3 + 2.0*(-0.8) + 0.1
print(y)    # tensor(0.)         negative vote → neuron stays silent
```

CHECK YOURSELF 1

In the code above, which single number would you change, and to what, so that the neuron outputs exactly 0.5?

2 Layers: many neurons at once is one matrix multiply

◆ INTUITION FIRST

One neuron gives you one number. Usually you want many. One neuron might ask “does this look like a wheel?”, another “does this look like a window?”, another “is there fur?”. A **layer** is a row of neurons. They all look at the same input at the same time. Each has its own weights and bias, and each produces its own number. Their outputs stack into a new vector, which becomes the input to the next layer.

☂ EVERYDAY PARALLEL

A panel of reviewers reads a loan application. One reviewer cares mostly about income, another about credit history, another about existing debt. They all read the *same* application, but they weigh it differently, and each writes down a score. The list of scores tells you more than any single opinion. A layer is that panel: the same input for everyone, different weights for each member, and a vector of scores out.

◆ NOW THE MATH

Stacking neurons needs no new tools. It is the matrix–vector multiply from Section 1. Put each neuron's weight vector as a *row* of a matrix $\mathbf{W}$. Collect the biases into a vector $\mathbf{b}$. Then one line computes the whole layer:

$$\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$$

In words: row i of $\mathbf{W}$ holds the weights of neuron i . So $\mathbf{W}\mathbf{x}$ computes every neuron's weighted vote in one multiply (one dot product per row). Add the bias vector and apply ϕ to every entry. The result is $\mathbf{h}$, the layer's output vector, with one number per neuron.

The shapes tell the whole story, so let's state them. Suppose the input has n features and the layer has m neurons:

$$\underbrace{\mathbf{W}}_{m \times n} \underbrace{\mathbf{x}}_{n \times 1} + \underbrace{\mathbf{b}}_{m \times 1} = \underbrace{\mathbf{z}}_{m \times 1}$$

In words: $\mathbf{W}$ has one row per neuron (m rows) and one column per input feature (n columns). By the shape rule (inner numbers match, outer numbers stay), an $(m \times n)$ matrix times an $(n \times 1)$ vector gives an $(m \times 1)$ vector. So a layer maps n numbers in to m numbers out. You choose m when you design the network.

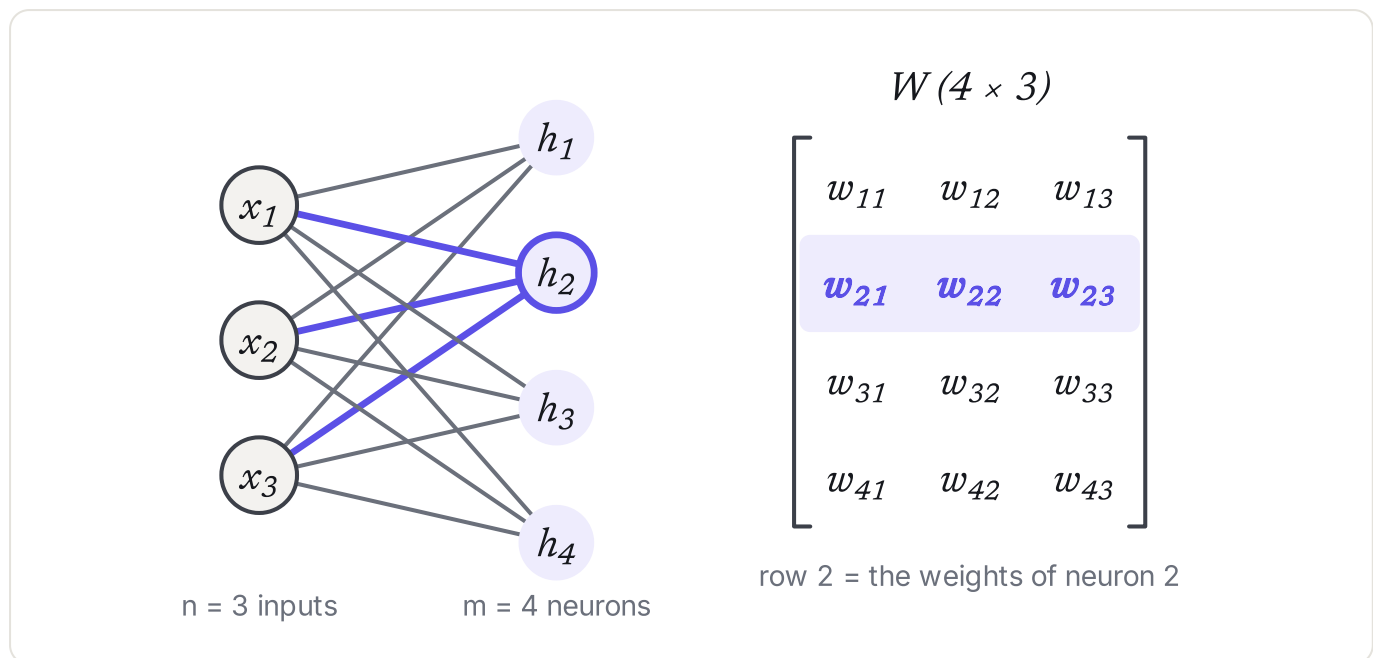


Figure 9.2 A layer with 3 inputs and 4 neurons. Every input connects to every neuron (a “fully connected” layer). The three highlighted edges into neuron 2 carry the three weights in row 2 of $\mathbf{W}$. One edge is one matrix entry.

In practice you rarely send one example at a time. You send a **batch** (a group of examples processed together for speed). Stack N input vectors as the rows of a matrix $\mathbf{X}$ of shape $(N \times n)$. The layer becomes:

$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W}^\top + \mathbf{b})$$

In words: each row of $\mathbf{X}$ is one example. Multiplying by $\mathbf{W}^\top$ gives an $(N \times m)$ matrix, one row of answers per example. The bias $\mathbf{b}$ has shape $(m,)$, so PyTorch adds it to every row. This automatic copying is called *broadcasting*; Section 13 explains it in detail.

This row-of-examples convention is why PyTorch's `nn.Linear` stores its weight with shape $(m \times n)$ and multiplies by $\mathbf{W}^\top$ inside its forward pass.

CHECK YOURSELF 2

A layer takes a 784-number image vector and has 128 neurons. What are the shapes of $\mathbf{W}$, $\mathbf{b}$ and $\mathbf{h}$? How many parameters does the layer have?

3 Activation functions: the shape of the switch

◆ INTUITION FIRST

The activation is the switch that each neuron's weighted vote passes through. Different switches behave differently. Some are soft and gradual. Some block negative values completely. Some let large values through unchanged. The choice decides two things. First, the range of values the output can take. Second, and important for learning, how the **gradient** behaves. (The gradient is the slope that tells training which way to change the weights. Section 10 computes it.) Where a switch's slope is flat, learning cannot pass through it.

☂ EVERYDAY PARALLEL

Think of a light dimmer with a knob. Near the middle, a small turn changes the brightness a lot. Near the two ends the knob still turns, but the light hardly changes. That is sigmoid and tanh: at both ends, turning the input changes the output very little. In math words, the slope is nearly zero. ReLU is a different kind of dimmer. Below zero the light is off, and turning the knob does nothing (slope 0). Above zero, each turn adds the same amount of light (slope 1). How much the light responds to a turn of the knob is the gradient.

◆ NOW THE MATH

The **sigmoid** σ squashes any number into the range $(0, 1)$. This is useful when you want an output you can read as a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{range } (0, 1)$$

In words: a large positive z gives an output near 1. A large negative z gives an output near 0. And $z = 0$ gives exactly 0.5. The problem: at both ends the curve becomes flat, so its slope (the gradient) shrinks to nearly zero. This is called **saturation**, and it slows learning.

The **tanh** is a rescaled sigmoid, centered on zero, with range $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \text{range } (-1, 1)$$

In words: the same S-shape as sigmoid, but symmetric around zero. A negative input gives a negative output, so the outputs are centered. This makes tanh a bit easier to train than sigmoid. But it still saturates at both ends.

The **ReLU** (“rectified linear unit”) is very simple. It keeps the input if it is positive, and outputs zero otherwise.

$$\text{ReLU}(z) = \max(0, z) \quad \text{range } [0, \infty)$$

In words: for any positive z the output is z and the slope is 1. The gradient passes through at full strength, so learning does not stall on the positive side. For negative z the output is 0 and the slope is 0 (the neuron is “off”). ReLU needs no costly e^z and does not saturate for positive inputs. That is why it became the default activation for hidden layers.

ReLU has one weak point. A neuron whose z is negative for every input has zero gradient everywhere, so it may never recover. This is a **dead neuron**. **Leaky ReLU** fixes this: on the negative side it lets a small fraction through instead of a hard zero.

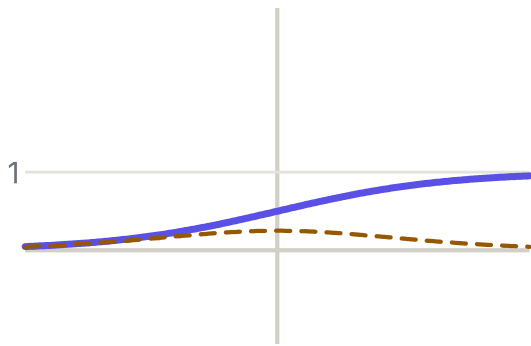
$$\text{LeakyReLU}(z) = \max(az, z), \quad a \approx 0.01 \quad \text{range } (-\infty, \infty)$$

In words: positive inputs behave like ReLU (slope 1). Negative inputs get a small slope a (for example 0.01) instead of 0. So the neuron always keeps a small gradient and can recover. The number a is the “leak.”

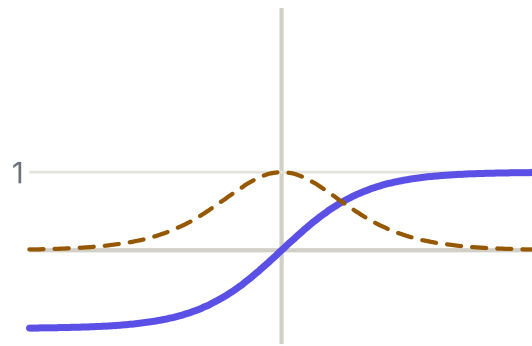
Finally, a name you should know: **GELU** (“Gaussian Error Linear Unit”). It is a smooth version of ReLU that bends gently through zero instead of having a sharp corner. It is the default activation inside transformer models (the architecture behind today's large language models). You do not need its formula yet. Just recognize the name when you see it later.

Activation	Formula	Output range	Gradient behavior
Sigmoid	$\frac{1}{1 + e^{-z}}$	$(0, 1)$	Saturates at both ends → tiny gradients, slow learning
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Zero-centered, but still saturates at both ends
ReLU	$\max(0, z)$	$[0, \infty)$	Slope 1 for $z > 0$, 0 for $z < 0$; cheap, no positive saturation
Leaky ReLU	$\max(az, z)$	$(-\infty, \infty)$	Like ReLU, but small slope a on the negative side → no dead neurons
GELU	smooth ReLU-like curve	$\approx [-0.17, \infty)$	Smooth everywhere; default in transformers

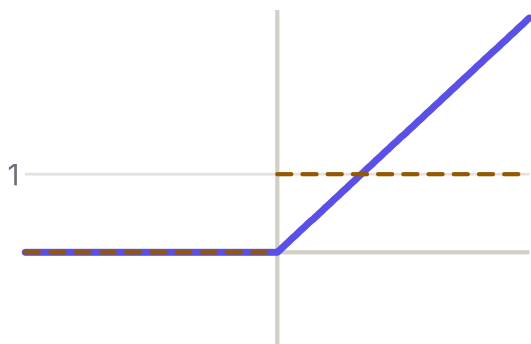
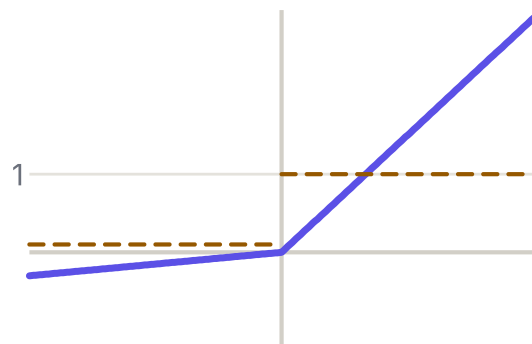
sigmoid



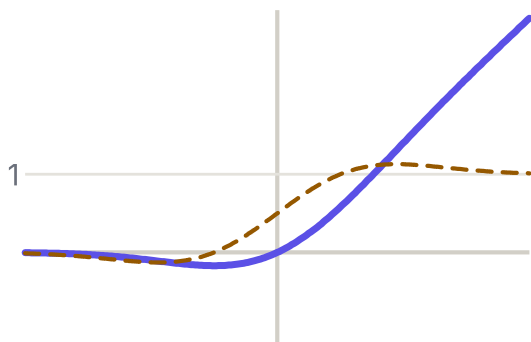
tanh



ReLU

leaky ReLU ($a = 0.1$)

GELU



Each panel: z from -3 to 3 .

— the function $\phi(z)$

- - - its slope $\phi'(z)$

Where the dashed line sits at 0, no gradient flows.

Figure 9.3 Each activation (solid) with its slope (dashed). Sigmoid and tanh have slopes that fall toward 0 at both ends: this is saturation. ReLU's slope is 1 for positive z and 0 for negative z . Leaky ReLU is drawn with $a = 0.1$ so you can see the leak; with the usual $a = 0.01$ it looks almost like ReLU.

◆ THE SAME THING IN PYTORCH

>_ EVERY SWITCH ON THE SAME INPUTS

```
import torch

z = torch.tensor([-2.0, -0.5, 0.0, 0.5, 2.0])

print(torch.sigmoid(z))           # tensor([0.1192, 0.3775, 0.5000, 0.6225, 0.8808])
print(torch.tanh(z))              # tensor([-0.9640, -0.4621,  0.0000,  0.4621,  0.9640])
print(torch.relu(z))              # tensor([0.0000, 0.0000, 0.0000, 0.5000, 2.0000])
print(torch.nn.functional.leaky_relu(z)) # tensor([-0.0200, -0.0050,  0.0000,  0.5000, 2.0000])
```

▲ WATCH OUT

Do not use sigmoid or tanh in deep hidden layers by default. They are flat at the ends. When you stack many of them, the gradients shrink layer after layer, and the early layers barely learn. This is the **vanishing gradient** problem (Section 12). Sigmoid still has a proper place: the *final* layer of a binary classifier, where you want a probability between 0 and 1. For the hidden layers, start with ReLU.

CHECK YOURSELF 3

A ReLU neuron has pre-activation $z = -3$. What is its output, and what gradient flows back through it? What would change with Leaky ReLU ($a = 0.01$)?

CHECK YOURSELF 4

Why can't we use a hard step function (output 1 if $z > 0$, else 0) as the activation and train it with gradients?

4 Why nonlinearity is necessary

◆ INTUITION FIRST

Here is the claim, and we will prove it. It is the reason activation functions exist. **if you stack layers with no activation between them, the whole stack equals a single layer.** A hundred plain matrix multiplies in a row have no more power than one. The switch is the only thing that makes depth useful.

☂ EVERYDAY PARALLEL

Place straight rulers end to end in one line, as many as you like. The chain is still one straight line. You never get a curve, no matter how many rulers you add. A linear layer is a straight ruler. To draw a curve you need a bend somewhere, and the activation is the only place where a bend can enter.

◆ NOW THE MATH

Take two layers with **no** activation. The first maps $\mathbf{x}$ to $\mathbf{h}$, and the second maps $\mathbf{h}$ to $\mathbf{y}$:

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{y} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$$

Now substitute the first equation into the second and simplify, one step at a time:

$$\begin{aligned} \mathbf{y} &= \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2 \\ &= \mathbf{W}_2 (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 && \text{(plug in } \mathbf{h} \text{)} \\ &= \mathbf{W}_2 \mathbf{W}_1 \mathbf{x} + \mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2 && \text{(multiply out)} \\ &= \underbrace{(\mathbf{W}_2 \mathbf{W}_1)}_{\mathbf{W}'} \mathbf{x} + \underbrace{(\mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2)}_{\mathbf{b}'} && \text{(group the constants)} \\ &= \mathbf{W}' \mathbf{x} + \mathbf{b}' \end{aligned}$$

In words: the two weight matrices multiply into one new matrix $\mathbf{W}' = \mathbf{W}_2 \mathbf{W}_1$. The two biases combine into one new bias $\mathbf{b}' = \mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2$. The result, $\mathbf{y} = \mathbf{W}' \mathbf{x} + \mathbf{b}'$, is a *single* linear layer. The two-layer network and this one-layer network compute the same function.

◆ THE PUNCHLINE

Nothing in that derivation depends on the number two. Chain 100 layers without activations, and the same collapse happens 99 times. The product $\mathbf{W}_{100} \cdots \mathbf{W}_1$ is some single matrix $\mathbf{W}'$, and all the biases combine into one $\mathbf{b}'$. So a 100-layer linear network is **exactly** as expressive as one layer. It can only draw straight lines (in more dimensions, flat planes). The extra parameters add nothing. The activation breaks the collapse. The expression $\phi(\mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}))$ cannot be rewritten as one matrix times $\mathbf{x}$, so each real layer adds new power.

So what *can* a network do once the nonlinearity is back? A lot. The **universal approximation theorem** says the following. Take a network with one hidden layer, a nonlinear activation (such as ReLU or sigmoid), and enough neurons. It can approximate any continuous function on a bounded range of inputs, as closely as you want. “Continuous” means the function has no sudden jumps; it does not need to be smooth. “Bounded range” means a limited region, for example every input between -10 and 10. We will not prove this here. The idea:

each neuron contributes one bend, and enough bends, placed well, can follow any curve. The theorem says nothing about how many neurons you need, or whether training will find the right weights. And without the activation, the theorem does not apply at all.

◆ THE SAME THING IN PYTORCH

>_ WATCH TWO LINEAR LAYERS COLLAPSE INTO ONE

```
import torch
torch.manual_seed(0)

W1 = torch.randn(4, 3); b1 = torch.randn(4)
W2 = torch.randn(2, 4); b2 = torch.randn(2)
x = torch.randn(3)

# two "layers" with NO activation between them
y_two = W2 @ (W1 @ x + b1) + b2

# the single equivalent layer, built from the proof
W_eq = W2 @ W1 # (2x4)(4x3) → (2x3)
b_eq = W2 @ b1 + b2 # combined bias
y_one = W_eq @ x + b_eq

print(torch.allclose(y_two, y_one)) # True → same function, one layer
```

CHECK YOURSELF 5

A colleague builds a 10-layer network but forgets the activation functions. The loss drops a little, then stops improving, and the model does no better than linear regression. Why? And what happens if you add ReLU only after layer 5?

5 A real layer in PyTorch

◆ INTUITION FIRST

You now have every piece: weights, bias, matrix multiply, activation. PyTorch combines the weighted sum and the bias in `nn.Linear`. It keeps the activation separate, so you combine the two yourself. Let's send a batch

through one hidden layer with ReLU and follow the shapes. If the shapes are right, the wiring is right.

☂ EVERYDAY PARALLEL

Think of a two-station assembly line. Station one (`Linear`) takes each part and combines its measurements into a rough result. Station two (`ReLU`) is a quality check: it sets every value below zero to zero and passes the rest unchanged. The same two stations process every item on the belt (every example in the batch).

◆ THE SAME THING IN PYTORCH

>_ A HIDDEN LAYER'S FORWARD PASS, WITH SHAPES

```
import torch
import torch.nn as nn

batch_size, in_features, out_features = 32, 784, 128

layer = nn.Linear(in_features, out_features)  # holds W (128x784) and b
relu = nn.ReLU()

x = torch.randn(batch_size, in_features)      # a batch: 32 examples of
print("input :", x.shape)                     # torch.Size([32, 784])

z = layer(x)                                  # weighted sum + bias: x (
print("linear:", z.shape)                     # torch.Size([32, 128])

h = relu(z)                                   # apply the switch to ever
print("relu :", h.shape)                     # torch.Size([32, 128])

# check that the activation removed the negatives:
print("min after relu:", h.min().item())      # 0.0 → nothing negative
```

In words: the batch goes in with shape (32, 784). `Linear` turns each 784-number example into 128 numbers, which gives (32, 128). `ReLU` keeps that shape and sets every negative entry to zero. Each row is one example. The batch dimension (32) passes through unchanged.

▲ WATCH OUT

The `in_features` of `nn.Linear` must equal the last dimension of its input. The next layer's `in_features` must equal this layer's `out_features`. If they do not match, PyTorch raises a shape error when you call the layer. When a network does not run, print the shape after every layer. The error is where two numbers that should be equal are different.

CHECK YOURSELF 6

You add a second layer `nn.Linear(784, 10)` after the ReLU above and call it on `h`. What happens, and what is the fix?

6 What to carry forward

- A **neuron** computes $\phi(\mathbf{w}^\top \mathbf{x} + b)$: a weighted vote (weights say how much each input matters) plus a bias (the default lean), passed through a nonlinear switch ϕ .
- A **layer** runs many neurons at once with one matrix multiply: $\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$. It maps n inputs to m outputs, and $\mathbf{W}$ has shape (m, n) .
- **Activations** differ in output range and gradient behavior. **ReLU** ($\max(0, z)$) is the default for hidden layers: cheap, and it does not saturate for positive inputs. Sigmoid and tanh saturate. Leaky ReLU avoids dead neurons. GELU is the transformer default.
- **The proof:** stacked linear layers with no activation collapse to one layer, $\mathbf{W}'\mathbf{x} + \mathbf{b}'$. Nonlinearity makes depth useful. With it, a network with one hidden layer can approximate any continuous function on a bounded input range.

We now have a network that can compute a forward pass. But its weights are random, so its answers are nonsense. Next we learn how to *correct* those weights: how the gradient flows back through every neuron and every activation to train the whole stack at once. That is backpropagation, and it is where the network starts to learn.

Answers to “Check yourself”

1. In the code above, which single number would you change, and to what, so that the neuron outputs exactly 0.5?

The easiest choice is the bias. The weighted sum without bias is $0.9 - 0.36 - 1.6 = -1.06$. For ReLU to output 0.5 you need $z = 0.5$, so $b = 0.5 + 1.06 = 1.56$. This shows the job of the bias: it moves the whole weighted sum up or down, without looking at the inputs. (Changing w_3 from -0.8 to -0.07 also works.)

2. A layer takes a 784-number image vector and has 128 neurons. What are the shapes of $\mathbf{W}$, $\mathbf{b}$ and $\mathbf{h}$? How many parameters does the layer have?

The layer maps $n = 784$ inputs to $m = 128$ neurons. So $\mathbf{W}$ is **(128, 784)**: one row per neuron, one column per input feature. The bias $\mathbf{b}$ is **(128,)**, one per neuron. The output $\mathbf{h}$ is **(128,)**. The parameter count is $128 \times 784 + 128 = 100,480$. Almost all of them are weights. This is the size of the first hidden layer of a small image classifier.

3. A ReLU neuron has pre-activation $z = -3$. What is its output, and what gradient flows back through it? What would change with Leaky ReLU ($a = 0.01$)?

With ReLU, the output is $\max(0, -3) = 0$ and the slope is **0**. So no gradient flows back on this example, and the neuron's weights learn nothing from it. If z is negative for every example, the neuron is dead. With Leaky ReLU, the output is $0.01 \times (-3) = -0.03$ and the slope is 0.01. The gradient is small but not zero, so the weights can still change and the neuron can recover.

4. Why can't we use a hard step function (output 1 if $z > 0$, else 0) as the activation and train it with gradients?

The step function is flat everywhere except at $z = 0$, where it jumps. So its slope is 0 at every point where it is defined. Gradient descent multiplies by this slope, so every weight gradient would be zero, and nothing would ever change. Sigmoid is a smooth version of the step: it has the same “off, then on” shape, but its slope is never exactly zero.

5. A colleague builds a 10-layer network but forgets the activation functions. The loss drops a little, then stops improving, and the model does no better than linear regression. Why? And what happens if you add ReLU only after layer 5?

Without activations, the ten layers collapse into one linear map $\mathbf{W}'\mathbf{x} + \mathbf{b}'$ (see the proof above). The model is a linear model, so it cannot beat one. It stops where a straight-line fit stops. With ReLU after layer 5 only, layers 1–5 collapse into one linear layer, and layers 6–10 collapse into another. So you get a network with one hidden layer: nonlinear, but still much less than ten real layers. The fix is an activation after every hidden layer.

6. You add a second layer `nn.Linear(784, 10)` after the ReLU above and call it on `h`. What happens, and what is the fix?

PyTorch raises a shape error. The tensor `h` has shape (32, 128), so its last dimension is 128, but the new layer expects 784 input features. The fix is `nn.Linear(128, 10)`: its `in_features` must equal the previous layer's `out_features`. The output then has shape (32, 10).

Backpropagation by Hand

Backprop is the chain rule with careful bookkeeping. We compute every gradient in a real network by hand, with real numbers, and show every step.

56 min read

YOU'LL LEARN

- the computation graph
- the chain rule flowing backward
- a full worked numerical example
- the local-gradient rule

Before you start: Section 2: Calculus for ML and Section 9: Neurons, Layers, Activations.

A neural network learns by changing its weights in the direction that lowers the loss. To change a weight the right way, we need one number for it: the **gradient**. It says how much the loss would change if we changed that weight by a tiny amount. A real network has millions of weights, so we need millions of these numbers, and we need them again at every training step.

Backpropagation (“backprop”) is the algorithm that computes all of them in one sweep backward through the network. Every later lesson in this book uses it.

◆ THE ONE IDEA TO HOLD ONTO

Backprop is the chain rule from Section 2 applied to a graph. The intermediate results are **stored and reused** instead of computed again. That reuse is the whole trick. It turns “millions of expensive experiments” into “one backward pass that costs about as much as two forward passes.” Everything below is careful bookkeeping of this idea.

1 The goal, and why the obvious method fails

◆ INTUITION FIRST

Suppose you want to know how each weight affects the loss. The obvious idea is to *try it*. Take one weight and add a tiny amount to it. Run the whole network again and see how much the loss changed. Divide the change in loss by the change in the weight. That ratio is (approximately) the gradient for that one weight. Then put the weight back and do the next one. This works, and it needs no calculus. The problem is speed.

🔧 EVERYDAY PARALLEL

Imagine a mixing desk with a million volume sliders that all feed one loudspeaker. You want to know how much each slider affects the final loudness. The slow way: move slider one a little, play the whole song, measure the loudness, and reset. Then do the same for slider two, and play the *entire* song again. That is a million playbacks for one reading of the desk. Backprop gets every slider's effect from one playback plus one pass back from the speaker to each slider.

◆ NOW THE MATH

Write the loss as L (one number that says how wrong the network is) and a weight as w . The brute-force (“finite difference”) estimate of the gradient is:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w)}{\varepsilon}$$

In words: $\partial L / \partial w$ means “how fast the loss L changes when we change the weight w .” We estimate it by adding a tiny amount ε (epsilon, a small number like 10^{-5}) to the weight. We see how much L changes, and divide by ε . The problem: computing $L(w + \varepsilon)$ means running the *entire* forward pass of the network.

Now count the cost. Say the network has P weights, and one forward pass costs C units of work. Then estimating every gradient this way costs about:

$$P \times C \quad \text{units of work.}$$

In words: one full forward pass *per weight*. A small model with a million weights ($P = 10^6$) needs a million forward passes for one learning step. Modern models have billions of weights and take hundreds of thousands of steps. At that size, the brute-force method would take far too long to be useful.

◆ WHAT BACKPROP GIVES YOU

Backprop computes **all** P gradients with one backward pass. The backward pass costs a small constant times the forward pass, usually about $2\times$, no matter how many weights there are. It reuses partial results: the gradient at an earlier layer is built from work already done for the later layers. Nothing is computed twice. This is an example of *dynamic programming* (solving a big problem by storing and reusing the answers to smaller, overlapping parts).

CHECK YOURSELF 1

A network has 5 million weights and one forward pass takes 2 ms. Roughly how long does one gradient step take with brute force, and with backprop?

2 The computation graph

◆ INTUITION FIRST

To organize the bookkeeping, we draw the network as a **computation graph**. It is a diagram of small boxes called *nodes*, one for each simple operation: multiply, add, apply the sigmoid, and so on. Arrows called *edges* carry numbers from one box to the next. We put the input in and let the numbers flow from left to right. This is the **forward pass**, and it produces the prediction and the loss. Then we go from right to left. This return trip is the **backward pass**, and it produces the gradients.

🔑 EVERYDAY PARALLEL

Think of a supply chain: farmer $\rightarrow$ mill $\rightarrow$ bakery $\rightarrow$ shop. In the forward direction, goods move toward the shop, and the shop's profit is the final number. In the backward direction, information moves toward the farmer: "one more unit from you changes my profit by this much." That number can be positive or negative. (If the shop already has too much bread, one more unit *lowers* its profit.) Each link receives this message, converts it with its own local rate, and passes it upstream. The next section makes this exact.

◆ NOW THE MATH

Here is the graph we will compute with. One input x flows through two layers. Each layer does a *linear step* (multiply by a weight, add a bias) and then a *nonlinear step* (the sigmoid, written σ). The final node compares the output with the target y and produces the loss L :

$$\begin{aligned}
 x &\rightarrow \underbrace{z_1 = w_1 x + b_1}_{\text{linear}} \rightarrow \underbrace{a_1 = \sigma(z_1)}_{\text{nonlinear}} \\
 &\rightarrow \underbrace{z_2 = w_2 a_1 + b_2}_{\text{linear}} \rightarrow \underbrace{a_2 = \sigma(z_2)}_{\text{nonlinear}} \rightarrow \underbrace{L = \frac{1}{2}(a_2 - y)^2}_{\text{loss}}
 \end{aligned}$$

In words: x is the input number. w_1 and b_1 are the first layer's weight and bias. z_1 is the layer's raw output (the *pre-activation*). a_1 is z_1 squashed by the sigmoid into the range $(0, 1)$ (the *activation*). The symbols w_2, b_2, z_2, a_2 mean the same for the second layer, and a_2 is the network's prediction. y is the target value for a_2 . L is the squared-error loss: half the squared gap. The $\frac{1}{2}$ is there for convenience; it cancels when we differentiate.

DEFINITION

The **forward pass** runs the graph from left to right. It computes and **stores** every intermediate value (z_1, a_1, z_2, a_2, L). The **backward pass** runs from right to left. It computes the gradient of L with respect to every stored value and every parameter. The forward pass must come first, because the backward pass *reuses* the stored values. That reuse is where the speed comes from.

CHECK YOURSELF 2

Why must we keep the forward-pass values in memory instead of discarding them once we have the loss?

3 The core rule at every node

◆ INTUITION FIRST

Every node in the graph follows the same small rule. Once you see it, you have seen all of backprop. A node receives a number from the node after it. The number says “this is how much the final loss changes per unit change of *my* output.” The node also knows one local fact: “how much does my output change when my input changes?” Multiply the two. The result is “how much the final loss changes per unit change of my *input*.” That is the message the node passes one step further back.

☂ EVERYDAY PARALLEL

Back to the supply chain. The shop tells the bakery: “each extra loaf you send raises my profit by 2 dollars” (the gradient coming in). The bakery knows a local fact: “each extra kilogram of flour lets me bake 3 loaves” (the local gradient). So the bakery tells the mill: “each extra kilogram of flour is worth $2 \times 3 = 6$ dollars of final profit.” If the shop had said “-2 dollars,” the message would be -6: less flour is better. Nobody needs to see the whole chain. Each link multiplies the message from downstream by its own local rate and passes it on.

◆ NOW THE MATH

This is the chain rule, written as a rule you can apply mechanically. Take any node that computes an output u from an input t . Let the loss be L . Then:

$$\underbrace{\frac{\partial L}{\partial t}}_{\text{gradient sent to input}} = \underbrace{\frac{\partial L}{\partial u}}_{\text{gradient from above}} \times \underbrace{\frac{\partial u}{\partial t}}_{\text{local gradient}}$$

In words: (gradient of the loss for my output) $\times$ (how my output responds to my input) = (gradient of the loss for my input). The last factor, $\partial u / \partial t$, is the **local gradient**. It depends only on what *this* node does. You can compute it without knowing anything about the rest of the network. This locality makes backprop modular: each operation only needs to know its own derivative.

◆ THE LOCAL-GRADIENT RULE

$$\text{gradient in (from above)} \times \text{local gradient} = \text{gradient out (to inputs)}$$

Every node applies this rule. Backprop starts at the loss, whose gradient with respect to itself is 1. Then it applies the rule node by node, back to the weights. If a value feeds *several* nodes, its gradient is the **sum** of the messages from all of them. (This is the multivariable chain rule: each path through which the value affects L adds its own contribution.)

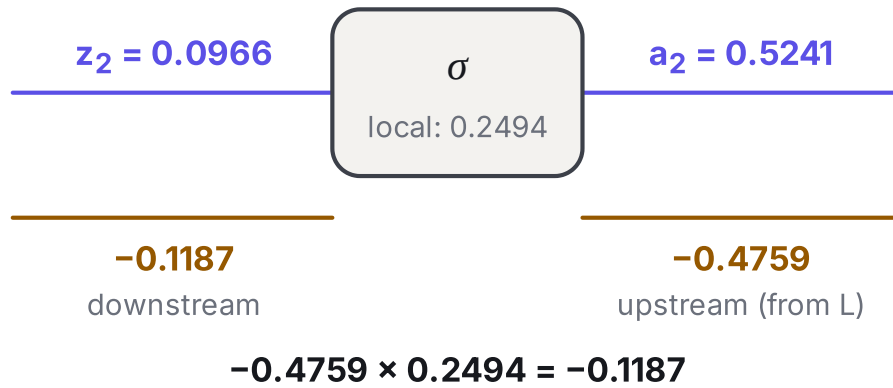


Figure 10.1 The rule at one node: the second sigmoid of the network in Section 4. Forward (violet): z_2 goes in and a_2 comes out. Backward (amber): the gradient -0.4759 arrives from the loss. The node multiplies it by its local gradient $a_2(1 - a_2) = 0.2494$ and sends -0.1187 back.

We need the local gradients of the three operations in our graph. The first two follow directly from the derivative rules:

$$\begin{aligned} \text{multiply } u = wt : \quad & \frac{\partial u}{\partial t} = w, & \frac{\partial u}{\partial w} = t \\ \text{add } u = t + b : \quad & \frac{\partial u}{\partial t} = 1, & \frac{\partial u}{\partial b} = 1 \end{aligned}$$

In words: a multiply node passes the gradient back scaled by the *other* input. So $z_2 = w_2 a_1$ sends gradient to a_1 scaled by w_2 , and to w_2 scaled by a_1 . An add node passes the gradient back unchanged (local gradient 1).

The third is the sigmoid. Section 6 derived its derivative; here is the short version. Write $\sigma(z) = (1 + e^{-z})^{-1}$ and use the chain rule:

$$\begin{aligned} \frac{d\sigma}{dz} &= \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\ &= \sigma(z) (1 - \sigma(z)) = a(1 - a) \end{aligned}$$

In words: the second fraction equals $1 - \sigma(z)$, because $\frac{e^{-z}}{1 + e^{-z}} = 1 - \frac{1}{1 + e^{-z}}$. So the sigmoid's local gradient is $a(1 - a)$. We can compute it from the activation a that we stored in the forward pass. There is no need to compute an exponential again on the way back.

CHECK YOURSELF 3

At a node $u = wt$, the gradient from above is $\partial L / \partial u = -3$, and the stored inputs are $w = 2$, $t = 5$. What gradients does the node send to t and to w ?

4 The main example: a full network, by hand

◆ INTUITION FIRST

Now we do the real thing. We take the two-layer graph from Section 2 and put in concrete numbers. We compute **every** value forward and **every** gradient backward. We skip nothing, so you can reproduce each number with a calculator. Figure 10.2 shows all the numbers at once; the steps below compute them one by one. These are the fixed ingredients:

Symbol	Meaning	Value
x	the single input	1.0
w_1, b_1	layer-1 weight and bias	0.5, 0.5
w_2, b_2	layer-2 weight and bias	1.5, -1.0
y	the target value for a_2	1.0

We round every result to four decimal places. The sigmoid is $\sigma(z) = 1/(1 + e^{-z})$.

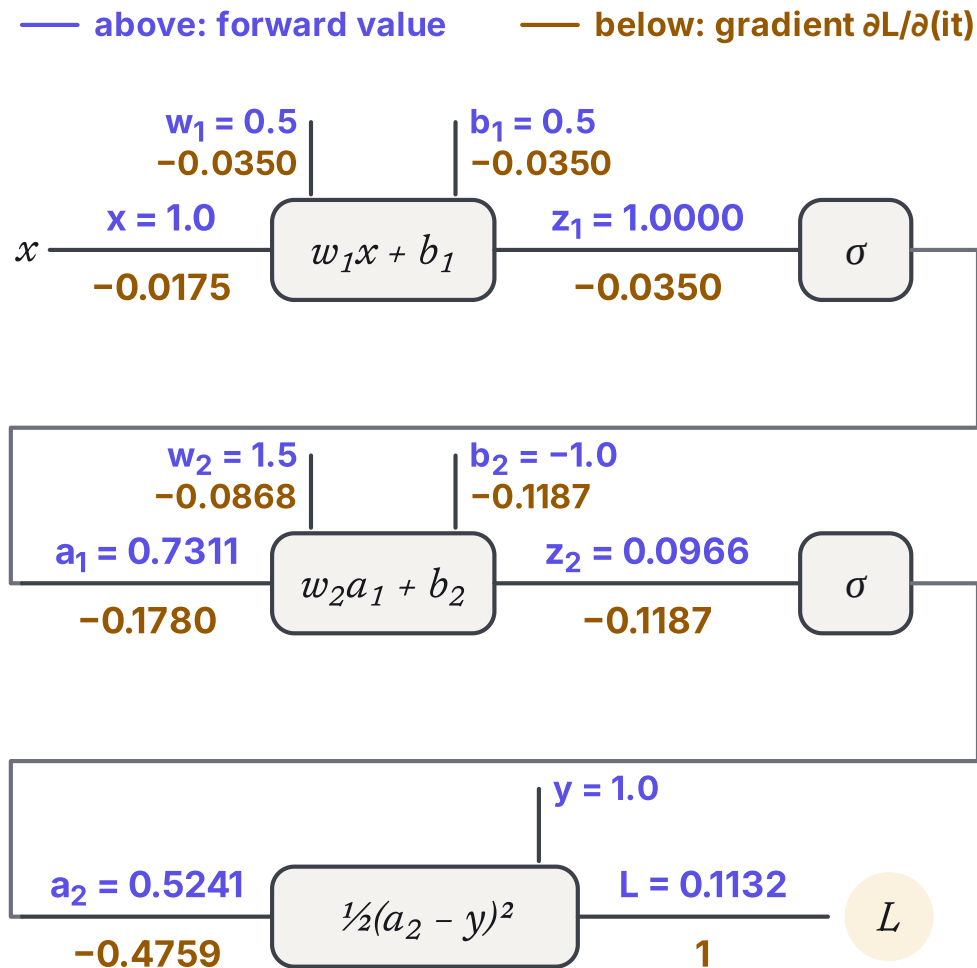


Figure 10.2 The whole worked example in one picture. Read the violet numbers from left to right (the forward pass), row by row. Then read the amber numbers from the end back to the start (the backward pass). Under each edge is the gradient of L with respect to the value on that edge. At every box, the amber number on its left equals the amber number on its right times the box's local gradient.

◆ NOW THE MATH

Forward pass: compute and store every value

Node 1: first linear step. Multiply the input by w_1 and add b_1 :

$$z_1 = w_1 x + b_1 = (0.5)(1.0) + 0.5 = \mathbf{1.0000}$$

Node 2: first sigmoid. Squash z_1 into $(0, 1)$:

$$a_1 = \sigma(1.0) = \frac{1}{1 + e^{-1.0}} = \frac{1}{1 + 0.3679} = \mathbf{0.7311}$$

Node 3: second linear step. Multiply the activation by w_2 and add b_2 :

$$z_2 = w_2 a_1 + b_2 = (1.5)(0.7311) + (-1.0) = 1.0966 - 1.0 = \mathbf{0.0966}$$

Node 4: second sigmoid. This is the network's prediction:

$$a_2 = \sigma(0.0966) = \frac{1}{1 + e^{-0.0966}} = \frac{1}{1 + 0.9079} = \mathbf{0.5241}$$

Node 5: the loss. Compare the prediction a_2 with the target $y = 1.0$:

$$\begin{aligned} L &= \frac{1}{2}(a_2 - y)^2 = \frac{1}{2}(0.5241 - 1.0)^2 \\ &= \frac{1}{2}(-0.4759)^2 = \frac{1}{2}(0.2265) = \mathbf{0.1132} \end{aligned}$$

In words: the network predicts 0.5241, but we want 1.0. It is off by 0.4759, and the loss is 0.1132. All five numbers are now stored. The backward pass will reuse them.

Backward pass: send the gradient back, node by node

We start at the loss and apply the local-gradient rule again and again. It helps to name the gradient at each pre-activation. The standard name is δ (delta): $\delta_2 = \partial L / \partial z_2$ and $\delta_1 = \partial L / \partial z_1$. Notice how each step reuses a stored forward value.

Step A: through the loss node. Differentiate $L = \frac{1}{2}(a_2 - y)^2$ with respect to a_2 . The $\frac{1}{2}$ cancels the 2 from the power:

$$\frac{\partial L}{\partial a_2} = (a_2 - y) = 0.5241 - 1.0 = \mathbf{-0.4759}$$

In words: the gradient is negative. Gradient descent moves *against* the gradient, so it will increase a_2 . That makes sense, because 0.5241 is below the target 1.0.

Step B: through the second sigmoid. The local gradient is $a_2(1 - a_2)$, computed from the stored a_2 . Multiply it by the gradient from above to get δ_2 :

$$\begin{aligned} \frac{\partial a_2}{\partial z_2} &= a_2(1 - a_2) = (0.5241)(0.4759) = 0.2494 \\ \delta_2 &= \frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} = (-0.4759)(0.2494) = \mathbf{-0.1187} \end{aligned}$$

In words: we took the message -0.4759 and scaled it by the sigmoid's local gradient 0.2494. That 0.2494 came directly from the stored a_2 , with no new exponentials. δ_2 is the gradient of the loss with respect to the layer-2 pre-activation.

Step C: the layer-2 weight and bias. Recall $z_2 = w_2 a_1 + b_2$. The multiply node sends gradient to w_2 scaled by the *other* input, the stored a_1 . The add node sends gradient to b_2 unchanged:

$$\begin{aligned}\frac{\partial L}{\partial w_2} &= \delta_2 \cdot \frac{\partial z_2}{\partial w_2} = \delta_2 \cdot a_1 = (-0.1187)(0.7311) = \mathbf{-0.0868} \\ \frac{\partial L}{\partial b_2} &= \delta_2 \cdot \frac{\partial z_2}{\partial b_2} = \delta_2 \cdot 1 = \mathbf{-0.1187}\end{aligned}$$

In words: two of our four parameter gradients are done. The weight gradient reused the stored activation $a_1 = 0.7311$. The bias gradient is δ_2 itself, because adding a bias passes the gradient through unchanged.

Step D: back into layer 1. The multiply node also sends gradient to its *other* input a_1 , scaled by w_2 :

$$\frac{\partial L}{\partial a_1} = \delta_2 \cdot \frac{\partial z_2}{\partial a_1} = \delta_2 \cdot w_2 = (-0.1187)(1.5) = \mathbf{-0.1780}$$

In words: we have crossed from layer 2 into layer 1. The gradient has reached a_1 , the first layer's activation. Now we pass it through the first sigmoid, in the same way as the second.

Step E: through the first sigmoid. Same rule as Step B, now with the stored a_1 :

$$\begin{aligned}\frac{\partial a_1}{\partial z_1} &= a_1(1 - a_1) = (0.7311)(0.2689) = 0.1966 \\ \delta_1 &= \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} = (-0.1780)(0.1966) = \mathbf{-0.0350}\end{aligned}$$

In words: δ_1 is the gradient of the loss with respect to the layer-1 pre-activation. It is much smaller than δ_2 . The signal shrank as it passed back through the sigmoid, whose local gradient is at most 0.25. Stack many sigmoid layers and the signal keeps shrinking. This is the *vanishing gradient* problem (Section 12).

Step F: the layer-1 weight and bias. Recall $z_1 = w_1 x + b_1$. This is the same pattern as Step C, now with the stored input x :

$$\begin{aligned}\frac{\partial L}{\partial w_1} &= \delta_1 \cdot \frac{\partial z_1}{\partial w_1} = \delta_1 \cdot x = (-0.0350)(1.0) = \mathbf{-0.0350} \\ \frac{\partial L}{\partial b_1} &= \delta_1 \cdot \frac{\partial z_1}{\partial b_1} = \delta_1 \cdot 1 = \mathbf{-0.0350}\end{aligned}$$

In words: all four parameter gradients are done. We went from the loss back to the first weight. At each node we did only one thing: "gradient from above $\times$ local gradient," using stored values. That whole sweep is one backward pass.

◆ ALL FOUR GRADIENTS, IN ONE PLACE

These are the numbers we came for: the slope of the loss along each parameter.

$\frac{\partial L}{\partial w_1}$	$\frac{\partial L}{\partial b_1}$	$\frac{\partial L}{\partial w_2}$	$\frac{\partial L}{\partial b_2}$
-0.0350	-0.0350	-0.0868	-0.1187

All four are negative. This means that increasing any of these parameters would *lower* the loss. So the update will increase all of them.

One gradient-descent step: does the loss drop?

Gradient descent moves each parameter a small step *against* its gradient (downhill on the loss). The step size is the *learning rate* α (alpha). Take $\alpha = 1.0$. The update rule for any parameter is $w \leftarrow w - \alpha \partial L / \partial w$:

$$w_1 \leftarrow 0.5 - (1.0)(-0.0350) = \mathbf{0.5350}$$

$$b_1 \leftarrow 0.5 - (1.0)(-0.0350) = \mathbf{0.5350}$$

$$w_2 \leftarrow 1.5 - (1.0)(-0.0868) = \mathbf{1.5868}$$

$$b_2 \leftarrow -1.0 - (1.0)(-0.1187) = \mathbf{-0.8813}$$

In words: subtracting a negative gradient means *adding*. Every parameter grew a little, as the signs predicted. Now the real test: does the loss fall? Run the forward pass again with the new parameters.

New forward pass:

$$z_1 = (0.5350)(1.0) + 0.5350 = 1.0700$$

$$a_1 = \sigma(1.0700) = 0.7446$$

$$z_2 = (1.5868)(0.7446) + (-0.8813) = 0.3002$$

$$a_2 = \sigma(0.3002) = 0.5745$$

$$L_{\text{new}} = \frac{1}{2}(0.5745 - 1.0)^2 = \frac{1}{2}(0.1811) = \mathbf{0.0905}$$

In words: the loss fell from 0.1132 to **0.0905**. The prediction moved toward the target, from 0.5241 to 0.5745. One backward pass and one step made the network measurably less wrong. Repeat this loop thousands of times and you have training.

EVERYDAY PARALLEL

You walk down a hill in thick fog. You cannot see the valley; you can only feel the slope under your feet. The gradient is that slope, and the learning rate is the length of your step. You take one step downhill, feel the slope again, and take another step. Backprop tells you the slope; gradient descent takes the step.

THE SAME THING IN PYTORCH

>_ MANUAL BACKPROP, NO AUTOGRAD: REPRODUCES EVERY NUMBER ABOVE

```

import torch

def sigmoid(z):
    return 1.0 / (1.0 + torch.exp(-z))

# fixed ingredients
x = torch.tensor(1.0)
w1, b1 = torch.tensor(0.5), torch.tensor(0.5)
w2, b2 = torch.tensor(1.5), torch.tensor(-1.0)
y = torch.tensor(1.0)

# ---- forward pass: compute and keep every value ----
z1 = w1 * x + b1          # 1.0000
a1 = sigmoid(z1)          # 0.7311
z2 = w2 * a1 + b2          # 0.0966
a2 = sigmoid(z2)          # 0.5241
L = 0.5 * (a2 - y)**2     # 0.1132

# ---- backward pass: gradient in x local gradient, node by node ----
dL_da2 = (a2 - y)         # -0.4759
d2 = dL_da2 * a2 * (1 - a2) # delta2 = -0.1187
dL_dw2 = d2 * a1           # -0.0868
dL_db2 = d2                # -0.1187
dL_da1 = d2 * w2           # -0.1780
d1 = dL_da1 * a1 * (1 - a1) # delta1 = -0.0350
dL_dw1 = d1 * x            # -0.0350
dL_db1 = d1                # -0.0350

print(dL_dw1, dL_db1, dL_dw2, dL_db2)
# tensor(-0.0350) tensor(-0.0350) tensor(-0.0868) tensor(-0.1187)

# ---- one gradient-descent step (learning rate 1.0) ----
alpha = 1.0
w1, b1 = w1 - alpha*dL_dw1, b1 - alpha*dL_db1
w2, b2 = w2 - alpha*dL_dw2, b2 - alpha*dL_db2

a2_new = sigmoid(w2 * sigmoid(w1*x + b1) + b2)
print(0.5 * (a2_new - y)**2) # tensor(0.0905) → the loss dropped

```

CHECK YOURSELF 4

Why is $\partial L / \partial b_1$ equal to $\partial L / \partial w_1$ here (both -0.0350)? Would they still be equal if the input were $x = 2.0$?

5 The same rules for a whole layer at once

◆ INTUITION FIRST

Real layers have many neurons. The weights form a matrix $\mathbf{W}$, and the activations form vectors. The rules do not change. “A multiply node swaps its inputs” becomes “multiply by the transpose of the matrix.” The number δ becomes a vector $\boldsymbol{\delta}$, with one entry per neuron. Everything you did by hand now runs on a whole layer in one matrix operation. GPUs are built to do such operations fast.

☂ EVERYDAY PARALLEL

Think of computing the payroll for a whole company at once instead of one employee at a time. The rule for each person (hours $\times$ rate) does not change. You put everyone in one table and do the arithmetic in one sweep. Vectorizing backprop is the same move: the same rule, applied to the whole layer with one matrix multiply.

◆ NOW THE MATH

Take a layer $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$. $\mathbf{W}$ has shape (out, in), as in Section 9. Let $\boldsymbol{\delta} = \partial L / \partial \mathbf{z}$ be the gradient that arrives at the layer's pre-activations. We derive the gradients one entry at a time, then collect them into matrices. Entry i of $\mathbf{z}$ is:

$$z_i = \sum_j W_{ij} x_j + b_i$$

In words: neuron i multiplies input j by the weight W_{ij} , sums over all inputs j , and adds its bias b_i . The weight W_{ij} appears in only one output, z_i . But each input x_j appears in every output $z_1, z_2, \dots$

Now apply the local-gradient rule to each kind of parameter:

$$\begin{aligned}\frac{\partial L}{\partial W_{ij}} &= \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial W_{ij}} = \delta_i x_j \\ \frac{\partial L}{\partial b_i} &= \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial b_i} = \delta_i \\ \frac{\partial L}{\partial x_j} &= \sum_i \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial x_j} = \sum_i \delta_i W_{ij}\end{aligned}$$

In words: the first two lines are Step C again, for each neuron: “ δ times the other input” for a weight, and “ δ itself” for a bias. The third line has a sum, because x_j feeds every neuron. By the sum rule from Section 3, its gradient adds up the messages from all of them. That is Step D, summed over neurons.

Now collect the entries into matrices. The grid of numbers $\delta_i x_j$ is the *outer product* $\boldsymbol{\delta} \mathbf{x}^\top$ (a column times a row). The sum $\sum_i \delta_i W_{ij}$ is entry j of $\mathbf{W}^\top \boldsymbol{\delta}$ (the little $\top$ means *transpose*: rows become columns). So:

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{W}} &= \boldsymbol{\delta} \mathbf{x}^\top \quad (\text{for the weight matrix}) \\ \frac{\partial L}{\partial \mathbf{b}} &= \boldsymbol{\delta} \quad (\text{for the bias vector}) \\ \frac{\partial L}{\partial \mathbf{x}} &= \mathbf{W}^\top \boldsymbol{\delta} \quad (\text{sent to the previous layer})\end{aligned}$$

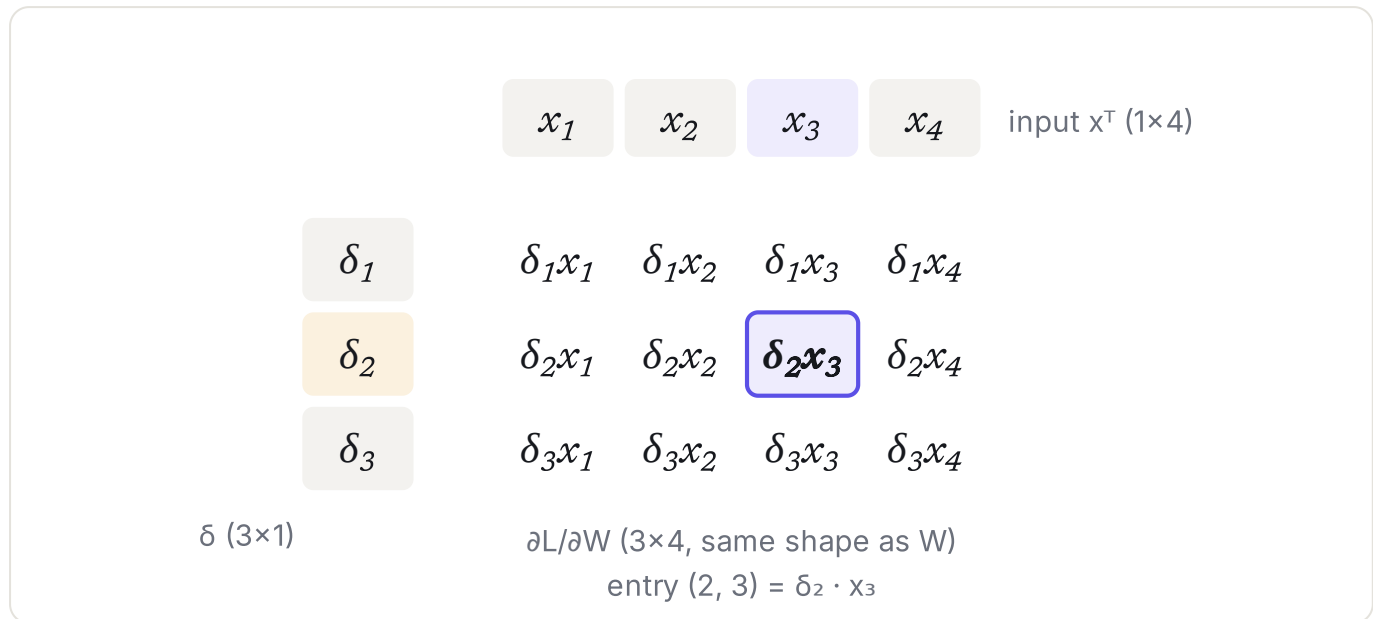


Figure 10.3 The weight gradient $\boldsymbol{\delta} \mathbf{x}^\top$ for a layer with 4 inputs and 3 neurons. Entry (i, j) is $\delta_i x_j$: the error signal of neuron i times the input that the weight W_{ij} multiplies. The result has the same shape as $\mathbf{W}$.

To continue one more layer back, remember where $\mathbf{x}$ came from. The input of this layer is the output of the previous one: $\mathbf{x} = \phi(\mathbf{z}_{\text{prev}})$, where ϕ is the previous layer's activation function. So pass $\partial L / \partial \mathbf{x}$ through ϕ , as in Step E. Multiply by the local gradient ϕ' , entry by entry:

$$\delta_{\text{prev}} = (\mathbf{W}^\top \delta) \odot \phi'(\mathbf{z}_{\text{prev}})$$

In words: $\odot$ means “multiply matching entries” (the *element-wise* product). Each neuron's incoming gradient is scaled by its own local slope ϕ' . This line, repeated from layer to layer, **is** the backward pass for a deep network. It is the same “gradient in $\times$ local gradient = gradient out” that you did by hand, now vectorized.

◆ THE SAME THING IN PYTORCH

>_ THE THREE LAYER GRADIENTS, CHECKED AGAINST AUTOGRAD

```
import torch
torch.manual_seed(0)

# a layer: W is (out=3, in=4), one input vector x of length 4
W = torch.randn(3, 4)
x = torch.randn(4)
delta = torch.randn(3)          # gradient arriving at this layer's z

dL_dW = torch.outer(delta, x) # delta x^T → shape (3, 4), same as W
dL_db = delta                  # → shape (3,)
dL_dx = W.T @ delta           # → shape (4,), sent to the previous layer
print(dL_dW.shape, dL_db.shape, dL_dx.shape)
# torch.Size([3, 4]) torch.Size([3]) torch.Size([4])

# check with autograd: a loss whose gradient at z is exactly delta
W.requires_grad_(); x.requires_grad_()
L = (delta * (W @ x)).sum()
L.backward()
print(torch.allclose(W.grad, dL_dW), torch.allclose(x.grad, dL_dx)) # T
```

▲ WATCH OUT

The transposes are necessary: they make the shapes fit. $\partial L / \partial \mathbf{W}$ must have the *same shape* as $\mathbf{W}$, because you subtract it from $\mathbf{W}$ in the update. And $\partial L / \partial \mathbf{x}$ must have the same shape as $\mathbf{x}$. If a backward pass gives a shape error, check the transposes first. When you are stuck, print the shape on every line, as in Section 1.

CHECK YOURSELF 5

A layer has $\delta = [1, 0, -2]$ and input $\mathbf{x} = [1, 2, 0, 1]$. Predict row 3 of $\partial L / \partial \mathbf{W}$. Which whole row and which whole column are zero, and why?

6 Trust, but verify: gradient checking

◆ INTUITION FIRST

It is easy to make a small mistake in hand-derived gradients: a missing transpose, or a wrong sign. There is a simple way to catch such mistakes. Compare your backprop gradient with the brute-force estimate from the start of this lesson. If they agree to many decimal places, your backprop is almost certainly correct. This is the **numerical gradient check**. It is one of the most useful tools for debugging a network written from scratch.

☂ EVERYDAY PARALLEL

You compute 47×23 in your head with a clever shortcut. To check it, you add 47 twenty-three times on paper. The slow method is too slow for everyday use, but it is hard to get wrong. So it is a good way to test the fast method once. Backprop is the fast shortcut; the numerical gradient is the slow, reliable check.

◆ NOW THE MATH

Use the **centered** difference: change the weight both up and down by ϵ . It is much more accurate than the one-sided version from Section 1:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$$

In words: move the weight up, move it down, and take the slope between the two points. The error of this estimate shrinks like ϵ^2 , while the one-sided error shrinks only like ϵ . It is far too slow to *train* with (one forward pass per weight, the problem that backprop solves). But it is a good one-time check that backprop is correct.

▲ WATCH OUT

Do gradient checks in **float64**. A float32 number keeps only about 7 significant digits. The difference $L(w + \varepsilon) - L(w - \varepsilon)$ is tiny, so most of those digits are lost in the subtraction. In float32, the code below prints about -0.0339 instead of -0.0350 .

◆ THE SAME THING IN PYTORCH

>_ NUMERICAL CHECK OF OUR HAND-COMPUTED GRADIENT FOR W1, IN FLOAT64

```
import torch

def loss(w1, b1, w2, b2, x=1.0, y=1.0):
    a1 = 1/(1 + torch.exp(-(w1*x + b1)))
    a2 = 1/(1 + torch.exp(-(w2*a1 + b2)))
    return 0.5 * (a2 - y)**2

w1, b1, w2, b2 = (torch.tensor(v, dtype=torch.float64) for v in (0.5, 0.5,
eps = 1e-5
num_grad_w1 = (loss(w1+eps, b1, w2, b2) - loss(w1-eps, b1, w2, b2)) / (2*eps)

print(num_grad_w1.item())    # -0.03500407346271328    (backprop: -0.03500407346271328)
```

In words: the numerical estimate is -0.0350 when rounded, the same value we computed by hand in Step F. In float64 it agrees with the exact backprop value to about 11 decimal places. That agreement shows our backward pass is correct. Section 15 turns this into a regular debugging habit.

◆ YOU WILL RARELY WRITE THIS BY HAND AGAIN

Deriving gradients by hand is how you *understand* backprop. In practice, a library does it for you. Autograd (Section 13) records the computation graph while your forward code runs. Then it applies the local-gradient rule you just used at every node, automatically. In Section 11 we write a full multi-layer network's backward pass from scratch in NumPy. You will see there is nothing hidden: only the bookkeeping you now understand.

CHECK YOURSELF 6

Your backprop gradient for a weight is 0.4000, but the centered numerical check gives -0.4000. What kind of bug does this mismatch point to?

7 What to carry forward

- Training needs $\partial L / \partial w$ for every parameter. Brute force costs one forward pass *per parameter*, which is far too slow. Backprop gets them **all** with one backward pass (about 2× the cost of a forward pass) by reusing stored forward values. This is dynamic programming.
- The **computation graph** is the network drawn as nodes (operations) and edges (values). The forward pass computes and stores values; the backward pass computes gradients.
- The **local-gradient rule** at every node: *gradient in (from above) × local gradient = gradient out (to inputs)*. If a value feeds several nodes, add up the gradients that come back.
- Useful local gradients: multiply swaps its inputs; add passes the gradient through unchanged; the sigmoid's is $a(1 - a)$, computed from the stored activation.
- We computed a whole two-layer network by hand: forward to $L = 0.1132$, backward to four gradients, one step, and the loss fell to 0.0905. The code confirms every number.
- For a whole layer, $\partial L / \partial W_{ij} = \delta_i x_j$, which gives $\partial L / \partial \mathbf{W} = \boldsymbol{\delta} \mathbf{x}^\top$, $\partial L / \partial \mathbf{b} = \boldsymbol{\delta}$ and $\partial L / \partial \mathbf{x} = \mathbf{W}^\top \boldsymbol{\delta}$.
- **Gradient checking** in float64 compares backprop with the centered difference. A sign error or a missing factor shows up at once.

Next, in Section 11, we stop working by hand. We build a full multi-layer network, forward and backward, from scratch in NumPy. You already know the one rule it runs on.

Answers to “Check yourself”

1. A network has 5 million weights and one forward pass takes 2 ms. Roughly how long does one gradient step take with brute force, and with backprop?

Brute force: one forward pass per weight, so $5,000,000 \times 2 \text{ ms} = 10,000,000 \text{ ms}$. That is about **2.8 hours** for one learning step, and you need thousands of steps. **Backprop:** one forward pass (2 ms) plus one backward pass (about $2 \times$ the forward pass, so about 4 ms). That is about **6 ms** in total. The gradients are the same. The cost differs by a factor of more than a million, because backprop reuses intermediate results.

2. Why must we keep the forward-pass values in memory instead of discarding them once we have the loss?

Because the backward pass needs them. As we will see, the gradient for w_2 is $\delta_2 \cdot a_1$: it multiplies by the stored activation a_1 . If we threw a_1 away, we would have to compute it again, which defeats the purpose. This is why training needs much more memory than running a model for predictions (*inference*). Training must keep the whole forward pass in memory for the return trip.

3. At a node $u = wt$, the gradient from above is $\partial L / \partial u = -3$, and the stored inputs are $w = 2, t = 5$. What gradients does the node send to t and to w ?

Apply the rule twice. To the input t : the local gradient is $w = 2$, so $\partial L / \partial t = -3 \times 2 = -6$. To the weight w : the local gradient is $t = 5$, so $\partial L / \partial w = -3 \times 5 = -15$. The multiply node “swaps” its inputs on the way back: each branch is scaled by the *other* value. Every linear layer is built from multiply nodes, so you will use this swap all the time.

4. Why is $\partial L / \partial b_1$ equal to $\partial L / \partial w_1$ here (both -0.0350)? Would they still be equal if the input were $x = 2.0$?

Both are δ_1 times a local gradient. For the bias the local gradient is 1, so $\partial L / \partial b_1 = \delta_1$. For the weight the local gradient is the input x , so $\partial L / \partial w_1 = \delta_1 \cdot x$. They are equal here only because $x = 1.0$. With $x = 2.0$ they would differ. (Careful: δ_1 itself would also change, because the forward values change. But the weight gradient would be $2\delta_1$ and the bias gradient δ_1 .) A weight's gradient always scales with the input that feeds it; a bias's gradient does not.

5. A layer has $\delta = [1, 0, -2]$ and input $\mathbf{x} = [1, 2, 0, 1]$. Predict row 3 of $\partial L / \partial \mathbf{W}$. Which whole row and which whole column are zero, and why?

Row 3 is $\delta_3 \mathbf{x}^\top = -2 \times [1, 2, 0, 1] = [-2, -4, 0, -2]$. Row 2 is all zeros, because $\delta_2 = 0$: neuron 2's output does not affect the loss here, so its weights get no update. Column 3 is all zeros, because $x_3 = 0$: a weight that multiplies a zero input had no effect on the output, so it gets no blame.

6. Your backprop gradient for a weight is 0.4000, but the centered numerical check gives -0.4000. What kind of bug does this mismatch point to?

A **sign error** in the backward pass. When the sizes match but the signs are opposite, you have almost certainly flipped a subtraction somewhere. A typical example is $y - a$ written where $a - y$ belongs. This is a common backprop bug, and a gradient check finds it at once. (The check compares gradients only, so it cannot find a sign error in the update step itself.) If instead the *sizes* were very different, look for a missing local-gradient factor (for example a forgotten $a(1 - a)$) or a wrong transpose.

Build an MLP in NumPy

No PyTorch yet. We use only NumPy and the backprop math from Section 10, and build a network that learns.

58 min read

YOU'LL LEARN

- forward pass
- backward pass
- the training loop
- watching a real network learn

Before you start: Section 10: Backpropagation by Hand.

So far each idea has had its own page: a neuron here, a gradient there, a loss function somewhere else. In this section we **put all the pieces in one file** and run it. There is no framework, only NumPy arrays and arithmetic you can read line by line. By the end you will have a small network that learns to solve a problem no straight line can solve. And you will understand every number that flows through it.

◆ THE ONE IDEA TO HOLD ONTO

A neural network is not a mysterious black box. It learns with four steps in a loop: **predict, measure, find the direction, step**. Predict an answer. Measure how wrong it is. Find the direction for each weight that reduces the error. Take a small step in that direction. Repeat a few thousand times and the predictions become good. In this section we write those four steps in plain NumPy and watch them work.

1 The plan: solve a problem a line cannot

◆ INTUITION FIRST

In Section 9 we saw that a single linear layer can only draw a **straight boundary** between classes. That is fine for data you can separate with a ruler, but real data is rarely that simple. The classic example is **XOR** (short for “exclusive or”). The rule is: output 1 when the two inputs *disagree*, and 0 when they agree.

input x_1	input x_2	label (XOR)
0	0	0 (agree)
0	1	1 (disagree)
1	0	1 (disagree)
1	1	0 (agree)

Try to draw one straight line with the two 1s on one side and the two 0s on the other. You cannot. The 1s sit on opposite corners of a square, and so do the 0s. No single line separates them. This is the smallest problem that a linear model cannot solve, so it is a good first test for a real network.

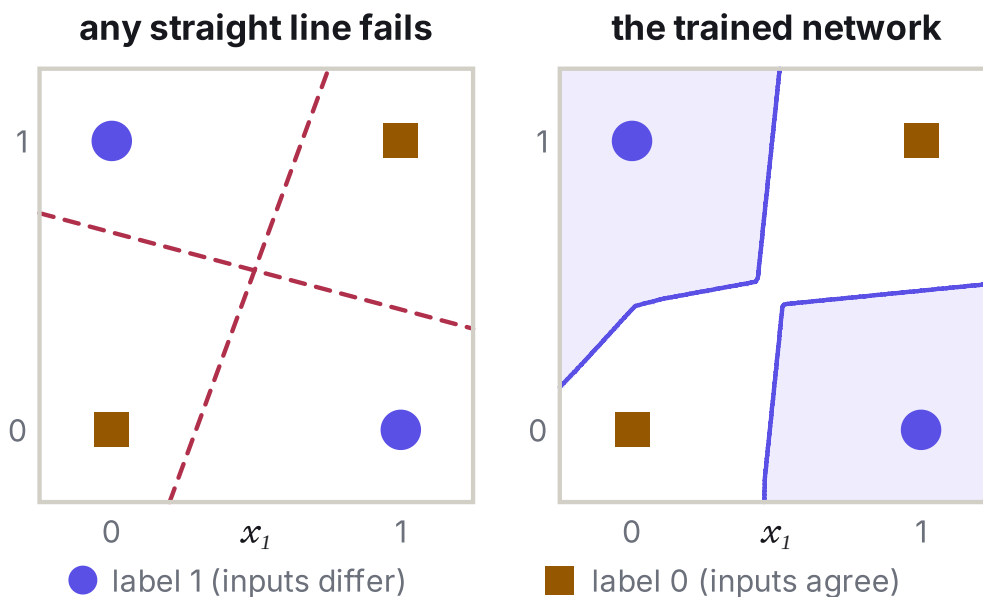


Figure 11.1 Left: every straight line puts at least one point on the wrong side. The two dashed lines are examples. Right: the regions learned by the network we train in this section (seed 42). The shaded area is where the network predicts label 1. Its boundary is made of straight pieces joined at bends. Each bend comes from a ReLU switching on or off.

☂ EVERYDAY PARALLEL

Think of a staircase light with one switch at the top and one at the bottom. The light is on only when the two switches are in *different* positions. Flip either switch and the light changes. “On when they disagree” is XOR. Flipping switch A has the opposite effect depending on where switch B is. So no rule of the form “more A means more light” can describe it. You need a rule that *bends*, and a network can bend.

The plan for this section: build a **2-layer MLP** in pure NumPy and train it until it classifies all four XOR points correctly. Every gradient we use comes from Section 10. Here we turn that math into code that runs.

DEFINITION

An **MLP** (multi-layer perceptron) is a network built from layers. Each layer does a matrix multiply and adds a bias. A **nonlinearity** (a bending function such as ReLU) sits between the layers. The nonlinearity lets the decision boundary bend instead of being one straight line.

CHECK YOURSELF 1

Suppose you add a third input $x_3 = x_1 \cdot x_2$ to each XOR point. Can a single linear layer now separate the classes?

2 The training loop, before any code

◆ INTUITION FIRST

Every network in this course, from this small one to large language models, learns with the same four-step loop. Remember these four steps and the code below will be easy to follow:

1. **Predict:** pass the inputs through the network to get an answer (the *forward pass*).
2. **Measure:** compare the answer with the true answer using a *loss* (one number that says how wrong we are).
3. **Find the direction:** for every weight, find which way to change it to make the loss smaller. This direction comes from the *gradient*, computed by the *backward pass*.
4. **Step:** change every weight a little in that direction. Then go back to step 1.

🌂 EVERYDAY PARALLEL

Think of learning a basketball free throw. You **shoot** (predict). You **see the miss**: short and to the left (measure). You work out the **correction**: a bit more arc, aim a little to the right (find the direction). You shoot again with that small change (step). No single throw is perfect, but after a hundred throws the ball goes in. A network trains in the same way, with numbers instead of muscles.

◆ ONE PASS = ONE EPOCH

Running the loop once over the whole dataset is called one *epoch*. We will run a few thousand epochs. Each one makes the weights a little better, and the printed loss goes down. That falling number is the network learning.

CHECK YOURSELF 2

You run a training loop and the printed loss is exactly the same number at every epoch. The update line is `for p, g in zip(params, grads): p = p - lr * g`. What is wrong?

3 Building it, one block at a time

We build the network in small labeled pieces, so every line has a reason. At the end (the full script), the same pieces are collected into one script you can copy and run.

(a) The dataset

First the data. XOR has only four points, so we can type them by hand. `X` holds the four inputs (one per row), and `y` holds their correct labels.

>_ THE FOUR XOR POINTS AND THEIR LABELS

```
import numpy as np

# each row is one example: [x1, x2]
X = np.array([[0.0, 0.0],
              [0.0, 1.0],
              [1.0, 0.0],
              [1.0, 1.0]])

# the correct answer for each row (XOR: 1 when inputs differ)
y = np.array([0, 1, 1, 0])

N, D = X.shape          # N = 4 examples, D = 2 features each
C = 2                   # number of classes: "0" and "1"
```

We treat this as a **classification** problem with two classes (“output is 0” and “output is 1”) instead of predicting a raw number. This choice lets us use softmax and cross-entropy, as a real classifier does.

Cross-entropy compares two *probability lists*. So the label 1 must also become a probability list: “0% class 0, 100% class 1.” This format is called **one-hot**: a list of zeros with a single 1 in the correct position.

>_ TURN EACH LABEL INTO A ONE-HOT ROW

```
# Y[i] is all zeros except a 1 in the correct class column
Y = np.zeros((N, C))
Y[np.arange(N), y] = 1.0
#   y = [0, 1, 1, 0] → Y = [[1, 0],
#                           [0, 1],
#                           [0, 1],
#                           [1, 0]]
```

☂ EVERYDAY PARALLEL

One-hot is a multiple-choice answer sheet where you fill in exactly one bubble. The label “the answer is B” becomes the row [0, 1, 0, 0]: a 1 under B and 0 everywhere else. In this form we can compare the answer, option by option, with the network's output, which says how sure it is about each option.

(b) Initialize the weights

The network has two layers, so it has two weight matrices and two bias vectors. Layer 1 turns the 2 input features into H hidden numbers. Layer 2 turns those H numbers into 2 class scores. We fill the weights with **small random** values and set the biases to zero.

▲ WATCH OUT: WEIGHT SHAPE IN THIS LESSON

In this NumPy lesson, a weight matrix has shape (**in**, **out**), and a layer computes $\mathbf{X} @ \mathbf{W} + \mathbf{b}$. For example, `W1` is (2, 8). In Section 9, Section 10 and PyTorch's `nn.Linear` (Section 14), $\mathbf{W}$ has shape (**out**, **in**) and the layer computes $\mathbf{X}\mathbf{W}^\top + \mathbf{b}$. The two forms compute the same thing; our $\mathbf{W}$ here is their $\mathbf{W}^\top$. We use (in, out) because the code is shorter without the transposes.

>_ TWO LAYERS: (2 → H) THEN (H → 2)

```
rng = np.random.default_rng(42)    # fixed seed so results repeat
H = 8                               # neurons in the hidden layer

# He initialization: random spread scaled by the number of inputs
W1 = rng.normal(0, np.sqrt(2.0 / D), size=(D, H))    # (2, 8)
b1 = np.zeros(H)                                     # (8,)
W2 = rng.normal(0, np.sqrt(2.0 / H), size=(H, C))    # (8, 2)
b2 = np.zeros(C)                                     # (2,)
```

Why *random* and not zeros? If every weight in a layer starts with the same value, every neuron in that layer computes the same thing. Each one then receives the same gradient and changes in the same way, forever. The neurons never become different, so they cannot learn different features. This is the **symmetry** problem, and a little randomness breaks it. The scale of the random values also matters (too big or too small both break training). He initialization chooses a scale that keeps the values in each layer at a similar size. Section 12 explains why.

☂ EVERYDAY PARALLEL

Give eight new interns the same starting notes and the same feedback. They will all reach the same conclusion. In effect you have hired one intern eight times. Give each one a slightly different starting point, and they explore different ideas. Random initialization is how the eight hidden neurons avoid becoming eight copies of the same neuron.

(c) The forward pass

The **forward pass** is the “predict” step: pass `x` through both layers to get class scores. Between the layers sits **ReLU** (from Section 9), which keeps positive numbers and replaces negative numbers with 0. That bend is what lets the network solve XOR.

◆ NOW THE MATH

We process all N examples at once, so each quantity is a matrix with one row per example. We write these batch matrices with capital letters:

$$\begin{aligned}\mathbf{Z}_1 &= \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1 \\ \mathbf{A}_1 &= \text{ReLU}(\mathbf{Z}_1) = \max(0, \mathbf{Z}_1) \\ \mathbf{Z}_2 &= \mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2 \\ \mathbf{P} &= \text{softmax}(\mathbf{Z}_2)\end{aligned}$$

In words: multiply the inputs by the first weight matrix and add the bias to get $\mathbf{Z}_1$ (the pre-activations, code name `z1`). Bend them with ReLU to get the hidden values $\mathbf{A}_1$ (`a1`). Multiply those by the second weight matrix and add the second bias to get the raw scores $\mathbf{Z}_2$ (`z2`). These scores are called *logits*: they are not normalized and can be any size, positive or negative. Finally, softmax turns each row of logits into probabilities $\mathbf{P}$ (`probs`). The bias vector is added to every row; NumPy does this automatically (broadcasting, Section 13).

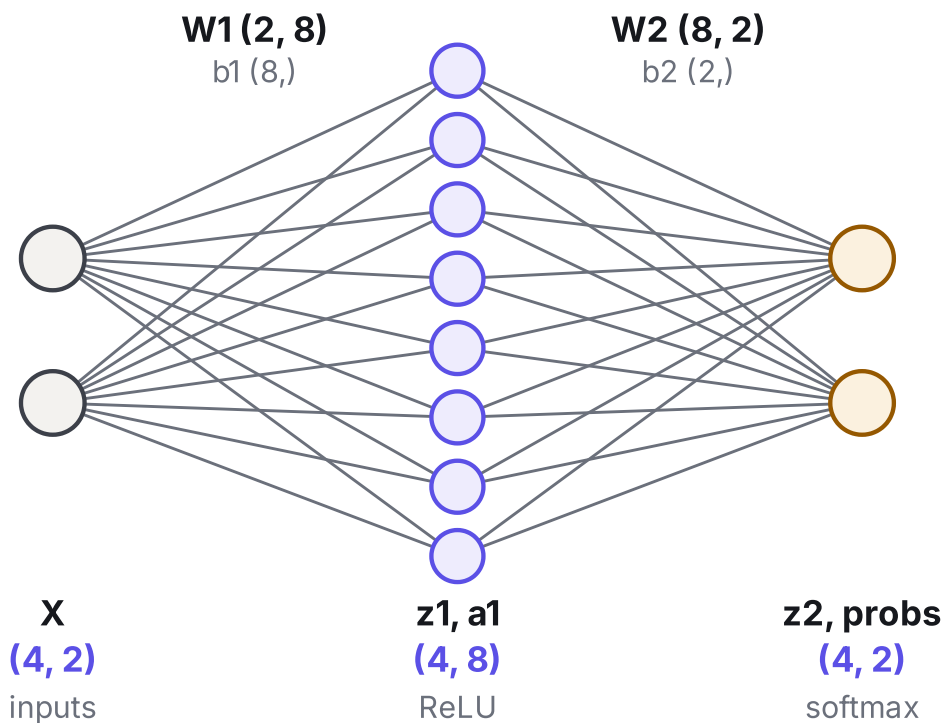


Figure 11.2 Our $2 \rightarrow 8 \rightarrow 2$ network. Every line is one weight: $2 \times 8 = 16$ in **W1** and $8 \times 2 = 16$ in **W2**. Below each layer is the shape of its array for the batch of 4 examples. The first number is always the batch size.

Softmax turns a row of raw scores into a row of probabilities. It makes every number positive with the exponential function. Then it divides by the total, so the row sums to 1.

$$\text{softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

In words: for each score z_j , compute e^{z_j} . This makes it positive and increases large scores much more than small ones. Then divide by the sum of all of them, so the results are fractions that add up to 1. The largest score gets the largest probability.

An example: scores of 8, 6 and 2 become about 88%, 12% and 0.2%. The gap between 8 and 6 is only 2, but $e^2 \approx 7.4$, so the first probability is about 7.4 times the second. Softmax exaggerates differences between scores. No probability is ever exactly zero.

◆ THE SAME THING IN NUMPY

>_ RELU, A NUMERICALLY SAFE SOFTMAX, AND THE FORWARD PASS

```
def relu(z):
    return np.maximum(0.0, z)

def softmax(z):
    # subtract each row's max before exp: same result, no overflow on big
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1          # (N,2)@(2,8) → (N,8)
    a1 = relu(z1)             # (N,8)
    z2 = a1 @ W2 + b2         # (N,8)@(8,2) → (N,2)
    probs = softmax(z2)       # (N,2), each row sums to 1
    return z1, a1, z2, probs  # keep z1 and a1: the backward pass needs t
```

Follow the shapes in the comments. `X @ W1` is (N, 2) times (2, 8), which gives (N, 8). Then `a1 @ W2` is (N, 8) times (8, 2), which gives (N, 2). The inner numbers match at every step (the shape rule from Section 1). The output is one pair of probabilities per example. We also return `z1` and `a1`, because the backward pass reuses these values instead of computing them again.

CHECK YOURSELF 3

Why does subtracting each row's maximum before `np.exp` not change the softmax result?

(d) The loss: cross-entropy

The loss is the “measure” step: one number that says how wrong the predictions are. **Cross-entropy** (from Section 4) looks only at the probability the network gave to the *correct* class. It penalizes that probability for being less than 1.

◆ NOW THE MATH

$$L = -\frac{1}{N} \sum_{i=1}^N \log(p_{i, y_i})$$

In words: for each example i , find the probability that the model gave to the *true* class y_i . Take its log, change the sign, and average over all examples. If the model is confident and right (

$p \approx 1$), then $\log 1 = 0$: no penalty. If it is confident and wrong ($p \approx 0$), then $-\log p$ becomes very large: a large penalty. Cross-entropy rewards being correct *and* sure.

◆ THE SAME THING IN NUMPY

>_ AVERAGE CROSS-ENTROPY OVER THE BATCH

```
def cross_entropy(probs, y):  
    # pick out prob of the correct class for each row, then -log, then average  
    correct = probs[np.arange(len(y)), y] # shape (N,)   
    return -np.mean(np.log(correct + 1e-12)) # +1e-12 guards log(0)
```

The tiny `1e-12` is a lower limit. It stops `log(0)` from becoming `-inf` if a probability is rounded to exactly zero. It changes the loss by a negligible amount, but it keeps the run from failing.

☂ EVERYDAY PARALLEL

Cross-entropy is like an exam that also scores your *confidence*. Mark the right answer and say “I am certain”: full marks. Mark the right answer but say “maybe”: partial marks. Mark the *wrong* answer and say “I am certain”: a very large penalty. So the network learns to be right, and also to be sure only when it is right.

(e) The backward pass

Now the central part. The **backward pass** is the “find the direction” step: it computes the gradient of the loss with respect to every weight. Every line is one use of the local-gradient rule from Section 10. We go backward through the same operations that the forward pass ran.

◆ NOW THE MATH

Notation. For any array $\mathbf{M}$ in the network, the code name `dM` means the gradient $\partial L / \partial \mathbf{M}$, with the same shape as $\mathbf{M}$. So `dz2` is $\partial L / \partial \mathbf{Z}_2$ and `dW1` is $\partial L / \partial \mathbf{W}_1$. As in Section 10, the gradient at a pre-activation is also called δ .

Step 1: the gradient at the logits. Softmax followed by cross-entropy has a very simple gradient. Let's derive it for one example. Call its logits $\mathbf{z}$, its probabilities $\mathbf{p} = \text{softmax}(\mathbf{z})$, its one-hot label $\mathbf{y}$, and its correct class c . Its loss is $-\log p_c$. Write out $\log p_c$ with the softmax formula:

$$\ell = -\log p_c = -\log \frac{e^{z_c}}{\sum_k e^{z_k}} = -z_c + \log \sum_k e^{z_k}$$

$$\frac{\partial \ell}{\partial z_j} = -y_j + \frac{e^{z_j}}{\sum_k e^{z_k}} = p_j - y_j$$

In words: the first term, $-z_c$, depends on z_j only when $j = c$. Its derivative is -1 for the correct class and 0 for the others, which is $-y_j$. The second term is the log of a sum. By the chain rule its derivative is e^{z_j} divided by the sum, which is p_j . So the gradient is *predicted probability minus one-hot truth*. We call the loss of one example ℓ here; L is the average of ℓ over the batch.

The full loss L averages over N examples, so each row gets a factor $1/N$. In matrix form:

$$\frac{\partial L}{\partial \mathbf{Z}_2} = \frac{1}{N} (\mathbf{P} - \mathbf{Y})$$

In words: if the network said 0.7 for a class whose true value is 1 , that entry is $0.7 - 1 = -0.3$ (before the $1/N$). The negative sign means “increase this logit.” Where the prediction is right, prediction minus truth is near zero, and the logit changes very little. You will see this gradient in almost every classifier.

Step 2: back through the layers. From [dz2](#) we pass the gradient back through each earlier operation. Each step pairs with one forward step and uses the rules from Section 10. First, a weight gradient. With the (in, out) layout, $\mathbf{Z}_2 = \mathbf{A}_1 \mathbf{W}_2 + \mathbf{b}_2$ means $Z_{ik} = \sum_j A_{ij} W_{jk} + b_k$ for example i . The weight W_{jk} is used by *every* example, so its gradient is a sum over examples:

$$\frac{\partial L}{\partial W_{jk}} = \sum_i A_{ij} \frac{\partial L}{\partial Z_{ik}} \implies \frac{\partial L}{\partial \mathbf{W}_2} = \mathbf{A}_1^\top \frac{\partial L}{\partial \mathbf{Z}_2}$$

In words: for one example, this is Section 10's rule “ δ times the input.” The matrix product $\mathbf{A}_1^\top (\cdot)$ does the sum over the examples i . The bias b_k is also added to every example's row, so its gradient is the column sum of $\partial L / \partial \mathbf{Z}_2$.

The same reasoning gives all six gradients. Here $1[\mathbf{Z}_1 > 0]$ is a mask: 1 where the entry of $\mathbf{Z}_1$ is positive, 0 elsewhere. And $\odot$ means “multiply matching entries.”

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{W}_2} &= \mathbf{A}_1^\top \frac{\partial L}{\partial \mathbf{Z}_2} & \frac{\partial L}{\partial \mathbf{b}_2} &= \sum_i \left(\frac{\partial L}{\partial \mathbf{Z}_2} \right)_i \\ \frac{\partial L}{\partial \mathbf{A}_1} &= \frac{\partial L}{\partial \mathbf{Z}_2} \mathbf{W}_2^\top & \frac{\partial L}{\partial \mathbf{Z}_1} &= \frac{\partial L}{\partial \mathbf{A}_1} \odot 1[\mathbf{Z}_1 > 0] \\ \frac{\partial L}{\partial \mathbf{W}_1} &= \mathbf{X}^\top \frac{\partial L}{\partial \mathbf{Z}_1} & \frac{\partial L}{\partial \mathbf{b}_1} &= \sum_i \left(\frac{\partial L}{\partial \mathbf{Z}_1} \right)_i \end{aligned}$$

In words, line by line: the gradient for $\mathbf{W}_2$ is the layer's input $\mathbf{A}_1$ (transposed) times the incoming gradient. The gradient for $\mathbf{b}_2$ is the incoming gradient summed over the batch (the rows i). To pass the gradient back into the hidden layer, multiply by $\mathbf{W}_2$ transposed; this gives $\mathbf{da}_1$. On the way forward, ReLU passed only positive values. So on the way back it passes gradient only where $\mathbf{Z}_1 > 0$; the mask does this. Its local gradient is 1 there and 0 elsewhere. Then $\mathbf{W}_1$ and $\mathbf{b}_1$ follow the same two rules as the second layer.

◆ THE SAME THING IN NUMPY

>_ THE BACKWARD PASS, ONE LINE PER OPERATION

```
def backward(X, y, z1, a1, probs, W2):
    N = len(y)
    Y = np.zeros_like(probs)
    Y[np.arange(N), y] = 1.0

    dz2 = (probs - Y) / N           # (N,2) softmax + cross-entropy: (P -
    dW2 = a1.T @ dz2                # (8,N)@(N,2) → (8,2) same shape as
    db2 = dz2.sum(axis=0)           # (2,) same shape as

    da1 = dz2 @ W2.T               # (N,2)@(2,8) → (N,8)
    dz1 = da1 * (z1 > 0)           # (N,8) ReLU mask: 0 where z1 ≤ 0
    dW1 = X.T @ dz1                # (2,N)@(N,8) → (2,8) same shape as
    db1 = dz1.sum(axis=0)           # (8,) same shape as

    return dW1, db1, dW2, db2
```

Use the shape comments as a checklist. Every gradient has the same shape as the array it updates: `dW2` is (8, 2), like `W2`, and so on. When your own backward pass has a bug, check this first. A gradient whose shape does not match its weight means a transpose is in the wrong place.

▲ WATCH OUT

The ReLU mask uses `z1`, the value *before* the bend. For plain ReLU, `(z1 > 0)` and `(a1 > 0)` give the same mask, so either works here. The rule to remember is “gradient flows only where the neuron was active.” Also keep the `/ N` in `dz2`. It must match the $1/N$ average in the loss. Without it, your gradients are N times too large and the steps are too big.

CHECK YOURSELF 4

One row of `probs` is `[0.9, 0.1]`, its label is 0, and the batch has $N = 4$ examples. What is that row of `dz2`? Which logit will the update push up?

CHECK YOURSELF 5

Why does `db2` sum over `axis=0`, and what would go wrong with `axis=1`?

(f) The update: gradient descent

This is the “step” part of the loop. For every weight we now know the direction that *increases* the loss (the gradient points that way). So we step the opposite way: we subtract a small fraction of the gradient. That fraction is the **learning rate**.

◆ NOW THE MATH

$$\mathbf{W} \leftarrow \mathbf{W} - \alpha \frac{\partial L}{\partial \mathbf{W}}$$

In words: replace each weight with itself minus the learning rate α (alpha) times its gradient. The gradient points uphill. The minus sign turns us downhill, toward a smaller loss. α sets the step size. If it is too small, learning is very slow. If it is too large, we jump past the minimum and bounce around. This update, repeated, is *gradient descent*.

◆ THE SAME THING IN NUMPY

>_ ONE GRADIENT-DESCENT STEP, IN PLACE

```
def gd_step(params, grads, lr):
    for p, g in zip(params, grads):
        p -= lr * g          # in place: move each weight downhill
```

Our dataset has only four points, so every step uses all of them. This is called **full-batch** gradient descent. Real datasets are too large for that. There, each step uses a small random group of examples (a *mini-batch*). The randomness gives the name **stochastic** gradient descent (SGD). The update rule is the same; only the data in each step changes.

(g) The epoch loop

Now we connect the four steps and run them a few thousand times. This is the loop from Section 2, in code:

>_ PREDICT → MEASURE → FIND THE DIRECTION → STEP, REPEATED

```
lr = 0.5
for epoch in range(2001):
    # 1. PREDICT (forward pass)
    z1, a1, z2, probs = forward(X, W1, b1, W2, b2)

    # 2. MEASURE (loss)
    loss = cross_entropy(probs, y)

    # 3. FIND THE DIRECTION (backward pass)
    dW1, db1, dW2, db2 = backward(X, y, z1, a1, probs, W2)

    # 4. STEP (update)
    gd_step([W1, b1, W2, b2], [dW1, db1, dW2, db2], lr)

    if epoch % 400 == 0:
        print(f"epoch {epoch:4d}  loss {loss:.4f}")
```

With seed 42, the loss falls toward zero:

>_ OUTPUT

```
epoch    0  loss 0.8471
epoch  400  loss 0.0059
epoch  800  loss 0.0022
epoch 1200  loss 0.0012
epoch 1600  loss 0.0008
epoch 2000  loss 0.0006
```

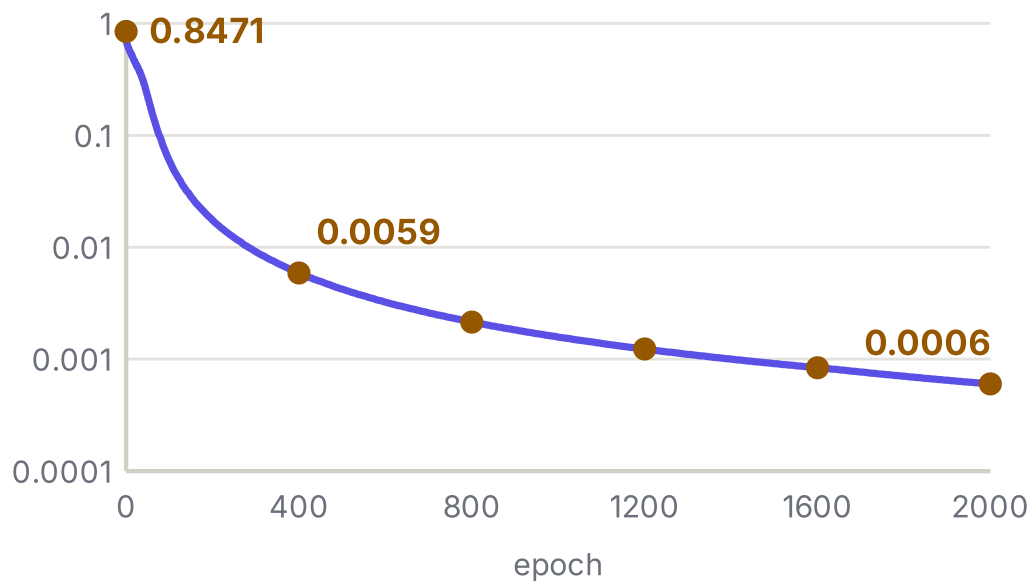



Figure 11.3 The loss at every epoch of the run above, on a log scale. The amber dots are the printed values. Most of the drop happens in the first 200 epochs. After that, the loss keeps shrinking slowly as the network becomes more and more sure of answers that are already correct.

Now the result: compare the final predictions with the truth.

>_ FINAL PREDICTIONS (CONTINUES THE LOOP ABOVE)

```
pred = forward(X, W1, b1, W2, b2)[-1].argmax(axis=1)
print(pred)                    # [0 1 1 0]
print("accuracy:", (pred == y).mean()) # accuracy: 1.0
```

◆ THE RESULT

[0 1 1 0]: all four XOR points are correct, 100% accuracy. A single linear layer *cannot* do this, however long you train it, because no straight line separates the classes. The hidden layer with its ReLU bend can. It learned this by itself, only by running the four-step loop about two thousand times. That is the central idea of deep learning, in code you can read from top to bottom.

CHECK YOURSELF 6

Suppose all weights and biases start at exactly 0. What are `dw1` and `dw2` on the first step? What happens during training?

CHECK YOURSELF 7

If you delete the ReLU line, so that `a1 = z1` with no bend between the layers, can the network still solve XOR?

4 The complete script

Here is everything above in one file. It needs only NumPy. Copy it into a `.py` file and run it: you will see the loss fall and the network reach 100% on XOR. You have already seen every line.

```
>_ MLP_XOR.PY: A 2-LAYER NETWORK, FROM SCRATCH
```

```

import numpy as np

# ---- 1. data -----
X = np.array([[0.0, 0.0],
              [0.0, 1.0],
              [1.0, 0.0],
              [1.0, 1.0]])
y = np.array([0, 1, 1, 0])      # XOR: 1 when the inputs differ
N, D = X.shape                  # 4 examples, 2 features
C = 2                           # two classes: "0" and "1"

# ---- 2. weights (small random; biases zero); shapes are (in, out) -----
rng = np.random.default_rng(42)
H = 8                           # hidden neurons
W1 = rng.normal(0, np.sqrt(2.0 / D), size=(D, H))    # (2, 8)
b1 = np.zeros(H)                      # (8,)
W2 = rng.normal(0, np.sqrt(2.0 / H), size=(H, C))    # (8, 2)
b2 = np.zeros(C)                      # (2,)

# ---- 3. building blocks -----
def relu(z):
    return np.maximum(0.0, z)

def softmax(z):
    z = z - z.max(axis=1, keepdims=True)      # stability, same res
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1                        # (N, 8)
    a1 = relu(z1)                          # (N, 8)
    z2 = a1 @ W2 + b2                      # (N, 2) logits
    probs = softmax(z2)                   # (N, 2) each row sums to 1
    return z1, a1, z2, probs

def cross_entropy(probs, y):
    correct = probs[np.arange(len(y)), y]
    return -np.mean(np.log(correct + 1e-12))

def backward(X, y, z1, a1, probs, W2):

```

```

N = len(y)
Y = np.zeros_like(probs)
Y[np.arange(N), y] = 1.0
dz2 = (probs - Y) / N           # (N, 2) softmax + cross-entropy
dW2 = a1.T @ dz2                # (8, 2)
db2 = dz2.sum(axis=0)           # (2,)
da1 = dz2 @ W2.T                # (N, 8)
dz1 = da1 * (z1 > 0)            # (N, 8) ReLU mask
dW1 = X.T @ dz1                 # (2, 8)
db1 = dz1.sum(axis=0)           # (8,)
return dW1, db1, dW2, db2

# ---- 4. train -----
lr = 0.5
for epoch in range(2001):
    z1, a1, z2, probs = forward(X, W1, b1, W2, b2)           # 1. predict
    loss = cross_entropy(probs, y)                             # 2. measure
    dW1, db1, dW2, db2 = backward(X, y, z1, a1, probs, W2)   # 3. find th
    for p, g in zip([W1, b1, W2, b2], [dW1, db1, dW2, db2]):
        p -= lr * g                                           # 4. step
    if epoch % 400 == 0:
        print(f"epoch {epoch:4d}  loss {loss:.4f}")

# ---- 5. results -----
pred = forward(X, W1, b1, W2, b2)[-1].argmax(axis=1)
print("predictions:", pred, " truth:", y)
print("accuracy:    ", (pred == y).mean())

```

>_ OUTPUT

```

epoch    0  loss 0.8471
epoch  400  loss 0.0059
epoch  800  loss 0.0022
epoch 1200  loss 0.0012
epoch 1600  loss 0.0008
epoch 2000  loss 0.0006
predictions: [0 1 1 0]  truth: [0 1 1 0]
accuracy:    1.0

```

▲ WATCH OUT

The seed in `np.random.default_rng(42)` makes the run reproducible: you get the loss values shown above. With a different seed, the starting weights change and so does the loss curve. Most seeds still reach 100%: in our test, 39 of the seeds 0 to 39 did. Seed 10 did not. It stalls at a loss of about 0.4775 and gets one point wrong. The reason: three hidden neurons have `z1 ≤ 0` on all four inputs from the start, and a fourth reaches that state during training. These are dead ReLUs (Section 9): their gradient is zero, so they never recover. The other four neurons are active only for the input (1, 0). If a run stalls, try another seed, or use Leaky ReLU (with seed 10 and $\alpha = 0.01$, it classifies all four points after 2000 epochs). Section 12 shows how initialization and learning rate decide whether training succeeds.

CHECK YOURSELF 8

In the stalled seed-10 run, four hidden neurons are dead and the other four are active only for the input (1, 0). Why must the network give the same answer for (0, 0), (0, 1) and (1, 1)? Why can't gradient descent fix this?

5 What to carry forward

- A network learns with one loop of four steps: **predict** → **measure** → **find the direction** → **step**, repeated.
- The **forward pass** is matrix multiplies with a nonlinearity between them. The ReLU bend is the reason two layers can do more than one. Remove it and the layers collapse into a single linear layer.
- **Softmax** turns raw scores (logits) into probabilities. **Cross-entropy** scores those probabilities against the one-hot truth.
- The **backward pass** is Section 10's chain rule, one line per operation. Softmax plus cross-entropy has the gradient $(\mathbf{P} - \mathbf{Y})/N$ at the logits. Every gradient has the same shape as the array it updates.
- In this lesson, weights have shape (in, out) and a layer computes `X @ W + b`; PyTorch stores the transpose.
- Weights start **small and random**. All zeros would never break the symmetry. Gradient descent then moves them downhill by the learning rate at each step.

You have now written by hand everything a deep-learning framework does for a network like this. In Section 12 we make training reliable: initialization, optimizers and learning-rate schedules. Then in Part 4 we give these same steps to **PyTorch**, which computes the backward pass for us automatically. When it does, you will know what it is doing.

Answers to “Check yourself”

1. Suppose you add a third input $x_3 = x_1 \cdot x_2$ to each XOR point. Can a single linear layer now separate the classes?

Yes. The label equals $x_1 + x_2 - 2x_3$: this gives 0, 1, 1, 0 for the four points. So a linear model with threshold 0.5 on this sum is correct. The new feature did the “bending” for us. A hidden layer does the same thing, but it *learns* useful features from the data instead of needing you to invent them.

2. You run a training loop and the printed loss is exactly the same number at every epoch. The update line is `for p, g in zip(params, grads): p = p - lr * g`. What is wrong?

Step 4 never happens. `p = p - lr * g` creates a *new* array and gives it the local name `p`. The original weight array is not changed. So every epoch predicts with the same weights and prints the same loss (0.8471 for our network). The fix is the in-place form `p -= lr * g`, which changes the array itself.

3. Why does subtracting each row's maximum before `np.exp` not change the softmax result?

Subtracting a constant c from every score multiplies every e^{z_j} by the same factor e^{-c} : $e^{z_j - c} = e^{z_j} e^{-c}$. The factor appears in the numerator and in the sum in the denominator, so it cancels. The result is the same, but the largest exponent is now $e^0 = 1$, so nothing overflows. Without the shift, a logit of 1000 would give `inf`.

4. One row of `probs` is [0.9, 0.1], its label is 0, and the batch has $N = 4$ examples. What is that row of `dz2`? Which logit will the update push up?

The one-hot row is [1, 0]. So the row is $([0.9, 0.1] - [1, 0])/4 = [-0.025, 0.025]$. Gradient descent moves against the gradient, so it pushes logit 0 (the correct class) up and logit 1 down. The numbers are small because the prediction is already mostly right.

5. Why does `db2` sum over `axis=0`, and what would go wrong with `axis=1`?

Axis 0 runs over the examples (rows). The same bias `b2` is added to every example's row, so it affects the loss through all N rows. By the sum rule, its gradient adds up the contributions of all rows. The result has shape (2,), like `b2`. With `axis=1` you would sum over the two classes instead and get shape (4,): one number per example. That does not match `b2`, and it mixes the gradients of different classes.

6. Suppose all weights and biases start at exactly 0. What are `dW1` and `dW2` on the first step? What happens during training?

Both are exactly zero. With zero weights, `z1` and `a1` are all 0. So `dW2 = a1.T @ dz2 = 0`. Also `da1 = dz2 @ W2.T = 0`, because `W2` is zero, so `dW1 = 0` too. (The ReLU mask is also 0 everywhere, since no `z1` is positive.) The logits are 0, so each row of `probs` is [0.5, 0.5]. XOR has two examples of each class, so even `db2` sums to 0. Every gradient is zero, and the network never changes. This is the symmetry problem at its most extreme.

7. If you delete the ReLU line, so that $a1 = z1$ with no bend between the layers, can the network still solve XOR?

No. Without the bend, the two layers compute $(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$. This multiplies out to $\mathbf{X}(\mathbf{W}_1\mathbf{W}_2)$ plus a constant: one matrix times $\mathbf{X}$, plus a shift. This is the collapse from Section 9. The two layers act as one linear layer, and one linear layer cannot separate XOR. The network would stop at an accuracy of at most 75%.

8. In the stalled seed-10 run, four hidden neurons are dead and the other four are active only for the input (1, 0). Why must the network give the same answer for (0, 0), (0, 1) and (1, 1)? Why can't gradient descent fix this?

For those three inputs, every hidden neuron has $z1 \leq 0$, so $a1$ is all zeros. Then $z2 = b2$ for all three, and they get the same probabilities (about [0.67, 0.33] here). But (0, 1) has label 1 and the others have label 0, so one of them must be wrong. Gradient flows through a neuron only on inputs where it is active. The dead neurons get no gradient at all. The live ones get gradient only from (1, 0), which is already correct, so nothing pushes them to become active on (0, 1).

Training Dynamics & Optimizers

The same network can learn well or fail completely, depending on how you start it and how you step. Here is why.

60 min read

YOU'LL LEARN

- weight initialization (Xavier/He)
- learning rates
- SGD, momentum, Adam
- LR schedules

Before you start: Section 11: Build an MLP in NumPy.

You have a network. You have a loss (a single number that says how wrong the network is). You have backpropagation to tell you which way each weight should move. So training is done, right? Not quite. Two choices decide whether the loop converges in a minute or never converges. They are the numbers you **start** the weights at, and the size of the **steps** you take. This section is about those two knobs. They turn a working design into a working model.

◆ THE ONE IDEA TO HOLD ONTO

Backprop tells you the *direction* to move each weight. It says nothing about *where to begin* or *how far to step*. Those two decisions (initialization and the optimizer) are what “training dynamics” means. If you get them wrong, a network that is otherwise correct will not learn.

1 Initialization: where the weights start

◆ INTUITION FIRST

Before the first step of training, every weight needs a starting value. It feels like this should not matter much, because training will fix the weights anyway. But the starting point controls two things. First, it decides whether the neurons can ever become *different* from each other. Second, it decides whether the signal can travel through a deep stack of layers without dying out or blowing up. If either goes wrong, the network learns slowly, badly, or not at all.

☂ EVERYDAY PARALLEL

Imagine a choir where every singer gets the same sheet music and the same instructions from the conductor. However long they rehearse, they all sing the same note. A choir of 40 then sounds like one loud singer. To get harmony, the singers must start on different parts. A layer of neurons is the same: the neurons must start *different* to end up doing different jobs.

Why not all zeros: the symmetry problem

◆ NOW THE MATH

The tempting “safe” choice is to set every weight to zero. Let us prove why this breaks the network. Take one layer. Each neuron j computes a weighted sum of the same inputs $\mathbf{x}$. Then it passes the sum through an activation function ϕ (a simple nonlinear function such as ReLU or tanh):

$$a_j = \phi \left(\sum_i w_{ji} x_i + b_j \right)$$

In words: to get neuron j 's output, multiply each input x_i by its weight w_{ji} , add them up, add the bias b_j , then apply ϕ . The weight w_{ji} connects input i to neuron j .

Now give every neuron in the layer the same weights and bias (for example, all zero). Then the sum inside is the same number for every neuron j . So every neuron produces the *same output*:

$$w_{ji} = c \text{ for all } j, i \Rightarrow a_1 = a_2 = \dots = a_n$$

That is already bad. The real problem is what backprop does next. From Section 10, the gradient for a weight is the error signal at the neuron times the input:

$$\frac{\partial L}{\partial w_{ji}} = \delta_j x_i, \quad \delta_j = \phi'(z_j) \sum_k w_{kj}^{\text{next}} \delta_k^{\text{next}}$$

In words: δ_j is the gradient of the loss with respect to neuron j 's pre-activation z_j (the sum before ϕ). It collects the error signals δ_k^{next} from the next layer, weighted by the *outgoing* weights w_{kj}^{next} .

If the outgoing weights are also identical for every j , then every neuron gets the same z_j and the same sum. So every δ_j is equal, and every weight gets the same gradient:

$$\frac{\partial L}{\partial w_{1i}} = \frac{\partial L}{\partial w_{2i}} = \dots = \frac{\partial L}{\partial w_{ni}}$$

In words: $\partial L / \partial w_{ji}$ means "how much the loss L changes if we nudge weight w_{ji} ." When every neuron is a copy, these numbers are all equal. So the optimizer moves every copy by the same amount.

Follow the consequence. The weights start equal and always move by the same amount, so they stay equal *forever*. A layer of 100 neurons behaves like a layer of *one*. This is the **symmetry problem**: nothing in the update rule can break the tie.

All zeros is even worse than "all equal." The outgoing weights are zero, so every δ_j in the hidden layers is zero. The inputs to the last layer are $\phi(0)$, which is 0 for ReLU. So in an all-zero ReLU network every weight gradient is exactly 0. Only the biases can move, and the network cannot start learning.

◆ THE FIX

Break the symmetry by starting the weights at *small random* numbers. Now each neuron sees a different weighted sum and receives a different gradient. So each neuron is free to specialize. Random initialization is not optional. It is what lets a wide layer be more useful than a single neuron. (Biases can start at zero; the random weights already break the tie.)

CHECK YOURSELF 1

You set every weight in a tanh layer to 0.5 (not zero). Is the symmetry problem solved?

Why not too large, and not too small

◆ INTUITION FIRST

So the weights must be random. But *how big*? This is the second job of initialization. A deep network passes the signal through many layers in a row. If each layer *magnifies* the signal a little, the effect compounds and the numbers explode. If each layer *shrinks* it a little, the signal fades to nothing before it reaches the end. Either way, learning stalls.

🌂 EVERYDAY PARALLEL

Think of 50 audio amplifiers connected in a row, each feeding the next. If each one has a gain of 1.1, the sound at the end is $1.1^{50} \approx 117$ times louder. If each has a gain of 0.9, only $0.9^{50} \approx 0.5\%$ of the sound is left. Only a gain very close to 1 keeps the sound at a normal level. Initialization sets that gain for each layer of the network.

This is the eigenvalue story from Section 1. Passing a signal through a stack of layers is repeated multiplication by weight matrices. Repeated multiplication grows or decays depending on whether each matrix stretches by more or less than 1. We want each layer to stretch by *about* 1, so the signal survives all the way through. Figure 12.1 shows what happens in a real 20-layer ReLU network.

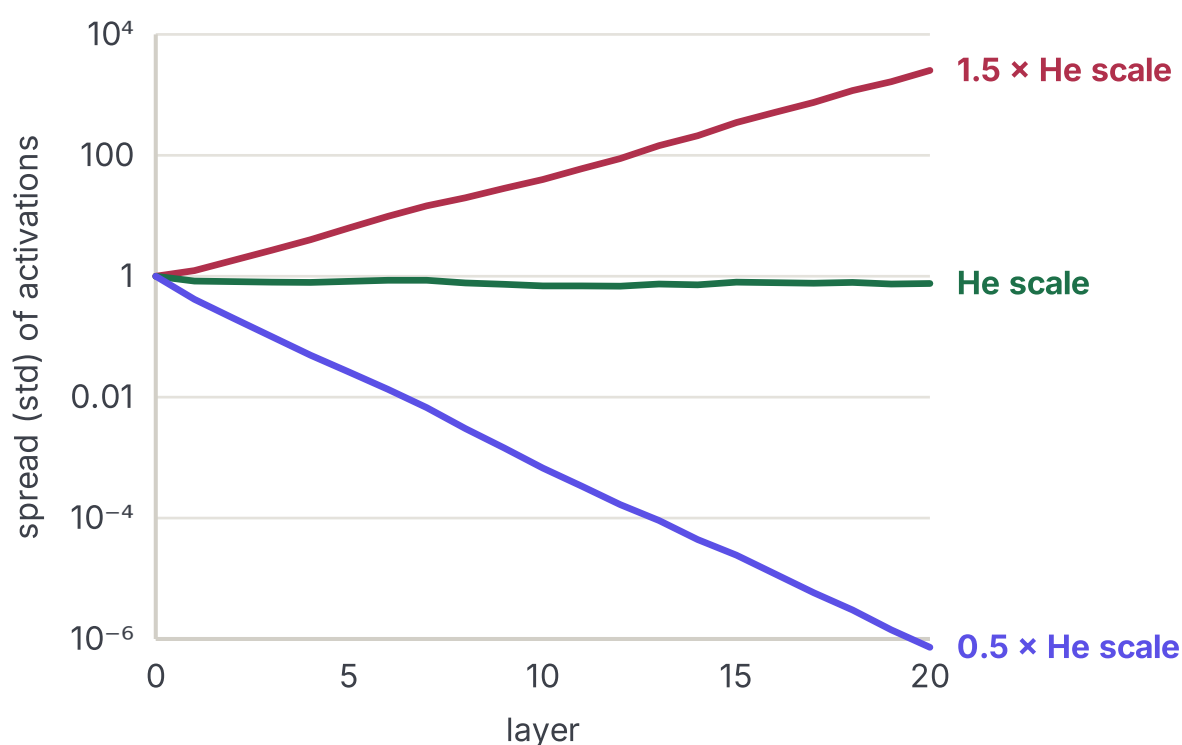


Figure 12.1 A 20-layer ReLU network with 256 neurons per layer and 1,000 random inputs (seed 0). With He scale, the spread of the activations stays near 0.8 at every layer. With half that weight scale, it shrinks by about $2\times$ per layer (to 10^{-6}). With $1.5\times$ the scale, it grows by about $1.5\times$ per layer (to about 2,500). The y-axis is logarithmic.

Deriving the right scale: variance preservation

◆ NOW THE MATH

Let us turn “stretch by about 1” into a formula. Consider one neuron's pre-activation z (the weighted sum before the activation function):

$$z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$$

In words: z is the sum over all n_{in} incoming connections of weight times input. n_{in} (the *fan-in*) is the number of inputs that feed this neuron.

We want z to have about the same *spread* as the inputs. We measure spread with the **variance** (the average squared distance of the values from their mean). Assume the weights and the inputs are independent and have mean zero. Then the variance of one term is:

$$\text{Var}(wx) = \mathbb{E}[w^2 x^2] - (\mathbb{E}[wx])^2 = \mathbb{E}[w^2] \mathbb{E}[x^2] - 0 = \text{Var}(w) \text{Var}(x)$$

The variances of independent terms add, and there are n_{in} terms:

$$\text{Var}(z) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$$

In words: each of the n_{in} terms adds $\text{Var}(w) \text{Var}(x)$. So one layer multiplies the input variance by the factor $n_{\text{in}} \text{Var}(w)$.

For the output spread to equal the input spread, that factor must be 1:

$$n_{\text{in}} \text{Var}(w) = 1 \implies \boxed{\text{Var}(w) = \frac{1}{n_{\text{in}}}}$$

In words: draw each weight from a distribution whose variance is 1 divided by the number of inputs. A neuron with more inputs must start with *smaller* weights, because it adds up more terms. Every modern init scheme starts from this line.

Xavier (Glorot) initialization notes that the signal flows *two* ways: forward (the activations) and backward (the gradients). The same argument on the backward pass sums over the n_{out} neurons that this one feeds (the *fan-out*). So the forward pass wants $\text{Var}(w) = 1/n_{\text{in}}$ and the backward pass wants $\text{Var}(w) = 1/n_{\text{out}}$. Glorot's compromise uses the average of the two fan numbers:

$$\text{Var}(w) = \frac{1}{\frac{1}{2}(n_{\text{in}} + n_{\text{out}})} = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

In words: replace n_{in} by the average of the fan-in and the fan-out. When the layer has as many outputs as inputs, this is the same as $1/n_{\text{in}}$. Xavier assumes an activation that is symmetric around zero and close to the identity near zero, such as $\tanh$.

He (Kaiming) initialization adapts this for ReLU. $\text{ReLU}(\max(0, z))$ sets every negative input to zero. That changes the step above, because the inputs to the next layer are no longer zero-mean. Let $x = \text{ReLU}(z)$, with z symmetric around zero. Half of the values of z are negative and become 0. So the second moment (the mean of the squares) is halved:

$$\mathbb{E}[x^2] = \frac{1}{2} \mathbb{E}[z^2] = \frac{1}{2} \text{Var}(z)$$

The next layer's weights still have mean zero, so for its pre-activation $z' = \sum_i w_i x_i$ we get $\text{Var}(w_i x_i) = \mathbb{E}[w_i^2] \mathbb{E}[x_i^2]$, and:

$$\text{Var}(z') = n_{\text{in}} \text{Var}(w) \mathbb{E}[x^2] = \frac{1}{2} n_{\text{in}} \text{Var}(w) \text{Var}(z)$$

Setting the factor $\frac{1}{2} n_{\text{in}} \text{Var}(w)$ to 1 gives:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}}$$

In words: ReLU halves the mean of the squares at each layer. The factor of 2 in the weight variance cancels that halving, so the spread stays the same from layer to layer. This is why the numerator is 2, not 1.

DEFINITION

Use **He (Kaiming)** initialization with **ReLU** and its relatives. Use **Xavier (Glorot)** with **tanh**. Sigmoid is not centered at zero. Xavier is still the usual choice for it, but there the derivation holds only roughly. Both schemes come in two versions. The “normal” version draws from a bell curve with the target variance. The “uniform” version draws from a flat range with the same variance. The difference rarely matters.

◆ THE SAME THING IN PYTORCH

>_ HE INIT FOR A RELU LAYER, COMPARED WITH THE PYTORCH DEFAULT

```

import torch
import torch.nn as nn

torch.manual_seed(0)
layer = nn.Linear(256, 128)  # fan_in = 256, fan_out = 128
print(layer.weight.var().item())  # 0.001297...  default: 1/(3*256) = 0.001297...

# He / Kaiming init, the right choice when a ReLU follows this layer
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')
nn.init.zeros_(layer.bias)  # biases can start at zero
print(layer.weight.var().item())  # 0.007741...  target: 2/256 = 0.0078125

# For a tanh layer you would use Xavier instead:
# nn.init.xavier_normal_(layer.weight)

```

Note the default. `nn.Linear` draws its weights uniformly from $[-1/\sqrt{n_{\text{in}}}, 1/\sqrt{n_{\text{in}}}]$. That gives $\text{Var}(w) = 1/(3n_{\text{in}})$, about 6× smaller than He. The default works well for shallow networks and for networks with normalization layers. In a deep plain ReLU stack, the signal shrinks layer by layer. Then you call `kaiming_normal_` by hand. You also use these functions when you build a custom layer.

▲ WATCH OUT

Two silent failures. **All-zero weights:** the symmetry trap above; the network never becomes better than one neuron per layer. **The wrong scale for a deep network:** if the weights are too big, the activations overflow and the loss becomes `NaN` within a few steps. If they are too small, the activations and gradients fade and the loss barely moves. If your loss goes to `NaN` at once, check the initialization and the learning rate (next) first.

CHECK YOURSELF 2

You build a 50-layer ReLU network with 256 inputs per layer, but you draw every weight from a standard normal (variance 1). Predict what happens in the forward pass.

2 The learning rate: how far to step

◆ INTUITION FIRST

Gradient descent moves each weight a little in the downhill direction that backprop found. The **learning rate** α (also called the *step size*) is the number that says *how big* “a little” is. It is usually the first hyperparameter to tune. If it is too big, the steps overshoot and the loss diverges. If it is too small, training crawls.

🚶 EVERYDAY PARALLEL

You walk downhill in thick fog to reach the bottom of a valley. You can only feel the slope under your feet. With *giant* strides you jump over the valley floor and land higher up the far slope. You bounce back and forth and never settle. With *tiny* steps you get there, but very slowly. The learning rate is the length of your stride.

◆ NOW THE MATH

The update rule for the whole parameter vector θ is:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

In words: the arrow means “replace the left side with the right side.” Take the current parameters θ and subtract the gradient $\nabla_{\theta} L$ (the uphill direction) times the learning rate α . Subtracting the uphill direction means stepping downhill. α sets the length of the step.

Why does a too-large α diverge? Take the simplest bowl, $L(\theta) = \frac{1}{2}k\theta^2$, where k is the curvature. Its gradient is $k\theta$, so one step gives:

$$\theta \leftarrow \theta - \alpha k \theta = (1 - \alpha k) \theta$$

In words: each step multiplies the distance to the minimum by $(1 - \alpha k)$. If $\alpha < 2/k$, the size of that factor is below 1 and θ shrinks toward 0. If $\alpha > 2/k$, the factor is below -1: each step jumps to the other side of the bowl and lands *farther* away. Then the loss grows every step.

▲ WATCH OUT

Learn the classic signs. Loss going to `NaN`, or rising: the learning rate is **too high**; divide it by 10. Loss falling very slowly, almost a flat line: it is **too low**; raise it. A good first guess for Adam is `1e-3`; for plain SGD, `1e-1` to `1e-2`. Tune the learning rate before you change anything else.

CHECK YOURSELF 3

On the bowl $L = \frac{1}{2}k\theta^2$ with $k = 10$, what happens with $\alpha = 0.15$? With $\alpha = 0.25$?

3 Optimizers: better ways to step

Plain gradient descent takes the raw gradient and steps. An **optimizer** is a rule that *reshapes* that step to make training faster and more stable. We will build up the three you meet most often (SGD, momentum and Adam). Each one fixes a weakness of the one before.

SGD and why we use minibatches

◆ INTUITION FIRST

The “true” gradient is the average gradient over the *entire* training set. With millions of examples, computing it for every step is very expensive. **Stochastic gradient descent (SGD)** takes a shortcut. (“Stochastic” means “random.”) It estimates the gradient from a small random group of examples, called a **minibatch**, and steps on that estimate. The estimate is noisy, but it is cheap. You can take hundreds of steps in the time that one full-dataset step would take.

☂ EVERYDAY PARALLEL

To learn how a national election is leaning, you do not phone all 50 million voters. You poll a random 1,000. The poll is a little off from the true number (that is the noise). But it asks 50,000 times fewer people, so you can repeat it every day. A minibatch is that poll: a small random sample that shows well enough which way to step.

◆ NOW THE MATH

SGD is the update from the last section, computed on a minibatch $\mathcal{B}$ instead of the whole set:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L_{\mathcal{B}}(\theta)$$

In words: the same downhill step as before, but the gradient $\nabla_{\theta} L_{\mathcal{B}}$ is measured on the current minibatch $\mathcal{B}$ only. It is a noisy but unbiased estimate of the true gradient. “Unbiased” means that its average over many batches is the true gradient.

The noise is not only a cost. Its small random kicks can push the optimizer out of shallow bad spots (poor local dips in the loss). A perfectly smooth gradient would stay there.

Momentum: remember where you were going

◆ INTUITION FIRST

Pure SGD knows only the slope *right now*. In a long narrow valley, it bounces from wall to wall instead of rolling along the floor. On flat patches it slows to a crawl. **Momentum** keeps a running memory of the recent gradients and steps in that accumulated direction. Directions that stay the same build up speed. Side-to-side wobbles cancel out.

☂ EVERYDAY PARALLEL

Roll a heavy bowling ball down a bumpy hill. A small dip or pebble does not stop it; its built-up speed carries it over. Random small bumps to the left and right mostly cancel, so it holds a steady line toward the bottom. That built-up “keep going this way” is what momentum adds to the raw gradient.

◆ NOW THE MATH

First, a tool we will use three times. An *exponential moving average (EMA)* of a sequence of values $g_1, g_2, \dots$ is a running average. It gives recent values more weight and lets old ones fade:

$$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta) g$$

In words: keep a fraction β of the old average and mix in a fraction $1 - \beta$ of the new value g . With $\beta = 0.9$, a value's weight shrinks by 10% per step, so the average “remembers” roughly the last $1/(1 - \beta) = 10$ values.

Momentum keeps a **velocity** vector $\mathbf{u}$. PyTorch's version is:

$$\begin{aligned}\mathbf{u} &\leftarrow \beta \mathbf{u} + \nabla_{\theta} L \\ \theta &\leftarrow \theta - \alpha \mathbf{u}\end{aligned}$$

In words: first line: the new velocity is β (typically 0.9) times the old velocity, *plus* the fresh gradient. Second line: step along the velocity, not the raw gradient. A steady downhill direction adds up step after step. Noisy sideways parts average toward zero.

This rule has no $(1 - \beta)$, so is it an EMA? Yes, up to a constant factor. Define $\bar{g} = (1 - \beta) \mathbf{u}$ and multiply the first line by $(1 - \beta)$:

$$(1 - \beta) \mathbf{u} \leftarrow \beta (1 - \beta) \mathbf{u} + (1 - \beta) \nabla_{\theta} L \quad \Longleftrightarrow \quad \bar{g} \leftarrow \beta \bar{g} + (1 - \beta) \nabla_{\theta} L$$

In words: the velocity $\mathbf{u}$ is the EMA of the gradients times $1/(1 - \beta)$. With $\beta = 0.9$ that factor is 10. So momentum steps along the average recent gradient. On a steady slope, its step is up to 10× larger than plain SGD with the same α .

CHECK YOURSELF 4

With momentum $\beta = 0.9$, the gradient is the same vector $\mathbf{g}$ at every step. What does the velocity $\mathbf{u}$ approach?

Adam: momentum plus a step size for every weight

◆ INTUITION FIRST

Momentum fixed the *direction*. A second problem remains: different weights need different step sizes. Some weights have large gradients and some have tiny ones. With one global learning rate, the first group jumps too far or the second group barely moves. **Adam** (“adaptive moment estimation”) solves both problems. It uses momentum for the direction, and it gives every parameter its own step size, tuned automatically. It is the default optimizer in much of deep learning.

🕒 EVERYDAY PARALLEL

A hiker walks down a mountain. She remembers the general downhill direction she has been walking, so she keeps her momentum. She also measures the ground under each step. Where the ground rises and falls sharply, she shortens her stride. Where it is gentle, she lengthens it. So her stride is about the same length everywhere, whatever the terrain. Adam does this for every weight.

◆ NOW THE MATH

Adam keeps *two* EMAs of the gradient $\mathbf{g} = \nabla_{\theta} L$. Let t be the step count, starting at 1. The **first moment** $\mathbf{m}$ is an EMA of the gradient itself. It tracks *which way* to go:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \mathbf{g}$$

In words: mix the running direction $\mathbf{m}$ with the fresh gradient. β_1 is usually 0.9. This is the momentum half: the smoothed direction.

The **second moment** $\mathbf{v}$ is an EMA of the gradient *squared* (each component squared, elementwise). It tracks how large each parameter's gradients have been:

$$\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) \mathbf{g}^2$$

In words: mix the running squared gradient v with the fresh squared gradient. β_2 is usually 0.999 (a very long memory). A parameter with large gradients builds a large v ; a parameter with small gradients has a small v .

There is a cold-start problem. Both m and v start at zero. So in early steps they are biased toward zero (too small). The **bias correction** scales them back up:

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \quad \hat{v} = \frac{v}{1 - \beta_2^t}$$

In words: divide by $1 - \beta^t$. At step $t = 1$ the divisor is small (0.1 for β_1 , 0.001 for β_2). So it scales the small early estimates back up. As t grows, β^t goes to 0 and the divisor goes to 1. For m this takes a few dozen steps ($1 - 0.9^{12} \approx 0.72$). For v it takes thousands ($1 - 0.999^{12} \approx 0.012$, $1 - 0.999^{1000} \approx 0.63$).

Finally, the update: the direction $\hat{m}$, divided by the per-parameter scale $\sqrt{\hat{v}}$:

$$\theta \leftarrow \theta - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

In words: step in the smoothed direction $\hat{m}$, but divide each parameter's step by the square root of its own $\hat{v}$. ϵ (about 10^{-8}) is a tiny number that prevents division by zero. If a gradient is steady at some value g , then $\hat{m} \approx g$ and $\sqrt{\hat{v}} \approx |g|$. So the step is about α , whatever the size of g . Adam removes the gradient's scale and keeps its direction.

◆ ADAM IN ONE BREATH

m is a smoothed *direction* (momentum). $\sqrt{v}$ is a per-parameter *scale* that Adam divides out, so every step is roughly α long. **Bias correction** fixes the too-small early estimates. Together they give an optimizer that usually works at learning rate **1e-3** without tuning each layer. That is why it is a common default.

Figure 12.2 shows the three optimizers on a narrow valley. The loss is $L = \frac{1}{2}(x^2 + 25y^2)$: steep across (y), gentle along (x).

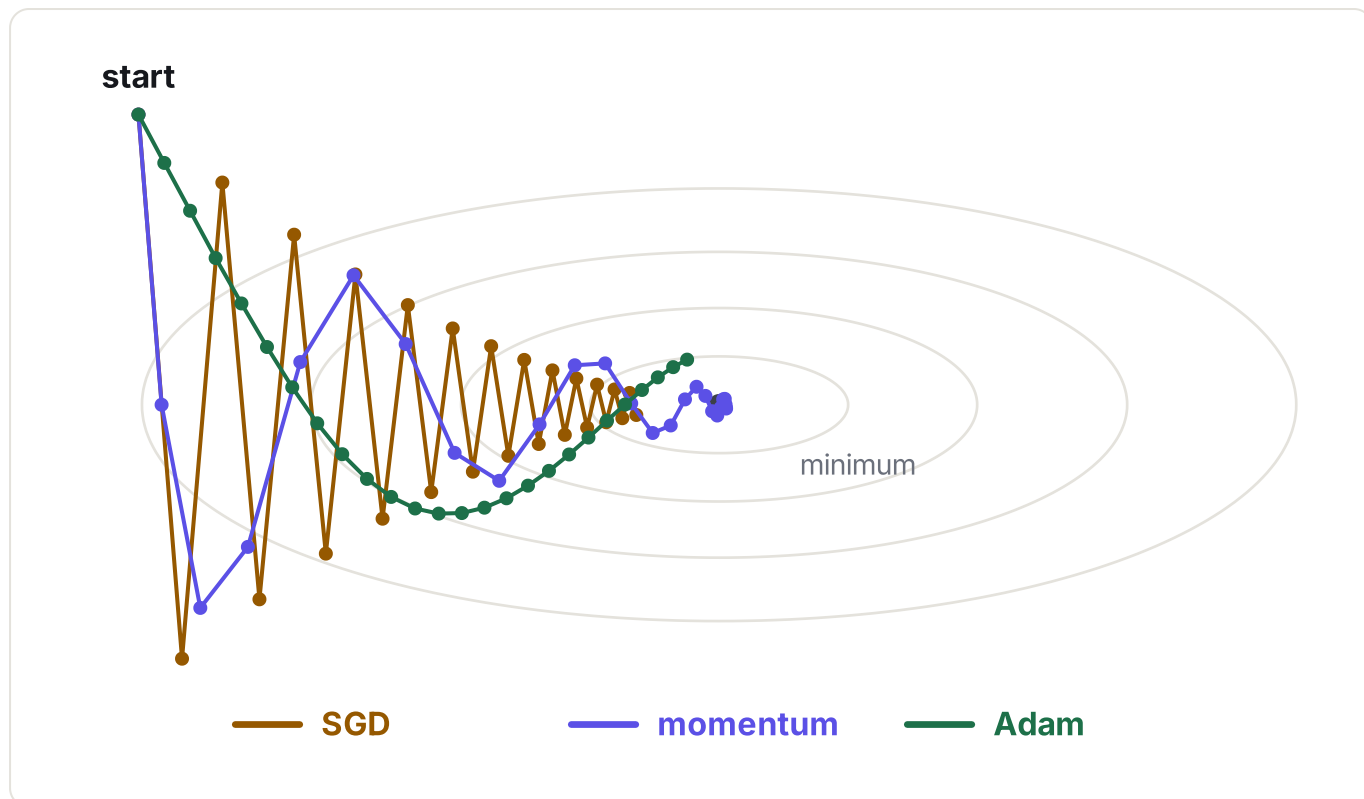


Figure 12.2 25 steps of each optimizer, computed with `torch.optim`, from the same start. **SGD** ($\alpha = 0.075$) zig-zags across the steep direction and is still at $x \approx -0.64$. **Momentum** ($\alpha = 0.04, \beta = 0.7$) builds speed along the floor and ends near the minimum. **Adam** ($\alpha = 0.2$) takes steps of similar size in both directions, so its path starts out diagonal. It ends near $(-0.25, 0.19)$.

◆ THE SAME THING IN PYTORCH

>_ THE THREE OPTIMIZERS SHARE ONE INTERFACE

```

import torch

model = torch.nn.Linear(64, 10)

# plain SGD
opt = torch.optim.SGD(model.parameters(), lr=1e-2)

# SGD with momentum (beta = 0.9)
opt = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)

# Adam: betas = (beta1, beta2), eps as above
opt = torch.optim.Adam(model.parameters(), lr=1e-3,
                        betas=(0.9, 0.999), eps=1e-8)

# AdamW: Adam with decoupled weight decay (see below)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

# the training step is the same whichever you chose:
loss = model(torch.randn(32, 64)).sum() # stand-in for a real loss
opt.zero_grad() # clear last step's gradients (they accumulate otherwise)
loss.backward() # backprop fills every .grad
opt.step() # the optimizer applies its update rule

```

The loop uses the same three calls for every optimizer: `zero_grad`, `backward`, `step`. The optimizer object hides the momentum and scaling math inside `step()`.

AdamW. Weight decay (L2 regularization, from Section 7) pulls each weight toward zero with strength λ . Plain Adam adds $\lambda\theta$ to the gradient, so the decay is also divided by $\sqrt{\hat{v}}$. Weights with large gradients then get almost no decay. AdamW *decouples* the two: it does the Adam step, then subtracts $\alpha\lambda\theta$ directly. This is the version most large models use.

▲ WATCH OUT

Do not forget `opt.zero_grad()`. PyTorch *adds* new gradients into `.grad` by design. If you skip the reset, this step's gradient is added to the old one. The updates are then wrong, and the model may not converge. Clear the gradients every step.

CHECK YOURSELF 5

Two weights have steady gradients: 100 for one, 0.01 for the other. With learning rate α , how far does SGD move each weight per step? How far does Adam move each (after the first few steps)?

4 Learning-rate schedules: change the step over time

◆ INTUITION FIRST

Even the best fixed learning rate is a compromise. Early in training you want *big* steps to cover ground fast. Near the bottom, you want *small* steps, so the weights settle into the minimum instead of bouncing around it. A **learning-rate schedule** changes α during training, usually from high to low.

☂ EVERYDAY PARALLEL

Parking a car. You drive toward the parking space at normal speed (big steps). Then you slow down for the last few centimeters, so you do not hit the curb (small steps). A schedule does the same: move fast first, then finish slowly and carefully.

◆ NOW THE MATH

Here are three common schedules. Let t count training steps (or epochs; an *epoch* is one full pass over the training data). **Step decay** multiplies the learning rate by a factor γ (for example 0.1 or 0.5) every s steps:

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

In words: start at α_0 . Every s steps, multiply the rate by γ . The floor $\lfloor t/s \rfloor$ counts how many drops have happened so far. The result is a staircase.

Cosine decay lowers the rate smoothly along a cosine curve, from α_0 to zero over the total training length T_{total} :

$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \frac{\pi t}{T_{\text{total}}} \right)$$

In words: at the start ($t = 0$) the cosine is 1, so $\alpha_t = \alpha_0$. At the end ($t = T_{\text{total}}$) the cosine is -1 , so $\alpha_t = 0$. In between, the rate follows a smooth S-shaped curve with no sudden drops. It is a very popular choice today.

Warmup is used only at the start. Begin with a tiny learning rate and *raise* it over the first few hundred or thousand steps. Then a decay schedule takes over. Why? Early on, the weights are random and the gradients are large and change fast. A big step at once can push the model into a bad region. Warmup also gives Adam's v time to collect many gradients before the steps get large.

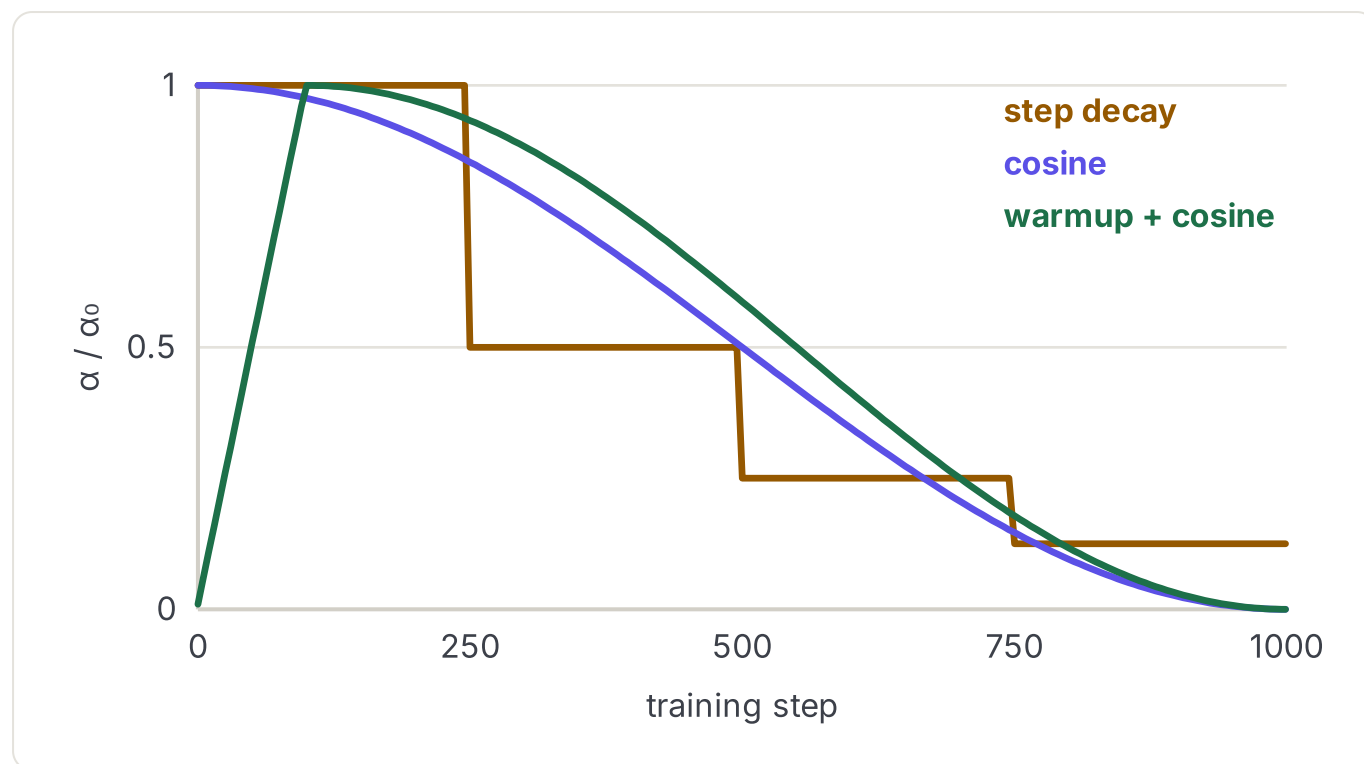


Figure 12.3 Three schedules over 1,000 steps, read from PyTorch schedulers. Step decay halves α every 250 steps. Cosine glides from α_0 to 0. Warmup + cosine climbs linearly for 100 steps, then follows a cosine down.

◆ WHY TRANSFORMERS WARM UP

Large transformer models are sensitive in their first few hundred steps. Without warmup, the loss often diverges early. So a standard recipe is **linear warmup, then cosine (or inverse-square-root) decay**: raise the learning rate gently, then lower it slowly. When you read a training setup for a transformer, you will usually find these two numbers: the warmup steps and the decay schedule.

◆ THE SAME THING IN PYTORCH

›_ ADAM WITH LINEAR WARMUP THEN COSINE DECAY, STEPPED EVERY MINIBATCH

```

import math
import torch
import torch.nn as nn

torch.manual_seed(0)
X, Y = torch.randn(512, 64), torch.randint(0, 10, (512,))  # toy data
train_loader = torch.utils.data.DataLoader(
    torch.utils.data.TensorDataset(X, Y), batch_size=32, shuffle=True)
model = nn.Linear(64, 10)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

epochs = 20
total = epochs * len(train_loader)  # 20 epochs x 16 batches = 320 steps
warmup = 32

def lr_factor(step):
    # multiplies the base lr of 1e-3
    if step < warmup:
        return (step + 1) / warmup  # linear warmup
    progress = (step - warmup) / (total - warmup)
    return 0.5 * (1 + math.cos(math.pi * progress))  # cosine decay

scheduler = torch.optim.lr_scheduler.LambdaLR(opt, lr_factor)

for epoch in range(epochs):
    for x, y in train_loader:
        opt.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        opt.step()  # update the weights
        scheduler.step()  # then move the lr one step along the scheduler
    if epoch % 5 == 0:
        print(epoch, f"{scheduler.get_last_lr()[0]:.2e}")

```

>_ OUTPUT


```
0 5.31e-04
5 8.83e-04
10 5.00e-04
15 1.17e-04
```

Call `opt.step()` every minibatch. Then call `scheduler.step()` at the rate the schedule is written for. Here the schedule counts minibatch steps, so we call it every minibatch; warmup needs this. Some schedules (like a step decay every 30 epochs) are written in epochs; then you call `scheduler.step()` once per epoch. The scheduler changes the `lr` that the optimizer reads on its next step.

CHECK YOURSELF 6

In the snippet above, you move `scheduler.step()` out of the inner loop, so it runs once per epoch. What does the learning rate do over the 20 epochs?

CHECK YOURSELF 7

Your model trains well with a fixed learning rate, but in the last third of training the loss stops improving and jitters. What single change is most likely to help, and why?

5 What to carry forward

- **Never initialize all weights to the same value** (such as zero). Identical neurons get identical gradients and stay copies forever: the symmetry problem. Use small random weights.
- Scale that randomness to **preserve variance** across layers: $\text{Var}(w) \approx 1/n_{\text{in}}$. Use **He** ($2/n_{\text{in}}$) with ReLU, because ReLU halves the mean square. Use **Xavier** ($2/(n_{\text{in}} + n_{\text{out}})$) with tanh.
- The **learning rate** is the first knob to tune: too high diverges, too low crawls. A good default for Adam is `1e-3`.
- **Optimizers** build on each other. SGD steps on a cheap, noisy minibatch gradient. **Momentum** steps along a running average of past gradients. **Adam** also divides by a per-parameter scale ($\sqrt{\hat{v}}$) and uses bias correction. **AdamW** applies weight decay separately.
- **Schedules** change the learning rate over time: step or cosine decay to settle, with **warmup** at the start for sensitive models like transformers.

You now have every part of the training loop: a model, a loss, gradients, a starting point, and a stepping rule. Next we move into PyTorch. First comes the tensor and autograd machinery (Section 13). Then we build and

train a real network (Section 14) and learn to read its diagnostics when it misbehaves (Section 15).

Answers to “Check yourself”

1. You set every weight in a tanh layer to 0.5 (not zero). Is the symmetry problem solved?

No. The problem is not the value zero, it is that all the values are *equal*. With every weight 0.5, every neuron computes the same sum, gets the same δ_j , and receives the same update. The neurons stay copies. Only random (different) starting values break the tie.

2. You build a 50-layer ReLU network with 256 inputs per layer, but you draw every weight from a standard normal (variance 1). Predict what happens in the forward pass.

The activations **explode**. Each layer multiplies the mean square of the signal by $\frac{1}{2}n_{\text{in}}\text{Var}(w) = 128$. After 50 layers that is about $128^{50} \approx 10^{105}$, far above the float32 maximum (about 3×10^{38}). The values overflow to `inf`, and the output becomes `NaN` on the first forward pass. (We ran this with `torch.manual_seed(0)`: the output is `nan`.) He init sets the factor to 1, and the signal survives all 50 layers.

3. On the bowl $L = \frac{1}{2}k\theta^2$ with $k = 10$, what happens with $\alpha = 0.15$? With $\alpha = 0.25$?

The factor is $1 - \alpha k$. With $\alpha = 0.15$ it is -0.5 : θ changes sign every step (a zig-zag) but halves in size, so it still converges. With $\alpha = 0.25$ it is -1.5 : each step jumps to the other side and $1.5\times$ farther away, so the loss grows. The limit is $\alpha = 2/k = 0.2$. A loss that zig-zags and trends upward means the learning rate is too high.

4. With momentum $\beta = 0.9$, the gradient is the same vector g at every step. What does the velocity $\mathbf{u}$ approach?

At the steady state, $\mathbf{u} = 0.9\mathbf{u} + g$, so $\mathbf{u} = g/0.1 = 10g$. The step becomes $10\alpha g$. On a steady slope, momentum moves $10\times$ faster than SGD with the same learning rate. When the gradient flips sign from step to step, the terms cancel and the velocity stays small.

5. Two weights have steady gradients: 100 for one, 0.01 for the other. With learning rate α , how far does SGD move each weight per step? How far does Adam move each (after the first few steps)?

SGD moves them by 100α and 0.01α : a factor of 10,000 apart. Adam moves both by about α . With a steady gradient, $\hat{m} \approx g$ and $\sqrt{\hat{v}} \approx |g|$, so the ratio is about 1. So for the large-gradient weight Adam takes a much *smaller* step than SGD, and for the small-gradient weight a much *larger* one. (In PyTorch, 5 Adam steps with `lr=1e-3` move both weights by `-0.0050`.)

6. In the snippet above, you move `scheduler.step()` out of the inner loop, so it runs once per epoch. What does the learning rate do over the 20 epochs?

The schedule counts in steps and expects 320 of them, but now it gets only 20. So the learning rate stays in the warmup phase the whole time. It climbs from $1/32 \cdot 10^{-3}$ to only about $20/32 \cdot 10^{-3} \approx 6 \times 10^{-4}$, and it

never decays. The step counter and the schedule must use the same unit.

7. Your model trains well with a fixed learning rate, but in the last third of training the loss stops improving and jitters. What single change is most likely to help, and why?

Add a **decay schedule** (step or cosine), so the learning rate shrinks late in training. The jitter means the steps are too large to settle into the minimum: each step overshoots a little. A smaller α lets the weights move closer to the bottom. Cosine decay to near zero is a common choice, and it often gives a small gain in final accuracy.

PART 4

PyTorch

Tensors & Autograd

In Section 10 you built the graph, applied the chain rule and ran the backward pass by hand. Now PyTorch does all of it for you. You will work through a small graph by hand, then watch autograd produce the same gradients.

30 min read

YOU'LL LEARN

- tensors
- broadcasting
- autograd
- the computation graph

Before you start: Section 10: Backpropagation by Hand.

A **tensor** is PyTorch's main data structure. Every input, weight and output you work with is a tensor. The idea itself is plain: a grid of numbers with a shape. What makes it useful is that PyTorch can record every operation you do on a tensor. On request, it gives you back the exact gradient: the thing you computed by hand in Section 10.

◆ THE ONE IDEA TO HOLD ONTO

A tensor is an n-dimensional array. **Autograd** is the part of PyTorch that builds a computation graph while you do arithmetic on tensors. It is the same kind of graph you drew by hand in Section 10. Then it runs the same backward pass to fill in every gradient, automatically.

1 Tensors: arrays that can grow another dimension

◆ INTUITION FIRST

A **tensor** is a number, a list of numbers, a grid of numbers, or a stack of grids. It is one idea that can keep adding dimensions. A single number is a *scalar* (rank 0; the *rank* is the number of dimensions). A list is a *vector* (rank 1; see Section 1). A grid is a *matrix* (rank 2). A stack of matrices is a rank-3 tensor, and so on. PyTorch allows any rank.

☂ EVERYDAY PARALLEL

Think of a spreadsheet. One cell is a scalar. One row is a vector. The whole sheet is a matrix. Now think of a *workbook*: several sheets of the same shape, one behind the other. That stack is a rank-3 tensor. Nothing new happened; you stacked the same kind of thing one level deeper.

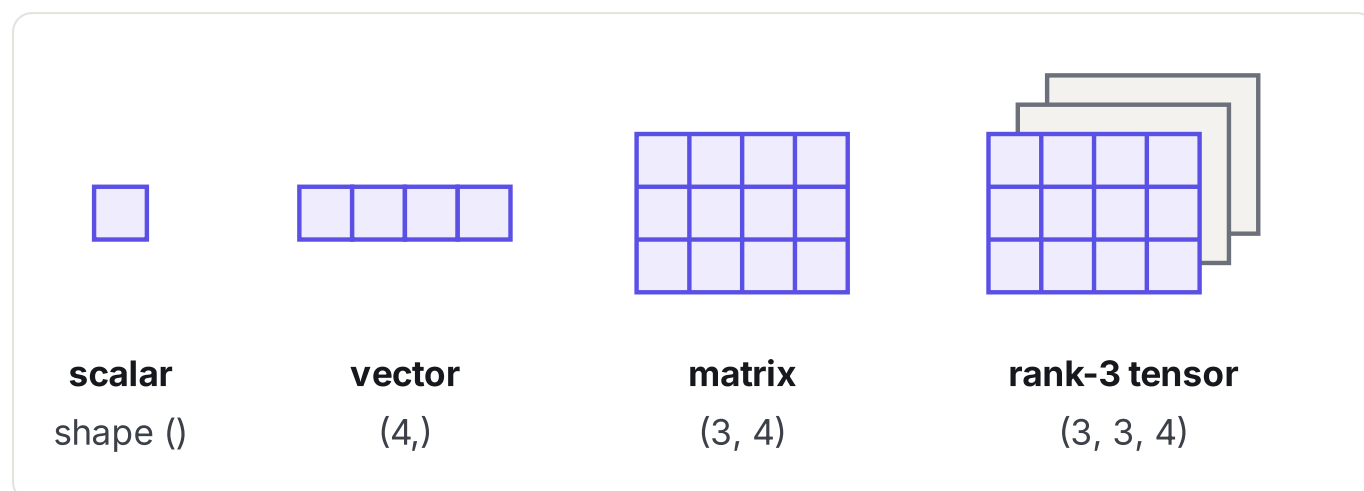


Figure 13.1 Each step adds one dimension. The shape lists the size of each dimension, from the outermost to the innermost. The rank is the length of the shape.

Every tensor carries three pieces of information. Check them whenever something breaks:

- **shape**: how many entries along each dimension. For example, `(64, 3, 28, 28)` for 64 color images of 28×28 pixels.
- **dtype**: what kind of number is stored (`float32`, `int64`, ...). Mixing dtypes in one operation is a common source of errors.
- **device**: where the numbers are stored, in the CPU's memory or in a GPU's memory. Section 14 shows how to move tensors and models between devices.

◆ THE SAME THING IN PYTORCH

›_ MAKING TENSORS AND READING THEIR SHAPE

```
import torch

scalar = torch.tensor(3.0)                # rank 0
vector = torch.tensor([1.0, 2.0, 3.0])    # rank 1, shape (3,)
matrix = torch.tensor([[1.0, 2.0],
                       [3.0, 4.0]])        # rank 2, shape (2, 2)
cube   = torch.zeros(2, 3, 4)             # rank 3, shape (2, 3, 4)

print(vector.shape, vector.dtype, vector.device)
# torch.Size([3]) torch.float32 cpu
```

CHECK YOURSELF 1

A batch of 32 grayscale images, each 28×28 pixels. What rank is the tensor, and what is a reasonable shape? What changes for color images?

2 Broadcasting: stretching a small tensor to fit a big one

◆ INTUITION FIRST

Broadcasting is the rule that lets you combine tensors of different shapes without writing a loop. Suppose one tensor has size 1 along some dimension. Then PyTorch treats it as if it were repeated along that dimension until the shapes match. No memory is copied; the tensor only behaves as if it were.

☂ EVERYDAY PARALLEL

A spreadsheet has one row per shop and one column per product type. Each product type has its own tax rate: 10% on electronics, 5% on clothes, 0% on food. You do not type the rates into every row. You write one row of three rates, and the spreadsheet applies it to every shop. Broadcasting does the same: one small row is applied to every row of a bigger table.

◆ NOW THE MATH

PyTorch compares two shapes from the *right-hand side*, one dimension at a time:

1. If one shape is shorter, add dimensions of size 1 on its *left* until both have the same length. So `(3,)` is read as `(1, 3)`.
2. Two dimensions are compatible if they are equal, or if one of them is 1. A size-1 dimension is stretched to match the other.
3. If every pair is compatible, the result takes the larger size in each position. Otherwise PyTorch raises an error.

$$(4, 3) \text{ and } (3,) \rightarrow (4, 3) \text{ and } (1, 3) \rightarrow (4, 3)$$

In words: a 4×3 matrix combined with a vector of length 3 works. The vector gets a missing leading dimension of size 1, so it acts like one row. That row is then stretched to all 4 rows.

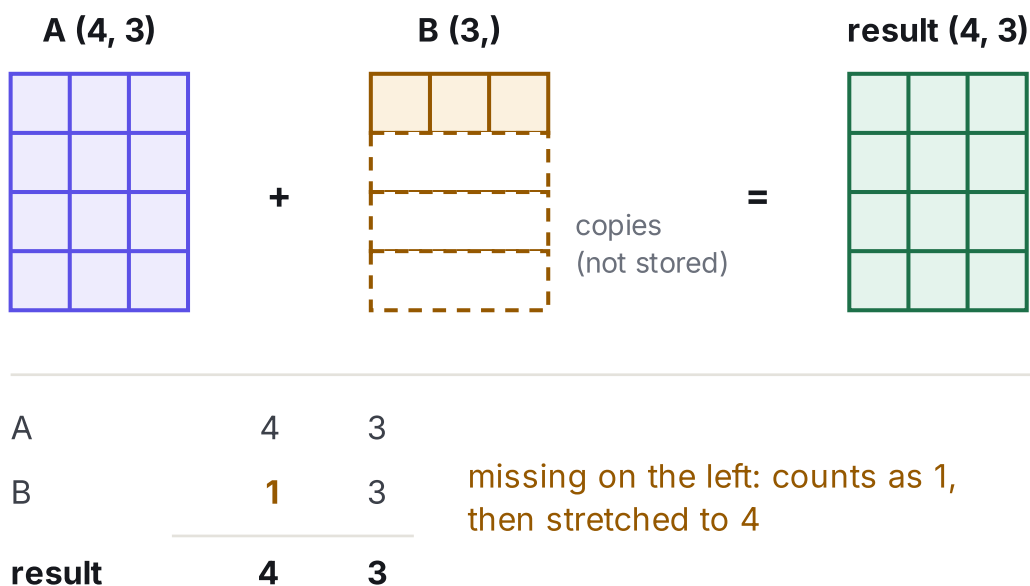


Figure 13.2 Shapes are lined up on the right. B has no first dimension, so it counts as 1 and is stretched to 4 (the dashed rows). PyTorch never stores these copies; it reads the same row four times.

◆ THE SAME THING IN PYTORCH

>_ ONE ROW OF TAX RATES APPLIED TO EVERY ROW

```
import torch

sales = torch.tensor([[100.0, 200.0, 300.0],  # 4 shops (rows),
                      [ 50.0,  75.0,  10.0],  # 3 product types (columns),
                      [400.0, 100.0,  20.0],
                      [ 10.0,  10.0,  10.0]]) # shape (4, 3)

tax_rate = torch.tensor([0.10, 0.05, 0.00]) # one rate per column, shape (3)

print(sales * tax_rate) # shape (4, 3): each column gets its own rate,
# tensor([[10.0000, 10.0000,  0.0000],
#         [ 5.0000,  3.7500,  0.0000],
#         [40.0000,  5.0000,  0.0000],
#         [ 1.0000,  0.5000,  0.0000]])
```

▲ WATCH OUT

The first broadcasting error you will see: `sales + torch.ones(4)` gives `RuntimeError: The size of tensor a (3) must match the size of tensor b (4) at non-singleton dimension 1`. Broadcasting stretches only dimensions of size 1. It never guesses how to match a 3 against a 4. Print both `.shape`s and find the dimension that does not line up. (Here, `torch.ones(4, 1)` would work: it is one column, stretched across the 3 columns.)

CHECK YOURSELF 2

`x` has shape `(64, 10)` (a batch of 64 rows of 10 scores) and `b` has shape `(10,)` (a bias per column). Does `x + b` broadcast, and to what shape?

CHECK YOURSELF 3

Predictions `pred` have shape `(64, 1)` and targets `y` have shape `(64,)`. What is the shape of `pred - y`? Why is this dangerous?

3 Autograd: Section 10, automated

◆ INTUITION FIRST

In Section 10 you built a computation graph by hand. It had a node for each operation and arrows for how values flow forward. Then you walked the arrows backward and applied the chain rule to get every gradient. **Autograd** does the same bookkeeping for you. When you do arithmetic on a tensor with `requires_grad=True`, PyTorch records the operation as a node in a graph. When you call `.backward()`, it walks that graph backward and fills in `.grad` on every leaf.

EVERYDAY PARALLEL

When you do your taxes by hand once, you learn how each deduction changes the final number. Tax software then does the same arithmetic for you every year after: same rules, same order. Section 10 was doing your taxes by hand. Autograd is the software. It is not a different, more clever method. It is the same method, run for you.

DEFINITION

A tensor with `requires_grad=True` tells PyTorch: “record every operation done with me, because I will want a gradient for this value later.” Calling `.backward()` on a final scalar output starts the backward pass. The gradient for each tracked tensor is stored in its `.grad` attribute.

◆ THE SAME THING IN PYTORCH

>_ THE SMALLEST WORKING EXAMPLE

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x          # y = x^2 + 2x

y.backward()               # walk the graph backward from y
print(x.grad)              # tensor(8.) because dy/dx = 2x + 2 = 2(3)
```

A small graph, by hand and by autograd

Take a small expression: multiply two values, add a third, then multiply the result by a fourth. (This example and its numbers come from Andrej Karpathy's micrograd, a tiny autograd engine that is worth reading.)

$$e = a \cdot b, \quad d = e + c, \quad L = d \cdot f$$

In words: three operations in a chain, ending in one number L . It has the same building blocks as a neuron: a multiply (like a weight times an input) and an add (like a bias). It has no nonlinearity, so it is not a neural network. But autograd treats every operation the same way, so the method carries over.

With $a = 2, b = -3, c = 10, f = -2$, the forward pass gives $e = -6, d = 4$ and $L = -8$. Now apply the chain rule backward from L , as in Section 10:

$$\begin{aligned}\frac{\partial L}{\partial d} &= f = -2, & \frac{\partial L}{\partial f} &= d = 4 \\ \frac{\partial L}{\partial e} &= \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial e} = -2 \cdot 1 = -2 \\ \frac{\partial L}{\partial c} &= \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial c} = -2 \cdot 1 = -2 \\ \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a} = -2 \cdot b = 6 \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial b} = -2 \cdot a = -4\end{aligned}$$

In words: start at the output and multiply local derivatives backward along the graph. Each gradient says how much the final number moves per unit change in that variable. For example, $\partial L / \partial a = 6$ means: if you raise a by a tiny amount, L rises by about 6 times that amount.

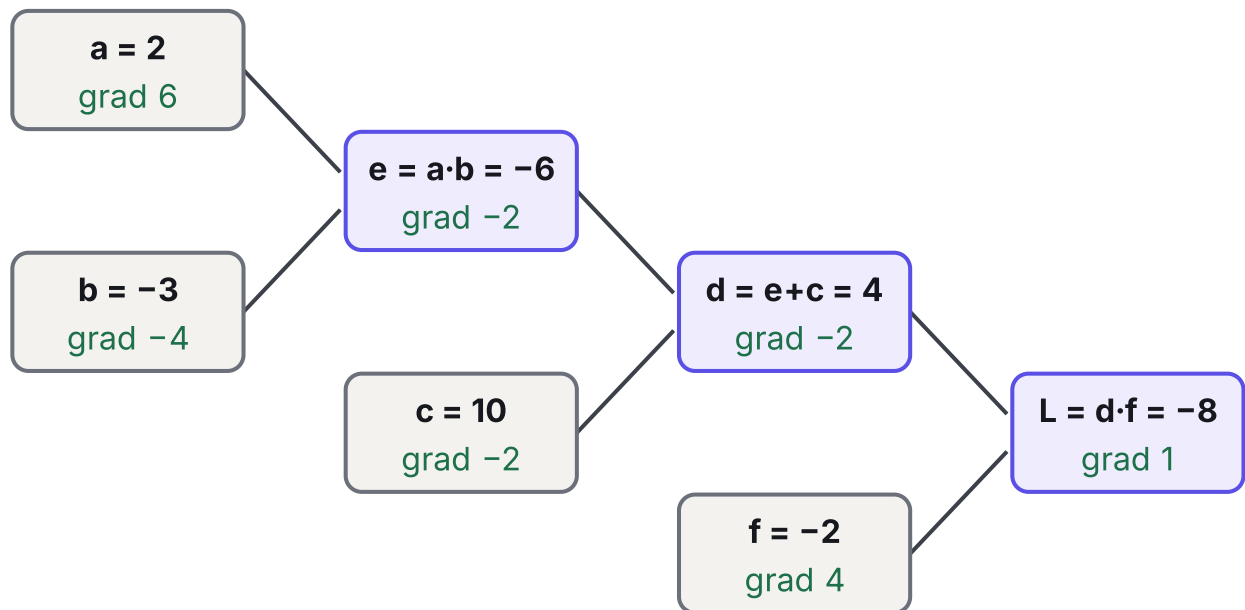


Figure 13.3 The graph autograd builds. Values flow forward along the arrows (left to right). Gradients (green) flow backward, from $\partial L / \partial L = 1$ at the right. Gray boxes are leaves: tensors you created. Violet boxes are results of operations.

Now build the same graph in PyTorch and let autograd do the same walk:

>_ AUTOGRAD REPRODUCES THE HAND DERIVATION

```
import torch

a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(-3.0, requires_grad=True)
c = torch.tensor(10.0, requires_grad=True)
f = torch.tensor(-2.0, requires_grad=True)

e = a * b
d = e + c
L = d * f

L.backward()

print(a.grad, b.grad, c.grad, f.grad)
# tensor(6.) tensor(-4.) tensor(-2.) tensor(4.) ← same as the hand der.
```

The numbers are the same. Autograd is not a different or approximate technique. It is the chain-rule walk from Section 10, done by a machine that never forgets a term.

CHECK YOURSELF 4

Change the last line to `L = d * f + a`, so a is used twice. Predict `a.grad` before you run it.

4 Graph mechanics: leaves, detaching, and not tracking at all

◆ INTUITION FIRST

A few practical rules keep autograd from doing more (or less) work than you want. If you know them, you avoid code that trains on wrong gradients or stops with a confusing error.

DEFINITION

A **leaf tensor** is one you created directly (like `a` and `b` above). It is not the result of an operation on other tracked tensors (like `e` or `d`). By default, only leaf tensors keep a `.grad`. The model's parameters (weights and biases) are always leaves. That is what you want, because those are the numbers you update.

There are two ways to stop autograd from tracking, for different reasons:

- `.detach()` returns a new tensor with the same numbers, but not connected to the graph. Use it when you want a value (for example, to log or plot) without its history.
- `torch.no_grad()` is a context manager that turns off graph building for everything inside it. Use it for *inference* (making predictions, not training). No backward pass follows, so a graph would only waste memory and time.

◆ THE SAME THING IN PYTORCH

>_ THREE WAYS A RESULT CAN BE TRACKED OR NOT

```
import torch

w = torch.tensor(1.5, requires_grad=True)
x = torch.tensor(2.0)

y = w * x
print(y.requires_grad)          # True -- built from a tracked tensor

with torch.no_grad():
    y_infer = w * x              # no graph built at all -- for prediction,
    print(y_infer.requires_grad) # False

y_detached = y.detach()         # same value as y, graph history dropped
print(y_detached.requires_grad) # False
```

▲ WATCH OUT

Gradients accumulate. Calling `.backward()` does not *overwrite* `.grad`; it **adds** to what is already there. If you run the backward pass twice without a reset, the two gradients are added together. The model still trains, but on the wrong numbers. This is why every training loop calls `optimizer.zero_grad()` before each backward pass. We build a full loop in Section 14.

>_ GRADIENTS ADD UP UNTIL YOU RESET THEM

```
import torch

x = torch.tensor(2.0, requires_grad=True)

y = x ** 2
y.backward()
print(x.grad)      # tensor(4.)    dy/dx = 2x = 4

y = x ** 2
y.backward()
print(x.grad)      # tensor(8.)    NOT 4 again -- it accumulated: 4 + 4

x.grad.zero_()     # reset before the next backward pass
y = x ** 2
y.backward()
print(x.grad)      # tensor(4.)    correct, now that we cleared it first
```

CHECK YOURSELF 5

Continue the last snippet (so `x.grad` is 4) with `(3 * x).backward()` and no reset. What is `x.grad` now? Then you call `x.grad.zero_()` and `(x ** 3).backward()`. What is it then?

5 What to carry forward

- A **tensor** is an n-dimensional array with a `shape`, a `dtype` and a `device`. Scalar, vector, matrix and higher ranks are all the same idea.
- **Broadcasting** lines shapes up on the right, counts missing leading dimensions as 1, and stretches size-1 dimensions. Watch for `(N, 1)` against `(N,)`.
- **Autograd** builds the same computation graph you built by hand in Section 10. `.backward()` runs the same chain-rule walk, and it matched the hand-computed gradients above.
- Only **leaf tensors** (like model parameters) keep `.grad`. Use `.detach()` to drop history and `torch.no_grad()` for inference.
- **Gradients accumulate** across calls to `.backward()`. Zero them before the next backward pass.

Next, we stop wiring graphs by hand. `nn.Module` will organize the layers, the parameters and the training loop (Section 14).

Answers to “Check yourself”

1. A batch of 32 grayscale images, each 28×28 pixels. What rank is the tensor, and what is a reasonable shape? What changes for color images?

Rank 3, shape `(32, 28, 28)`: one dimension for “which image in the batch” (32), and two for the pixel grid (28, 28). Color images need a channel dimension for red, green and blue. That gives rank 4: `(32, 3, 28, 28)`.

2. `x` has shape `(64, 10)` (a batch of 64 rows of 10 scores) and `b` has shape `(10,)` (a bias per column). Does `x + b` broadcast, and to what shape?

Yes. `b` is read as `(1, 10)`. From the right: 10 and 10 match; 1 and 64 are compatible. The result has shape `(64, 10)`: the same bias vector added to each of the 64 rows. This is how a linear layer's bias `b` in $\mathbf{XW}^T + \mathbf{b}$ is applied to a whole batch at once.

3. Predictions `pred` have shape `(64, 1)` and targets `y` have shape `(64,)`. What is the shape of `pred - y`? Why is this dangerous?

`y` is read as `(1, 64)`. Then `(64, 1)` and `(1, 64)` both stretch, and the result is `(64, 64)`: every prediction minus every target. There is no error. A loss like `((pred - y)**2).mean()` still returns one number, but it is the wrong number. Fix it with `pred.squeeze(1)` or `y.unsqueeze(1)`, so both shapes match.

4. Change the last line to `L = d * f + a`, so `a` is used twice. Predict `a.grad` before you run it.

`tensor(7.)`. `a` now reaches `L` along two paths. Through `e` it gets 6, as before. Through the new “+ `a`” it gets 1, because addition passes the upstream gradient (here 1) unchanged. When a value is used in several places, the chain rule *adds* the gradients from all paths, and autograd does the same: $6 + 1 = 7$.

5. Continue the last snippet (so `x.grad` is 4) with `(3 * x).backward()` and no reset. What is `x.grad` now? Then you call `x.grad.zero_()` and `(x ** 3).backward()`. What is it then?

First `tensor(7.)`: the new gradient 3 is added to the old 4. After the reset, `tensor(12.)`: the derivative of x^3 is $3x^2 = 3 \cdot 4 = 12$. If a training loss behaves strangely, a missing `zero_grad()` is one of the first things to check.

nn.Module & the Training Loop

Five calls (zero_grad, forward, loss, backward, step) train every PyTorch model. Once you can explain each one, you can read other people's training code.

30 min read

YOU'LL LEARN

- nn.Module
- Dataset & DataLoader
- the standard training loop
- GPU basics

Before you start: Section 13: Tensors & Autograd.

You now know tensors and autograd. You can build a computation graph and get gradients from it. `nn.Module` is a clean way to organize many such graphs (the weights, the layers, the forward computation) into one reusable object. From here on, the new ideas are about organization, not new math.

◆ THE ONE IDEA TO HOLD ONTO

`nn.Module` is a box that remembers its own parameters and knows how to compute its own output. The **training loop** is a short, fixed procedure. It feeds data through that box again and again, measures how wrong the output is, and moves the parameters to make it less wrong. Every PyTorch project is built on these two ideas.

1 nn.Module: a container that tracks its own parts

◆ INTUITION FIRST

`nn.Module` is PyTorch's base class for “a thing that has learnable numbers and a way to use them.” You write a subclass of it. You declare your layers in `__init__`, and you write how data flows through them in `forward`. In return, PyTorch finds every parameter you declared, however deeply nested. You never have to keep a list of them by hand.

☂ EVERYDAY PARALLEL

Lego bricks. A single brick (one `nn.Linear` layer) does one small thing. Connect several bricks in a fixed order and you get something bigger, such as a wall or a car. `nn.Module` is both the brick and the rule for connecting bricks. A module can contain other modules, and from the outside the result looks like one bigger module.

◆ NOW THE MATH

A tiny MLP (multi-layer perceptron: linear layers with a nonlinearity between them) with one hidden layer computes:

$$\mathbf{h} = \phi(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \hat{\mathbf{y}} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$

In words: transform the input $\mathbf{x}$ with the first layer's weights and bias. Pass the result through a nonlinearity ϕ (here ReLU), so the network can learn curves, not only straight lines. Then transform the hidden result $\mathbf{h}$ with a second layer to get the prediction $\hat{\mathbf{y}}$. Each $\mathbf{W}$ and $\mathbf{b}$ is a learnable leaf tensor, as in Section 13. $\mathbf{W}_1$ has shape (out, in), like every `nn.Linear` weight.

◆ THE SAME THING IN PYTORCH

>_ A TWO-LAYER MLP AS AN `NN.MODULE`

```

import torch
import torch.nn as nn

class TinyMLP(nn.Module):
    def __init__(self, in_features, hidden, out_features):
        super().__init__() # required: sets up P
        self.layer1 = nn.Linear(in_features, hidden) # holds W1, b1 -- c
        self.act = nn.ReLU() # the nonlinearity
        self.layer2 = nn.Linear(hidden, out_features) # holds W2, b2

    def forward(self, x):
        h = self.act(self.layer1(x)) # h = phi(W1 x + b1)
        return self.layer2(h) # y_hat = W2 h + b2

model = TinyMLP(in_features=784, hidden=64, out_features=10)

x = torch.randn(32, 784) # a batch of 32 flattened 28x28 images
y_hat = model(x) # calls forward() for you
print(y_hat.shape) # torch.Size([32, 10])

for name, p in model.named_parameters():
    print(name, p.shape) # found automatically:
# layer1.weight torch.Size([64, 784])
# layer1.bias torch.Size([64])
# layer2.weight torch.Size([10, 64])
# layer2.bias torch.Size([10])

```

DEFINITION

Calling `model(x)` runs `forward`. Do not call `model.forward(x)` directly. `model(x)` goes through `__call__`, which also runs some bookkeeping (like hooks) that a direct call would skip.

CHECK YOURSELF 1

A classmate writes `h = nn.Linear(784, 64)(x)` inside `forward` instead of creating `self.layer1` in `__init__`. The code runs. What goes wrong during training?

2 Dataset & DataLoader: feeding the model in small batches

◆ INTUITION FIRST

A **Dataset** is an object that knows two things: how many examples it has, and how to fetch the n -th one. A **DataLoader** wraps it and does the detailed work. It groups examples into minibatches. It shuffles their order each epoch (one full pass over the data). And it gives you one batch at a time in a simple loop.

☂ EVERYDAY PARALLEL

A Dataset is a filing cabinet of numbered cards: ask for card 47 and you get it. A DataLoader is an assistant. At the start of each round, the assistant decides a new random order for the cards. Then the assistant hands you the cards in small stacks, in that order, until every card has been used once. You only say “next stack, please.”

Why use small stacks instead of all the data at once? As Section 12 explained, the exact gradient over the whole dataset is slow and uses a lot of memory. A **minibatch** (a random, modest-sized part of the data) gives a noisy but cheap estimate of that gradient. Many small, noisy steps usually reach a good model faster than a few exact ones.

Dataset: examples 0 to 9

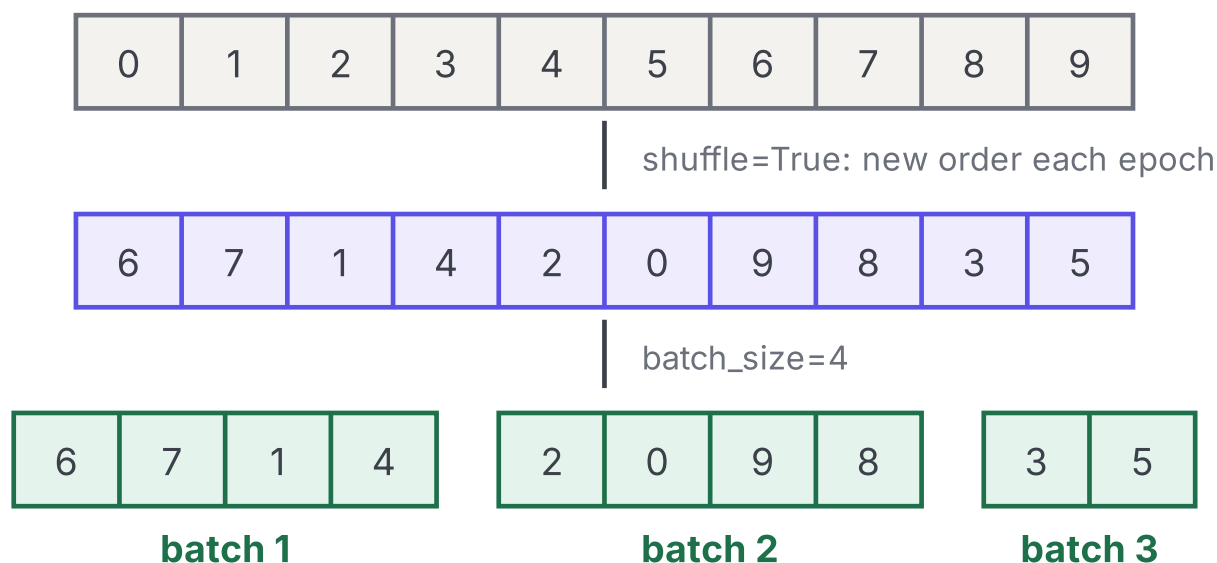


Figure 14.1 What a DataLoader does in one epoch (real order from `torch.manual_seed(0)`). It shuffles the indices, then cuts them into batches. Each example is used once per epoch. The last batch is smaller when the batch size does not divide the dataset size (use `drop_last=True` to skip it).

◆ THE SAME THING IN PYTORCH

>_ A DATASET AND A DATALOADER

```
import torch
from torch.utils.data import Dataset, DataLoader

class ToyDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y

    def __len__(self):
        return len(self.X)           # how many examples in total

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx] # one example, by index

torch.manual_seed(0)
X = torch.randn(1000, 784)          # 1000 fake flattened images
y = torch.randint(0, 10, (1000,))   # 1000 fake labels, classes 0-9

dataset = ToyDataset(X, y)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

for batch_x, batch_y in loader:
    print(batch_x.shape, batch_y.shape) # torch.Size([32, 784]) torch.S
    break
```

CHECK YOURSELF 2

The dataset has 1,000 examples and `batch_size=32`. How many batches are in one epoch, and how big is the last one? Why do we shuffle for training but not for validation?

3 The standard training loop

◆ INTUITION FIRST

Every PyTorch training script, however large the model, repeats the same few lines. Take a batch. Make a guess. Measure how wrong the guess was. Work out how to improve. Take a small step in that direction. Below is the whole loop, with every line explained. Learn this shape well: you will write a version of it in every project.

☂ EVERYDAY PARALLEL

Learning to throw darts. Throw (forward pass). See how far you missed the center (loss). Work out which way your arm was off, and by how much (backward pass). Adjust your stance a little (optimizer step). Throw again, many times. Training does the same with the data. One pass through all the examples is an epoch. The next epoch uses the same examples again, in a new order. The adjustments stay small, so each throw improves your aim a little.

◆ NOW THE MATH

Each step moves every parameter θ a small amount against its gradient. With plain SGD (from Section 12) the update is:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

In words: for each parameter, subtract the learning rate α times its gradient. `optim.SGD` performs this line for every parameter at once. The loop below uses `optim.Adam`, which does a scaled version: it steps along $\hat{m}/(\sqrt{\hat{v}} + \epsilon)$ instead of the raw gradient. Autograd supplies the gradient either way.

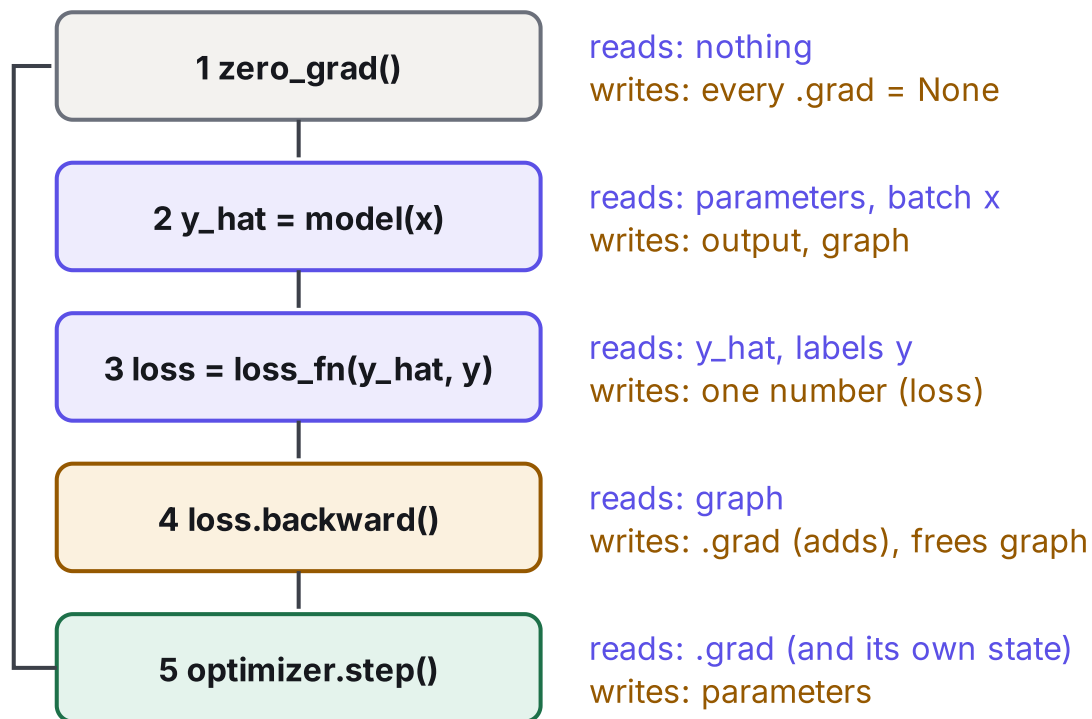


Figure 14.2 The five calls inside the batch loop. Each call reads what an earlier call wrote: `backward` needs the graph from the forward pass, and `step` needs the `.grad` from `backward`. That is why the order is fixed.

◆ THE SAME THING IN PYTORCH

>_ THE TRAINING LOOP (CONTINUES THE TINYMLP AND DATALOADER SNIPPETS ABOVE)


```

import torch.optim as optim

torch.manual_seed(0)
model = TinyMLP(in_features=784, hidden=64, out_features=10)
loss_fn = nn.CrossEntropyLoss()      # multi-class loss: softmax + log-loss
optimizer = optim.Adam(model.parameters(), lr=1e-3) # Adam, from Section 11

for epoch in range(3):                # one full pass over the data
    for batch_x, batch_y in loader:    # one minibatch at a time
        optimizer.zero_grad()         # 1. clear old gradients (see Section 11)
        y_hat = model(batch_x)         # 2. forward pass: raw score
        loss = loss_fn(y_hat, batch_y) # 3. how wrong was the guess
        loss.backward()                # 4. backward pass: fills .grad
        optimizer.step()               # 5. move every parameter using gradients

    print(f"epoch {epoch}: loss = {loss.item():.4f}") # loss of the last batch
# epoch 0: loss = 2.3341
# epoch 1: loss = 1.9191
# epoch 2: loss = 1.4914

```

The labels here are random, so the falling loss only shows that the model is memorizing the 1,000 training examples. On validation data it cannot do better than chance, as we will see below.

▲ WATCH OUT

`nn.CrossEntropyLoss` wants the **raw scores** (logits) from the last layer. It applies the softmax itself, in a numerically stable way. In Section 11 we applied the softmax by hand. Do not do that here: if you pass softmax outputs, the softmax is applied twice. The model still trains, but badly.

▲ WATCH OUT

Why does `zero_grad()` come first? As you saw in Section 13, `.backward()` **adds** to the gradient already in `.grad`; it never overwrites it. Without `optimizer.zero_grad()`, each batch's gradient would be added to the last one, and the optimizer would step on the wrong numbers. Clearing the gradients before every backward pass makes each step depend only on the current batch.

Here is a summary of the five calls:

Line	What it does
<code>optimizer.zero_grad()</code>	Clears every parameter's <code>.grad</code> (sets it to <code>None</code>) before this batch.
<code>y_hat = model(batch_x)</code>	Forward pass: runs <code>forward()</code> and builds the computation graph.
<code>loss = loss_fn(y_hat, batch_y)</code>	Compares the guess to the true labels and gives one number to minimize.
<code>loss.backward()</code>	Backward pass: walks the graph and fills <code>.grad</code> on every parameter.
<code>optimizer.step()</code>	Uses each parameter's new <code>.grad</code> to update it (with Adam's rule here).

Measuring on validation data

To check how well the model does on data it has not trained on, run a separate loop. Two things change. First, `model.eval()` switches some layers (such as dropout and batch normalization) to their evaluation behavior. Our TinyMLP has none, but the habit matters. Second, `torch.no_grad()` stops autograd from building a graph, because no backward pass follows.

>_ AN EVALUATION LOOP (CONTINUES THE TRAINING LOOP ABOVE)

```
val_set = ToyDataset(torch.randn(200, 784), torch.randint(0, 10, (200,)))
val_loader = DataLoader(val_set, batch_size=64, shuffle=False)

model.eval()                                # evaluation behavior for dropout, ba
correct, total = 0, 0
with torch.no_grad():                        # no graph, no .grad: faster, less me
    for batch_x, batch_y in val_loader:
        logits = model(batch_x)
        correct += (logits.argmax(dim=1) == batch_y).sum().item()
        total += len(batch_y)
print(f"val accuracy = {correct / total:.3f}")    # val accuracy = 0.110
model.train()                                # back to training behavior before th
```

CHECK YOURSELF 3

You swap two lines, so each iteration runs `zero_grad()`, forward, loss, `step()`, then `backward()`. What happens to the model?

4 GPU basics: same code, different device

◆ INTUITION FIRST

A GPU (graphics processing unit) does the same tensor arithmetic as a CPU. But it has thousands of small cores that work in parallel, instead of a few fast ones. That gives a large speedup for the big matrix multiplications that neural networks do all the time. Moving your model to a GPU does not change the code above. It only changes *where* the numbers are stored.

☂ EVERYDAY PARALLEL

A CPU is like a few expert mathematicians who can each solve any problem. A GPU is like a stadium of students who can each only multiply and add. For one hard problem, the experts are better. For a million small multiplications, the stadium finishes first, because everyone works at the same time. Matrix multiplication is that kind of job: the same simple operation, repeated a very large number of times.

◆ THE ONE RULE THAT MATTERS

A tensor on the CPU and a tensor on the GPU cannot be combined in one operation; PyTorch raises an error. The model's parameters and the data you feed it must be on the **same device**. Move both.

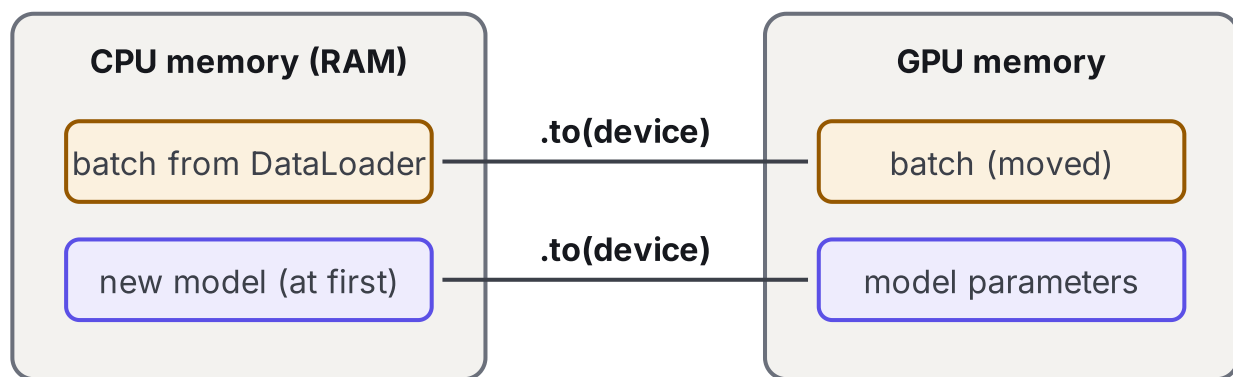


Figure 14.3 CPU and GPU have separate memory. A new model and every batch from the DataLoader start in CPU memory. `.to(device)` copies them across. The model moves once; the batches must move every iteration.

◆ THE SAME THING IN PYTORCH

>_ TRAINING ON THE GPU WHEN ONE IS AVAILABLE (CONTINUES THE SNIPPETS ABOVE)

```
device = "cuda" if torch.cuda.is_available() else "cpu"

model = TinyMLP(in_features=784, hidden=64, out_features=10).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3)  # build it AFTER the model

for batch_x, batch_y in loader:
    batch_x = batch_x.to(device)  # data must move too, every batch
    batch_y = batch_y.to(device)

    optimizer.zero_grad()
    y_hat = model(batch_x)  # model and data are both on `device`
    loss = loss_fn(y_hat, batch_y)
    loss.backward()
    optimizer.step()
```

Note the order. The optimizer holds references to the parameters it was given. If you build a new model, you must build a new optimizer for it. An optimizer made for an old model updates the old model's parameters, so the new one never trains. Creating the optimizer after `.to(device)` is the safe habit.

▲ WATCH OUT

The device error you will see first: `RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cuda:0 and cpu!` You moved the model but not the batch, or the reverse. The fix: find the tensor on the wrong device and add `.to(device)`.

CHECK YOURSELF 4

You call `model.to("cuda")` once before training, but inside the loop you forget to move `batch_x` and `batch_y`. What happens?

5 What to carry forward

- **nn.Module** is a container. Declare layers in `__init__`, define the computation in `forward`, and PyTorch finds every parameter for you.
- **Dataset** answers “how many, and give me item *i*.” **DataLoader** turns that into shuffled minibatches.
- The **training loop** is always `zero_grad` → forward → loss → `backward` → `step`. Zeroing first matters because gradients accumulate. Feed `CrossEntropyLoss` raw logits.
- For evaluation, use `model.eval()` and `torch.no_grad()`, then switch back with `model.train()`.
- **GPU basics:** `.to(device)` moves tensors. Model and data must be on the same device, and the optimizer must be built for the model you train.

Next, Section 15 covers the debugging habits and shape checks that keep this loop from training the wrong thing without any error.

Answers to “Check yourself”

1. A classmate writes `h = nn.Linear(784, 64)(x)` inside `forward` instead of creating `self.layer1` in `__init__`. The code runs. What goes wrong during training?

Every forward pass creates a brand-new layer with new random weights. Those weights are not attributes of the module, so `model.parameters()` does not list them and the optimizer never updates them. Even if it did, they would be thrown away at the next call. So the first layer never learns. Parameters must be created once, as attributes in `__init__`, so they persist and can be found.

2. The dataset has 1,000 examples and `batch_size=32`. How many batches are in one epoch, and how big is the last one? Why do we shuffle for training but not for validation?

$1000/32 = 31.25$, so there are 32 batches: 31 full ones and a last one with $1000 - 31 \cdot 32 = 8$ examples (`len(loader)` is 32). Shuffling in training stops the model from learning anything about the *order* of the examples (for example, if all class-0 examples came first). It also gives each epoch different batches. In validation we update no weights; we only measure. The order does not change the result, and a fixed order makes runs easier to compare.

3. You swap two lines, so each iteration runs `zero_grad()`, `forward`, `loss`, `step()`, then `backward()`. What happens to the model?

It never changes. Since PyTorch 2.0, `zero_grad()` sets every `.grad` to `None` by default. When `step()` runs next, it finds no gradient and skips every parameter. Then `backward()` fills `.grad`, but the next `zero_grad()` clears it before any step can use it. The loss is printed every epoch, but the weights stay at their initial values. The order `forward` → `loss` → `backward` → `step` is required, because each call uses what the previous one wrote.

4. You call `model.to("cuda")` once before training, but inside the loop you forget to move `batch_x` and `batch_y`. What happens?

The forward pass raises the device-mismatch `RuntimeError` above. The first layer's weights are on the GPU, and the batch is on the CPU. Moving the model once is not enough. Every batch from the DataLoader starts on the CPU and must be moved in every iteration.

Debugging & Good Habits

Most training bugs show up in a few typical ways. Learn to recognize them, and a broken run becomes a problem you can find and fix.

38 min read

YOU'LL LEARN

- shape discipline
- gradient checking
- overfit-a-single-batch
- logging & checkpointing

Before you start: Section 14: nn.Module & the Training Loop.

A neural network that fails to train usually fails without an error message. The loss does not crash with a red error. It stops going down, and nothing in the code looks wrong. This section is practical, not theory. It gives the sequence of checks to run, in order, when a model is not learning.

◆ THE ONE IDEA TO HOLD ONTO

Debugging a model is like debugging any other program, except that the failures are silent. So make them visible on purpose. Print shapes, check gradients numerically, and prove that the model *can* learn something easy before you trust it with something hard.

1 Shape discipline: print them everywhere

◆ INTUITION FIRST

A very common bug in deep learning code is a **shape mismatch**. A tensor that comes out of one layer does not have the shape the next layer expects. We met the rule behind this in Section 1. To multiply an $(m \times n)$

matrix by an $(n \times p)$ matrix, the inner numbers must match. Every new layer, model or loss function depends on this rule holding all the way through. When it fails, PyTorch tells you, and the error message says where.

EVERYDAY PARALLEL

Think of water pipes of different diameters. Pipe A ends in a 5 cm opening, and pipe B starts with an 8 cm fitting. They do not connect, however good each pipe is. A shape mismatch is the same: two correct layers that do not connect, because one delivers a different size than the next one accepts.

◆ THE SAME THING IN PYTORCH

Say you build a tiny two-layer network, but you type the wrong input size for the second layer:

>_ A TWO-LAYER NETWORK WITH A WRONG INPUT SIZE

```
import torch
import torch.nn as nn

layer1 = nn.Linear(784, 128)    # 784 pixels in, 128 features out
layer2 = nn.Linear(64, 10)     # BUG: should be nn.Linear(128, 10)

x = torch.randn(32, 784)      # a batch of 32 images
h = layer1(x)                  # h has shape (32, 128) -- correct so far
out = layer2(h)                # crashes
```

>_ WHAT PYTORCH PRINTS

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x128 and 64x10)
```

Read it from left to right. `mat1` is your data, `h`. Its shape is (32, 128), as expected. `mat2` comes from the weight of `layer2`. `nn.Linear` stores its weight as (out, in), here (10, 64), and multiplies by its transpose, (64, 10). That is what the message shows. Its first number, 64, should match `h`'s last dimension, 128. So the message is a diagnosis, not a general complaint. Read both numbers on each side.

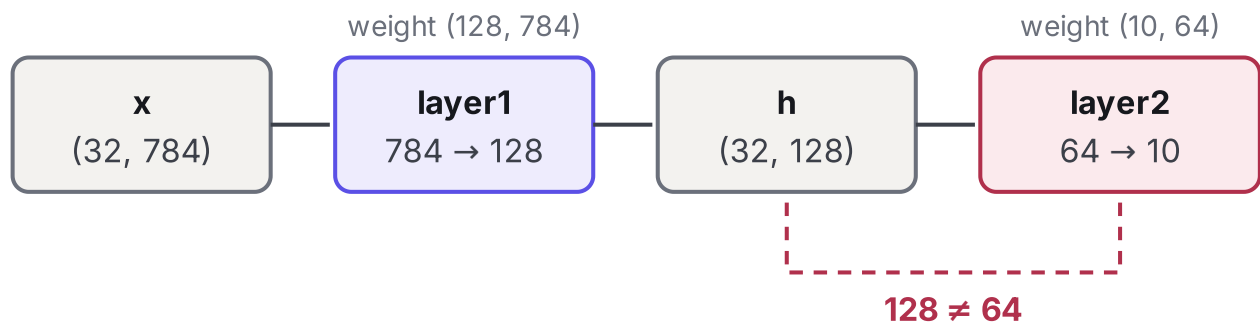


Figure 15.1 Follow the last dimension through the network. Each layer's input size must equal the last dimension of the tensor that flows into it. Here `h` brings 128 features, but `layer2` was built for 64.

▲ WATCH OUT

Do not guess a fix by trial and error. While you debug, add `print(x.shape)` after every line that changes a tensor's shape. Delete the prints once the model runs. A few seconds of printing saves many minutes of guessing.

▲ WATCH OUT

Some shape bugs raise no error at all, because of broadcasting (Section 13). If predictions have shape `(N, 1)` and targets have shape `(N,)`, then `pred - y` has shape `(N, N)`. The loss is then a wrong number, with no error. In the example below the predictions are perfect, but the loss is 2.5 instead of 0. (`F.mse_loss` gives a warning in this case; a hand-written loss does not.)

>_ A SILENT BROADCASTING BUG IN A LOSS

```

import torch

pred = torch.tensor([[1.0], [2.0], [3.0], [4.0]])  # shape (4, 1): perfect
y = torch.tensor([1.0, 2.0, 3.0, 4.0])           # shape (4,)

print((pred - y).shape)                           # torch.Size([4, 4]) -- not what we want
print(((pred - y) ** 2).mean())                     # tensor(2.5000) wrong
print(((pred.squeeze(1) - y) ** 2).mean())          # tensor(0.) right
  
```

CHECK YOURSELF 1

In the buggy network, you fix `layer2` and then add `layer3 = nn.Linear(10, 5)` after it. What shape does `layer3(layer2(h))` have, and what is the shape of `layer3.weight`?

2 Gradient checking: does the math match reality?

◆ INTUITION FIRST

Backpropagation (Section 10) computes gradients *analytically*, by applying calculus rules. That is fast. But if you write a custom loss or layer with its own backward pass, it is easy to get one sign or one factor wrong. **Gradient checking** computes the same gradient a second, much slower way: it changes one number by a tiny amount and measures how much the output moves. Then it compares the two answers. Section 10 introduced this check (see its section 6); here we run it in PyTorch.

☂ EVERYDAY PARALLEL

You compute a 15% tip in your head with a shortcut rule (the analytic way). To check it, you add one dollar to the bill. Then you see whether the tip goes up by about 15 cents. This is the numeric way: change the input and observe. If the two disagree, trust the direct check. Your shortcut has a mistake in it.

◆ NOW THE MATH

The definition of the derivative tells us how to check it. Change the input by a tiny amount ϵ in both directions and see how the loss L moves:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$$

In words: raise the weight w by a tiny amount ϵ (epsilon) and compute the loss. Lower it by the same amount and compute the loss again. The difference, divided by 2ϵ , estimates the slope at that point: the numeric gradient. Using both sides (a “centered” difference) cancels more of the error than a one-sided nudge. Its error shrinks like ϵ^2 (see Section 10).

Run the check in **float64**. In float32, the rounding error of each loss value (about 10^{-7} relative) is divided by the tiny 2ϵ and becomes large. With float32 and $\epsilon = 10^{-4}$, the check below gives about `[2.0027, 4.0007, 5.9938]` instead of `[2, 4, 6]`.

◆ THE SAME THING IN PYTORCH

>_ A NUMERIC GRADIENT CHECK IN FLOAT64

```
import torch

def loss_fn(w):
    return (w ** 2).sum()    # a toy loss; the true gradient is 2*w

w = torch.tensor([1.0, 2.0, 3.0], dtype=torch.float64, requires_grad=True)

# 1) the analytic gradient, from autograd
loss = loss_fn(w)
loss.backward()
analytic = w.grad.clone()

# 2) the numeric gradient, one entry at a time
eps = 1e-6
numeric = torch.zeros_like(w)
for i in range(len(w)):
    w_plus = w.detach().clone(); w_plus[i] += eps
    w_minus = w.detach().clone(); w_minus[i] -= eps
    numeric[i] = (loss_fn(w_plus) - loss_fn(w_minus)) / (2 * eps)

print(analytic)    # tensor([2., 4., 6.], dtype=torch.float64)
print(numeric)    # tensor([2.0000, 4.0000, 6.0000], dtype=torch.float64)
print((analytic - numeric).abs().max().item())    # about 8e-10: they agree
```

For a real model, PyTorch has a ready-made version of this check: `torch.autograd.gradcheck`. It also expects float64 inputs.

CHECK YOURSELF 2

You wrote a custom layer with a hand-written backward pass. The analytic gradient is 1000 times larger than the numeric one, for every entry. The loss averages over a batch of 1000. Where is the bug: in the loss, or in your backward code?

3 A strong debugging test: overfit one batch

◆ INTUITION FIRST

Before you trust a model on your real dataset, give it a much easier task. Ask it to memorize **one small batch** (8 or 16 examples) and train on that same batch for a few hundred steps. A working model and training loop should drive the loss close to zero (or accuracy to 100%) on that batch. There is nothing to generalize; the model only has to remember 16 answers.

☂ EVERYDAY PARALLEL

Before a long car trip, you drive once around the block. If the car cannot even do that, the problem is not the long road; it is the car. You would rather find that out near home than halfway through the trip. Overfitting one batch is the drive around the block.

◆ WHY THIS TEST IS SO USEFUL

Suppose the model **cannot** drive the loss near zero on one tiny batch. Then the problem is almost never that the model is too small or needs more data. It is a real bug. Typical causes are the wrong loss function or a detached computation graph (so gradients do not flow). Others are labels that do not match their inputs, or a learning rate that is far off. You find it here, on one batch, in seconds, and not hours into a real training run.

◆ THE SAME THING IN PYTORCH

›_ TRAIN ON ONE FIXED BATCH UNTIL THE LOSS IS NEAR ZERO

```

import torch
import torch.nn as nn

torch.manual_seed(0)
model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 5))
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

xb = torch.randn(16, 20)          # ONE batch of 16 examples...
yb = torch.randint(0, 5, (16,))    # ...with 5 classes; reuse it every s
# (with real data: xb, yb = next(iter(train_loader)))

for step in range(301):
    optimizer.zero_grad()
    loss = criterion(model(xb), yb)
    loss.backward()
    optimizer.step()
    if step % 100 == 0:
        print(step, f"{loss.item():.4f}")
# 0 1.5749
# 100 0.2215
# 200 0.0356
# 300 0.0143    healthy: the loss goes toward 0
# A broken run stays at a high value -- go and find the bug.

```

CHECK YOURSELF 3

You run the overfit-one-batch test. The loss drops for a while, then stays at a high value and never gets close to zero, even after 500 steps. Name three concrete things to check.

4 Logging and reading loss curves

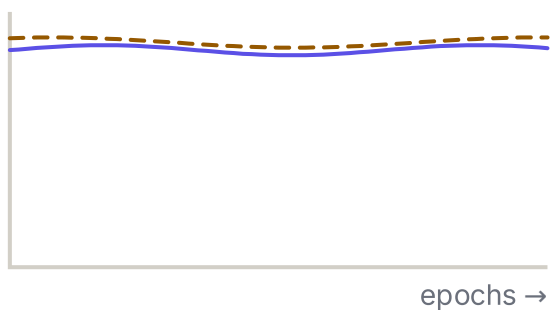
◆ INTUITION FIRST

Once the single-batch test passes, log the loss (and any accuracy metric) every epoch. Do it for both the training set and a held-out validation set, and plot them. The shape of the curve is a diagnosis, not only a number.

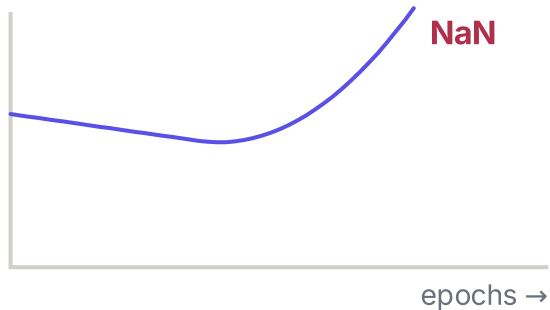
☂ EVERYDAY PARALLEL

A doctor does not look at your temperature only once. The doctor looks at the *trend* over the day. A fever that is rising, one that stays high, and one that is falling mean different things and need different responses. A loss curve is the same kind of chart for your training run.

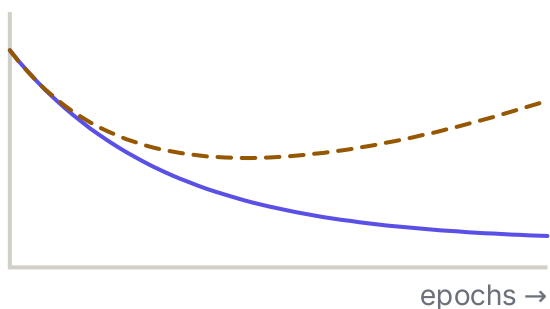
(a) loss stays high



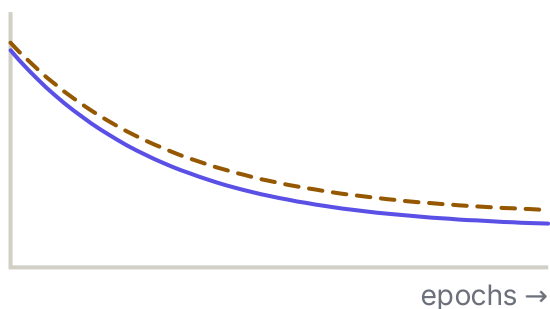
(b) loss rises or NaN



(c) overfitting



(d) healthy



— train loss

- - - validation loss

Figure 15.2 Sketches (not real runs) of the four patterns in the table below. Solid: training loss. Dashed: validation loss. In (c) the gap between the two curves grows; in (d) the curves stay close.

What you see	What it usually means
(a) Loss stays high, barely moves	Something is broken. Run the overfit-one-batch test if you have not. Also suspect a learning rate that is far too small.
(b) Loss goes up, or to <code>NaN</code>	Diverging. Usually the learning rate is too high, or an unstable operation is involved (for example <code>log(0)</code> , or exploding gradients). Lower the learning rate or add gradient clipping.
(c) Training loss keeps falling, validation loss flattens or rises	A growing gap between training and validation: overfitting. The model memorizes the training set instead of learning general patterns (Section 7).
(d) Both curves fall together and flatten near the same value	Healthy. This is what a well-behaved training run looks like.

◆ THE SAME THING IN PYTORCH

>_ A MINIMAL LOGGING LOOP ON A TOY PROBLEM

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(0)
X = torch.randn(600, 20)
y = (X[:, 0] + 0.5 * torch.randn(600) > 0).long()    # a learnable toy l
train_loader = DataLoader(TensorDataset(X[:500], y[:500]), batch_size=32,
val_loader = DataLoader(TensorDataset(X[500:], y[500:]), batch_size=100)
model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 2))
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

def run_one_epoch(model, loader, optimizer=None):
    total, count = 0.0, 0
    for xb, yb in loader:
        loss = loss_fn(model(xb), yb)
        if optimizer is not None:    # training: update the weigh
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        total += loss.item() * len(yb)
        count += len(yb)
    return total / count    # average loss per example

history = {"train_loss": [], "val_loss": []}
for epoch in range(5):
    model.train()
    train_loss = run_one_epoch(model, train_loader, optimizer)
    model.eval()
    with torch.no_grad():
        val_loss = run_one_epoch(model, val_loader)    # no optimizer: onl
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    print(f"epoch {epoch}: train={train_loss:.4f}  val={val_loss:.4f}")

```

>_ OUTPUT


```
epoch 0: train=0.6949 val=0.6849
epoch 1: train=0.6518 val=0.6541
epoch 2: train=0.6198 val=0.6243
epoch 3: train=0.5864 val=0.5944
epoch 4: train=0.5530 val=0.5646
```

Both losses fall together, so this run looks like pattern (d) so far. It has not converged yet; more epochs would help.

CHECK YOURSELF 4

After 50 epochs, the training loss is 0.05 and still falling. The validation loss reached its lowest value, 0.40, at epoch 18 and has risen to 0.65 since. Which pattern is this, and which checkpoint would you keep?

5 Checkpointing, and train() vs eval()

◆ INTUITION FIRST

A **checkpoint** is a saved snapshot of your model's learned numbers. If a run crashes, or when it finishes, you do not lose the training you already did. Saving is only half of the job. When you load a model to evaluate it, you must also tell it to *behave differently* than during training, with `model.eval()`.

☂ EVERYDAY PARALLEL

Think of a theater group. In rehearsals, the director swaps in random understudies for some roles, so that no actor depends too much on one partner. On opening night, the full regular cast plays every role, the same way every night. Training mode is rehearsal (dropout switches off random units). Evaluation mode is opening night: the full network, every time.

◆ THE SAME THING IN PYTORCH

>_ SAVE AND LOAD A CHECKPOINT (CONTINUES THE LOGGING LOOP ABOVE)

```

torch.save({"epoch": epoch,
          "model": model.state_dict(),
          "optimizer": optimizer.state_dict()}, # Adam's m and v, step
          "checkpoint.pt")

# later, or in a new program: build the same objects, then load the number
ckpt = torch.load("checkpoint.pt")
model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 2))
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
model.load_state_dict(ckpt["model"])
optimizer.load_state_dict(ckpt["optimizer"])
start_epoch = ckpt["epoch"] + 1 # 5: continue where we stopped

```

Save the optimizer state too if you want to continue training. Adam keeps its running averages m and v for every parameter (Section 12). If you restart with a fresh optimizer, those averages start again at zero, and the first steps after the restart behave differently.

Some layers behave differently in training and in evaluation. Two common ones are dropout and batch normalization (Section 20). Dropout sets random units to zero, but only during training. Batch norm uses statistics of the current batch during training, and a running average during evaluation. `model.train()` and `model.eval()` tell PyTorch which mode to use. They do not change the weights; they change only the *behavior* of a few layer types.

>_ DROPOUT IN TRAINING MODE AND IN EVALUATION MODE

```

import torch
import torch.nn as nn

torch.manual_seed(0)
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Dropout(p=0.5), nn.Linear(8, 4))
x = torch.randn(1, 4)

net.train() # dropout active: random units are zeroed
print(net(x).item(), net(x).item()) # -0.1568 -0.1155 (different each time)

net.eval() # dropout off; batchnorm would use running statistics
with torch.no_grad(): # also skip building the graph: no backward pass
    print(net(x).item(), net(x).item()) # -0.1362 -0.1362 (the same every time)

```

CHECK YOURSELF 5

You load a checkpoint and run inference. The predictions are worse than expected, and they change a little each time you run the same input. What did you most likely forget?

6 The usual suspects: a checklist

When a model does not train, go through this list before you look for anything unusual:

- **Forgot `optimizer.zero_grad()`**. By default, gradients accumulate (add up) across calls to `backward()`. If you skip the reset, each step's gradient includes every earlier step's gradient.
- **Forgot `model.eval()` for evaluation.** Dropout and batch norm keep their training behavior and distort the validation and test numbers.
- **Learning rate too high.** It shows as a diverging or `NaN` loss. The fix is usually to lower it (or to warm it up slowly).
- **Wrong loss reduction.** With `sum` instead of `mean`, the gradient is N times larger (N = batch size). With SGD this acts like an N times higher learning rate. With Adam it has almost no effect, because Adam divides out the gradient's scale.
- **Silent broadcasting.** A `(N, 1)` prediction against a `(N,)` target gives an `(N, N)` loss matrix and no error (section 1).
- **Data on the wrong device.** Mixing a CPU tensor with a GPU model raises a clear error. But a *new* tensor that you create inside `forward` is easy to miss. Watch for `RuntimeError: Expected all tensors to be on the same device`, and create such tensors with `device=x.device`.

CHECK YOURSELF 6

Find the bug: `for xb, yb in loader: loss = loss_fn(model(xb), yb); loss.backward(); optimizer.step()`. The model trains well for a few steps, then the loss jumps around and grows. Why?

7 What to carry forward

- Print shapes often. When shapes do not match, PyTorch's error message names the two tensors that failed to combine. Read both sides. Also watch for silent broadcasting.

- **Gradient checking** compares the analytic gradient (from `backward()`) with a numeric one, from changing weights by $\pm\epsilon$. Run it in float64 whenever you write a custom backward pass.
- **Overfit one small batch first.** If the loss does not go near zero on a batch of 16, the bug is in your code. It is not in your data or in the size of your model.
- Log training and validation curves every epoch. A growing gap between them means overfitting. A flat, high loss means something is broken. A rising loss means the learning rate is too high.
- Checkpoint the model's and the optimizer's `state_dict()`. Use `model.eval()` to evaluate and `model.train()` to train.

Next we put these habits to work on real architectures, starting with convolutional networks for images in Section 16.

Answers to “Check yourself”

1. In the buggy network, you fix `layer2` and then add `layer3 = nn.Linear(10, 5)` after it. What shape does `layer3(layer2(h))` have, and what is the shape of `layer3.weight`?

After the fix, `layer2 = nn.Linear(128, 10)` maps (32, 128) to (32, 10). `layer3` maps that to (32, 5). Its weight is stored as (out, in) = (5, 10). The batch size 32 passes through every layer unchanged; only the last dimension changes.

2. You wrote a custom layer with a hand-written backward pass. The analytic gradient is 1000 times larger than the numeric one, for every entry. The loss averages over a batch of 1000. Where is the bug: in the loss, or in your backward code?

In your hand-written backward code. The numeric check calls the same loss function. So if the loss used `.sum()` instead of `.mean()`, both gradients would grow by 1000 together and still agree. A constant ratio on every entry means the analytic path has a **scale** bug. Most likely your backward pass forgets the $1/N$ from the mean (with $N = 1000$). Or it applies a constant factor that should not be there.

3. You run the overfit-one-batch test. The loss drops for a while, then stays at a high value and never gets close to zero, even after 500 steps. Name three concrete things to check.

(1) Do the labels match the inputs? A data-pipeline bug can shuffle `xb` and `yb` separately. (2) Is the loss right for the task, and do gradients flow? For example, do not pass softmax outputs to `CrossEntropyLoss`, and look for an accidental `.detach()`, `.item()` or `.data` that cuts the graph. (3) Is the learning rate reasonable? A loss that stays flat (rather than diverging) can mean the learning rate is too *small* for the number of steps.

4. After 50 epochs, the training loss is 0.05 and still falling. The validation loss reached its lowest value, 0.40, at epoch 18 and has risen to 0.65 since. Which pattern is this, and which checkpoint would you keep?

Pattern (c), overfitting. The model got better on unseen data until epoch 18; after that it memorized the training set. Keep the checkpoint from epoch 18, where the validation loss was lowest. (This is “early stopping.”) To do better, add regularization or more data.

5. You load a checkpoint and run inference. The predictions are worse than expected, and they change a little each time you run the same input. What did you most likely forget?

`model.eval()`. Without it, the model is still in training mode. Dropout sets random units to zero on every call, so the same input gives different outputs. Batch normalization uses the statistics of the small batch you passed in, not the stable running averages from training. Both effects make predictions worse and random.

6. Find the bug: `for xb, yb in loader: loss = loss_fn(model(xb), yb); loss.backward(); optimizer.step()`. The model trains well for a few steps, then the loss jumps around and grows. Why?

There is no `optimizer.zero_grad()`. Each `backward()` adds to the gradients already in `.grad`. After k steps, `.grad` holds the sum of k batch gradients, so the steps grow and point partly in old directions. At first the error is small, which is why the first steps look fine. Add `optimizer.zero_grad()` at the top of the loop.

PART 5

Deep Learning Architectures

Convolutional Networks

A convolution is a dot product that slides across an image. When you see it that way, you can derive the shape of every layer in a CNN by hand.

36 min read

YOU'LL LEARN

- convolution as an operation
- output-shape math
- pooling
- ResNets & skip connections

Before you start: Section 14: nn.Module & the Training Loop.

Every image is a grid of numbers. So in principle you could feed it into the fully-connected layers from Section 1: flatten the grid into one long vector and multiply by a big matrix. In practice this is a bad idea. The reasons why are the same reasons convolutional networks exist. Once you see why fully-connected layers struggle with images, the convolution stops looking like a new trick. It becomes the obvious fix.

◆ THE ONE IDEA TO HOLD ONTO

A convolution takes one small, reusable pattern detector (a **kernel**). It slides the kernel over every position in the input and asks the same question everywhere: “*how much of my pattern is here?*” Because the same detector is used everywhere, CNNs need few parameters, and they find a cat wherever it appears in the photo.

1 Why fully-connected layers struggle with images

◆ INTUITION FIRST

A fully-connected layer gives every input pixel its own private weight, for every output neuron. This has two bad results for images. First, images have many pixels, so the weight matrix becomes enormous. Second, and less obvious: the layer does not know that “a cat's ear near the top of the photo” and “a cat's ear near the bottom” are the *same feature*. The two ears land on different input slots with different weights. So the layer learns them as two unrelated facts.

☂ EVERYDAY PARALLEL

Imagine you teach someone to spot a stop sign with one photo. You say: “the stop sign is the red octagon in the upper-left corner of *this exact photo*.” Now show them a second photo where the sign is in the bottom-right. They are lost. They never learned “stop sign.” They learned “red shape at one pixel location.” A fully-connected layer learns in the same way. It has a separate weight for each position. Nothing makes it reuse what it learned about a shape at one place when the shape appears at another.

◆ NOW THE MATH

Take a small color image: 224×224 pixels with 3 color channels (red, green, blue). Flattened into a vector, that is:

$$224 \times 224 \times 3 = 150,528 \text{ numbers}$$

In words: one image is already a vector with over 150,000 entries. Connect it to a hidden layer of only 1,000 neurons. The weight matrix **W** then has shape (1,000 × 150,528), which is over **150 million** numbers to learn. That is for one layer, before the network has seen a single example. And each of those weights is tied to one exact pixel position.

Convolution fixes both problems with one move. A fully-connected layer uses a giant matrix that touches every pixel once. A convolution uses a *tiny* matrix (the kernel) that touches every pixel *many times*, once per position it slides to. The next section makes that precise.

CHECK YOURSELF 1

A fully-connected layer from a 64×64 grayscale image to 500 neurons: how many weights does it have? What happens to that number if the image becomes 128×128?

2 Convolution: a sliding dot product

◆ INTUITION FIRST

A **kernel** (also called a **filter**) is a small grid of numbers, for example 3×3 . To apply it, place it in the top-left corner of the input. Multiply it element by element with the patch of input under it, and add up the products into one number. That is a dot product (Section 1), computed on a 2D patch instead of a flat list. Then slide the kernel one step and repeat, until you have covered the whole input. The result is a new grid with one number per position. Each number tells you how strongly the pattern showed up there.

☂ EVERYDAY PARALLEL

Imagine you search a page by sliding a small stencil over it. At each position you check how well the stencil's shape lines up with the drawing under it. Move the stencil one step right and check again. At the end of the row, move down and do the next row. You never redraw the stencil. It is the same small shape at every spot, and at each spot you ask “does this pattern match *here*?”

◆ NOW THE MATH

Three numbers control how a convolution slides:

DEFINITIONS

The **kernel size** K is the width (and height) of the small grid of weights. The **stride** S is how many pixels the kernel moves between positions (stride 1 checks every position; stride 2 skips every other one). **Padding** P is a border of zeros added around the input. It lets the kernel center itself near the edges, and it controls how much the output shrinks.

First, the operation itself. Write the input as $x[c, r, s]$: channel c , row r , column s . (Channels are the separate grids of an image, such as red, green and blue; more on them below.) The kernel has weights $w[c, u, v]$ for every input channel c and every offset u, v from 0 to $K - 1$, plus one bias b . The output at row i , column j is:

$$y[i, j] = \sum_c \sum_u \sum_v w[c, u, v] \times x[c, iS+u, jS+v] + b$$

In words: the sums run over every input channel c (from 1 to C_{in}) and every offset u and v (from 0 to $K - 1$). Put the kernel's top-left corner at input row iS and column jS . Multiply each

kernel weight by the input value under it, in every channel. Add all $C_{in}K^2$ products and the bias. The same w and b are used for every output position (i, j) ; only the patch of x changes. (Here x is the input after padding.)

▲ WATCH OUT

In mathematics, a “convolution” flips the kernel before sliding it ($x[i - u]$ instead of $x[i + u]$). The formula above does not flip. Its proper name is *cross-correlation*. Deep learning libraries, including PyTorch's `nn.Conv2d`, compute cross-correlation and call it convolution. For learning this makes no difference: the network learns the weights, so a flipped kernel is still a kernel.

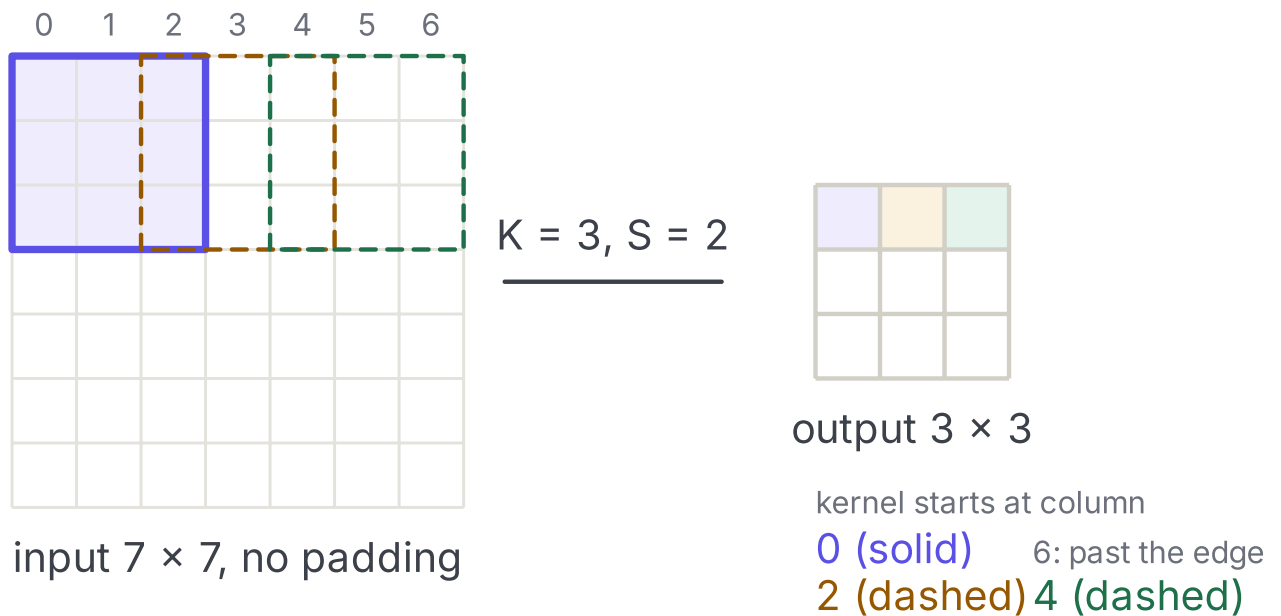


Figure 16.1 The kernel (violet) starts at column 0, then jumps 2 columns to start at 2 (amber), then at 4 (green). A start at column 6 would run past the edge. So the first output row has 3 values, one per position. Rows work the same way, giving a 3×3 output.

Now let's derive the size of the output grid for an input of width W . (Assume a square input and kernel; the same formula gives the height.) Padding adds P zeros on each side, so the padded width is $W + 2P$. The kernel's left edge can start at position 0, then S , then $2S$, and so on. The last start is the largest one where the kernel still fits inside the padded input:

$$\begin{aligned} \text{start} + K &\leq W + 2P \\ \text{start} &\leq W + 2P - K \end{aligned}$$

In words: the kernel's start position plus its width K cannot run past the padded edge. That limits how far right the kernel can start.

The start positions are $0, S, 2S, \dots$ up to that limit. The *number* of start positions is the output size. It is the limit divided by the stride, rounded down, plus one for the start at 0:

$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

In words: take the input width, subtract the kernel width, add twice the padding, and divide by the stride. Round down ($\lfloor \cdot \rfloor$ means “round down”, because a position cannot be a fraction). Then add 1 to count the first position. This one formula gives the output shape of any convolution before you run it.

>_ WORKED EXAMPLE

An input of width $W = 7$, kernel $K = 3$, stride $S = 2$, no padding ($P = 0$), as in Figure 16.1:

$$\text{out} = \left\lfloor \frac{7 - 3 + 0}{2} \right\rfloor + 1 = \lfloor 2 \rfloor + 1 = 3$$

In words: the kernel fits at start positions 0, 2 and 4. It covers pixels 0–2, 2–4 and 4–6. That is three positions and three output values, as the formula says.

A second, very common case keeps the output the *same size* as the input (“same” padding). Take $W = 5$, $K = 3$, $S = 1$, $P = 1$:

$$\text{out} = \left\lfloor \frac{5 - 3 + 2}{1} \right\rfloor + 1 = 4 + 1 = 5$$

In words: a 3-wide kernel would shrink the output by 2 pixels. Padding of 1 on each side adds those 2 pixels back, so the output stays 5×5 . This is why “kernel 3, padding 1” appears together so often in real architectures.

Channels, feature maps, and parameter sharing

A color image does not have one grid of numbers. It has three (red, green, blue), called *channels*. So a kernel for this input is not 3×3 but $3 \times 3 \times 3$. It spans the full depth of the input, and the sum over C in the formula above adds up all three channels at once. One kernel produces one output grid, called a *feature map*: a map of how strongly the pattern fired at every position. A convolutional layer normally learns many kernels in parallel, say 64. They produce 64 stacked feature maps, and these become the 64 input channels of the next layer.

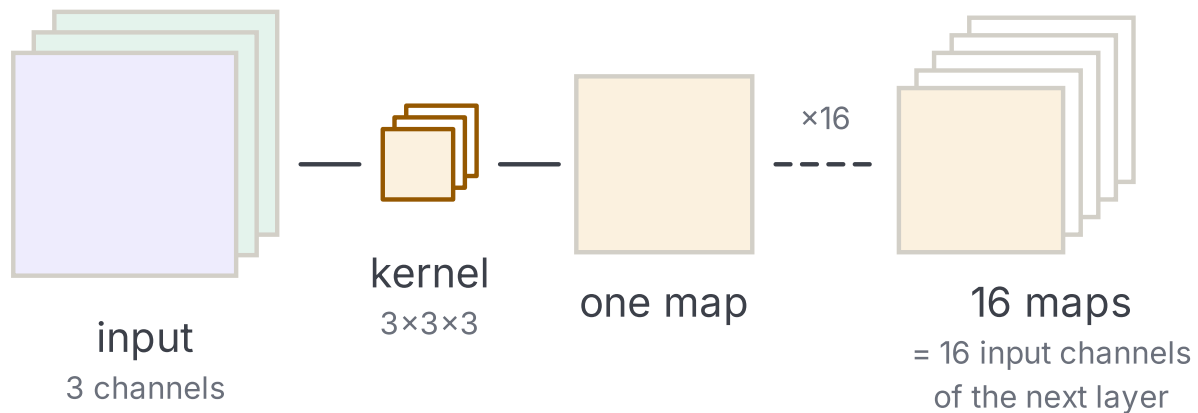


Figure 16.2 One kernel always spans all input channels and gives one feature map. The number of kernels, not the number of input channels, sets how many maps come out. So `nn.Conv2d(3, 16, 3)` holds 16 kernels of shape $3 \times 3 \times 3$.

◆ PARAMETER SHARING

The key trick: the *same* kernel weights are used at every position in the image. This one choice gives two things.

First, **translation equivariance**. Shift the input image, and the feature map shifts by the same amount; its values do not change. A pattern the kernel learned in one corner is found anywhere else, because the same weights do the checking. (Equivariance means “the output moves with the input.” **Invariance** means “the output does not change at all.” Convolution gives equivariance. Pooling, in the next section, adds a small amount of invariance.)

Second, far fewer parameters. A 3×3 kernel over 3 input channels, with 64 output feature maps, needs $3 \times 3 \times 3 \times 64 = 1,728$ weights. That is true whether the image is 32×32 or 3200×3200 . The fully-connected layer at the start of this lesson needed over 150 million weights. That is about 87,000 times more, or roughly five orders of magnitude.

◆ THE SAME THING IN PYTORCH

> `NN.CONV2D, THE SHAPE FORMULA, AND THE SUM BY HAND`

```

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
# batch of 8 RGB images, 32x32 pixels: shape (batch, channels, height, width)
x = torch.randn(8, 3, 32, 32)

conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
y = conv(x)
print(y.shape) # torch.Size([8, 16, 32, 32])
# height/width: floor((32 - 3 + 2*1)/1) + 1 = 32 → "same" padding, confidence
# channels: 16, one feature map per learned kernel

print(conv.weight.shape) # torch.Size([16, 3, 3, 3]) → (out_ch, in_ch, kernel_h, kernel_w)
print(sum(p.numel() for p in conv.parameters())) # 16*3*3*3 + 16 bias = 160

# the triple sum y[i,j] = sum_c sum_u sum_v w[c,u,v] * x[c, i+u, j+v] + b
with torch.no_grad():
    xp = F.pad(x[0], (1, 1, 1, 1)) # image 0, padded with P
    i, j = 5, 7 # any output position (S)
    by_hand = (conv.weight[0] * xp[:, i:i+3, j:j+3]).sum() + conv.bias[0]
    print(torch.allclose(by_hand, y[0, 0, i, j])) # True

# a downsampling conv: stride 2, no padding
down = nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=0)
print(down(x).shape) # torch.Size([8, 16, 15, 15])
# floor((32 - 3 + 0)/2) + 1 = floor(29/2) + 1 = 14 + 1 = 15

```

CHECK YOURSELF 2

An input feature map is 28×28 with 8 channels. You apply a convolution with kernel size 5, stride 1, padding 2, and 32 output channels. Predict the output shape and the number of weights (without biases).

3 Pooling: shrinking on purpose

◆ INTUITION FIRST

After a few convolutions, you often want to make the feature maps smaller on purpose. There are fewer numbers to carry forward. The network also gets a little tolerance when a pattern moves by a pixel or two. **Pooling** does this with no learned weights. It slides a small window over the input and keeps one summary number at each position.

☂ EVERYDAY PARALLEL

Think of a weather report for a country. The report does not list the temperature of every street. It gives one number per region. **Max pooling** reports the highest temperature in each region. It answers “did it get hot anywhere in this region?” **Average pooling** reports the regional mean. It is a smoother summary. In both cases, a heat spot that moves a little inside its region gives the same report.

◆ NOW THE MATH

Pooling uses the same sliding window as convolution, so the same output-size formula applies. Usually the kernel size K and the stride S are equal, so the windows do not overlap. There are no weights. For max pooling, the output at (i, j) in each channel c is:

$$y[c, i, j] = \max_{0 \leq u, v < K} x[c, iS+u, jS+v]$$

$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

In words: the same sliding window as the convolution sum, but the window takes the largest value instead of a weighted sum. Each channel is pooled on its own, so the number of channels does not change. Average pooling replaces the max with the mean of the K^2 values.

◆ THE SAME THING IN PYTORCH

›_ 2x2 MAX POOLING HALVES HEIGHT AND WIDTH

```
import torch
import torch.nn as nn

x = torch.randn(8, 16, 32, 32)

pool = nn.MaxPool2d(kernel_size=2, stride=2)
y = pool(x)
print(y.shape)  # torch.Size([8, 16, 16, 16])
# floor((32 - 2 + 0)/2) + 1 = 15 + 1 = 16, applied to height and width
```

CHECK YOURSELF 3

A 2×2 max pool with stride 2 sees a strong activation at column 0 of a window. The feature map shifts one pixel to the right. Does the pooled output stay the same? What if the activation was at column 1?

4 ResNets and skip connections

◆ INTUITION FIRST

With convolutions in hand, the next idea was to stack many more of them. A deeper network should never be worse: its extra layers could copy the input through unchanged if they did not help. But in practice, plain stacks of convolutions became *harder* to train past a certain depth. This was not overfitting. Even the *training* accuracy got worse. He et al. (2015), who introduced ResNets, called this the **degradation problem**.

What caused it? Their networks used batch normalization, which keeps gradients at a healthy size. So the authors argued that vanishing gradients (Section 12) were probably not the main cause. The problem was optimization. A stack of plain layers finds it hard to learn even the identity function (“output = input”). So the extra layers could not learn to “do nothing”, and they made things worse instead.

☂ EVERYDAY PARALLEL

Imagine you edit a good essay with a team. The plain way: each editor must rewrite the whole essay from memory. Even if an editor has nothing to add, they must reproduce every sentence perfectly, and small errors pile up. The residual way: each editor gets the essay and writes only *corrections* in the margin. An editor with nothing to add writes nothing, and the essay passes on unchanged. A residual block works like the second editor: it learns a correction $F(\mathbf{x})$ that is added to the input.

◆ NOW THE MATH

A plain layer computes some function of its input, $F(\mathbf{x})$, and passes only that forward. A **residual block** computes:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x})$$

In words: $F(\mathbf{x})$ (usually two convolutions) is still computed and used. But it is *added to* the input $\mathbf{x}$ instead of replacing it. To “do nothing”, the block only needs $F(\mathbf{x}) \approx 0$. That is easy: set the weights near zero. A plain layer would have to learn the identity map exactly, which is much harder.

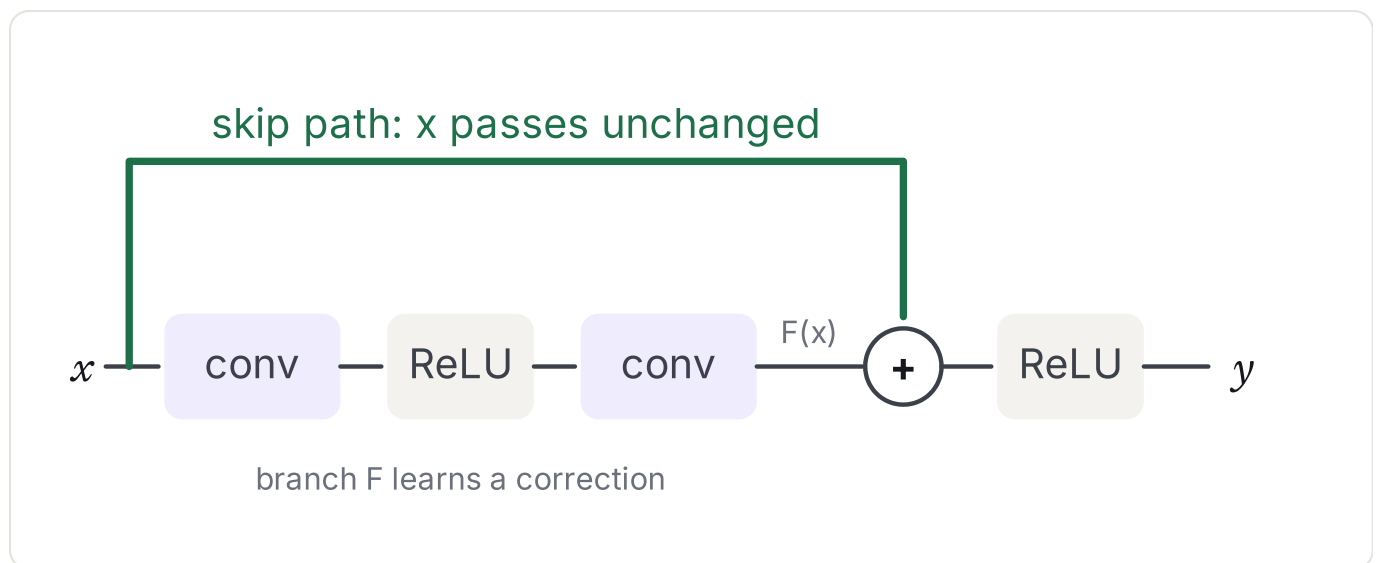


Figure 16.3 The residual block in the code below. The green skip path carries $\mathbf{x}$ around the two convolutions and adds it back. In the backward pass, the gradient can flow along this path without passing through any weights.

The skip path also helps the backward pass. By the chain rule (Section 10), the derivative of the output with respect to the input is:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{x}}{\partial \mathbf{x}} + \frac{\partial F(\mathbf{x})}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}}$$

In words: the gradient splits into two paths that add up. One is the **identity** path (I , the identity matrix: the derivative of $\mathbf{x}$ with respect to itself). The other is whatever the convolutions inside F contribute. Stack many residual blocks, and the gradient through the stack is a product of factors like $(I + \partial F / \partial \mathbf{x})$. Even if every $\partial F / \partial \mathbf{x}$ is tiny, each factor stays close to I instead of shrinking toward zero. So the gradient reaches the early layers largely intact. This is the same shrink-or-explode story as the repeated matrix products in Section 1 and the

recurrent networks in Section 17. The skip connection fixes it through the structure of the network, not through careful tuning.

◆ THE SAME THING IN PYTORCH

>_ A MINIMAL RESIDUAL BLOCK

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.relu(self.conv1(x))
        out = self.conv2(out)          # out is F(x)
        return self.relu(x + out)      # skip connection, then ReLU: ReLU(x + F(x))

block = ResidualBlock(channels=16)
x = torch.randn(8, 16, 32, 32)
y = block(x)
print(y.shape)  # torch.Size([8, 16, 32, 32]) → padding=1, kernel=3 keep.
```

In words: this is the original 2015 ResNet block. It applies a ReLU *after* the addition, so it computes $\text{ReLU}(\mathbf{x} + F(\mathbf{x}))$, not exactly $\mathbf{x} + F(\mathbf{x})$. A later version (He et al., 2016, “pre-activation” ResNets) moves the ReLUs inside F and drops the last one. Then the skip path is a pure identity.

CHECK YOURSELF 4

In the block above, training has pushed F 's weights and biases to nearly zero. What does the block output for an input value of 2.0? For -2.0? What is the gradient through the skip path?

5 What to carry forward

- Fully-connected layers give images too many parameters and no reuse of patterns across positions. Convolution fixes both.
- A **convolution** slides a small kernel over the input and computes a dot product (summed over all input channels) at each position. The output size is $\lfloor (W - K + 2P)/S \rfloor + 1$. Derive it; do not memorize it.
- **Parameter sharing** (the same kernel used everywhere) gives translation *equivariance* and cuts the parameter count by orders of magnitude compared with a fully-connected layer.
- **Pooling** shrinks feature maps with no learned weights, uses the same sliding-window formula, and adds a little shift *invariance*.
- **Skip connections** ($\mathbf{y} = \mathbf{x} + F(\mathbf{x})$) make “do nothing” easy to learn and give gradients an identity path, so very deep networks train.

Convolutions assume a grid with local structure. That is great for images. Text and audio have a different structure: order along a single sequence. Next we will see how networks handle that.

Answers to “Check yourself”

1. A fully-connected layer from a 64×64 grayscale image to 500 neurons: how many weights does it have? What happens to that number if the image becomes 128×128?

$64 \times 64 \times 500 = 2,048,000$ weights (plus 500 biases). Doubling the width and height gives four times as many pixels, so the count grows four times, to 8,192,000. The size of a fully-connected layer grows with the image. Each of those weights also belongs to one fixed pixel. So a pattern learned at one place teaches the layer nothing about the same pattern at another place. A convolution, as you will see next, has the same number of weights for any image size.

2. An input feature map is 28×28 with 8 channels. You apply a convolution with kernel size 5, stride 1, padding 2, and 32 output channels. Predict the output shape and the number of weights (without biases).

Output size: $\lfloor (28 - 5 + 4) / 1 \rfloor + 1 = 28$. Padding 2 with a 5-wide kernel is again “same” padding, so the output is 32 maps of 28×28. Each kernel spans all 8 input channels, so it has $5 \times 5 \times 8 = 200$ weights. There are 32 kernels, so the layer has $200 \times 32 = 6,400$ weights (plus 32 biases). This number does not depend on the 28×28 size: that is parameter sharing.

3. A 2×2 max pool with stride 2 sees a strong activation at column 0 of a window. The feature map shifts one pixel to the right. Does the pooled output stay the same? What if the activation was at column 1?

At column 0, the shift moves the activation to column 1, which is still inside the same window (columns 0–1). The window reports the same maximum, so the output often stays the same. At column 1, the shift moves it to column 2, which is in the *next* window. Now the output changes. So max pooling gives tolerance to some one-pixel shifts, not to all of them. The invariance is partial. A fully-connected layer has no such tolerance at all: any shift changes which weights multiply which values.

4. In the block above, training has pushed F's weights and biases to nearly zero. What does the block output for an input value of 2.0? For -2.0? What is the gradient through the skip path?

With $F(\mathbf{x}) \approx 0$, the block outputs $\text{ReLU}(\mathbf{x})$. So 2.0 comes out as 2.0: the block passes it through unchanged. But -2.0 comes out as 0, because of the final ReLU. The block is an exact identity only for values that are not negative. (The pre-activation version, with no ReLU after the add, is an identity for every value.) Through the addition itself, the gradient is $I + \partial F / \partial \mathbf{x} \approx I$. Where the final ReLU is active, the gradient passes at full strength. This is why very deep ResNets train. A block that has not learned anything useful yet mostly passes its input on. It does not damage the signal.

Sequence Models: RNNs & LSTMs

A recurrent network reads one step at a time and carries a memory forward. The same design that makes this possible also makes it hard to train over long sequences.

32 min read

YOU'LL LEARN

- RNNs & hidden state
- unrolling
- vanishing/exploding gradients
- LSTMs
- why we moved on

Before you start: Section 14: nn.Module & the Training Loop.

The convolutional networks in Section 16 work on a grid, where nearby pixels matter but no direction comes “first”. Text, audio and time series are different. They are **sequences**, and their *order* carries meaning. “Dog bites man” and “man bites dog” use the same words in a different order, and they say very different things. Recurrent networks were the first architecture built to respect that order.

◆ THE ONE IDEA TO HOLD ONTO

A recurrent network reads a sequence one element at a time. It keeps a running **hidden state**, a summary of everything seen so far, and updates it at every step. One update rule, used again and again, is the source of its power. As you will see, it is also the source of its biggest weakness.

1 RNNs: a hidden state that carries memory forward

◆ INTUITION FIRST

A **recurrent neural network** (RNN) processes a sequence step by step: one word, one audio sample, or one time tick at a time. At each step it looks at two things: the current input, and a running summary of everything before it (the **hidden state**). From these it makes an updated summary and carries it to the next step.

☂ EVERYDAY PARALLEL

Think of reading a sentence aloud, word by word, to someone who cannot see the page. When you reach “bank” in “I walked along the river bank”, you already have a sense of the sentence from the earlier words. That sense tells you “bank” means the side of a river, not a place for money. You do not re-read the sentence from the start at every word. You carry a summary forward and update it. That summary is the hidden state.

◆ NOW THE MATH

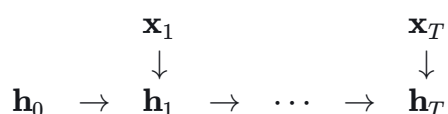
At each time step t , the network takes the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$. It produces the new hidden state $\mathbf{h}_t$:

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$$

In words: $\mathbf{W}$ is a matrix that transforms the current input (the same $\mathbf{W}\mathbf{x}$ as in Section 1). $\mathbf{U}$ is a matrix that transforms the previous hidden state, which brings in what the network remembers. $\mathbf{b}$ is a bias vector. The $\tanh$ squashes each entry into the range -1 to 1 , so the hidden state cannot grow without limit. The key point: the *same* $\mathbf{W}$, $\mathbf{U}$ and $\mathbf{b}$ are used at every time step. One small set of weights handles a sequence of any length. This is like one convolution kernel being used at every position in Section 16.

Unrolling and backpropagation through time

To picture the RNN, and to train it, we draw it “unrolled”. We make one copy of the update rule per time step and lay the copies out left to right. Each copy passes its hidden state to the next one, and each copy also receives its own input:



In words: $\mathbf{h}_0$ is the starting state (usually all zeros), and T is the sequence length. The unrolled picture looks like a very deep feedforward network, except that every copy shares the same weights $\mathbf{W}$ and $\mathbf{U}$. We train it with ordinary backpropagation (Section 10) on this unrolled picture. This is called **backpropagation through time** (BPTT). The error at the last step must flow backward through every earlier step, through the same matrix $\mathbf{U}$ each time, to reach the start of the sequence.

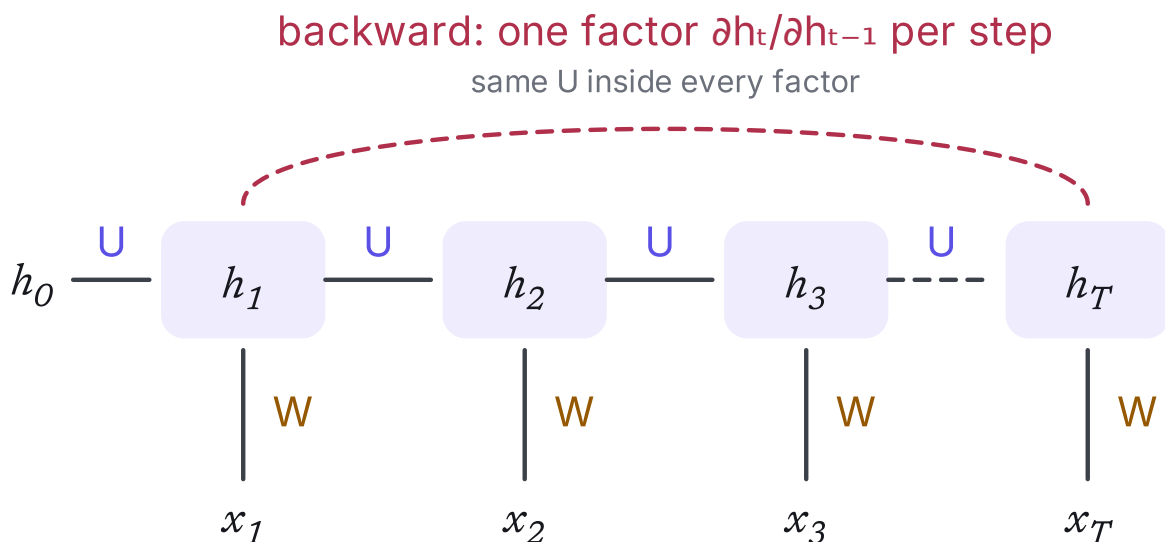


Figure 17.1 The unrolled RNN. The same $\mathbf{W}$ (amber) and $\mathbf{U}$ (violet) appear at every step. The gradient from the last step (rose, dashed) must pass back through every copy of $\mathbf{U}$ to reach the first input.

◆ THE SAME THING IN PYTORCH

>_ AN RNN OVER A BATCH OF SEQUENCES

```

import torch
import torch.nn as nn

torch.manual_seed(0)
rnn = nn.RNN(input_size=8, hidden_size=16, batch_first=True)

x = torch.randn(4, 10, 8)          # (batch=4, sequence length=10, features=8)
h0 = torch.zeros(1, 4, 16)         # (layers=1, batch=4, hidden=16), all zeros

outputs, h_last = rnn(x, h0)
print(outputs.shape)    # torch.Size([4, 10, 16]) → hidden state at each time step
print(h_last.shape)     # torch.Size([1, 4, 16]) → only the final hidden state
print(torch.allclose(outputs[:, -1], h_last[0])) # True: the last output is the final hidden state

```

In words: `batch_first=True` changes the layout of `x` and `outputs` only. The hidden state `h0` keeps the shape (layers, batch, hidden) either way. The first axis counts stacked RNN layers; here there is one.

CHECK YOURSELF 1

Predict the parameter count of `nn.RNN(input_size=8, hidden_size=16)`. Does it change if you feed it a sequence of length 500 instead of 10?

2 The failure mode: vanishing and exploding gradients

◆ INTUITION FIRST

Here is the problem with using the same weight matrix at every step. Backpropagation through time multiplies by that *same* matrix once per time step, all the way back to the start. Repeated multiplication does not stay neutral. It compounds, like repeated stretching or repeated shrinking.

☂ EVERYDAY PARALLEL

A message passes down a long line of people. Each person repeats it at 90% of the volume they heard. After 10 people it is at 35% of the first volume. After 50 people it is at 0.5%, and you can no longer hear it. If each person instead repeats it 10% louder, after 50 people it is over 100 times louder. A small change per step, repeated along a long chain, makes the signal fade away or blow up.

◆ NOW THE MATH

Let's find how the last hidden state depends on an early one. Write $\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$, so $\mathbf{h}_t = \tanh(\mathbf{a}_t)$. One step back, the chain rule gives a Jacobian (the matrix of all partial derivatives):

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(1 - \mathbf{h}_t^2) \mathbf{U}$$

In words: the derivative of $\tanh$ is $1 - \tanh^2$. So the Jacobian is $\mathbf{U}$, with each row scaled by the $\tanh$ slope of that unit. $\text{diag}(\cdot)$ puts a vector on the diagonal of a matrix. Each slope is between 0 and 1.

Going back T steps multiplies T of these Jacobians:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_0} = \prod_{t=T}^1 \text{diag}(1 - \mathbf{h}_t^2) \mathbf{U}$$

In words: the gradient that connects the last hidden state to the first passes through T copies of $\mathbf{U}$, each scaled by slopes of at most 1. Recall from Section 1 that a matrix's **eigenvalues** are its stretch factors, and that repeated multiplication raises them to a power. If the largest eigenvalue of $\mathbf{U}$ (in absolute value) is below 1, the product shrinks toward zero as T grows. The $\tanh$ slopes can only shrink it more. This is the **vanishing gradient** problem. It is the same effect as the eigenvalue example in Section 1 and the depth problem in Section 12. If the largest eigenvalue is above 1, the product *can* blow up: the **exploding gradient** problem. It does not always blow up, because the $\tanh$ slopes can pull it back down. So an eigenvalue above 1 is needed for explosion, but it is not enough on its own (Pascanu et al., 2013).

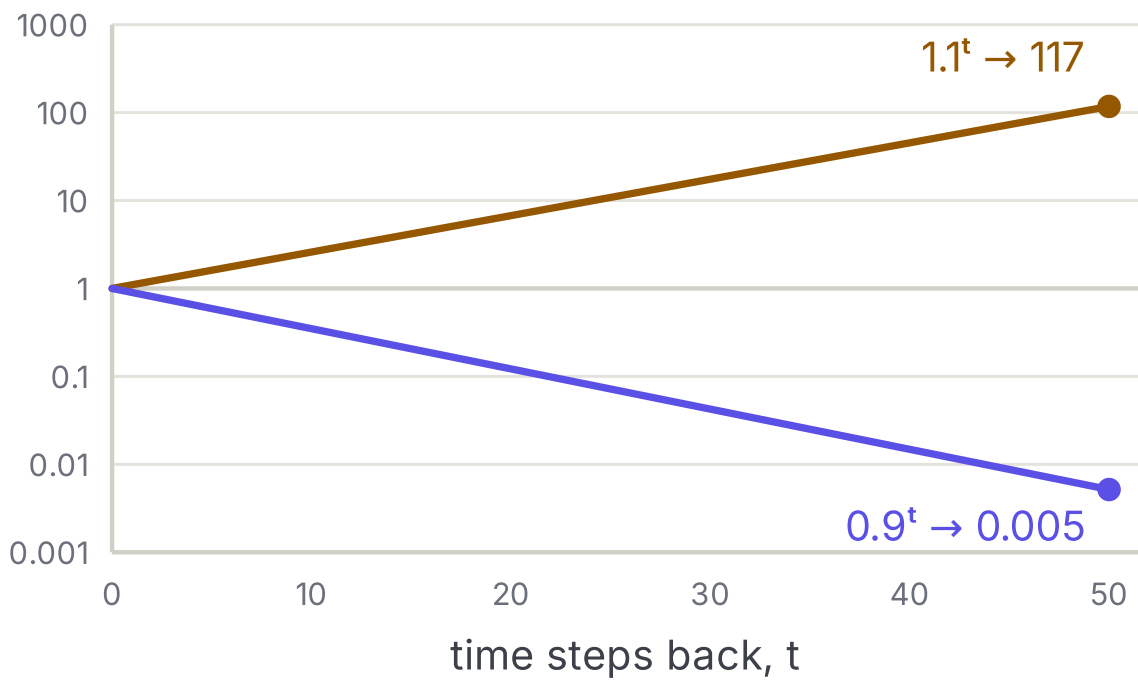


Figure 17.2 Repeated factors of 0.9 and 1.1, on a log scale. Both start at 1. After 50 steps, one has shrunk about 200 times and the other has grown over 100 times. A gradient that passes through 50 copies of $\mathbf{U}$ behaves like one of these lines.

▲ WATCH OUT

This is why plain RNNs do badly on long-range tasks. An example: “does the verb at the end of a long paragraph agree with the subject at the start?” The gradient must pass back through dozens of update steps to link the two. By then, it has often faded to nearly nothing. At prediction time the same happens to the hidden state: information about the early subject gets overwritten step by step.

CHECK YOURSELF 2

Suppose each backward step scales the gradient by about 0.9. How much weaker is the signal from 50 steps back than from 1 step back? What does that mean for learning?

3 LSTMs: gates that choose what to remember

◆ INTUITION FIRST

The **LSTM** (Long Short-Term Memory) is an RNN variant designed against the vanishing gradient. A plain RNN pushes everything through one $\tanh$ update every step. An LSTM instead keeps a separate **memory cell**. It uses three learned **gates** to decide, at every step, what to forget, what to add, and what to read out.

📌 EVERYDAY PARALLEL

Imagine you keep a notebook while you read a long report. On each page you make three decisions. Which old notes do you *cross out* because they no longer matter? (The **forget gate**.) Which new fact do you *write down*? (The **input gate**.) How much of the notebook do you *read back* to use right now? (The **output gate**.) The notebook itself, the memory cell, stays the same from page to page except where you edit it. You do not rewrite it from scratch on every page.

DEFINITION

A **gate** in an LSTM is a small learned layer followed by the sigmoid function $\sigma(\cdot)$, so each output is between 0 and 1. It works like a volume knob for each element. Near 0 it blocks information; near 1 it lets information through unchanged. The forget, input and output gates are three separate knobs, each learned for its own job.

◆ NOW THE MATH

At step t , the LSTM reads $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$. It computes three gates and one candidate memory. Each has its own weights, and each has the same form as the RNN update:

$$\begin{aligned}\mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)\end{aligned}$$

In words: $\mathbf{f}_t$ is the forget gate, $\mathbf{i}_t$ the input gate, $\mathbf{o}_t$ the output gate, and $\tilde{\mathbf{c}}_t$ the candidate memory. All four read the same inputs. The three gates use a sigmoid, so they are knobs between 0 and 1. The candidate $\tilde{\mathbf{c}}_t$ uses $\tanh$, so it is new content between -1 and 1 .

Then the cell and the hidden state are updated:

$$\begin{aligned}\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)\end{aligned}$$

In words: $\odot$ means element-by-element multiplication. The new memory is the old memory, scaled by the forget gate, *plus* new content, scaled by the input gate. The hidden state is a

gated readout of the memory.

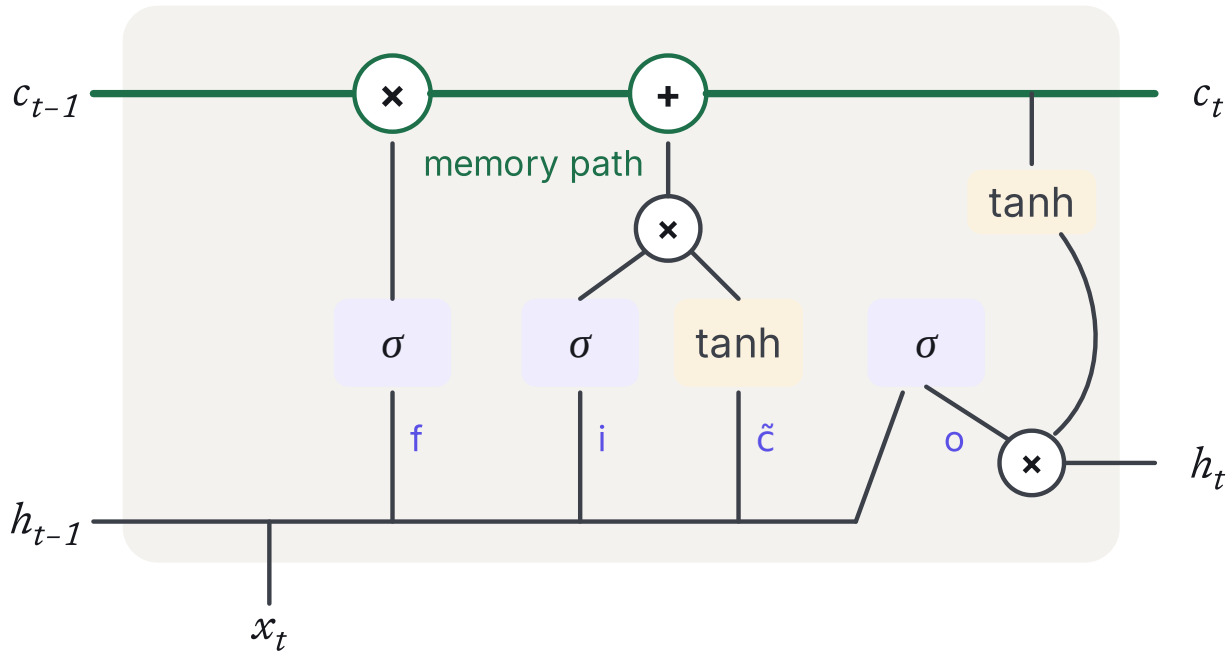


Figure 17.3 One LSTM step. Along the green memory path, c_{t-1} meets only an element-wise multiply by f_t and an addition. There is no matrix $\mathbf{U}$ and no squashing on this path. That is why gradients can travel along it for many steps.

Now see why this helps the gradient. Follow only the memory path and treat the gates as given. Then one step back is:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \text{diag}(\mathbf{f}_t)$$

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_0} = \text{diag}(\mathbf{f}_T \odot \cdots \odot \mathbf{f}_1)$$

In words: on the memory path, each backward step multiplies the gradient by the forget gate. No $\mathbf{U}$, no $\tanh$ slope. If the network learns to keep a fact (f_t near 1 for that unit), the gradient for that unit passes back almost unchanged. Compare the RNN's product of $\text{diag}(1 - \mathbf{h}_t^2) \mathbf{U}$ terms. (The gates also depend on $\mathbf{h}_{t-1}$, which adds more paths. We ignore those here; the memory path is the one that carries long-range gradients.)

This additive update is similar to the $\mathbf{x} + F(\mathbf{x})$ skip connection of Section 16. Both give the gradient a path with no repeated matrix multiplication. The LSTM eases the vanishing gradient, but it does not remove it. The forget gates must learn values near 1, and a product of many values like 0.95 still shrinks. The code below shows how much the forget gate matters.

◆ THE SAME THING IN PYTORCH

›_ AN LSTM, AND THE FORGET GATE CONTROLLING THE GRADIENT PATH

```
import torch
import torch.nn as nn

torch.manual_seed(0)
lstm = nn.LSTM(input_size=8, hidden_size=16, batch_first=True)
x = torch.randn(4, 10, 8)          # (batch=4, sequence length=10, feat
h0 = torch.zeros(1, 4, 16)         # (layers, batch, hidden): batch_fir
c0 = torch.zeros(1, 4, 16)         # LSTMs carry a memory cell next to

outputs, (h_last, c_last) = lstm(x, (h0, c0))
print(outputs.shape) # torch.Size([4, 10, 16])
print(c_last.shape)  # torch.Size([1, 4, 16]) → the memory cell, separa
print(lstm.weight_hh_l0.shape) # torch.Size([64, 16]) → 4 blocks (i, f,

def grad_to_first_input(model, T=50):
    x = torch.randn(1, T, 8, requires_grad=True)
    _, (h, c) = model(x)
    c.sum().backward() # how much does step 1 still affect
    return x.grad[0, 0].norm().item()

print(f"{grad_to_first_input(lstm):.1e}") # 1.7e-11 tiny: forget gates
with torch.no_grad():
    lstm.bias_ih_l0[16:32] += 5.0 # rows 16..31 are the forget gate: f
print(f"{grad_to_first_input(lstm):.1e}") # 1.0e+00 the cell path now
```

In words: PyTorch stores the four blocks stacked in one matrix, in the order input gate, forget gate, candidate (called **g**), output gate. The experiment changes only the forget-gate bias. With gates near 0.5, the gradient from step 1 to step 50 is about 10^{-11} . With gates near 0.99, it is about 1. This is why many LSTM implementations start the forget-gate bias at 1 or higher.

CHECK YOURSELF 3

At the start of training, the forget-gate bias is near 0, so $\mathbf{f}_t \approx \sigma(0) = 0.5$. Estimate the memory-path gradient factor over 50 steps. Is that better than the plain RNN with factor 0.9?

4 Why we moved on: the case for attention

◆ INTUITION FIRST

LSTMs eased the gradient problem. But they did not change one other limit. An RNN must process a sequence **one step at a time**. Step 50 needs the hidden state from step 49, which needs step 48, and so on back to the start. You cannot compute step 50 before step 49 is done.

☂ EVERYDAY PARALLEL

Compare two ways to read a book. In the first, one person reads it aloud, page by page. In the second, you give every page to a different reader in a room. They all read at the same time, then compare notes. The second way only works if each reader can look at any other page when needed. They cannot depend only on what the previous reader passed along.

This order of steps has a concrete cost. GPUs are built to do very many independent calculations *at the same time*. An RNN cannot use that parallel power along the time axis. Step 50 must wait for step 49, even when many processors are idle. So training on long sequences is slow. And even LSTMs still lose information over very long distances.

◆ THE OPENING FOR ATTENTION

In an RNN, information from position 3 must pass through every step to reach position 300. What if position 300 could look at position 3 directly? There would be no relay, and nothing would stop us from computing many positions at once. That is the idea behind Section 18: attention and the transformer. They replace the step-by-step hidden state with a way for every position to look directly at every other position.

CHECK YOURSELF 4

An RNN reads a 1,000-token sequence on a GPU that can run 1,000 independent calculations at once. How many steps must run one after another? What would it be if every position could read every other position directly?

5 What to carry forward

- An **RNN** reads a sequence step by step. It keeps a **hidden state** that is updated by the same weights at every step: $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$.
- **Unrolling** turns the recurrence into a deep chain of identical steps. Training it is backpropagation through time.
- The gradient over T steps is a product of T Jacobians $\text{diag}(1 - \mathbf{h}_t^2) \mathbf{U}$. So it tends to **vanish** when $\mathbf{U}$'s largest eigenvalue is below 1, and it can **explode** when it is above 1.
- **LSTMs** add gates and an additive memory cell, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. On the memory path the gradient is scaled only by the forget gates. This eases the problem but does not remove it.
- RNNs and LSTMs are **sequential**: they cannot run in parallel across time steps. That, and their limited long-range memory, is what motivates attention and transformers.

Next, we drop the recurrence entirely and see how a model can look at a whole sequence at once.

Answers to “Check yourself”

1. Predict the parameter count of `nn.RNN(input_size=8, hidden_size=16)`. Does it change if you feed it a sequence of length 500 instead of 10?

$\mathbf{W}$ is $16 \times 8 = 128$ numbers and $\mathbf{U}$ is $16 \times 16 = 256$. PyTorch keeps two bias vectors of 16 each (one for the input part, one for the hidden part). So the total is $128 + 256 + 32 = 416$. The count does not change with the sequence length. The weights describe one *update step*, not the whole sequence. A 500-step sequence runs the same update 500 times instead of 10.

2. Suppose each backward step scales the gradient by about 0.9. How much weaker is the signal from 50 steps back than from 1 step back? What does that mean for learning?

$0.9^{50} \approx 0.005$, so about half a percent of the signal survives. Compared with one step back (0.9), the signal is roughly 170 times weaker. The weight update is a sum over all steps, so the short-range terms fill it almost completely. The network learns short-range patterns well and long-range ones barely at all. This is why plain RNNs have a “short memory” in practice, even though the hidden state could in principle carry information from every step.

3. At the start of training, the forget-gate bias is near 0, so $\mathbf{f}_t \approx \sigma(0) = 0.5$. Estimate the memory-path gradient factor over 50 steps. Is that better than the plain RNN with factor 0.9?

$0.5^{50} \approx 9 \times 10^{-16}$. That is far *worse* than $0.9^{50} \approx 0.005$. The additive memory path is not magic: it only helps when the forget gates are close to 1. The LSTM's advantage is that the network can *learn* to set $\mathbf{f}_t$ near 1 for the units that must remember. Nothing else on the path shrinks the gradient. In a plain RNN, $\mathbf{U}$ and the tanh slopes are in the way at every step. This is why the forget-gate bias is often set to 1 or more at the start.

4. An RNN reads a 1,000-token sequence on a GPU that can run 1,000 independent calculations at once. How many steps must run one after another? What would it be if every position could read every other position directly?

The RNN needs 1,000 steps in a row, one per token, because each needs the previous hidden state. The extra processors do not help along the time axis. If each position can read the others directly, the positions do not wait for each other. All 1,000 can be computed in one parallel step per layer. The gradient path between any two positions is also one step long, instead of up to 1,000.

Attention & Transformers

The architecture behind modern large language models, derived step by step, with the shape of every matrix written out.

68 min read

YOU'LL LEARN

- scaled dot-product attention (from scratch)
- multi-head attention
- positional encodings
- the full transformer block, dimension by dimension

Before you start: Section 14: `nn.Module` & the Training Loop and Section 17: Sequence Models: RNNs & LSTMs.

Most large language models you have heard of (GPT, Claude, Llama, Gemini) are a tall stack of one repeated part: the **transformer block**. The central operation inside that block is called **attention**. When you understand attention, the rest of a modern language model becomes engineering, not magic. So we will build it here from the dot product you already know. We will write down the shape of every matrix as we go, because in this subject the shapes *are* the understanding.

◆ THE ONE IDEA TO HOLD ONTO

Attention lets **every position in a sequence look directly at every other position** and take the information it needs. All positions do this at the same time, in parallel. This is what replaced the slow, forgetful recurrent networks of Section 17. Everything below is the detail of how that lookup is computed.

1 Why we needed something new

◆ INTUITION FIRST

In Section 17 we met **recurrent neural networks** (RNNs). They read a sentence one word at a time and carry a running summary, the *hidden state* (a fixed-size memory vector), from each word to the next. They work, but they have two problems.

- **Long-range forgetting.** To connect the last word of a paragraph to the first, the information must survive every step in between. It fades. This is the vanishing-gradient problem from Section 17.
- **No parallelism.** Word 50 cannot be processed until word 49 is done, because it needs the hidden state from word 49. So the work cannot be spread across the thousands of cores in a modern GPU, and training is slow.

We want two things at once. First, **direct connections between far-apart words**, so nothing has to survive a long relay. Second, the ability to **process every word at the same time**, so training is fast. Attention gives both, with an idea that is surprisingly simple.

☂ EVERYDAY PARALLEL

Imagine reading a mystery novel in two ways. The **RNN way**: you may keep only one small note of memory, and you must rewrite it after every sentence. By the last chapter, the early clues are lost. The **attention way**: the whole book lies open on a large table. To understand any sentence, you may look directly at any other sentence, however far back. Nothing fades on the way. And a room full of readers can each study a different sentence at the same time.

CHECK YOURSELF 1

In “The trophy didn't fit in the suitcase because it was too big,” what does “it” refer to? Is the hard part here the distance between the words, or something else?

2 Scaled dot-product attention, from scratch

◆ INTUITION FIRST

Here is the whole mechanism in one picture, and it is a picture of a **lookup**. Every position asks a question. Every position also shows a label that says what it is about, and every position offers some content. A position

builds its answer as a *blend* of everyone's content. The blend is weighted by how well each label matches its question. That is attention. Three roles do all the work:

- The **query (q)**: what this position is looking for. ("I'm a pronoun; I need the noun I refer to.")
- The **key (k)**: a label each position shows about itself. ("I'm a concrete singular noun.")
- The **value (v)**: the content a position hands over when it is selected. ("Here is my meaning: *trophy*.")

☂ EVERYDAY PARALLEL

Think of a library search. Your **query** is the phrase you type into the catalog. Each book has a spine label, its **key**, and its pages, its **value**. A normal search picks the one book whose label matches best and gives you that book. Attention is the *soft* version. It returns a blend: 70% of this book's pages, 20% of that one's, 10% of a third. The shares depend on how well each label matched your query. So attention is a **soft dictionary lookup**. A dictionary returns one value for an exact key. Attention returns a weighted mix of all values, graded by how similar each key is to the query.

◆ NOW THE MATH

We start from an input we already have: a sequence of vectors, one per position (per word). We stack them as the rows of a matrix $\mathbf{X}$.

$$\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$$

In words: $\mathbf{X}$ has one row per position. T is the *sequence length*: how many words (or tokens) we process. d_{model} is the *model dimension*: the length of the vector for each word (512 in the original transformer). So $\mathbf{X}$ has T rows, and each row is a vector of length d_{model} .

Now the one trick. We do not use $\mathbf{X}$ directly as query, key and value. We *project* it into three different spaces with three learned weight matrices. Then the model can learn what makes a good question, a good label, and good content. The three need not be the same.

$$\begin{aligned}\mathbf{Q} &= \mathbf{X}\mathbf{W}_Q^\top \\ \mathbf{K} &= \mathbf{X}\mathbf{W}_K^\top \\ \mathbf{V} &= \mathbf{X}\mathbf{W}_V^\top\end{aligned}$$

In words: as in the rest of this course, we store a weight matrix with shape (out, in), like `nn.Linear`. For a stack of row vectors $\mathbf{X}$, the layer is $\mathbf{X}\mathbf{W}^\top$ (see Section 14). Each row of $\mathbf{Q}$ is one position's query, and so on.

Track every shape with the shape rule from Section 1 (inner dimensions cancel, outer dimensions stay):

Object	Shape	What it is
$\mathbf{W}_Q$	$d_k \times d_{\text{model}}$	learned: how to build a query
$\mathbf{W}_K$	$d_k \times d_{\text{model}}$	learned: how to build a key
$\mathbf{W}_V$	$d_v \times d_{\text{model}}$	learned: how to build a value
$\mathbf{Q}$	$T \times d_k$	one query per position
$\mathbf{K}$	$T \times d_k$	one key per position
$\mathbf{V}$	$T \times d_v$	one value per position

In words: d_k is the length of each query and each key. They must have the same length, because we are about to compare them. d_v is the length of each value, and it can be any size. Very often $d_k = d_v$, and we will assume that from now on to keep the picture clean.

Step 1: scores (how well does each query match each key?)

To compare query i with key j , we use the **dot product** from Section 1, our alignment meter. A large dot product means “this query and this key point the same way: a strong match.” We want *every* query against *every* key, and that is one matrix multiply:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$$

$$(T \times d_k)(d_k \times T) = (T \times T)$$

In words: $\mathbf{K}^\top$ is $\mathbf{K}$ transposed (rows and columns swapped), so the shared d_k lines up and cancels. Entry S_{ij} is the dot product of query i with key j . It is one number that says how much position i should attend to position j . The result is a square $T \times T$ table: every position scored against every position. This one matrix is where the promise “look at everything at once” is kept.

Step 2: the scaling, and why we need it

We do not use $\mathbf{S}$ as it is. We divide it by $\sqrt{d_k}$. This looks like a fudge factor, but statistics forces it on us. Let's derive it.

Suppose the entries of a query $\mathbf{q}$ and a key $\mathbf{k}$ are independent random numbers, each with mean 0 and variance 1. (This is a fair assumption for freshly initialized weights.) Their dot product is a sum of d_k products:

$$\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$$

Take one term $q_i k_i$. Because q_i and k_i are independent with mean 0:

$$\begin{aligned}\mathbb{E}[q_i k_i] &= \mathbb{E}[q_i] \mathbb{E}[k_i] = 0 \\ \text{Var}(q_i k_i) &= \mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2 \\ &= \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] - 0 = 1\end{aligned}$$

In words: $\mathbb{E}$ is the expected value (the long-run average). Var is the variance (how spread out the values are). Each small product has average 0 and variance 1. We used $\mathbb{E}[q_i^2] = \text{Var}(q_i) = 1$, which holds because the mean is 0.

Now add up d_k of these independent terms. Variances of independent terms add:

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$$

In words: the raw score has variance d_k , so its standard deviation (typical size) is $\sqrt{d_k}$. With $d_k = 64$ the scores spread about ± 8 instead of ± 1 : eight times too large.

Why does a large score hurt? The next step is a **softmax** (defined below). Softmax of large numbers *saturates*: one weight goes to nearly 1 and the others go to nearly 0. Look at the softmax derivative. With weights $a_j = \text{softmax}(\mathbf{z})_j$:

$$\frac{\partial a_j}{\partial z_m} = a_j (\delta_{jm} - a_m)$$

In words: δ_{jm} is 1 when $j = m$ and 0 otherwise. If one weight is near 1 and all others are near 0, every term is near 0. For $j = m$ it is $a_j(1 - a_j)$, which is $1 \cdot 0$ or $0 \cdot 1$. For $j \neq m$ it is $-a_j a_m$, and at least one factor is near 0. So the **gradient** (the slope the network learns from; see Section 2) is almost zero, and there is almost no learning signal for the scores.

Dividing by $\sqrt{d_k}$ brings the variance back to 1. This keeps the scores in the middle range of the softmax, where the gradient is large:

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = \frac{\text{Var}(\mathbf{q} \cdot \mathbf{k})}{d_k} = 1$$

In words: dividing a random quantity by a constant divides its variance by that constant *squared*. So dividing by $\sqrt{d_k}$ divides the variance by d_k , from d_k back to 1. That is the reason for the word “scaled” in *scaled* dot-product attention.

▲ WATCH OUT

If you drop the $\sqrt{d_k}$, a small model with small d_k may still train, only worse. The larger d_k is, the larger the unscaled scores are at the start. Then the softmax starts close to one-hot, its gradients are tiny, and training is slow or gets stuck. This is the reason the original transformer paper (Vaswani et al., 2017) gives for the scaling. The code at the end of this section measures the effect.

Step 3: softmax (turn scores into a blend that sums to 1)

The scaled scores are still any real numbers. Some are negative, and they do not add up to anything useful. We want *weights*: all positive and summing to 1, so they describe a real blend. The **softmax** function does this. We apply it to each row separately.

DEFINITION

The **softmax** of a list of numbers $z_1, \dots, z_n$ takes the exponential of each one (this makes it positive and makes the largest one stand out). Then it divides by the total, so the results sum to 1:

$$\text{softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_m e^{z_m}}$$

It turns any list of scores into probabilities. We met it in Section 4. Here it turns match scores into attention weights.

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)$$

In words: $\mathbf{A}$ has shape $T \times T$, and each row sums to 1. Row i of $\mathbf{A}$ is position i 's attention distribution: how much of its answer it takes from each of the T positions. “Softmax over each row” matters. We normalize across the keys that one query is compared with, so each query spends a total attention budget of exactly 1.

Step 4: the weighted blend of values

Finally, each position builds its output by mixing all the values with its attention weights. This is another matrix multiply:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\mathbf{A}}_{T \times T} \underbrace{\mathbf{V}}_{T \times d_v}$$

In words: multiply the $T \times T$ weight table by the $T \times d_v$ stack of values. The shared T cancels, and the result is $T \times d_v$: one new vector per position. Row i of the output is $\sum_j A_{ij} \mathbf{v}_j$, a weighted average of all the values, using position i 's weights. Positions it matched strongly contribute most; the rest contribute little.

◆ THE WHOLE FORMULA IN ONE LINE

$$\begin{aligned} \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \\ = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \end{aligned}$$

Read it left to right. Compare every query with every key ($\mathbf{Q}\mathbf{K}^\top$). Bring the numbers back to a normal size (divide by $\sqrt{d_k}$). Turn them into a blend (softmax). Then mix the values with that blend (multiply by $\mathbf{V}$). Four steps in one line.

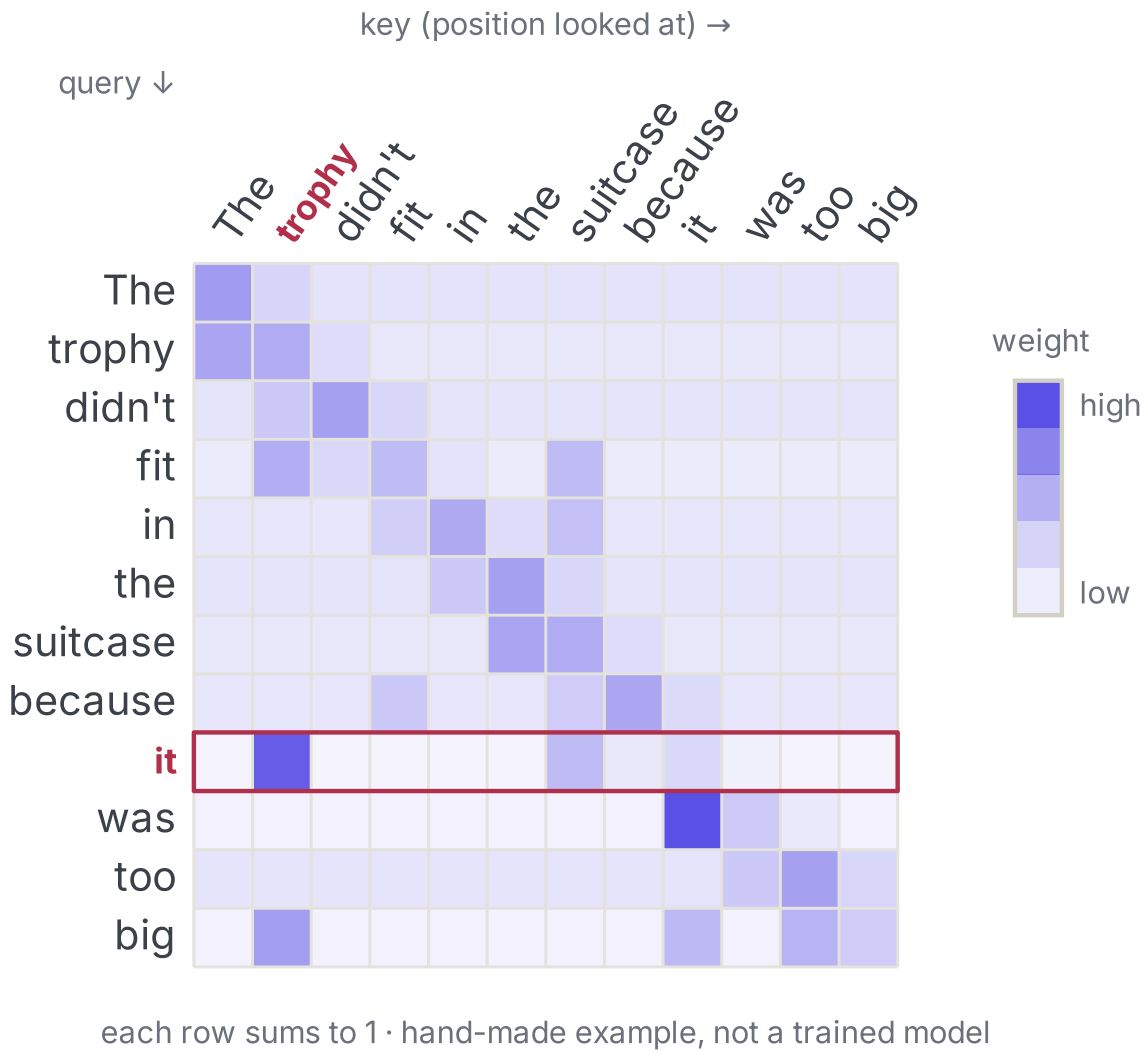


Figure 18.1 What a matrix $\mathbf{A}$ can look like. Each row is one query position and shows how it spreads its weight over the keys. A stronger color means more weight. The row for "it" (outlined) puts 0.54 on "trophy" and 0.18 on "suitcase". *These numbers are made by hand to show the idea; they are not from a trained model.* Real heads often put much weight on the token itself, on neighbors, or on the first token.

A worked example: $T = 3$, $d_k = 4$

Let's make it concrete with small numbers. Take three positions ($T = 3$) and queries, keys and values of length four ($d_k = d_v = 4$). Then:

- $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ are each 3×4 : three rows, four numbers per row.
- $\mathbf{QK}^\top$ is $(3 \times 4)(4 \times 3) = 3 \times 3$. The shared 4 cancels. That gives nine scores: every position against every position.
- Divide all nine by $\sqrt{4} = 2$.
- Softmax each of the 3 rows. The result is a 3×3 table whose rows each sum to 1. Say row 1 is $[0.7, 0.2, 0.1]$. Then position 1 takes 70% of its answer from position 1, 20% from position 2, and 10% from position 3.

- Multiply: $(3 \times 3)(3 \times 4) = 3 \times 4$. The shared 3 cancels, and we get three new vectors of length 4, one for each position. Position 1's output is $0.7 \mathbf{v}_1 + 0.2 \mathbf{v}_2 + 0.1 \mathbf{v}_3$.

Notice that the output has the *same shape* as the value input (3×4). Attention updates each position's vector with context from the other positions. It does not change the number of positions. This is what lets us stack these blocks dozens deep.

◆ THE SAME THING IN PYTORCH

›_ SCALED DOT-PRODUCT ATTENTION IN FOUR LINES, AND A TEST OF THE $\sqrt{D_K}$ ARGUMENT

```

import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V):
    # Q, K: [..., T, d_k]   V: [..., T, d_v]   (leading dims can be batch size)
    d_k = Q.size(-1)           # length of each query/key
    scores = Q @ K.transpose(-2, -1)         # [..., T, T]   QKT: every query dot key
    scores = scores / d_k ** 0.5              # [..., T, T]   the  $\sqrt{d_k}$  scaled dot product
    weights = F.softmax(scores, dim=-1)       # [..., T, T]   each ROW sums to 1
    return weights @ V                     # [..., T, d_v]   blend the values

torch.manual_seed(0)
Q, K, V = torch.randn(3, 4), torch.randn(3, 4), torch.randn(3, 4)   # T = 4
out = scaled_dot_product_attention(Q, K, V)
print(out.shape)                                                       # torch.Size([3, 4])
print(torch.allclose(out, F.scaled_dot_product_attention(Q, K, V), atol=1e-6))

# test the  $\sqrt{d_k}$  argument: random unit-variance q, k with d_k = 512
d_k = 512
q, k = torch.randn(10_000, d_k), torch.randn(10_000, d_k)
print((q * k).sum(-1).var().item())                                   # 517.6...   about d_k
print(((q * k).sum(-1) / d_k ** 0.5).var().item())                   # 1.01...   about 1

# what that does to softmax: one query against 10 keys
z = torch.randn(10, d_k) @ torch.randn(d_k)                         # 10 raw scores, s_i
J = torch.autograd.functional.jacobian
for name, s in [("unscaled", z), ("scaled", z / d_k ** 0.5)]:
    w = F.softmax(s, dim=-1)
    g = J(lambda t: F.softmax(t, dim=-1), s).norm()                  # size of all  $\partial w_i / \partial s_j$ 
    print(f"{name:8s} largest weight {w.max():.3f}   Jacobian norm {g:.1e}")
# unscaled largest weight 1.000   Jacobian norm 9.1e-10
# scaled largest weight 0.368   Jacobian norm 3.4e-01

```

In words: `transpose(-2, -1)` swaps the last two axes to form $\mathbf{K}^T$. `dim=-1` makes softmax normalize across the keys (the last axis), which is “over each row”. The leading `...` is on purpose: the same function will handle batches and heads later. Our function gives the same result as PyTorch's built-in `F.scaled_dot_product_attention`. In the last experiment, the unscaled softmax is one-hot and its derivative is almost zero. The exact numbers change with the seed, but in our runs the unscaled Jacobian was always the smaller one, often by many orders of magnitude.

CHECK YOURSELF 2

A query's scaled scores against three keys are $[5, 5, 5]$ (a perfect tie). What weights does softmax give, and what is the output? What changes if the scores are $[5, 5, 9]$?

CHECK YOURSELF 3

A model uses $d_k = 256$ and forgets the scaling. At the start of training, what is the typical size of a score? Predict what the attention rows look like.

3 Multi-head attention: several lookups at once

◆ INTUITION FIRST

One attention operation gives one blend per position, so it captures one kind of relationship. But a word relates to its neighbors in many ways at once. There is grammar (which noun does this verb agree with?), position (what came just before?), and meaning (what does this pronoun refer to?). One query cannot capture all of these at once. So we run several attentions in parallel. Each one is called a **head**, and each has its own $\mathbf{W}_Q$, $\mathbf{W}_K$ and $\mathbf{W}_V$. Each head can specialize in a different kind of relationship. Then we join their answers back together.

🌂 EVERYDAY PARALLEL

Think of marking up a text with several colored pens, one reading at a time. On the first reading you mark only which verb goes with which subject. On the second, you mark only which pronoun points to which name. On the third, you mark only dates and places. Each reading looks at the same text but asks a different question. At the end you combine all the marks into one set of notes. Each head is one reading with its own question ($\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$). Combining the notes is the final matrix $\mathbf{W}_O$.

◆ NOW THE MATH

We fix a total width d_{model} and split it across h heads. Each head gets a slice of size:

$$d_k = \frac{d_{\text{model}}}{h}$$

In words: with $d_{\text{model}} = 512$ and $h = 8$ heads, each head works in a small space of size $d_k = 64$. Eight 64-wide lookups cost about the same as one 512-wide lookup. So we get several kinds

of relationship at about the same cost as one.

For each head $i = 1, \dots, h$, we run ordinary scaled dot-product attention in its own slice:

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$$

$$\mathbf{Q}_i = \mathbf{X}(\mathbf{W}_Q^{(i)})^\top, \text{ etc.}$$

In words: each head has its own projection matrices $\mathbf{W}_Q^{(i)}$, $\mathbf{W}_K^{(i)}$ and $\mathbf{W}_V^{(i)}$, each of shape $d_k \times d_{\text{model}}$. So each head outputs a $T \times d_k$ matrix.

We lay all h head outputs side by side. This is called **concatenation**: joining matrices along their width.

$$\text{concat}(\text{head}_1, \dots, \text{head}_h)$$

$$\text{shape: } (T \times h d_k) = (T \times d_{\text{model}})$$

In words: because $h \cdot d_k = d_{\text{model}}$ by construction, joining the heads gives back the full width. Size-wise, we are where we started. But each vector now holds h different views.

One last step. After concatenation, each head's result is in its own separate lanes. Nothing has yet let a finding of head 3 combine with a finding of head 7. A final learned matrix $\mathbf{W}_O$ mixes them:

$$\begin{aligned} \text{MultiHead}(\mathbf{X}) \\ = \text{concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O^\top \end{aligned}$$

In words: $\mathbf{W}_O$ has shape $d_{\text{model}} \times d_{\text{model}}$. It takes the joined head outputs and blends them into one d_{model} -wide vector per position. In the colored-pen picture, it combines the separate sets of marks into one set of notes. The final output shape is $T \times d_{\text{model}}$, the same as the input, so blocks can be stacked.

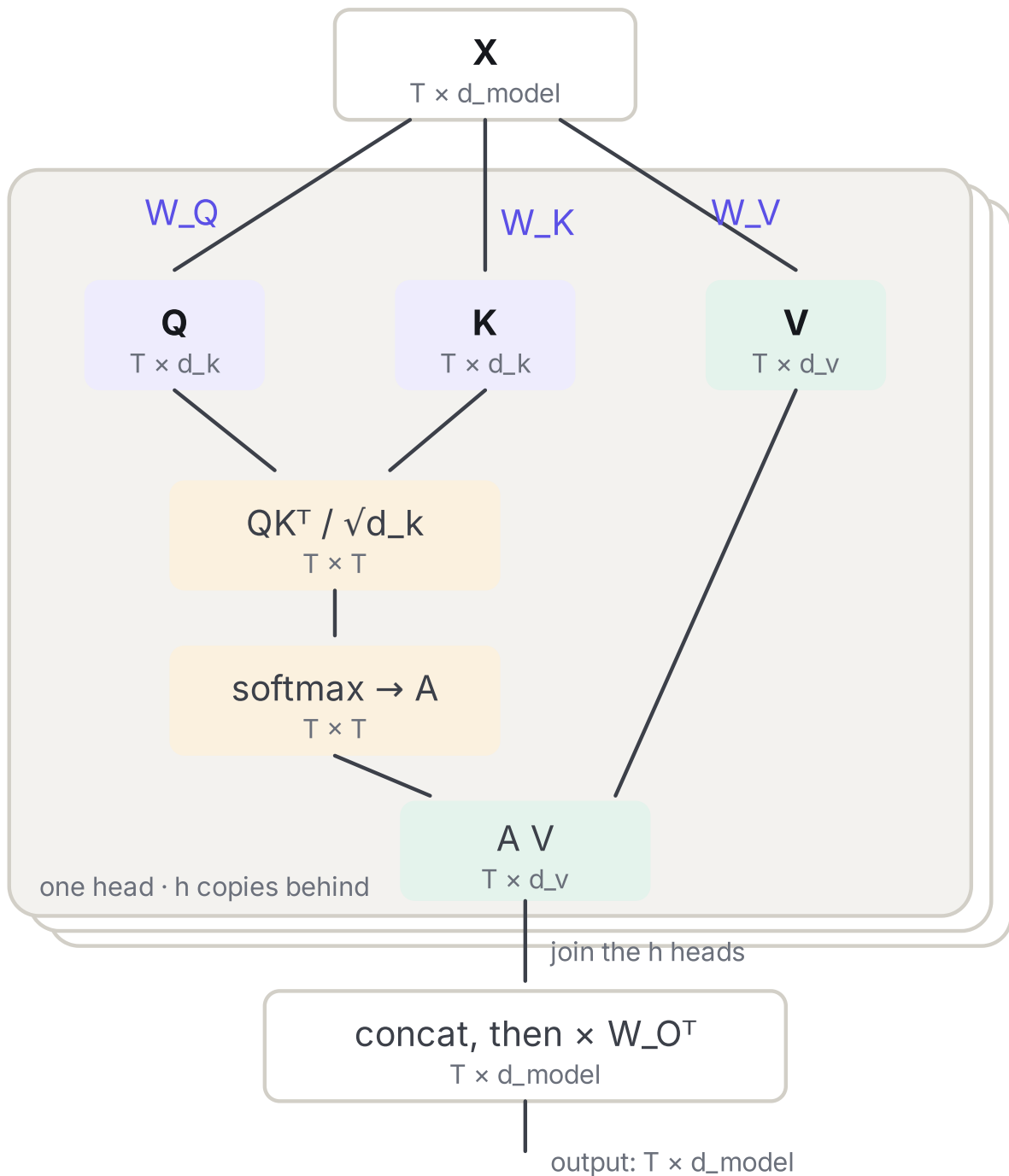


Figure 18.2 The data flow of multi-head attention, with the shape at each step. Inside the shaded frame is one head; there are h copies, each with its own W_Q , W_K , W_V . The only $T \times T$ objects are the scores and the weights A . Everything else keeps one row per position.

◆ WHY HEADS ARE WORTH IT

When researchers inspect trained transformers, some heads do show clear roles. Some track the previous word. Some link verbs to their subjects. Some connect a pronoun to the noun it refers to. No one programmed these roles. The split into heads gave the model room to learn several kinds of relationship, instead of averaging them all into one unclear signal. (Many other heads have no role that is easy to name.)

CHECK YOURSELF 4

A model has $d_{\text{model}} = 768$ and $h = 12$ heads. What is d_k , and what shape is one head's output for 100 tokens? Then: how many parameters does the attention layer have (the four matrices with biases)? Does that number change if you use 24 heads instead?

4 Positional encodings: attention can't tell order

◆ INTUITION FIRST

Attention has a hidden weakness. Look again at how a position builds its output: $\sum_j A_{ij} \mathbf{v}_j$, a sum over the other positions. A sum does not care about order. Shuffle the input words, and you get the same outputs, shuffled in the same way. So attention treats a sentence as a *bag* of words. But “dog bites man” and “man bites dog” are very different sentences. We must give the model some sense of position, because the mechanism itself has none.

☂ EVERYDAY PARALLEL

Think of loose book pages that fell on the floor. To keep their order, a printer can print a faint page-number watermark into the paper of each page. The watermark takes no extra space: it is blended into the page, on top of the text. A careful reader can still read both. Positional encodings work like this. We *add* a position pattern to each word vector, in the same space, instead of attaching extra numbers at the end.

◆ NOW THE MATH

Let $\mathbf{P}$ be a **permutation matrix**: an identity matrix with its rows shuffled. Then $\mathbf{P}\mathbf{X}$ reorders the input positions. Let's follow $\mathbf{P}\mathbf{X}$ through attention. Each projection acts on each row separately, so it keeps the

shuffle: $(\mathbf{P}\mathbf{X})\mathbf{W}_Q^\top = \mathbf{P}\mathbf{Q}$, and the same for $\mathbf{K}$ and $\mathbf{V}$. Then:

$$\begin{aligned}(\mathbf{P}\mathbf{Q})(\mathbf{P}\mathbf{K})^\top &= \mathbf{P}\mathbf{Q}\mathbf{K}^\top\mathbf{P}^\top \\ \text{softmax}(\mathbf{P}\mathbf{S}\mathbf{P}^\top) &= \mathbf{P}\mathbf{A}\mathbf{P}^\top \\ \mathbf{P}\mathbf{A}\mathbf{P}^\top\mathbf{P}\mathbf{V} &= \mathbf{P}\mathbf{A}\mathbf{V}\end{aligned}$$

In words: the scores are shuffled in both rows and columns. Softmax works on each row, and shuffling the entries of a row only shuffles its softmax, so the weights are shuffled in the same way. In the last line, $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$, because shuffling and then unshuffling changes nothing. So:

$$\text{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\text{Attention}(\mathbf{X})$$

In words: reorder the inputs, and you get the same outputs in the new order. Nothing else changes. This property is called *permutation equivariance*. (A *permutation* is a reordering. “Equivariant” means the output moves with the input; “invariant” would mean the output does not change at all.) It is right for a set of items with no natural order. It is wrong for language, where order carries meaning. The fix is not to change attention. The fix is to put each word's position *into* its vector before attention sees it.

So we build a position vector $\mathbf{PE}_{\text{pos}}$ for each slot $\text{pos} = 0, 1, 2, \dots$ and add it to the word's vector. The original transformer uses a fixed pattern of sines and cosines at many frequencies:

$$\begin{aligned}\mathbf{PE}_{(\text{pos}, 2i)} &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \\ \mathbf{PE}_{(\text{pos}, 2i+1)} &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)\end{aligned}$$

In words: pos is the slot in the sentence (0, 1, 2, ...), and i numbers the pairs of dimensions inside the vector. Even dimensions use a sine; the odd dimension next to it uses the matching cosine. The term $10000^{2i/d_{\text{model}}}$ makes the wavelength longer as i grows. The first dimensions change quickly from one position to the next; the last dimensions change slowly.

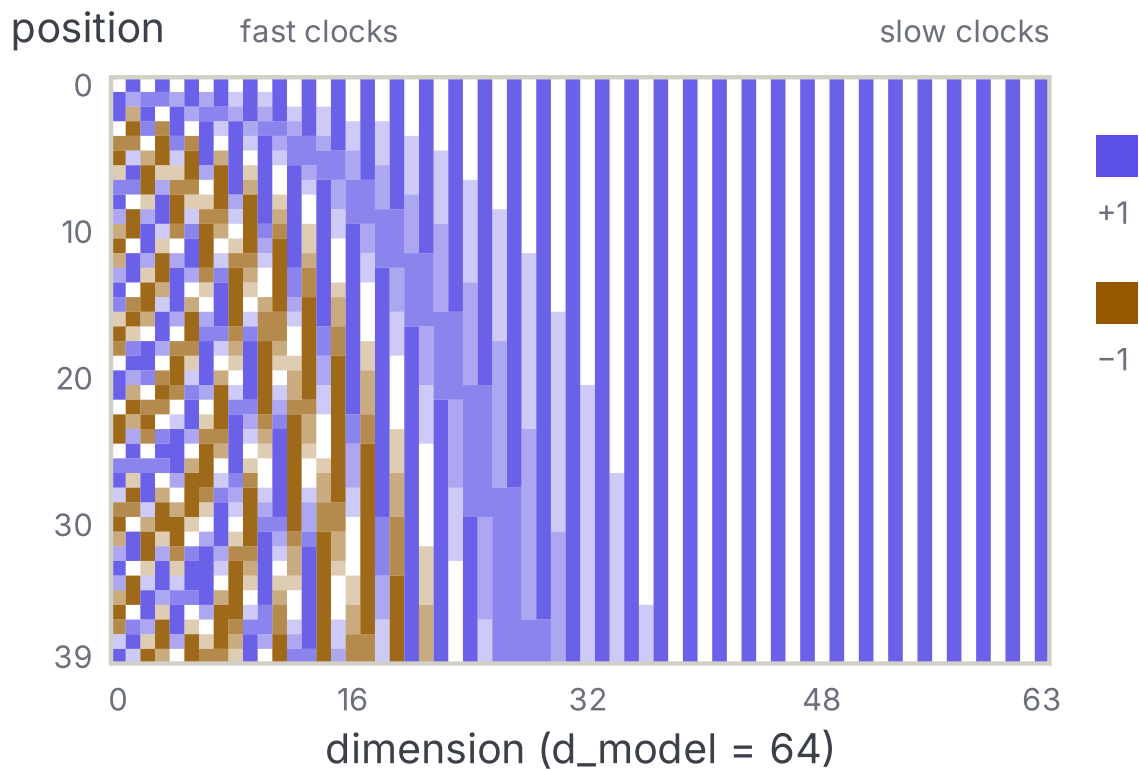


Figure 18.3 The real sinusoidal encoding for positions 0–39 in a model with $d_{\text{model}} = 64$ (violet: positive, amber: negative, stronger color: larger size). Read one row to get one position's vector. The left columns change sign every few positions; the right columns hardly change over 40 positions. Together, the columns give every position a different pattern.

◆ WHY SINES AT MANY FREQUENCIES

Think of a set of clocks whose hands turn at different speeds: seconds, minutes, hours, days. Read all the hands together and you can identify one exact moment. The fast hand separates nearby moments; the slow hand separates moments far apart. The sinusoidal encoding works like this. Each pair of dimensions is a clock at its own speed. The set of all hand positions is a pattern that no other position shares (over the lengths used in practice).

There is also a bonus. Write ω for the frequency of one pair. Moving from position p to $p + k$ turns that pair's "hand" by a fixed angle $\theta = \omega k$. With $s_p = \sin \omega p$ and $c_p = \cos \omega p$:

$$\begin{bmatrix} s_{p+k} \\ c_{p+k} \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} s_p \\ c_p \end{bmatrix}$$

This follows from the sum formulas for sine and cosine. The matrix depends only on the offset k , not on p . So the code for " k positions later" is a fixed linear function of the current code. This makes it easier for the model to learn *relative* offsets such as "three words back."

Other ways to give position are common:

- **Learned positional embeddings.** Keep a table with one trainable vector per position, and let training decide what each should be. GPT-2 and BERT use this. It works well for the lengths seen in training, but it has no vector for longer positions.
- **Rotary position embeddings (RoPE).** Instead of adding a vector to the input, RoPE rotates each query and key, pair by pair, by an angle proportional to the position. It uses the rotation above directly. The dot product of a query at p and a key at p' then depends only on the offset $p - p'$. Most recent open models, such as Llama, use RoPE.

The sine formula is defined for every pos, so it gives a vector even for positions longer than any seen in training. But in practice, models with sinusoidal encodings do *not* work well on sequences much longer than their training sequences (Press et al., 2021). Handling longer sequences needs other methods. For the additive methods the shape is the same ($T \times d_{\text{model}}$) and the step is the same: add it to the input.

◆ THE SAME THING IN PYTORCH

›_ SINUSOIDAL POSITIONAL ENCODINGS, ADDED TO THE INPUT

```

import torch

def positional_encoding(T, d_model):
    pos = torch.arange(T).unsqueeze(1)           # [T, 1]      slot
    i = torch.arange(0, d_model, 2)              # [d/2]      even
    freq = 1.0 / (10000 ** (i / d_model))         # [d/2]      odd
    pe = torch.zeros(T, d_model)                 # [T, d_model]
    pe[:, 0::2] = torch.sin(pos * freq)           # even dims → sin
    pe[:, 1::2] = torch.cos(pos * freq)           # odd  dims → cos
    return pe

torch.manual_seed(0)
X = torch.randn(10, 512)                        # 10 token vectors
X = X + positional_encoding(10, 512)             # [10, 512] pos. enc.
print(X.shape)                                   # torch.Size([10, 512])
print(positional_encoding(10, 512)[1, :4])       # tensor([0.8415, 0.5403, 0.8415, 0.5403])

```

In words: we add; we do not concatenate. The position travels inside the existing d_{model} width, so nothing later changes shape. The last line shows position 1: $\sin 1 = 0.8415$ and $\cos 1 = 0.5403$ for the fastest pair, then a slightly slower pair. After the addition, the same word in two different slots has two different vectors. The permutation equivariance is broken, which is what we want.

CHECK YOURSELF 5

Without positional encodings, feed the block “dog bites man” and then “man bites dog”. Compare the output vector for “dog” in the two runs. Are they the same or different? Why?

5 The full transformer block, dimension by dimension

◆ INTUITION FIRST

We now have the two main parts. **Multi-head attention** lets positions share information across the sequence: the “mixing across words” step. But it is mostly a weighted average, and it does little computation on each word by itself. So each block pairs it with a **feed-forward network** (FFN). This is a small two-layer network applied to each position separately: the “think hard about this one word” step. Around both, we add two

support devices: **residual connections** and **LayerNorm**. That is the complete transformer block. Stack it N times and you have a transformer.

EVERYDAY PARALLEL

Think of a team working on a problem in rounds. Each round has two phases. First, a **meeting**: everyone shares what they know and updates their view. That is attention, mixing information between people. Second, **desk work**: each person works alone on what they just heard. That is the FFN, computation for each position. After many rounds, the team shares a rich understanding. A transformer is that team, and one block is one round.

◆ NOW THE MATH

Residual connections. We do not replace the input with a sub-layer's output. We *add* the output to the input: $\mathbf{x} + \text{Sublayer}(\mathbf{x})$. This is the skip connection that made very deep image networks trainable in Section 16. It helps here for two reasons. It gives each sub-layer an easy “do nothing” option: output zero, and the input passes through. And it gives the gradient a direct path back through the addition, so the signal reaches the first layers without fading. The two sides of the addition must have the same shape. That is why every sub-layer keeps the width d_{model} .

LayerNorm. Before each sub-layer we *normalize* each position's vector. We subtract its mean and divide by its standard deviation. Then we rescale and shift with two learned vectors, γ and β , each of length d_{model} :

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

In words: μ and σ^2 are the mean and the variance of the d_{model} numbers in this one vector. ϵ is a small constant (such as 10^{-5}) that prevents division by zero. $\odot$ is element-by-element multiplication. This keeps the numbers that flow through the network at a steady size, so they do not grow or shrink from layer to layer. Section 20 treats normalization properly.

The feed-forward network. Two linear layers with a nonlinearity between them, applied in the same way to every position. For one position's vector $\mathbf{x}$ (a column vector of length d_{model}):

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

The shape widens and then narrows on purpose:

Step	Shape in → out	Matrix
expand	$d \rightarrow 4d$	$\mathbf{W}_1: 4d \times d$
nonlinearity	$4d \rightarrow 4d$	ReLU / GELU (no weights)
project back	$4d \rightarrow d$	$\mathbf{W}_2: d \times 4d$

In words: here d is short for d_{model} . The FFN makes each position's vector four times wider. It applies a nonlinearity: $\text{ReLU}(z) = \max(0, z)$ keeps positive numbers and sets negative ones to zero. Then it makes the vector narrow again. The wide middle gives the network room to compute richer features for each word. The factor 4 is the standard choice from the original paper. Importantly, the FFN acts on each position *independently*. It does no mixing between positions; all of that happened in attention.

Assembling the block (pre-norm order)

Most modern transformers put the LayerNorm *before* each sub-layer. This is called “pre-norm”, and it trains more stably than the original “post-norm” order, which normalized after the addition. The full block is two lines:

$$\begin{aligned}\mathbf{x} &\leftarrow \mathbf{x} + \text{MultiHead}(\text{LayerNorm}(\mathbf{x})) \\ \mathbf{x} &\leftarrow \mathbf{x} + \text{FFN}(\text{LayerNorm}(\mathbf{x}))\end{aligned}$$

In words: normalize, mix across words with attention, and add the result to the input. Then normalize again, process each word with the FFN, and add that result too. The input shape is $T \times d_{\text{model}}$, and the output shape is the same. This is on purpose, so the next block can take the output directly.

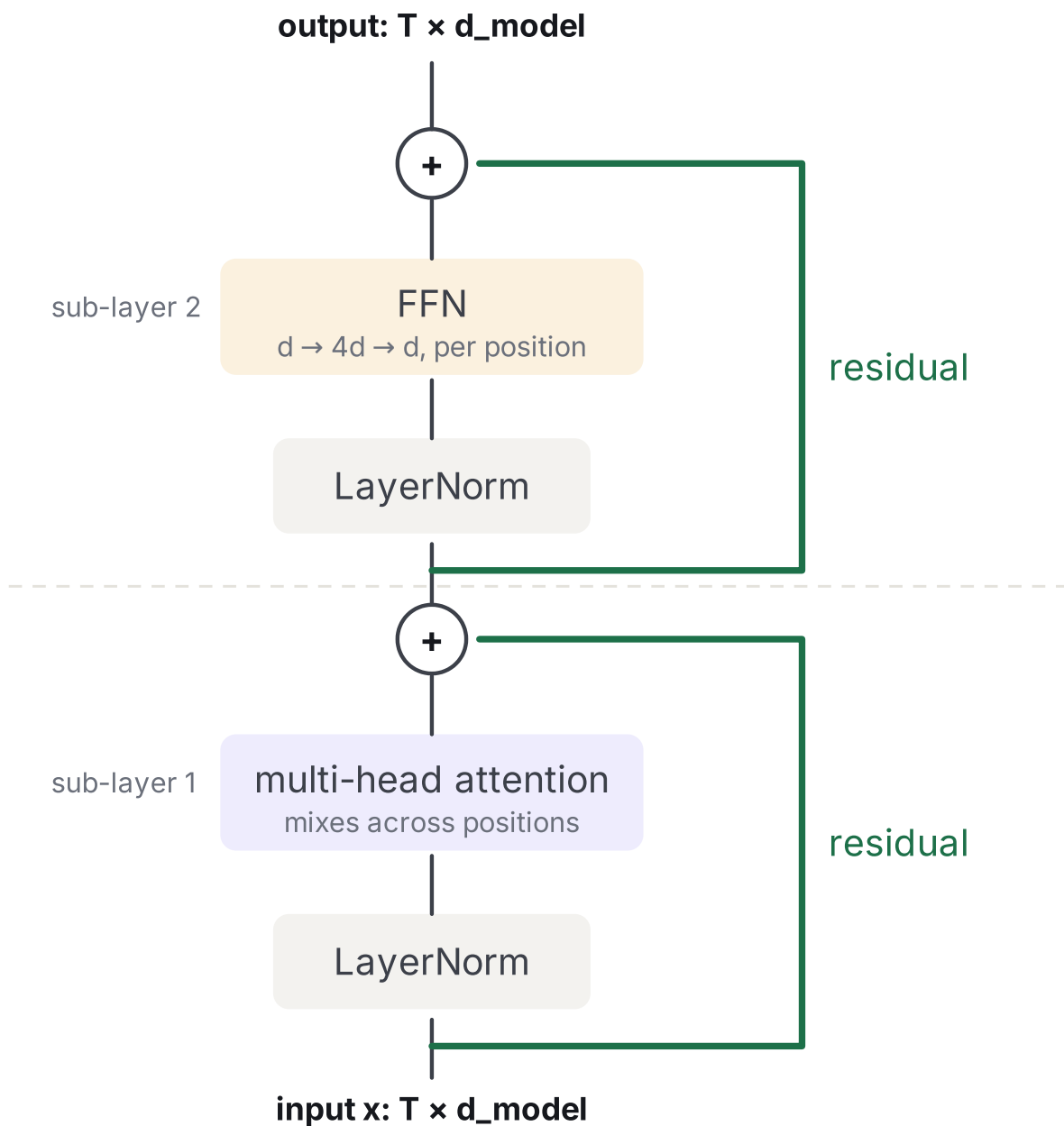


Figure 18.4 One pre-norm transformer block, read from bottom to top. Each sub-layer works on a normalized copy of $\mathbf{x}$, and its result is added back to the unnormalized $\mathbf{x}$ (green paths). The green residual path from input to output never passes through a LayerNorm or a weight matrix.

◆ **THE SHAPE NEVER CHANGES, AND THAT IS THE POINT**

Every sub-layer takes $T \times d_{\text{model}}$ in and gives $T \times d_{\text{model}}$ out. Attention keeps it, the FFN keeps it, LayerNorm keeps it, and the residual addition requires it. Because of this, you can stack 12, 48 or 96 identical blocks. The whole depth of a large model is this one shape, repeated.

▲ WATCH OUT

The block in this lesson is the classic design. Recent large models change some parts. Many use **RMSNorm** instead of LayerNorm: it only divides by the root mean square and does not subtract the mean. Many replace the ReLU/GELU FFN with **SwiGLU**, which multiplies two linear projections, one passed through a smooth gate, before projecting back. And most use RoPE for position (Section 4 above). The structure is the same: pre-norm, attention, residual, pre-norm, FFN, residual.

◆ THE SAME THING IN PYTORCH

›_ MULTI-HEAD ATTENTION AND A FULL TRANSFORMER BLOCK, WITH SHAPES ON EVERY LINE

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0 # heads must divide d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads # slice size per head
        # one big Linear does all heads at once (cheaper than n_heads separate)
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model) # the final mixing layer

    def forward(self, x): # x: [B, T, d_model]
        B, T, _ = x.shape
        # project, then split the width into heads: [B, T, d_model] → [B, T, n_heads, d_k]
        def split(t):
            return t.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        Q, K, V = split(self.W_q(x)), split(self.W_k(x)), split(self.W_v(x))

        scores = Q @ K.transpose(-2, -1) / self.d_k ** 0.5 # [B, n_heads, T, T]
        weights = F.softmax(scores, dim=-1) # [B, n_heads, T, T]
        out = weights @ V # [B, n_heads, T, d_k]

        # concat heads back together: [B, n_heads, T, d_k] → [B, T, d_model]
        out = out.transpose(1, 2).contiguous().view(B, T, self.n_heads * self.d_k)
        return self.W_o(out)

class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff=None):
        super().__init__()
        d_ff = d_ff or 4 * d_model # the standard 4x rule
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(

```

```

        nn.Linear(d_model, d_ff),
        nn.GELU(),
        nn.Linear(d_ff, d_model),
    )

    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        x = x + self.ffn(self.norm2(x))
        return x

torch.manual_seed(0)
block = TransformerBlock(d_model=512, n_heads=8)
x = torch.randn(2, 10, 512)
print(block(x).shape)
print(sum(p.numel() for p in block.parameters()))

# permutation equivariance: shuffle the 10 positions, outputs shuffle the
perm = torch.randperm(10)
print(torch.allclose(block(x[:, perm]), block(x)[:, perm], atol=1e-5))

```

In words: the batch dimension `B` travels in front the whole way. That is why our earlier `scaled_dot_product_attention` used a leading `...`. The code has no loop over heads. Reshaping to `[B, n_heads, T, d_k]` lets one batched matrix multiply do all heads at once: this is the parallelism promised at the start of the lesson. The parameter count is 1,050,624 for attention (four 512×512 matrices with biases), 2,099,712 for the FFN, and 2,048 for the two LayerNorms. The last line checks the equivariance from Section 4 on a whole block.

◆ WHERE THIS GOES NEXT

This block is the *encoder* form: every position may look at every other. To turn a transformer into a text *generator*, we add one thing: a **causal mask** that stops each position from looking at words that come after it. That change is the subject of Section 19. In very large models, the FFN is often replaced by a **mixture of experts**. This is many FFNs, with a router that picks a few of them for each token. That is Section 22. There, only the feed-forward sub-layer changes; the masked self-attention works as it does here.

CHECK YOURSELF 6

Find the bug: someone changes the FFN's last layer to `nn.Linear(d_ff, 2 * d_model)`. What error do you get, on which line, and why?

6 What to carry forward

- **Attention is a soft lookup.** Queries ask, keys label, values are the content. Each position gets a weighted blend of all values, graded by how well its query matches each key.
- **The formula** is $\text{softmax}(\mathbf{QK}^\top / \sqrt{d_k})\mathbf{V}$: compare, rescale, blend, mix. Shapes: $(T \times d_k)(d_k \times T) \rightarrow (T \times T)$, then $(T \times T)(T \times d_v) \rightarrow (T \times d_v)$.
- **The $\sqrt{d_k}$ is derived, not decoration.** Raw scores have variance d_k . Dividing by $\sqrt{d_k}$ brings the variance back to 1 and keeps the softmax away from its flat, near-zero-gradient region.
- **Multi-head** attention splits d_{model} into h heads of size d_{model}/h . It runs attention in each head, concatenates the results, and mixes them with $\mathbf{W}_O$. You get several kinds of relationship for the parameter count of one.
- **Attention is permutation-equivariant**, so we give it position: sinusoidal or learned vectors added to the input, or rotations of queries and keys (RoPE).
- **The block** is: LayerNorm $\rightarrow$ multi-head attention $\rightarrow$ residual add $\rightarrow$ LayerNorm $\rightarrow$ FFN ($d_{\text{model}} \rightarrow 4d_{\text{model}} \rightarrow d_{\text{model}}$) $\rightarrow$ residual add. The shape in equals the shape out, so blocks stack.

You now know the core of a modern language model. Next we add a causal mask and turn this machine into something that *writes*. That is Section 19, language modeling.

Answers to “Check yourself”

1. In “The trophy didn't fit in the suitcase because it was too big,” what does “it” refer to? Is the hard part here the distance between the words, or something else?

“It” means the **trophy**. Change “big” to “small” and it means the suitcase. The distance is short: “trophy” is only 7 words before “it”. The hard part is *knowledge*: you must know that a big thing does not fit into a smaller one. Sentences like this are called *Winograd schemas*, and they test common sense, not memory span. Attention helps in one way: “it” can compare itself with both “trophy” and “suitcase” directly, in one step. Choosing the right one still depends on what the model learned in training. For a truly long-range example, imagine the trophy was named three paragraphs earlier. An RNN would have to carry it through hundreds of steps; attention still reaches it in one.

2. A query's scaled scores against three keys are $[5, 5, 5]$ (a perfect tie). What weights does softmax give, and what is the output? What changes if the scores are $[5, 5, 9]$?

Softmax of three equal numbers is uniform: $[1/3, 1/3, 1/3]$. (The factor e^5 cancels above and below the line.) The output is the plain average of the three values, $\frac{1}{3}(\mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3)$. A query with no reason to prefer any position listens to all of them equally. With $[5, 5, 9]$, only the differences matter: the weights are $e^0 : e^0 : e^4 = 1 : 1 : 54.6$, so about $[0.018, 0.018, 0.965]$. The output is almost $\mathbf{v}_3$. Attention becomes selective only when the scores differ, which is why the learned $\mathbf{W}_Q$ and $\mathbf{W}_K$ matter so much.

3. A model uses $d_k = 256$ and forgets the scaling. At the start of training, what is the typical size of a score? Predict what the attention rows look like.

The variance of a raw score is $d_k = 256$, so the typical size is $\sqrt{256} = 16$. Two scores in the same row often differ by 10 or more, and $e^{10} \approx 22,000$. So most rows are close to one-hot: nearly all the weight on one key. The softmax derivative is then close to zero, and the scores get almost no gradient. With the scaling, the typical size is 1 and the rows are smooth blends.

4. A model has $d_{\text{model}} = 768$ and $h = 12$ heads. What is d_k , and what shape is one head's output for 100 tokens? Then: how many parameters does the attention layer have (the four matrices with biases)? Does that number change if you use 24 heads instead?

$d_k = 768/12 = 64$, and each head outputs shape (100, 64). Joining the 12 heads gives (100, 768), and $\mathbf{W}_O$ maps that to (100, 768). These are the sizes inside BERT-base and GPT-2 small, two well-known early transformers. The four matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) are each 768×768 plus a bias of 768, so the layer has $4 \times (768^2 + 768) = 2,362,368$ parameters. With 24 heads the count is *the same*. The heads only split the same width into narrower slices ($d_k = 32$). The number of heads changes how the width is used, not how many weights there are.

5. Without positional encodings, feed the block “dog bites man” and then “man bites dog”. Compare the output vector for “dog” in the two runs. Are they the same or different? Why?

They are exactly the same. The second input is a reordering of the first, so by permutation equivariance the outputs are the same vectors, reordered. “Dog” gets the same vector whether it is the biter or the one bitten, so the model cannot tell who bit whom. The code in Section 5 checks this property on a full block. Adding positional encodings gives “dog” in slot 0 a different input vector from “dog” in slot 2, so the two sentences now give different results. One nuance: the text-generating models of Section 19 use a causal mask, which already breaks the symmetry. Such models can learn some position information even without positional encodings (Haviv et al., 2022). The block in this lesson, with no mask, cannot.

6. Find the bug: someone changes the FFN's last layer to `nn.Linear(d_ff, 2 * d_model)`. What error do you get, on which line, and why?

The error comes from the **residual addition** `x = x + self.ffn(self.norm2(x))`. The left side is `[B, T, d_model]` and the right side is `[B, T, 2*d_model]`. The last dimensions (512 and 1024) are different and neither is 1, so broadcasting fails and PyTorch raises a `RuntimeError` about the sizes. Even if the addition were allowed, the *next* block expects width `d_model`, so the stack would break there. This is why the FFN can widen to `4*d_model` in the middle but must project back to `d_model` at the end. The residual path fixes the width from input to output.

Language Modeling & GPT

How a stack of transformer blocks becomes something that writes: from characters to sampled text, end to end.

62 min read

YOU'LL LEARN

- tokenization
- embeddings
- next-token prediction
- the GPT-style decoder & causal masking

Before you start: Section 18: Attention & Transformers.

A model that writes essays, answers questions, and drafts code sounds like magic. It is not. Underneath, it does one small thing over and over: given the text so far, it **guesses the next piece**. Do that guess well enough, feed each guess back in, and coherent paragraphs come out. This section builds that machine from the ground up: from raw characters, to numbers, to a stack of transformer blocks, to sampled words on a screen.

◆ THE ONE IDEA TO HOLD ONTO

A language model is a **next-token predictor**. Everything else (tokenization, embeddings, attention, the causal mask, sampling) exists to make that one prediction accurate, and then to run it in a loop. Once you see the loop, the field stops looking mysterious.

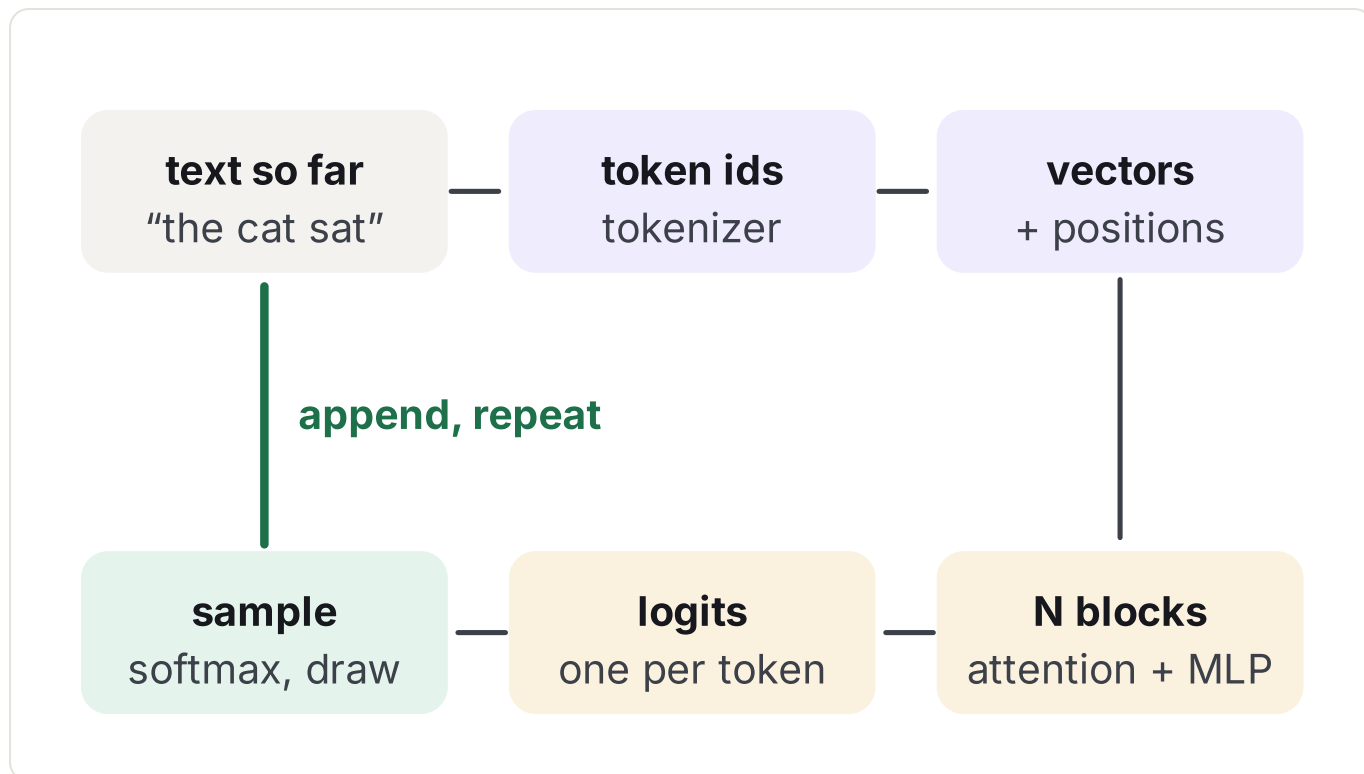


Figure 19.1 The whole machine in one loop. Text becomes ids, ids become vectors, the blocks turn vectors into scores, and one sampled token is added to the text. Then the loop runs again.

1 What a language model is

◆ INTUITION FIRST

A **language model** is a function that reads a stretch of text. It outputs a probability for *every possible next piece of text*. Given “the cat sat on the”, it says: “mat” is very likely, “roof” is plausible, “democracy” is almost impossible. That is the entire job. A *token* is the small unit it predicts. For now, think of it loosely as a word; we make it precise in a moment.

☂ EVERYDAY PARALLEL

Your phone keyboard's autocomplete. You type “I’ll be there in five” and it offers “minutes” before you touch another key. It has learned from many past messages which word tends to come next. A language model plays the same guessing game. It is much larger, and it is trained to keep the guesses coherent across whole paragraphs, not one word.

◆ NOW THE MATH

Write the text as a sequence of T tokens $x_1, x_2, \dots, x_T$. The model estimates the probability of the whole sequence by multiplying together the probability of each token given everything before it:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

In words: the chance of a whole sentence is a product. Take the chance of the first token. Multiply by the chance of the second *given* the first. Then multiply by the chance of the third *given* the first two, and so on. The big $\prod$ ("product") means "multiply all these together." This is the chain rule of probability, so it holds for any sequence. The one factor we train the model to produce is $P(x_t \mid x_{<t})$: the next token given the past. Get that factor right and the whole product is right.

DEFINITION

An **autoregressive** model generates a sequence one step at a time. It feeds each output back in as input for the next step. "Auto" means self, and "regressive" means predicting from earlier values. Every GPT-style model works this way: predict a token, append it, predict the next.

CHECK YOURSELF 1

Why do we condition each token on the tokens *before* it, rather than modeling $P(x_t)$ on its own?

2 Tokenization: turning text into tokens

◆ INTUITION FIRST

A computer cannot multiply matrices by the letter "q". Before anything else, we must cut raw text into pieces from a fixed list and give each piece a number. That step is **tokenization**. The obvious question is: what should a piece be? A letter, a word, or something in between?

DEFINITION

A **token** is one unit of text the model reads and predicts. The **vocabulary** is the fixed, finite set of all tokens the model knows (for example 50,000 of them). Each token has a unique integer id from 0 to $V - 1$, where V is the vocabulary size. Tokenizing means rewriting a string as a list of these ids.

There are three natural choices for “what is a piece”, and each has a cost:

Scheme	A token is...	Vocabulary size	The catch
Character	one letter or symbol	tiny (about 100)	Sequences get very long. The model must learn spelling from scratch, which uses capacity on how letters form words.
Word	one whole word	huge (millions)	Any word not seen in training becomes an <i>unknown token</i> (one catch-all id for “I have never seen this”). The model cannot read new names, typos, and rare words.
Subword	a frequent chunk of characters	fixed, chosen (about 30k to 256k)	The common choice today. Examples: 50,257 in GPT-2, about 128k in Llama 3, about 256k in Gemma.

Most modern models use **subword** tokenization. A common algorithm is **BPE** (*Byte-Pair Encoding*, used by GPT and Llama). It starts from single characters and repeatedly merges the most frequent adjacent pair into a new token. Common words like “the” end up as a single token. A rare word like “antidisestablishmentarianism” is split into several familiar pieces. Other subword methods exist: BERT uses *WordPiece*, and T5 uses a *Unigram* model (both through libraries such as SentencePiece). The ideas are the same.

Can a subword tokenizer meet an unknown piece? It depends on the base alphabet. If the merges start from *bytes* (byte-level BPE, as in GPT-2), every possible string is a list of the 256 byte values. So no text is ever unknown. A tokenizer that starts from characters can still meet a character it never saw, such as a rare symbol. Many such tokenizers add a “byte fallback” to fix this.

🔧 EVERYDAY PARALLEL

Think of LEGO. Whole-word tokens are like pre-built models: great until you need a shape the set does not include. Single-character tokens are like having only 1×1 bricks: you can build anything, but it takes a very long time. Subword tokens are the normal LEGO box, with small and medium bricks. Frequent shapes come pre-assembled, and you build unusual shapes from smaller pieces. The box of bricks is fixed, but you can build any object.

◆ WHY SUBWORD WINS, IN ONE LINE

It gives you a **fixed vocabulary** (so the model's size is bounded) and **handles rare words by breaking them into common pieces** the model already knows. With a byte-level base, it also has **no unknown tokens**. Small vocabulary, full coverage.

To keep this section self-contained and runnable with no downloads, we use the simplest possible tokenizer for all the code below: **character-level**. The ideas are the same for BPE; only the vocabulary is bigger.

>_ A CHARACTER-LEVEL TOKENIZER: TEXT $\rightleftharpoons$ TOKEN IDS (PLAIN PYTHON)

```
text = "hello world"

# the vocabulary = the set of distinct characters, sorted for a stable order
chars = sorted(set(text))          # [' ', 'd', 'e', 'h', 'l', 'o', 'r', 'w']
vocab_size = len(chars)            # 8

stoi = {c: i for i, c in enumerate(chars)}  # string  $\rightarrow$  integer id
itos = {i: c for c, i in stoi.items()}      # integer id  $\rightarrow$  string

def encode(s): return [stoi[c] for c in s]   # text  $\rightarrow$  list of ids
def decode(ids): return "".join(itos[i] for i in ids) # ids  $\rightarrow$  text

print(encode("hello"))                # [3, 2, 4, 4, 5]
print(decode([3, 2, 4, 4, 5]))        # hello (round-trips exactly)
```

CHECK YOURSELF 2

A word-level model is trained on news from 2010. Then it reads a 2024 article that mentions “COVID”. What happens, and how does a byte-level subword tokenizer help?

CHECK YOURSELF 3

Run the tokenizer above on the string `"hello!"`. What happens, and why?

3 Embeddings: token ids become vectors

◆ INTUITION FIRST

A token id like `3` is only a name tag. The number itself carries no meaning (id 3 is not “more” than id 2 in any useful sense). The model needs each token to be a **vector** (a list of numbers; see Section 1) that it can learn to fill with meaning. An **embedding** is a lookup table that maps each id to its own learned vector.

☂ EVERYDAY PARALLEL

A library catalog. A call number like “519.5” means nothing by itself; it only points to a card that describes the book. The token id is the call number, and the embedding row is the card. The difference: here the descriptions are learned. Training rewrites each card so that tokens used in similar ways get similar descriptions.

◆ NOW THE MATH

The embedding table is a matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$, where V is the vocabulary size. d is the *embedding dimension*: how many numbers describe each token (the code calls it `d_model`). To embed token id i , you pick out row i of the table:

$$\text{embed}(i) = \mathbf{E}[i, :] \in \mathbb{R}^d$$

In words: the embedding of token i is the i -th row of the table $\mathbf{E}$. There is no multiplication, only a row lookup. Those rows are ordinary parameters, so backpropagation changes them during training. Tokens that play similar roles move toward similar vectors.

Embeddings alone do not capture one thing: **order**. A plain lookup gives “dog bites man” and “man bites dog” the same set of vectors, only in a different order. The attention that follows (Section 18) is *permutation-equivariant*. If you shuffle its inputs, its outputs are shuffled in the same way, and nothing else changes. So attention alone cannot tell which word came first. We therefore *add* a second vector that encodes each token's **position**:

$$\mathbf{h}_t = \underbrace{\mathbf{E}[x_t, :]}_{\text{what the token is}} + \underbrace{\mathbf{P}[t, :]}_{\text{where it sits}}$$

In words: the vector that enters the network at position t is the token's meaning vector plus a position vector $\mathbf{P}[t]$. That vector says “this is slot 0, slot 1, slot 2...”. Now the model can tell a first word from a last word. The details of positional encoding are in Section 18. Here we only need to know that it is added in.

◆ THE SAME THING IN PYTORCH

>_ TOKEN + POSITION EMBEDDINGS

```
import torch
import torch.nn as nn

vocab_size, d_model, max_len = 8, 32, 16  # 32 numbers per token, up to 16

tok_emb = nn.Embedding(vocab_size, d_model)  # the lookup table E (8 x 32)
pos_emb = nn.Embedding(max_len, d_model)      # one vector per position

ids = torch.tensor([[3, 2, 4, 4, 5]])        # (batch=1, seq_len=5): "hello"
B, T = ids.shape

positions = torch.arange(T)                  # [0, 1, 2, 3, 4]
h = tok_emb(ids) + pos_emb(positions)        # (1, 5, 32): meaning + position
print(h.shape)                               # torch.Size([1, 5, 32])
```

In words: `tok_emb(ids)` looks up a 32-number vector for each of the 5 tokens, giving shape `(1, 5, 32)`. `pos_emb(positions)` has shape `(5, 32)`. PyTorch *broadcasts* it across the batch (Section 13), so it is added to every sequence. Every token now carries both what it is and where it sits.

CHECK YOURSELF 4

You remove `pos_emb` from the code above. The tokens “l” at positions 2 and 3 of “hello” now enter the network as what?

4 The objective: next-token prediction

◆ INTUITION FIRST

How do we *train* a next-token predictor? We never need a human to label anything, because the text labels itself. At every position, the correct answer is the token that came next. We move along the text and, at each position, ask the model to predict the following token. Then we penalize it for being wrong. This is why language models can learn from raw web text: the supervision is already in ordinary text.

☂ EVERYDAY PARALLEL

Flashcard practice where you make the cards from a book you are reading. Cover the next word, guess it, then move your hand to see if you were right. The book grades you for free. A sentence of 20 tokens gives about 20 cards, and one book gives hundreds of thousands. A language model studies the same way: the next token is always the answer key.

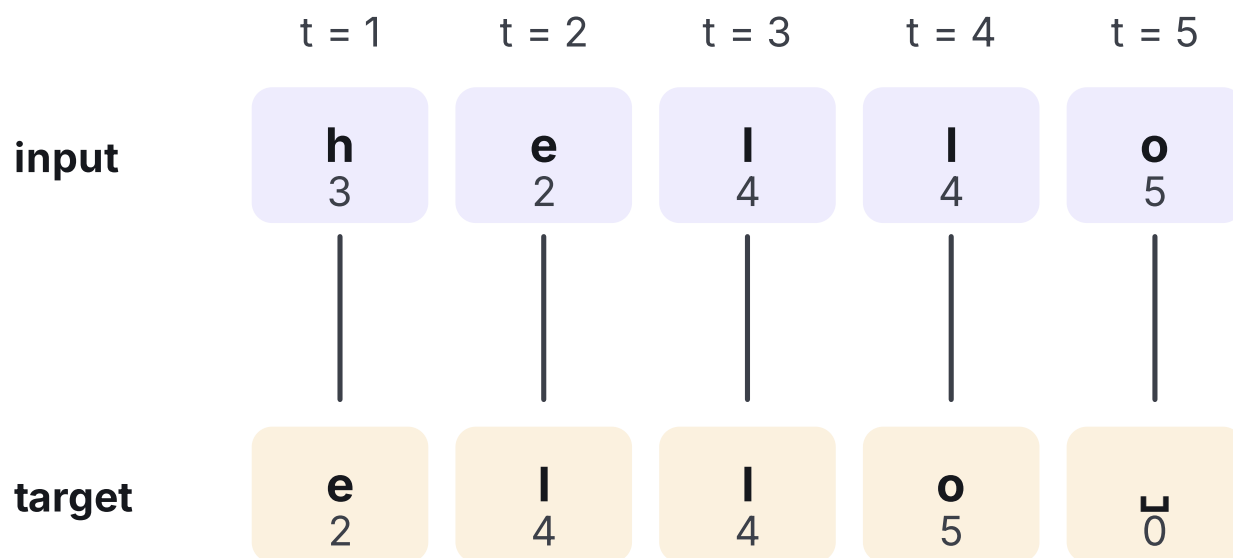


Figure 19.2 Inputs and targets come from the same text, shifted by one token. Position t reads the tokens up to t and must predict token $t + 1$. The small numbers are the token ids from the tokenizer above.

◆ NOW THE MATH

At each position the model outputs *logits*: one raw score per vocabulary token (a *logit* is a score before it is turned into a probability). We pass the logits through **softmax**. Softmax maps a list of scores to positive numbers that sum to 1, that is, a probability distribution. The result is the predicted probability of each possible next token:

$$\hat{p}_t = \text{softmax}(\mathbf{z}_t), \quad \hat{p}_{t,j} = \frac{e^{z_{t,j}}}{\sum_{k=1}^V e^{z_{t,k}}}$$

In words: $\mathbf{z}_t$ is the row of V raw scores at position t . Softmax exponentiates each score (making it positive) and divides by the total. So the outputs are all between 0 and 1 and add to 1: a probability for every token in the vocabulary.

We want the model to give high probability to the *true* next token. The standard penalty for that is **cross-entropy loss** (introduced in Section 4). It is the negative log probability of the correct token, averaged over positions. A sequence of T tokens has a known next token only at positions 1 to $T - 1$, so it gives $T - 1$ predictions:

$$L = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log \hat{p}_{t, x_{t+1}}$$

In words: for each position t , look up the probability the model gave to the token that came next (x_{t+1}). Take its log, flip the sign, and average over all positions. If the model was confident and right, that probability is near 1, its log is near 0, and the loss is small. If it was confident and *wrong*, the probability is near 0, its log goes toward $-\infty$, and the loss is large. Minimizing this pushes the model to put probability on the real next token.

DEFINITION

Teacher forcing means that during training the model always reads the *true* previous tokens, not its own earlier guesses. When predicting token 5, it sees the real tokens 1 to 4, even if it would have guessed token 4 wrong. So every position can be predicted in parallel from correct context, which makes training fast and stable. At generation time there is no answer key, so the model must read its own outputs. This mismatch is a known weakness, but teacher forcing is still worth it.

◆ THE SAME THING IN PYTORCH

>_ NEXT-TOKEN CROSS-ENTROPY WITH A SHIFTED TARGET

```

import torch
import torch.nn.functional as F

ids = torch.tensor([[3, 2, 4, 4, 5, 0]])      # "hello " with the tokenization
inputs, targets = ids[:, :-1], ids[:, 1:]    # "hello" → "ello "
print(targets)                               # tensor([[2, 4, 4, 5, 0]])

# logits: the model's raw scores, shape (batch, seq_len, vocab_size).
# All zeros = an untrained model with no preference between the 8 tokens.
logits = torch.zeros(1, 5, 8)

# cross_entropy wants (N, vocab) logits and (N,) targets, so flatten batch
B, T, V = logits.shape
loss = F.cross_entropy(logits.reshape(B * T, V), targets.reshape(B * T))
print(loss)      # tensor(2.0794) = log(8); training drives it down

```

In words: `F.cross_entropy` does softmax *and* the negative-log-probability step in one numerically safe call. So we give it raw logits, never softmaxed ones. We flatten the 5 positions into 5 independent (prediction, true next token) pairs and average their losses. The targets are the inputs shifted left by one, as in Figure 19.2. With all-zero logits every token gets probability $1/8$, so the loss is $\log 8 \approx 2.08$. Random logits (`torch.randn`) give a higher loss, about 2.5 on average, because a random guess is also confidently wrong some of the time.

CHECK YOURSELF 5

A GPT-2-sized model has $V = 50257$. At the very start of training it spreads probability almost evenly over all tokens. Predict the first loss value. Why is that a useful sanity check?

5 The GPT-style decoder, and the causal mask

◆ INTUITION FIRST

Now we assemble the model. A **GPT** (Generative Pre-trained Transformer) is *decoder-only*. It is a stack of identical **transformer blocks** (the attention-plus-feed-forward units from Section 18). One final linear layer turns each position's vector into V logits. Embeddings go in at the bottom and logits come out at the top. Every block in between lets each position gather information from the others.

◆ THE WHOLE SHAPE, TOP TO BOTTOM

token ids $\rightarrow$ **embeddings + positions** $\rightarrow$ **N transformer blocks** (each = masked self-attention, then a feed-forward layer, with residual connections) $\rightarrow$ **final linear layer** to size V $\rightarrow$ **logits** $\rightarrow$ softmax $\rightarrow$ probabilities. Training minimizes next-token cross-entropy over those probabilities; generation samples from them.

But there is a trap. Attention lets every position look at every other position. During training we give the model the whole sentence at once (teacher forcing). Suppose position 3 may read position 4 while it predicts position 4. Then it will copy the answer and learn nothing. We must forbid looking ahead.

🔑 EVERYDAY PARALLEL

An exam where the answers are printed on the next page. If students may turn the page, the test measures nothing: everyone “predicts” perfectly by reading the answer. The **causal mask** is the exam rule: you may reread everything you have already written, but you may not turn the page. Each word must be guessed from the words before it, never the ones after.

◆ NOW THE MATH

Recall from Section 18 that self-attention first computes a grid of *attention scores*. This is a matrix $\mathbf{S} \in \mathbb{R}^{T \times T}$. Entry S_{ij} says how much the token at position i (the one asking) wants to attend to the token at position j (the one being read):

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}$$

In words: $\mathbf{Q}$ holds each position's “query” (what am I looking for?) and $\mathbf{K}$ holds each position's “key” (what do I offer?). Their dot products fill the grid $\mathbf{S}$. Dividing by $\sqrt{d_k}$ keeps the numbers from getting too large (d_k is the length of each key vector). Row i of $\mathbf{S}$ holds the scores of position i for all positions j .

“Position i must not attend to a future position $j > i$ ” means: remove every entry to the right of the diagonal. Softmax is applied to each row to turn scores into weights. Softmax sends a score of $-\infty$ to a weight of exactly 0, because $e^{-\infty} = 0$. So we do not delete the future entries. We set their *scores* to $-\infty$ before the softmax:

$$S_{ij}^{\text{masked}} = \begin{cases} S_{ij} & j \leq i \quad (\text{past or self}) \\ -\infty & j > i \quad (\text{future}) \end{cases}$$

In words: keep a score if the column j is at or before the row i (a token may read itself and everything earlier). Replace it with $-\infty$ if j is later. After softmax those entries have zero weight, so no information flows backward in time.

For a length-5 sequence the mask is this lower-triangular pattern. A ✓ means “allowed to attend” and $-\infty$ means “blocked”:

$i \backslash j$	0	1	2	3	4
0	✓	$-\infty$	$-\infty$	$-\infty$	$-\infty$
1	✓	✓	$-\infty$	$-\infty$	$-\infty$
2	✓	✓	✓	$-\infty$	$-\infty$
3	✓	✓	✓	✓	$-\infty$
4	✓	✓	✓	✓	✓

In words: row 0 (the first token) may look only at itself. Row 4 (the last token) may look at all five. Every row sees itself and everything to its left, nothing to its right. This lower-triangular shape is *the* defining feature of a decoder. It makes the model *causal*: each output depends only on the past.

◆ THE SAME THING IN PYTORCH

>_ BUILDING THE CAUSAL MASK AND APPLYING IT TO SCORES

```
import torch

torch.manual_seed(0)
T = 5
# lower-triangular ones: allowed[i, j] = 1 when j ≤ i, else 0
allowed = torch.tril(torch.ones(T, T))
print(allowed)
# tensor([[1., 0., 0., 0., 0.],
#         [1., 1., 0., 0., 0.],
#         [1., 1., 1., 0., 0.],
#         [1., 1., 1., 1., 0.],
#         [1., 1., 1., 1., 1.]])

scores = torch.randn(T, T) # raw attention
scores = scores.masked_fill(allowed == 0, float("-inf")) # blocked → -inf
weights = torch.softmax(scores, dim=-1) # each row sums to 1
print(weights[0]) # tensor([1., 0., 0., 0., 0.]) token 0 attends only itself
print(weights[1]) # tensor([0.7326, 0.2674, 0.0000, 0.0000, 0.0000])
```

In words: `torch.tril` keeps the lower triangle (past and self) and sets the upper triangle (future) to zero. `masked_fill` writes $-\infty$ at every future entry. Softmax over the last dimension then turns each row into weights that sum to 1, with the future entries at exactly 0. Row 0 attends only to itself, as the mask math promised.

▲ WATCH OUT

Apply the mask to the *scores*, before softmax, not to the weights after. If you zero the weights afterward, the future is still in the softmax denominator. So the weights on past tokens still depend on the future tokens' scores, and information leaks from the future. The rows also no longer sum to 1. With the seed above, row 1 becomes `[0.3931, 0.1435, 0, 0, 0]`, which sums to 0.54. Mask first, softmax second.

CHECK YOURSELF 6

Without the causal mask, a decoder trains to almost zero loss in a few steps but generates nonsense at test time. Why?

6 Generation: from logits to written text

◆ INTUITION FIRST

A trained model is a next-token predictor, and writing means running it in a loop. Feed in the prompt, get logits for the next token, turn them into probabilities, and **pick one**. Append it to the sequence and feed the whole sequence back in. Repeat until you have enough text. The only real decision is *how* you pick from the probabilities.

☂ EVERYDAY PARALLEL

Telling a story aloud, one word at a time. After each word your mind holds a shortlist of plausible next words with rough odds. If you always say the most likely word, the story is flat and repeats itself. If you sometimes take a less likely word, it is more lively but riskier. **Temperature** is the dial between “play it safe” and “surprise me”.

◆ NOW THE MATH

Greedy decoding always takes the highest-probability token ($\arg \max$). It is deterministic and safe, but it tends to repeat itself and sound mechanical. **Sampling** instead draws a token at random *in proportion to its probability*: likely tokens are picked often, rare ones sometimes. This gives variety. **Temperature** (τ) reshapes the distribution before we sample. It divides the logits:

$$\hat{p}_j = \text{softmax}\left(\frac{\mathbf{z}}{\tau}\right)_j = \frac{e^{z_j/\tau}}{\sum_k e^{z_k/\tau}}$$

In words: divide every logit by τ before softmax. When $\tau < 1$ the gaps between scores grow, so the top token dominates: the model is *more confident and closer to greedy*. When $\tau > 1$ the gaps shrink and the probabilities move toward equal: the model is *more random*. As $\tau \rightarrow 0$ sampling becomes greedy. At $\tau = 1$ you sample from the model's own distribution.

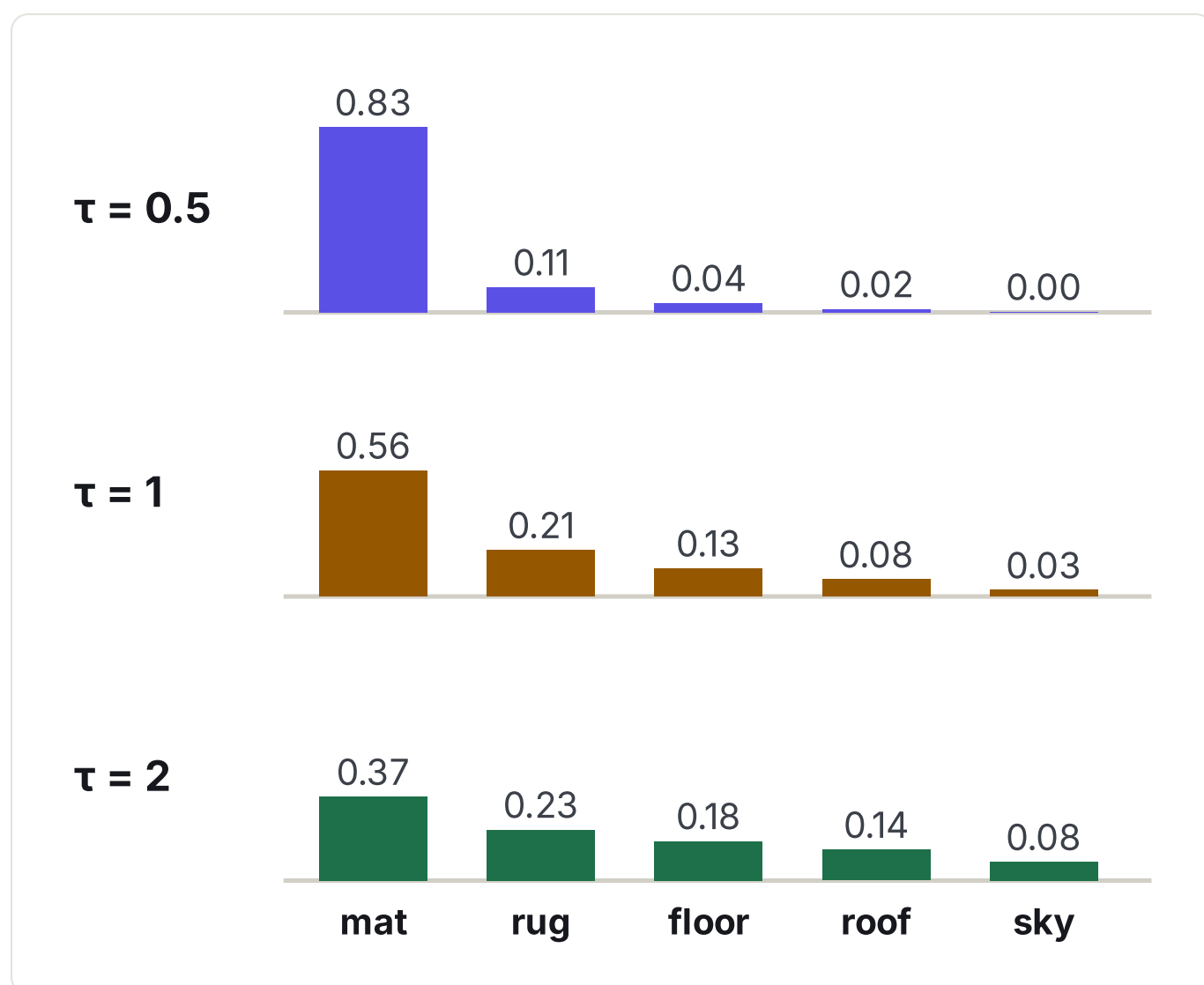


Figure 19.3 The same five logits (2, 1, 0.5, 0, -1) after softmax at three temperatures (values computed with `torch.softmax`). At $\tau = 0.5$ the top token takes 0.83 of the probability. At $\tau = 2$ it takes only 0.37, and even "sky" gets 0.08.

DEFINITION

Temperature is a single positive number that controls randomness in sampling. Low temperature (for example 0.2) gives confident, focused, repetitive text. High temperature (for example 1.3) gives varied text with more errors. It is the most common setting you change when you use a language model.

◆ THE SAME THING IN PYTORCH

›_ SAMPLE TOKENS ONE AT A TIME, WITH TEMPERATURE

```
import torch

@torch.no_grad()  # generation is inference: no gradients
def generate(model, idx, max_new_tokens, block_size, temperature=1.0):
    # idx: (batch, current_length) tensor of token ids, the prompt
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -block_size:]  # keep only the last block
        logits = model(idx_cond)  # (batch, length, vocab)
        logits = logits[:, -1, :] / temperature  # take the LAST position,
        probs = torch.softmax(logits, dim=-1)  # (batch, vocab): a distribution
        next_id = torch.multinomial(probs, num_samples=1)  # sample one token
        idx = torch.cat([idx, next_id], dim=1)  # append and loop
    return idx
```

In words: at each step we run the model and keep only the logits at the *last* position (the prediction for what comes next). We divide by temperature and apply softmax to get a probability distribution. Then `torch.multinomial` draws one token in proportion to those probabilities. We append it to the sequence and repeat. `idx_cond` cuts the input to the model's maximum context length `block_size`, so it never gets too long. Replace the sampling line with `logits.argmax(dim=-1, keepdim=True)` and you have greedy decoding.

▲ WATCH OUT

Only the logits at the **last** position matter for generation: they are the model's guess for the next token. Beginners sometimes sample from every position or from position 0. Those are predictions for tokens you already have. Always slice `logits[:, -1, :]`.

Top-k and top-p: cut off the unlikely tail

Plain sampling has one weakness. With $V = 50,000$ tokens, the thousands of very unlikely tokens together can hold a few percent of the probability. So now and then the sampler picks a nonsense token, and one bad token can derail the rest of the text. Two simple filters remove that tail before sampling:

- **Top-k sampling** keeps only the k most likely tokens (for example $k = 50$). It sets all other logits to $-\infty$ and samples from the rest.
- **Top-p sampling** (also called *nucleus sampling*) keeps the smallest set of most likely tokens whose probabilities add up to at least p (for example $p = 0.9$). When the model is sure, this set is small; when it is unsure, the set is larger.

In both cases softmax runs again on the kept logits, so their probabilities again sum to 1. Temperature, top-k, and top-p are often used together.

>_ TOP-K AND TOP-P FILTERING OF ONE STEP'S LOGITS

```
import torch

def filter_logits(logits, top_k=None, top_p=None):
    # logits: (batch, vocab). Returns logits with the removed tokens set
    if top_k is not None:
        kth = torch.topk(logits, top_k, dim=-1).values[:, -1:] # k-th l
        logits = logits.masked_fill(logits < kth, float("-inf"))
    if top_p is not None:
        sorted_logits, order = torch.sort(logits, dim=-1, descending=True)
        probs = torch.softmax(sorted_logits, dim=-1)
        mass_before = probs.cumsum(dim=-1) - probs # mass of the
        drop_sorted = mass_before ≥ top_p # set already
        drop = drop_sorted.scatter(-1, order, drop_sorted) # back to voc
        logits = logits.masked_fill(drop, float("-inf"))
    return logits

z = torch.tensor([[2.0, 1.0, 0.5, 0.0, -1.0]])
print(torch.softmax(z, dim=-1))
# tensor([[0.5630, 0.2071, 0.1256, 0.0762, 0.0280]])
print(torch.softmax(filter_logits(z, top_k=2), dim=-1))
# tensor([[0.7311, 0.2689, 0.0000, 0.0000, 0.0000]])
print(torch.softmax(filter_logits(z, top_p=0.8), dim=-1))
# tensor([[0.6285, 0.2312, 0.1402, 0.0000, 0.0000]])
```

In words: with $k = 2$ only the two best tokens survive, and their probabilities are rescaled to sum to 1. With $p = 0.8$ the top two tokens hold only 0.77, so the third token is also kept ($0.77 + 0.13 = 0.90$). The last two tokens are removed. To use this in `generate`, call `filter_logits` on the logits after dividing by temperature and before the softmax.

CHECK YOURSELF 7

You set `temperature = 0.01`. How does the output behave, and how does it compare with greedy decoding?

CHECK YOURSELF 8

What does top-k sampling with $k = 1$ do? And what does top-p sampling with $p = 1$ do?

7 Putting it together: a minimal GPT

Here is a transformer block with *built-in causal masking*, plus the small model head that produces logits. It reuses the attention idea from Section 18; the only new part is the mask we derived above. In Section 22 you will replace its feed-forward layer with a mixture of experts.

◆ THE SAME THING IN PYTORCH

>_ A MINIMAL GPT: CAUSAL ATTENTION, BLOCK, AND MODEL

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)  # produce Q, K, V in
        self.proj = nn.Linear(d_model, d_model)      # mix heads back togeth
        # a constant lower-triangular mask, saved with the model but not
        self.register_buffer("mask", torch.tril(torch.ones(block_size, blo

    def forward(self, x):                             # x: (B, T, d_model)
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)         # each (B, T, C)
        # reshape into heads: (B, n_heads, T, head_dim)
        q = q.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2)
        k = k.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2)
        v = v.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2)

        att = (q @ k.transpose(-2, -1)) / (k.size(-1) ** 0.5)  # scores
        att = att.masked_fill(self.mask[:T, :T] == 0, float("-inf"))  # b
        att = F.softmax(att, dim=-1)  # weights, future →
        y = att @ v  # (B, h, T, head_dim)
        y = y.transpose(1, 2).reshape(B, T, C)  # re-merge the heads
        return self.proj(y)

class Block(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)  # normalize before a
        self.attn = CausalSelfAttention(d_model, n_heads, block_size)
        self.ln2 = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(          # position-wise feed
            nn.Linear(d_model, 4 * d_model), nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )

```

```

def forward(self, x):
    x = x + self.attn(self.ln1(x))           # residual add-on and
    x = x + self.mlp(self.ln2(x))           # residual add-on and
    return x

class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model=64, n_heads=4, n_layers=2, block_size=16):
        super().__init__()
        self.block_size = block_size
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(block_size, d_model)
        self.blocks = nn.ModuleList(
            [Block(d_model, n_heads, block_size) for _ in range(n_layers)]
        )
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size) # final linear → logits

    def forward(self, idx):                    # idx: (B, T) token indices
        B, T = idx.shape
        pos = torch.arange(T, device=idx.device)
        x = self.tok_emb(idx) + self.pos_emb(pos) # (B, T, d_model)
        for block in self.blocks:
            x = block(x)
        x = self.ln_f(x)
        return self.head(x)                  # (B, T, vocab_size)

# smoke test: shapes must line up end to end
model = MiniGPT(vocab_size=8)
idx = torch.zeros(1, 5, dtype=torch.long) # a length-5 prompt
logits = model(idx)
print(logits.shape)                        # torch.Size([1, 5, 8])

```

In words: `idx`s become token plus position vectors. Each `Block` runs masked self-attention and then a feed-forward layer. Each of the two is wrapped in a *residual connection* (`x + ...`), which lets gradients flow well; see Section 18). Each is also preceded by *layer norm*, which rescales each vector to a stable range. The final linear `head` maps every position to `vocab_size` logits. The smoke test confirms the shape `(batch, seq_len, vocab)`. Feed those logits into cross-entropy to train the model, or into `generate` to write text.

CHECK YOURSELF 9

You call `model(torch.zeros(1, 40, dtype=torch.long))` on the `MiniGPT` above. What goes wrong, and which line in `generate` prevents it?

◆ THAT'S A REAL LANGUAGE MODEL

Trained on enough text with the cross-entropy loss and run through the generation loop, the code above does write text. Large GPT models keep this core design, but they are much bigger. GPT-3 has 96 blocks instead of 2, and vectors of 12,288 numbers instead of 64. It was also trained on far more data. Newer models change some parts: rotary position encodings (RoPE), gated SwiGLU feed-forward layers, RMSNorm, and grouped-query attention. We will not cover these here; each is a small change to one part of the block. In Section 22 we replace each block's single feed-forward layer with a *mixture of experts*. That adds capacity without raising the cost of every token.

8 What to carry forward

- A **language model** predicts the next token given the previous ones; run that predictor in a loop and it writes.
- **Subword tokenization** (BPE, WordPiece, Unigram) gives a fixed vocabulary and handles rare words by breaking them into familiar pieces. A byte-level base means no text is ever unknown.
- **Embeddings** turn each token id into a learned vector via a lookup table; we *add* positional vectors so order matters.
- Training is **next-token prediction**: cross-entropy over the vocabulary at every position, with **teacher forcing** feeding the true past. Targets are the inputs shifted by one.
- A **GPT** is a stack of transformer blocks plus a final linear layer to logits. The **causal mask** sets future scores to $-\infty$ before softmax, so no position can read ahead.
- **Generation** = logits $\rightarrow$ softmax $\rightarrow$ sample $\rightarrow$ append $\rightarrow$ repeat. **Temperature** moves between confident and random; **top-k** and **top-p** remove the unlikely tail.

You now have every moving part of a GPT. Next, Section 20 covers what changes when you train such models at scale: normalization, dropout, mixed precision, and many GPUs.

Answers to “Check yourself”

1. Why do we condition each token on the tokens *before* it, rather than modeling $P(x_t)$ on its own?

Because language depends on context. On its own, “bank” could be a river edge or a place for money. After “I deposited the check at the”, “bank” is almost surely the money kind. Conditioning on the past $(x_1 \dots x_{t-1})$ lets the model use context to sharpen every prediction. A model of each token in isolation would ignore that context and produce random word lists.

2. A word-level model is trained on news from 2010. Then it reads a 2024 article that mentions “COVID”. What happens, and how does a byte-level subword tokenizer help?

“COVID” never appeared in 2010 training text, so a word-level model has no id for it. It becomes the single *unknown* token, and all information about the word is lost. A subword tokenizer instead spells it from pieces it already knows, for example `"CO" + "VID"`, or letter by letter. The model still receives real input and can use the parts of the new word. With a byte-level base this works for any string, including names, typos, and emoji.

3. Run the tokenizer above on the string `"hello!"`. What happens, and why?

It raises `KeyError: '!'`. The vocabulary was built only from the characters in `"hello world"`, and `"!"` is not one of them. This is the unknown-token problem in its smallest form. A byte-level vocabulary avoids it, because every character is made of bytes that are always in the vocabulary.

4. You remove `pos_emb` from the code above. The tokens “l” at positions 2 and 3 of “hello” now enter the network as what?

As two identical vectors: both are row 4 of the table. Without the position vector, nothing in the input tells them apart. Attention would then treat “hello” and “hlelo” as the same set of vectors in a different order. That is why the position vector is added.

5. A GPT-2-sized model has $V = 50257$. At the very start of training it spreads probability almost evenly over all tokens. Predict the first loss value. Why is that a useful sanity check?

If every token gets probability $1/V$, the loss at each position is $-\log(1/V) = \log V$. For $V = 50257$ that is $\log 50257 \approx 10.8$. A first loss near $\log V$ that then falls tells you the model starts sensibly and the loss is wired correctly. If it starts far above $\log V$, something is wrong: often the initial logits are too large, or logits and targets are misaligned.

6. Without the causal mask, a decoder trains to almost zero loss in a few steps but generates nonsense at test time. Why?

With no mask, position t can attend to position $t + 1$, which holds the token it must predict. The model learns the shortcut “copy the next input”, so the training loss collapses. At generation time the future does not exist

yet (that is what we are producing), so the shortcut has nothing to copy and the output is poor. The causal mask forces the model to *learn to predict* instead. That is the only skill that transfers to generation.

7. You set `temperature = 0.01`. How does the output behave, and how does it compare with greedy decoding?

Dividing logits by 0.01 multiplies them by 100, so the gaps between scores become very large. After softmax, the top token gets almost all the probability, and `multinomial` picks it almost every time. The output is then almost deterministic and matches greedy decoding in practice, because $\tau \rightarrow 0$ is the limit that turns sampling into `arg max`. (If two tokens have nearly equal top logits, the sampler can still pick either one.)

8. What does top-k sampling with $k = 1$ do? And what does top-p sampling with $p = 1$ do?

With $k = 1$ only the most likely token survives, so it gets probability 1. That is greedy decoding. With $p = 1$ the kept set must hold all the probability, so no token is removed (except tokens with probability exactly 0). That is plain sampling.

9. You call `model(torch.zeros(1, 40, dtype=torch.long))` on the `MiniGPT` above. What goes wrong, and which line in `generate` prevents it?

The model has `block_size=32`, so `pos_emb` has only 32 rows (positions 0 to 31). Position 32 and later have no row, and the lookup raises an `IndexError`. The mask buffer is also only 32×32 . In `generate`, the line `idx_cond = idx[:, -block_size:]` keeps only the last 32 tokens, so the model never sees a longer input.

Training at Scale

What changes when a model trains on real hardware, in real time, on real data: normalization, dropout, half-precision numbers, and splitting work across machines.

40 min read

YOU'LL LEARN

- batch norm vs layer norm
- dropout
- mixed precision
- gradient accumulation
- distributed training

Before you start: Section 18: Attention & Transformers.

So far we assumed you can call `loss.backward()` and `optimizer.step()` and be done. At real scale, you need a few practical techniques more. They make the model train stably, fit in memory, and finish in time. None of them is exotic, and all of them are necessary.

◆ THE ONE IDEA TO HOLD ONTO

Every technique in this section solves one of two problems. Some keep the numbers flowing through the network **well-behaved** (normalization, dropout, mixed precision). Others make the most of **limited hardware** (gradient accumulation, distributed training). Know which problem you are solving, and the right tool follows.

1 Normalization: the general recipe

◆ INTUITION FIRST

As data flows through a deep network, the scale of the numbers at each layer can drift. They can grow too large, shrink too small, or move off center, and this makes training unstable. **Normalization** is a layer that brings the numbers back to a well-behaved range: centered around zero, with a controlled spread. Then it lets the network learn whatever scale and offset it needs.

🌂 EVERYDAY PARALLEL

A sound engineer first sets every microphone to the same baseline level before a concert. So no instrument is too loud only because it started too loud. Then the engineer turns some instruments up and others down on purpose, to get the mix they want. Reset to a common baseline, then adjust on purpose: that is normalization.

◆ NOW THE MATH

The recipe is always the same two steps: first normalize, then let the network scale and shift the result by its own learned amount.

$$z = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$
$$y = \gamma z + \beta$$

In words: μ (mu) is the mean and σ^2 (sigma squared) is the variance, which measures how spread out the values are. Subtracting the mean and dividing by the standard deviation σ gives z : a version of x with mean 0 and spread 1. ϵ (epsilon) is a tiny constant, for example 10^{-5} , so we never divide by zero. Then γ (gamma) and β (beta) are *learned* numbers that rescale and shift the normalized value. They let the network undo the normalization, partly or fully, if that works better. With $\gamma = \sigma$ and $\beta = \mu$ (ignoring ϵ), the layer gives back x exactly.

DEFINITION

The only real design choice in a normalization layer is *which numbers you average together* to compute μ and σ^2 . Batch norm and layer norm use the same formula above, over two different groups of numbers.

CHECK YOURSELF 1

A layer outputs the values 2, 4, 6. Normalize them with $\epsilon = 0$, $\gamma = 1$, $\beta = 0$. Then say what changes if $\gamma = 3$ and $\beta = 1$.

2 Batch norm vs layer norm: which numbers get averaged

◆ INTUITION FIRST

Batch normalization computes μ and σ^2 for each feature separately. It averages *across all examples in the mini-batch*. **Layer normalization** computes μ and σ^2 for each example separately. It averages *across all features of that one example*. The formula is the same; the difference is the direction in which you slice the data.

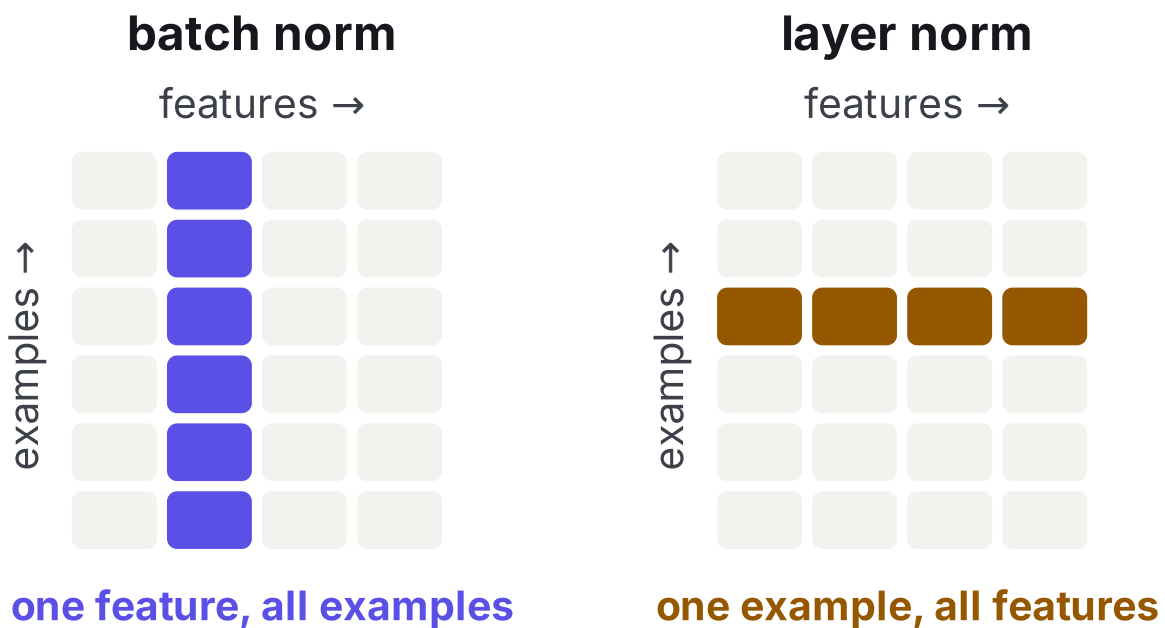


Figure 20.1 The same batch (rows are examples, columns are features). Batch norm computes one mean and variance per *column*, so it mixes examples. Layer norm computes one per *row*, so each example is handled alone.

☂ EVERYDAY PARALLEL

Batch norm is grading on a curve, per subject. Your math score is adjusted by how the *whole class* did on that test, so it depends on your classmates. **Layer norm** compares one student with their own report card: your math score against your own scores in history, art, and sports. It depends only on you, never on anyone else in the room.

◆ THE SAME THING IN PYTORCH

>_ BATCH NORM VS LAYER NORM ON THE SAME BATCH

```
import torch, torch.nn as nn

torch.manual_seed(0)
x = torch.randn(32, 4)  # 32 examples in the batch, 4 features each

bn = nn.BatchNorm1d(num_features=4)
y_bn = bn(x)  # mean/var for each of the 4 features, across a

ln = nn.LayerNorm(normalized_shape=4)
y_ln = ln(x)  # mean/var for each example, across its own 4 f

print(y_bn.std(dim=0, correction=0))  # tensor([1.0000, 1.0000, 1.0000, 1.0000])
print(y_ln.std(dim=1, correction=0)[:3])  # tensor([1.0000, 1.0000, 1.0000, 1.0000])
```

In words: after batch norm, each *column* (feature) has spread 1. After layer norm, each *row* (example) has spread 1. Both means are 0 in the same directions. (The `...` stands for the `grad_fn` part of the printout.)

◆ WHY TRANSFORMERS USE LAYER NORM

Layer norm's statistics never depend on other examples in the batch. This gives transformers (Section 18) three things they need:

- It works with a **batch size of 1**. Batch norm cannot train there, because one example has no spread across the batch.
- It does not care that sequences in a batch have **different lengths**, because it never mixes examples.
- It behaves **the same at training and test time**. Batch norm needs extra “running statistics” for evaluation to match training.

▲ WATCH OUT

At test time, batch norm may see a single example, but it must still behave as in training. So during training it keeps a *running average* of μ and σ^2 . At evaluation time it uses that running average instead of batch statistics. This is what `model.eval()` switches on (see Section 15). Layer norm has no such difference. It computes its statistics from the actual input every time, in training and in evaluation, so there is nothing extra to get wrong.

CHECK YOURSELF 2

You call `nn.BatchNorm1d(4)` in training mode on a batch of shape `(1, 4)`. What would the formula give, and what does PyTorch do?

3 Dropout: forcing redundancy

◆ INTUITION FIRST

Dropout randomly sets a fraction of a layer's units to zero on every forward pass during training. The network cannot depend too much on any one unit, because that unit may be gone on the next step. So it must spread the useful signal across many units, and this reduces overfitting.

☂ EVERYDAY PARALLEL

A sports team that rotates who starts each game, even its best player, gets a squad where several players can fill any role. A team that always depends on one star is in trouble when that player is injured. Dropout is that rotation policy, applied to the units of a neural network.

◆ NOW THE MATH

During training, each unit is set to zero with probability p , independently of the others. The surviving units are scaled up by $1/(1 - p)$, so the expected value stays the same:

$$\text{dropout}(x_i) = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1 - p} & \text{with probability } 1 - p \end{cases}$$

In words: flip a biased coin for every unit. With chance p the unit outputs 0 this step. Otherwise its value is divided by $1 - p$ to make up for the zeroed units. The expected output is then $p \cdot 0 + (1 - p) \cdot x_i / (1 - p) = x_i$, the same as without dropout. This rescaling is called *inverted dropout*. Because of it, dropout can do nothing at test time, with no extra work.

◆ THE SAME THING IN PYTORCH

>_ DROPOUT IN TRAINING MODE AND IN EVAL MODE

```
import torch, torch.nn as nn

torch.manual_seed(0)
drop = nn.Dropout(p=0.5)
x = torch.ones(8)

drop.train()    # inside a model, model.train() does this for every layer
print(drop(x))  # tensor([0., 0., 2., 0., 0., 0., 2., 2.]) survivors doubled

drop.eval()     # inside a model, model.eval() does this
print(drop(x))  # tensor([1., 1., 1., 1., 1., 1., 1., 1.]) identity at eval
```

In words: in training mode each entry becomes 0 or 2. Here 3 of 8 survived; on average half do. In eval mode the layer passes x through unchanged. When dropout sits inside a model,

`model.train()` and `model.eval()` set this mode for all its layers.

CHECK YOURSELF 3

Someone writes their own dropout: in training it zeros each unit with $p = 0.5$ but does *not* divide by $1 - p$. In eval it passes x through. What goes wrong at test time, and how would you fix it without changing training?

4 Mixed precision: same math, half the bits

◆ INTUITION FIRST

Every number in a network is normally stored as a 32-bit float (*fp32*). Modern GPUs run much faster with a 16-bit format, and they can store about twice as many numbers in the same memory. There are two common 16-bit formats, *fp16* and *bf16*, which divide the 16 bits in different ways. **Mixed precision** training runs most of the math in 16 bits. It keeps a few sensitive parts in 32 bits, such as the master copy of the weights. You get the speed without losing correctness.

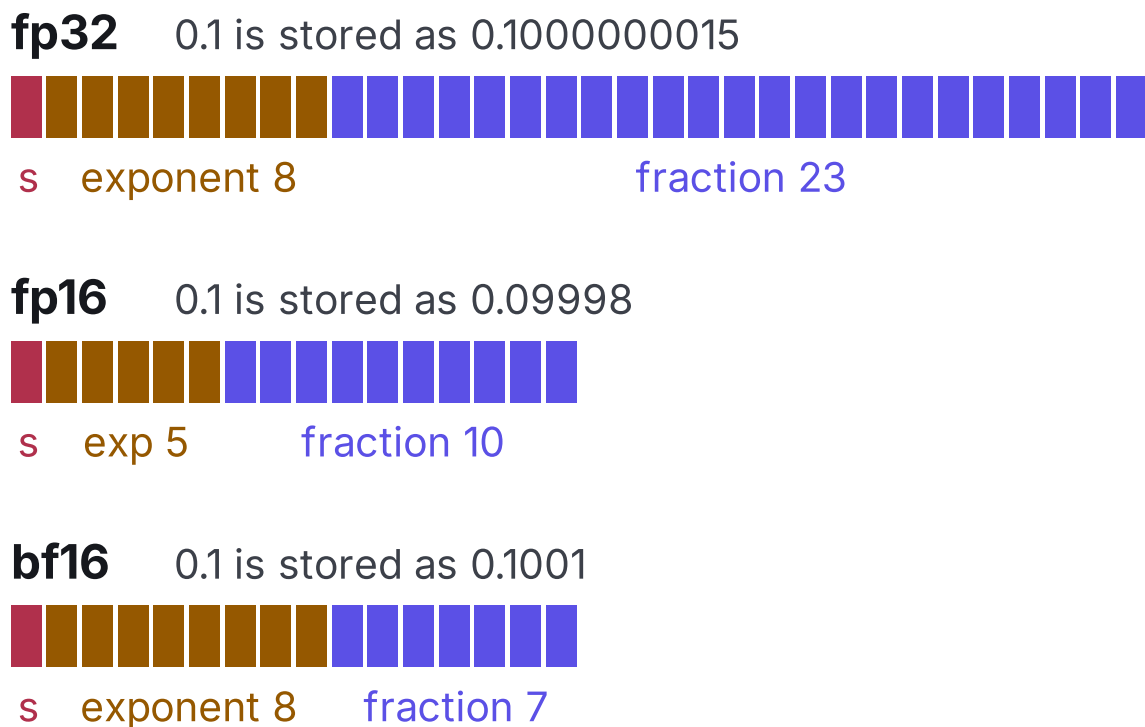


Figure 20.2 How each format splits its bits. The exponent bits (amber) set the *range*. The fraction bits (violet) set the *precision*. *bf16* keeps *fp32*'s 8 exponent bits, so it has the same range. *fp16* keeps more fraction bits but has only 5 exponent bits. The stored values of 0.1 were computed with PyTorch.

☂ EVERYDAY PARALLEL

A shop rounds each receipt to the whole dollar, but it keeps the account balance to the cent. Rounding receipts is fast and good enough. But suppose the balance itself were kept in whole dollars: a purchase of 40 cents would round to zero and vanish, again and again. Mixed precision works the same way. The forward and backward math uses 16 bits (the receipts), while the master weights stay in fp32 (the balance). Many small updates can then still add up.

You can see that last point directly. In fp16, the numbers near 1.0 are about 0.001 apart. So `1.0 + 1e-4` rounds back to `1.0`, and a small weight update is lost. In fp32 it gives `1.0001`.

◆ NOW THE MATH

fp16 has only 5 exponent bits, so its range is narrow. Its largest value is 65,504. Its smallest normal value is $2^{-14} \approx 6.1 \times 10^{-5}$. Below that, fp16 has *subnormal* numbers, which reach down to $2^{-24} \approx 6 \times 10^{-8}$ but with fewer significant digits. Anything smaller **underflows**: it rounds to exactly 0. Many gradients during training are this small, for example 10^{-8} . In fp16 such a gradient becomes 0, and a real gradient is lost without any warning. **Loss scaling** fixes this with one multiplication and one division:

$$L_{\text{scaled}} = s \cdot L, \quad \nabla_{\text{used}} = \frac{1}{s} \nabla L_{\text{scaled}}$$

In words: multiply the loss L by a large scale factor s (for example $2^{16} = 65536$) before calling `backward()`. By the chain rule, every gradient computed from the scaled loss is also multiplied by s . This moves tiny gradients out of the underflow zone into the range fp16 can store. Before the optimizer uses them, divide by the same s in fp32, so the weight update is correct. The format bf16 has the same exponent range as fp32 (it drops precision, not range), so it rarely needs this trick.

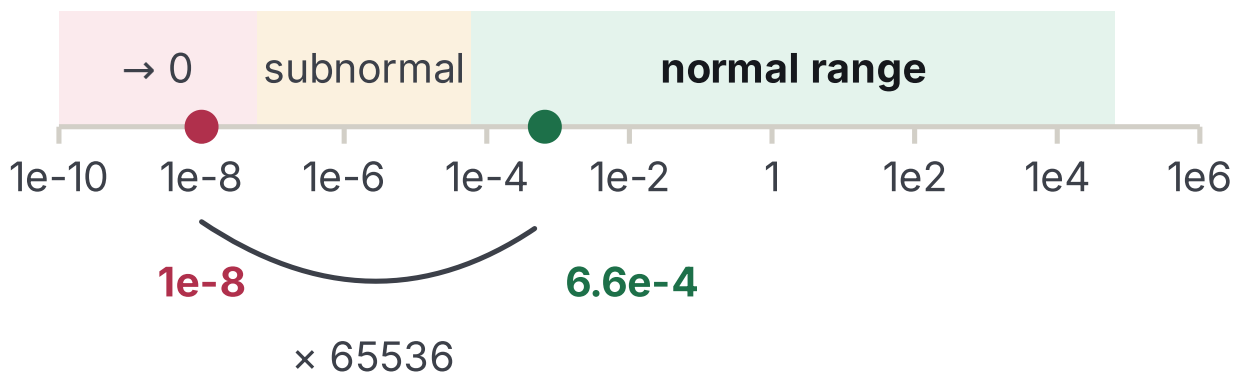


Figure 20.3 What fp16 can store, on a log scale. A gradient of 10^{-8} falls in the zone that rounds to 0. Multiplied by the loss scale 65536, it becomes about 6.6×10^{-4} , well inside the normal range. For bf16 the normal range goes down to about 10^{-38} , far off the left edge.

◆ THE SAME THING IN PYTORCH

>_ UNDERFLOW AND LOSS SCALING, CHECKED ON THE CPU

```
import torch

g = torch.tensor(1e-8) # a small but real fp32 gradient
print(g.half()) # tensor(0., dtype=torch.float16)
s = 65536.0 # loss scale, 2**16
print((g * s).half()) # tensor(0.0007, dtype=torch.float16)
print((g * s).half().float() / s) # tensor(9.9972e-09) unscaled
print(g.bfloat16()) # tensor(1.0012e-08, dtype=torch.bfloat16)

w = torch.tensor(1.0, dtype=torch.float16)
print(w + torch.tensor(1e-4, dtype=torch.float16)) # tensor(1., dtype=torch.float16)
print(torch.tensor(1.0) + 1e-4) # tensor(1.0001) keep
```

In a real training loop, PyTorch does the scaling for you. The loop below assumes a CUDA GPU and the `model`, `optimizer`, `criterion`, and `loader` from Section 14.

>_ A MIXED-PRECISION TRAINING LOOP WITH A GRADIENT SCALER

```

scaler = torch.amp.GradScaler("cuda")      # starts with s = 65536 and ada

for xb, yb in loader:
    optimizer.zero_grad()
    with torch.autocast("cuda", dtype=torch.float16):  # run the forward
        out = model(xb)
        loss = criterion(out, yb)

    scaler.scale(loss).backward()  # scale the loss up before backward
    scaler.step(optimizer)         # unscales gradients, then steps (skip)
    scaler.update()               # adjusts the scale factor for the ne.

```

In words: `autocast` runs each operation in fp16 when that is safe and in fp32 when it is not. The weights themselves stay in fp32. If a scaled gradient overflows to `inf`, the scaler skips that step and lowers s . If many steps go well, it raises s again. With `dtype=torch.bfloat16` you can usually drop the scaler.

CHECK YOURSELF 4 A gradient is 3×10^{-8} . What does fp16 store for it? And what happens if you use a scale of only $s = 1024$?

5 Gradient accumulation: a bigger batch on smaller memory

◆ INTUITION FIRST

Sometimes the batch size you want does not fit in GPU memory. **Gradient accumulation** builds a large batch from several small *micro-batches*. It processes them one at a time and adds up their gradients without resetting them. It calls `optimizer.step()` only once every few micro-batches, as if they were all one big batch.

☂ EVERYDAY PARALLEL

You survey 128 people, but your room holds only 32 at a time. So you run four sessions. To get the average answer of all 128, you add the four session averages and divide by 4. If you forget to divide, you report 4 times the true average. Gradient accumulation is the same: each micro-batch loss is already an average, so you divide it by the number of micro-batches.

◆ NOW THE MATH

Split a batch of N examples into M micro-batches of N/M examples each. Let L_m be the mean loss of micro-batch m . The mean loss of the full batch is then the mean of the micro-batch means:

$$L = \frac{1}{N} \sum_{n=1}^N \ell_n = \frac{1}{M} \sum_{m=1}^M \underbrace{\frac{M}{N} \sum_{n \in m} \ell_n}_{L_m} = \sum_{m=1}^M \frac{L_m}{M}$$

In words: ℓ_n is the loss of example n . Because gradients add up, $\nabla L = \sum_m \nabla(L_m/M)$. So if each micro-batch calls `backward()` on L_m/M , the summed `.grad` is exactly the gradient of one full batch.

◆ THE SAME THING IN PYTORCH

>_ FOUR MICRO-BATCHES GIVE THE SAME GRADIENT AS ONE BATCH OF 128

```

import torch, torch.nn as nn

torch.manual_seed(0)
model = nn.Linear(10, 1)
criterion = nn.MSELoss()
X, Y = torch.randn(128, 10), torch.randn(128, 1)

# (a) one true batch of 128
model.zero_grad()
criterion(model(X), Y).backward()
g_full = model.weight.grad.clone()

# (b) 4 micro-batches of 32, each loss divided by 4
accumulation_steps = 4
model.zero_grad()
for xb, yb in zip(X.split(32), Y.split(32)):
    loss = criterion(model(xb), yb) / accumulation_steps
    loss.backward() # gradients ADD UP in .grad (no .data)
g_acc = model.weight.grad.clone()

print(torch.allclose(g_full, g_acc, atol=1e-6)) # True
print((g_acc * accumulation_steps).norm() / g_full.norm()) # tensor(1.)

```

In words: the accumulated gradient matches the full-batch gradient. Without the division it would be 4 times larger. In a real loop you call `optimizer.step()` and `optimizer.zero_grad()` once every `accumulation_steps` micro-batches, for example with `if (i + 1) % accumulation_steps == 0:`.

▲ WATCH OUT

Dividing the loss by `accumulation_steps` is not optional. Without it, four accumulated micro-batch gradients add up to 4 times the gradient of a true batch of 128. With plain SGD this acts like a 4 times larger learning rate. It can make training unstable in a way that looks like an unrelated bug.

CHECK YOURSELF 5

You forget to divide by 4, but your optimizer is Adam, not SGD. Does the mistake still matter as much?

6 Distributed training: an overview

◆ INTUITION FIRST

When a model or dataset is too large for one machine, the work is split across several. There is no single “distributed training” technique. You can split different things, and real systems often combine several splits. Every split costs communication between devices, and that communication is often the real limit, not the computation.

☂ EVERYDAY PARALLEL

A school must grade a large stack of exams. It can give each teacher the same answer key and a different pile of exams (data parallelism). Or each teacher can grade only one page and pass the exam to the next teacher (pipeline parallelism). Either way, the teachers must coordinate: meet to combine notes, or wait for the exam from the previous teacher. That coordination costs time.

Data parallelism, the most common setup

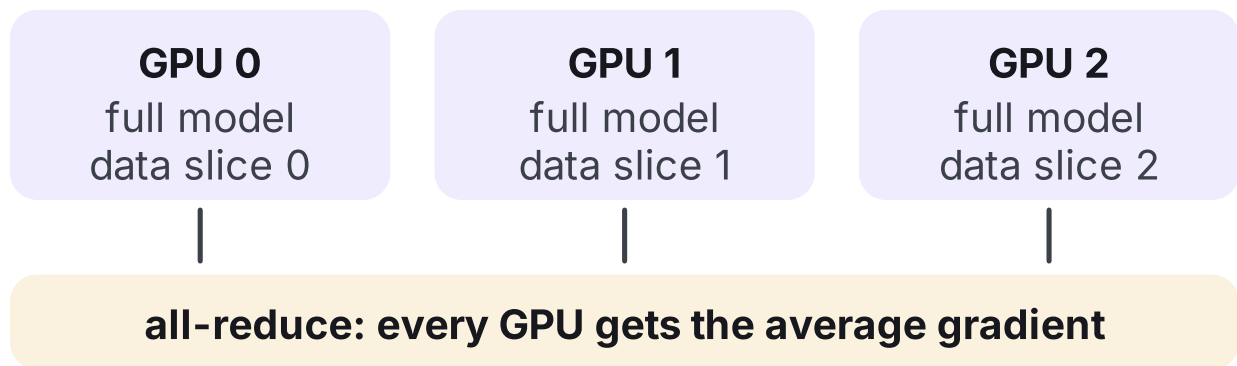
In **data parallelism**, every device (GPU) holds a full copy of the model. Each device processes a different slice of the data and computes its own gradients. Then, before any weights change, all devices combine their gradients with an **all-reduce**. An all-reduce is a group operation after which every device holds the same combined (usually averaged) gradient. So every copy applies the same update and the copies stay the same. With 4 GPUs of 32 examples each, this is the gradient of one batch of 128, as in gradient accumulation.

A full copy on every GPU wastes memory. With Adam, each parameter also needs two optimizer values and a gradient. *Sharded* data parallelism fixes this: each GPU stores only its share of the weights, gradients, and optimizer state, and collects the rest when it needs them. This is *ZeRO* (from DeepSpeed) and *FSDP* (Fully Sharded Data Parallel, in PyTorch).

When the model itself is split, there are other options:

- **Tensor parallelism**: single layers, or even single matrices, are split across devices. One forward pass then needs those devices to exchange intermediate results.
- **Pipeline parallelism**: different layers live on different devices. Data flows through them like an assembly line, and each device passes its output to the next.
- **Expert parallelism**: in a mixture-of-experts model, different experts live on different devices, and each token is sent to the device of its expert (Section 22).

Data parallelism



Pipeline parallelism

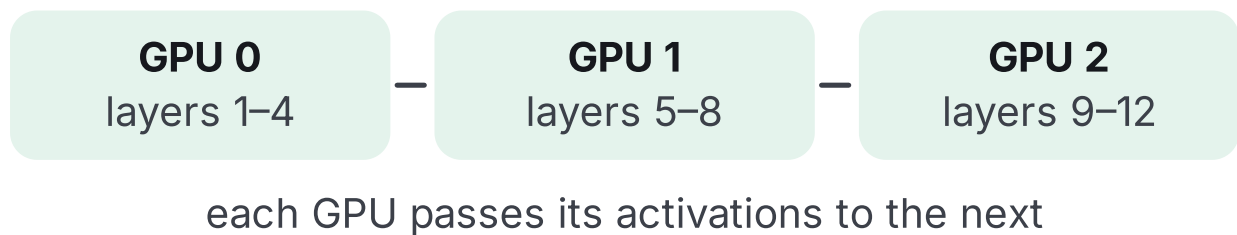


Figure 20.4 Two ways to split the work. In data parallelism the devices exchange *gradients* once per step. In pipeline parallelism they pass *activations* forward (and their gradients backward) between layers.

◆ THE REAL BOTTLENECK

Adding devices does not automatically make training proportionally faster. Every strategy needs devices to send data to each other: gradients in data parallelism, activations in tensor and pipeline parallelism. The links between devices are usually slower than the computation. Past a certain scale, training speed depends on how fast devices can communicate, not on how fast each one computes. This is why the interconnect (the physical links between devices) matters as much as raw compute in large-scale training.

CHECK YOURSELF 6

A model fits on one GPU for inference but runs out of memory in training with Adam. You have 8 GPUs. Does plain data parallelism help? What would?

7 What to carry forward

- Every normalization layer is **normalize, then scale and shift**:

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

The only difference between variants is which numbers μ and σ^2 are computed over.

- **Batch norm** normalizes each feature across the batch (it depends on other examples). **Layer norm** normalizes each example across its own features (it is independent of the batch), which is why transformers use layer norm.
- **Dropout** randomly zeros units during training so the network does not depend on any one of them. `model.eval()` switches it off.
- **Mixed precision** trades precision (and, for fp16, range) for speed and memory. **Loss scaling** keeps tiny fp16 gradients from rounding to zero, and fp32 master weights keep small updates.
- **Gradient accumulation** builds a large batch on limited memory by summing micro-batch gradients, each divided by the number of micro-batches, before one optimizer step.
- **Distributed training** splits the data (data parallelism, optionally sharded with FSDP or ZeRO) or the model (tensor, pipeline, or expert parallelism) across devices. Communication between devices, not raw compute, is often the real limit.

Next we move from building to reading: how to get a real algorithm out of a research paper, in Section 21. It is the last skill before the capstone.

Answers to “Check yourself”

1. A layer outputs the values 2, 4, 6. Normalize them with $\epsilon = 0$, $\gamma = 1$, $\beta = 0$. Then say what changes if $\gamma = 3$ and $\beta = 1$.

The mean is $\mu = 4$. The variance is $\sigma^2 = (4 + 0 + 4)/3 = 8/3$, so $\sigma \approx 1.63$. Then $z \approx (-1.22, 0, 1.22)$. With $\gamma = 3$ and $\beta = 1$, the output is $y = 3z + 1 \approx (-2.67, 1, 4.67)$. The normalization fixed the scale; the learned γ and β then chose a new one.

2. You call `nn.BatchNorm1d(4)` in training mode on a batch of shape `(1, 4)`. What would the formula give, and what does PyTorch do?

With one example, each feature's mean is that single value, so $x - \mu = 0$ exactly and the variance is 0. The normalized value is $z = 0/\sqrt{\epsilon} = 0$, so every output would be β , whatever the input was. The layer would carry no information. PyTorch refuses to do this and raises `ValueError: Expected more than 1 value per channel when training`. In `eval()` mode it works, because it uses the running statistics. Layer norm has no problem: it averages over the 4 features of the one example.

3. Someone writes their own dropout: in training it zeros each unit with $p = 0.5$ but does *not* divide by $1 - p$. In eval it passes x through. What goes wrong at test time, and how would you fix it without changing training?

In training, the next layer sees inputs whose expected value is $0.5x$, and it learns weights for that scale. At test time it suddenly gets x , twice as large on average, so its outputs are off. One fix is to multiply by $1 - p = 0.5$ at test time (this is the original, non-inverted dropout). Inverted dropout moves that factor into training instead, so evaluation needs no change.

4. A gradient is 3×10^{-8} . What does fp16 store for it? And what happens if you use a scale of only $s = 1024$?

It does not become 0. It rounds to the smallest fp16 subnormal, $2^{-24} \approx 5.96 \times 10^{-8}$, which is about twice the true value. So the gradient is kept, but with a large error. With $s = 1024$ the scaled value is about 3.1×10^{-5} . That is still below the smallest normal value 6.1×10^{-5} , so it is subnormal and has only a few correct digits. A larger scale such as 65536 moves it to about 2×10^{-3} , fully in the normal range. After `backward()` the gradients are divided by s again, so the update is the same as without scaling.

5. You forget to divide by 4, but your optimizer is Adam, not SGD. Does the mistake still matter as much?

Much less. Adam divides each gradient by a running estimate of its own size (Section 12). If every gradient is 4 times larger, that estimate is also 4 times larger, and the ratio is almost unchanged. Only the small ϵ in the denominator breaks the cancellation. In a quick test, three Adam steps moved the weights by the same amount

with and without the factor 4, while SGD moved them about twice as far. It is still a bug: gradient clipping, weight decay, and logged gradient norms all see the wrong size.

6. A model fits on one GPU for inference but runs out of memory in training with Adam. You have 8 GPUs. Does plain data parallelism help? What would?

Plain data parallelism does not help with memory. Every GPU still holds the full weights, gradients, and Adam state, so each one runs out of memory the same way. It only helps with speed. Sharded data parallelism (FSDP or ZeRO) does help. Each of the 8 GPUs stores about one eighth of the gradients and optimizer state (and, in the full version, of the weights). Smaller micro-batches with gradient accumulation also reduce activation memory.

PART 6

Capstone: Paper from Scratch

Reading a Paper Like an Engineer

You do not need to read every word of a research paper to get what is useful from it. You need a method: a few minutes to decide, an hour to understand, and more time only for the papers you build on.

32 min read

YOU'LL LEARN

- the three-pass reading method
- what to extract from any paper
- dissecting an MoE / Switch Transformer paper
- habits that make it stick

Before you start: Section 18: Attention & Transformers.

Many people read a paper the way they read a novel. They start at the top, read every sentence in order, and hope that understanding arrives by the last page. It usually does not, and it is slow. You can spend two hours on a minor paragraph before you even know if the paper is relevant. Engineers read papers differently: in passes. Each pass goes deeper than the last and has one clear goal. This section gives you that method. Then it applies the method to the papers behind our capstone project.

◆ THE ONE IDEA TO HOLD ONTO

Do not read a paper once, from start to finish. Read it in **passes of increasing depth**: skim, then structure, then reproduce. Stop as soon as the current pass has given you what you need.

1 The three-pass method

◆ INTUITION FIRST

This method is close to the one computer scientist S. Keshav described in “How to Read a Paper” (2007). Many working engineers arrive at something similar on their own. The idea: make three passes over the same document. Each pass asks a different question and takes more time than the last. You may stop after pass one if the paper does not matter to you.



Figure 21.1 The three passes as a funnel. Each pass costs more time, so fewer papers reach it. Most papers stop after pass 1, and that is a good result, not a failure.

☂ EVERYDAY PARALLEL

Learning a new city. First you look at a map for a few minutes: where is the center, where is the river, is this the right city at all (pass one)? Then you walk the main streets and find the one square everything connects to (pass two). Only for the city where you will live do you learn the side streets by finding your own way (pass three). Nobody learns every side street of every city they look at on a map.

1. **Pass 1: skim (about 5 minutes).** Read only the title, the *abstract* (the short summary at the top of the paper), every figure with its caption, and the conclusion. Skip the method section entirely. Goal: decide

what the paper claims and whether it is relevant to you. You may not be able to state its one-sentence claim after this pass. Then the paper is either poorly written, or you are not its intended reader. Both are useful to know before you spend more time.

2. **Pass 2: structure and the key equation (30–60 minutes).** Now read the body, in order, but do not follow every citation or check every derivation. Note the structure: what problem section 1 sets up, what section 3 proposes, what section 4 measures. Find the *one* equation that the paper's claim depends on. Usually there is a single formula that everything else exists to justify. Also find the main architecture or results figure. Redraw that figure from memory on paper. If you cannot, you have not understood it yet.
3. **Pass 3: reproduce (hours to days, only for papers you build on).** Derive the key equation yourself, line by line. Then implement the smallest unit of the method in code. Not the whole system: only the one component that carries the paper's idea. This pass turns “I read it” into “I understand it”. Code and math show you at once where your understanding has gaps; a quick read-through does not.

CHECK YOURSELF 1

You finish a 5-minute pass-1 skim and still cannot state the paper's main claim in one sentence. What should you do next?

CHECK YOURSELF 2

A search gives you 20 papers on a topic, and you will build on about one of them. With 5, 45, and 600 minutes for the three passes, roughly how long does the funnel take if 4 papers reach pass 2? How does that compare with a full pass 3 on all 20?

2 What to extract from any paper

◆ INTUITION FIRST

In almost any field, a research paper answers the same six questions. If you write down clear answers to these six, you have the paper's engineering value. You may never need the rest of the text again.

- **The problem.** What did not work before, in one sentence? Not “language models are hard”, but something testable, such as “dense models need proportionally more compute for every increase in parameters.”
- **The key idea, in one sentence.** If you cannot compress the contribution to one sentence, you have not found it yet. You are still describing the paper's title.
- **The central equation.** The formula without which the rest of the paper would make no sense. Usually there is only one.
- **The architecture diagram.** The picture that shows data flowing through the proposed system. If the paper has no diagram, draw one yourself from the method section. This shows you anything you skipped over.

- **The ablations:** what they tell you the idea is *for*.
- **What they do not tell you:** what the idea *costs*.

DEFINITION

An **ablation** is an experiment where the authors remove or replace one piece of their system and measure performance again. It shows whether that piece was necessary. (“Ablate” means to cut away.)

☂ EVERYDAY PARALLEL

A driver wants to know whether new tires make her car faster. She drives the same lap, on the same day, with the old tires and then with the new ones, and changes nothing else. The difference in lap time is the effect of the tires. That second lap is her ablation: it is evidence, not a claim. A paper's ablation table does the same thing. It trains the same model several times, each time with one part removed, so you can see which parts matter.

▲ WATCH OUT

Papers report what worked. They rarely report what it cost to get there. How many failed runs came before the reported one? How much hyperparameter tuning was needed? What hardware, and how much time, did the “efficient” method use? Which results did not reproduce on a different dataset? Read the “Limitations” section if there is one; it is often the most candid paragraph in the paper. If there is none, assume the costs exist and were left out. Look for them in the appendix or in the released code.

CHECK YOURSELF 3

A paper proposes a new component X and reports one number: “model with X gets 91% accuracy.” There is no ablation table. What is missing, and why does it matter?

3 Dissecting the MoE / Switch Transformer papers

◆ INTUITION FIRST

A **Mixture of Experts** (MoE) model replaces one big feed-forward layer with many smaller feed-forward layers, called *experts*. A small decision-maker, the *router*, picks which one or two experts handle each token.

The goal is to grow the model's total number of parameters (its capacity to store knowledge) without growing the compute for each token. Each token visits only a few experts, not one very large layer.

🔼 EVERYDAY PARALLEL

A large call center has dozens of specialists: billing, technical support, returns, legal. A caller never talks to all of them. A receptionist (the router) listens to the first sentence of the call and sends the caller to the one or two specialists (the experts) most likely to help. The center can add many specialists, while each call still takes the time of only one or two people. The router learns to make the receptionist's decision.

Two papers define most of the vocabulary you will meet here. The first is Shazeer et al., 2017, “*Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*”. It introduced the modern MoE layer with *noisy top- k gating*. The router adds learned noise to its scores and keeps a small number k of experts per token. Its experiments use values such as 2 and 4. The second is Fedus, Zoph & Shazeer, 2021, “*Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*”. It simplifies routing to one expert per token and uses it inside a Transformer. In between, GShard (Lepikhin et al., 2020) used top-2 routing in a large translation Transformer.

DEFINITION

The **router** is a small learned layer, usually one linear layer plus a softmax. It looks at a token's vector and outputs a probability for every expert: “how well does this token suit expert 1, expert 2, ...?” These probabilities decide the routing, and they also weight each chosen expert's output.

◆ NOW THE MATH

The one equation worth deriving yourself from the Switch Transformer paper is the **auxiliary load-balancing loss**. It is added to the normal training loss. It stops the router from sending almost every token to the same one or two experts:

$$L_{\text{aux}} = \lambda_{\text{aux}} \cdot E \sum_{i=1}^E f_i P_i$$

In words: E is the number of experts. f_i is the fraction of tokens in the batch that were sent to expert i , so $\sum_i f_i = 1$. P_i is the average probability the router gave to expert i over the batch. λ_{aux} is a small weight (the paper uses 0.01; it calls it α). The factor E makes the balanced value the same for any number of experts, as we show next.

Two cases make the formula concrete. With $E = 4$ experts and perfectly even routing, $f_i = P_i = 0.25$ for every expert:

$$\sum_i f_i P_i = 4 \times (0.25 \times 0.25) = 0.25, \quad L_{\text{aux}} = \lambda_{\text{aux}} \times 4 \times 0.25 = \lambda_{\text{aux}}$$

If instead every token goes to expert 1 ($f_1 = P_1 = 1$, the rest zero):

$$\sum_i f_i P_i = 1, \quad L_{\text{aux}} = \lambda_{\text{aux}} \times 4 \times 1 = 4 \lambda_{\text{aux}}$$

In words: the balanced case costs λ_{aux} and the fully collapsed case costs E times more. When f and P agree, balance is the best case. With $f = P$ the sum is $\sum_i f_i^2$, and because the f_i add up to 1, $\sum_i f_i^2 \geq 1/E$ (the Cauchy–Schwarz inequality), with equality only when every $f_i = 1/E$.

How does minimizing this loss change the router? The fractions f_i come from a hard choice (which expert had the top score), so they have no gradient. The gradient flows only through the probabilities P_i :

$$\frac{\partial L_{\text{aux}}}{\partial P_i} = \lambda_{\text{aux}} E f_i$$

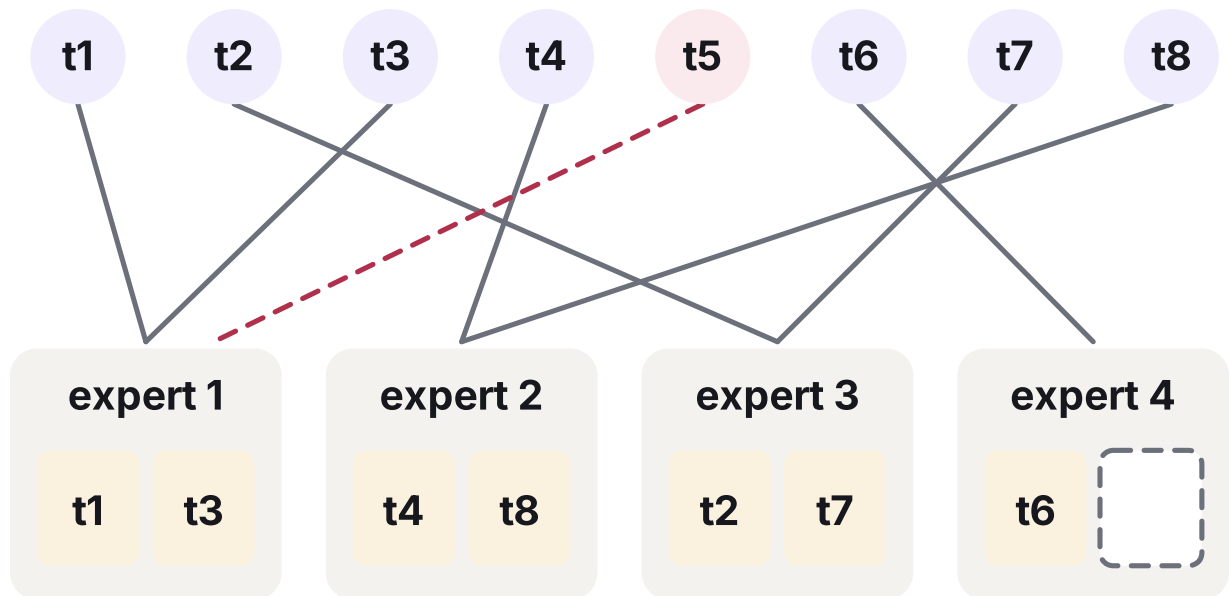
In words: the more tokens expert i received (f_i large), the harder gradient descent pushes its router probability P_i down. Experts that received few tokens are pushed down only a little, so they gain probability in comparison. Over many steps this moves tokens from busy experts toward idle ones.

DEFINITION

The **capacity factor** sets how many tokens each expert may process in one batch. With T tokens in the batch, k experts per token, E experts, and capacity factor c , each expert may take C tokens:

$$C = \frac{T \cdot k}{E} \times c$$

A factor of 1.0 gives each expert exactly its fair share, with no extra room. Tokens that arrive after an expert is full *overflow*: they skip that expert and pass through unchanged via the residual connection. A higher factor (say 1.25) gives extra room for uneven routing. The cost is reserved compute that is sometimes not used.



t5: expert 1 is full (capacity 2).
It skips the expert; the residual carries it on.

Figure 21.2 Top-1 routing with $T = 8$ tokens, $k = 1$, $E = 4$ experts, and capacity factor 1.0, so each expert has $8 \cdot 1/4 = 2$ slots. Three tokens choose expert 1, so the last one (t5) overflows. Expert 4 has an empty slot: uneven routing wastes compute and drops tokens at the same time.

DEFINITION

Top- k routing sends each token to its k highest-scoring experts. Their outputs are combined, weighted by the router's probabilities. GShard uses top-2. Earlier work had assumed that a token must go to at least two experts for the router to learn well. Switch Transformer tested **top-1** (one expert per token) and found that quality stayed good. Top-1 also reduces expert compute and the communication between devices.

Concrete questions to ask when you open one of these papers:

- How many experts (E), and how does the compute per token compare with a dense model with the same total number of parameters?
- What is k (top-1, top-2, more), and what is the argument for that choice?
- What is the capacity factor, and what happens to tokens that overflow?
- How is λ_{aux} chosen, and what happens in their ablation when it is set to zero?
- Do they report a dropped-token rate or routing collapse? Either is a sign that load balancing was not fully solved.

- Is the router trained end to end with the rest of the model, or does it use a fixed rule (for example, hashing tokens to experts)?

In Section 22 we implement exactly this from scratch in PyTorch: a router, a small set of expert feed-forward networks, and this load-balancing loss. Keep these six questions in mind; the next section answers each one in code.

CHECK YOURSELF 4

With $E = 4$ experts and $\lambda_{\text{aux}} = 0.01$, routing gives $f = P = (0.7, 0.1, 0.1, 0.1)$. Compute the auxiliary loss and compare it with the balanced case.

CHECK YOURSELF 5

A batch has $T = 1024$ tokens, $E = 8$ experts, top-1 routing, and capacity factor 1.25. What is each expert's capacity? If 200 tokens choose expert 3, what happens?

4 Practical habits that make it stick

◆ INTUITION FIRST

The method above tells you how to read. These three habits help you keep what you learned.

- **Read the code alongside the paper.** Many major papers release code, or a faithful reimplementation exists (search for the paper's title plus “github”). When the text says “we route tokens to their top-1 expert”, find the three lines of code that do it. Code cannot be vague the way prose can.
- **Reproduce the smallest unit, not the whole system.** You do not need to train a trillion-parameter Switch Transformer to understand it. You need to implement one router that chooses among four tiny experts on made-up data. If that ten-line version behaves as the paper predicts, you understand the mechanism.
- **Keep a short note per paper:** one line for the key idea, the central equation copied by hand (not pasted), and one open question you still cannot answer. Look at the open questions again after you read the next paper in the area. Papers often answer each other's questions.

☂ EVERYDAY PARALLEL

A musician who cannot play one fast passage does not play the whole piece again and again. She practices those four bars alone, slowly, until she knows why her fingers get stuck. Then she puts them back into the piece. Do the same here: build the router alone, on toy data, before you build the model it belongs to.

◆ THE SAME THING IN PYTORCH

>_ THE SMALLEST REPRODUCIBLE UNIT: A TOP-1 ROUTER AND ITS BALANCING LOSS

```
import torch

torch.manual_seed(0)
E = 4          # number of experts
d_model = 8

router = torch.nn.Linear(d_model, E)      # the whole "decision-maker"
tokens = torch.randn(6, d_model)         # 6 toy tokens, nothing real

logits = router(tokens)                  # (6, 4): one score per expert, 1
probs = torch.softmax(logits, dim=-1)
chosen = probs.argmax(dim=-1)             # (6,): top-1 routing decision
print(chosen)                            # tensor([1, 2, 1, 3, 2, 1])

f = torch.bincount(chosen, minlength=E) / len(chosen)  # fraction of tokens
P = probs.mean(dim=0)                                # mean router probabilities
lam_aux = 0.01
aux = lam_aux * E * (f * P).sum()
print(f)                                              # tensor([0.0000, 0.5000, 0.3333, 0.1667])
print(round(aux.item(), 4))                           # 0.0102
```

In words: this is the core routing decision and the balancing loss, nothing more. Expert 1 gets half the tokens and expert 0 gets none, yet the loss is only a little above the balanced value 0.01. The reason: the untrained router's probabilities P are still almost even (about 0.23 to 0.27 each). Section 22 adds the experts, the output weighting, the capacity limit, and training.

CHECK YOURSELF 6

In the code above, `f` is computed from `argmax`. If you call `aux.backward()`, which tensor does the gradient reach the router through, and why?

5 What to carry forward

- **Three passes, increasing depth:** skim for relevance, read the structure and the one key equation, then reproduce the smallest unit to understand it.
- Extract six things from any paper: the **problem**, the **key idea** in one sentence, the **central equation**, the **architecture diagram**, the **ablations**, and what it does not tell you.
- An **ablation** shows a component matters by removing it and measuring again. A single number with no matched baseline shows nothing.
- MoE vocabulary: a **router** scores each token against the E experts. **Top-k routing** sends it to k of them (top-1 in Switch Transformer, top-2 in GShard). The **capacity factor** limits how many tokens each expert takes. The **load-balancing loss** $\lambda_{\text{aux}} E \sum_i f_i P_i$ penalizes uneven routing.

You now have the method and the vocabulary. In Section 22 we stop reading about a Mixture of Experts and build one.

Answers to “Check yourself”

1. You finish a 5-minute pass-1 skim and still cannot state the paper's main claim in one sentence. What should you do next?

Do not start a full pass 2 and hope it becomes clear. First look again at the pass-1 material, only for the claim. Reread the last sentences of the abstract and the first paragraph of the conclusion; the claim is usually stated plainly there. If it is still unclear, the paper may not suit you right now. Check whether it assumes background you do not have, or whether it is not the paper you need. Deciding this in 5 minutes is the purpose of pass one. It is much cheaper than finding out after an hour of pass two.

2. A search gives you 20 papers on a topic, and you will build on about one of them. With 5, 45, and 600 minutes for the three passes, roughly how long does the funnel take if 4 papers reach pass 2? How does that compare with a full pass 3 on all 20?

Pass 1 on all 20: 100 minutes. Pass 2 on 4: 180 minutes. Pass 3 on 1: 600 minutes. That is about 880 minutes, or roughly 15 hours. A deep reading of all 20 papers would take $20 \times 600 = 12,000$ minutes, about 200 hours. The early passes are cheap filters that protect your time for the one paper that matters.

3. A paper proposes a new component X and reports one number: “model with X gets 91% accuracy.” There is no ablation table. What is missing, and why does it matter?

The missing number is “model without X, everything else the same”. Without that baseline, 91% tells you nothing about whether X caused it. A larger learning rate or three extra epochs might have given 91% anyway. An ablation is not an optional extra; it is the evidence for the paper's claim. Treat a single number with no matched baseline with doubt, until you find the ablation or run it yourself.

4. With $E = 4$ experts and $\lambda_{\text{aux}} = 0.01$, routing gives $f = P = (0.7, 0.1, 0.1, 0.1)$. Compute the auxiliary loss and compare it with the balanced case.

$\sum_i f_i P_i = 0.7^2 + 3 \times 0.1^2 = 0.49 + 0.03 = 0.52$. So $L_{\text{aux}} = 0.01 \times 4 \times 0.52 = 0.0208$. The balanced case gives $L_{\text{aux}} = \lambda_{\text{aux}} = 0.01$, so this uneven routing costs about twice as much. The gradient on P_1 is $0.01 \times 4 \times 0.7 = 0.028$, seven times the gradient on each other expert. So training pushes expert 1's probability down the most.

5. A batch has $T = 1024$ tokens, $E = 8$ experts, top-1 routing, and capacity factor 1.25. What is each expert's capacity? If 200 tokens choose expert 3, what happens?

$C = 1024 \cdot 1/8 \times 1.25 = 160$ tokens. Expert 3 processes 160 of its 200 tokens. The other 40 overflow: they skip the expert and continue unchanged through the residual connection. They are not sent to another expert and not tried again. The balancing loss exists to make such overflows rare.

6. In the code above, `f` is computed from `argmax`. If you call `aux.backward()`, which tensor does the gradient reach the router through, and why?

Only through `P.argmax` and `P.bincount` return integer results with no gradient, so `f` acts as a constant. `P` is the mean of the softmax output, which depends smoothly on the router weights. This is the equation $\partial L_{\text{aux}} / \partial P_i = \lambda_{\text{aux}} E f_i$ from above, now in code.

Mixture of Experts from Scratch

The technique behind several of today's largest models. Build the router, derive the balancing loss, and put it into the transformer you already understand.

70 min read

YOU'LL LEARN

- conditional computation
- top-k router math
- the load-balancing auxiliary loss (derived)
- a full annotated MoE transformer, dense vs MoE

Before you start: Section 19: Language Modeling & GPT and Section 21: Reading a Paper Like an Engineer.

Every model you have built so far runs **all** of its weights on **every** input. A bigger model means more weights, and more weights mean more arithmetic per token. Cost and capacity are chained together. This lesson breaks that chain. A **Mixture of Experts** (MoE) lets you grow the number of parameters a model *owns* without growing the work it does *per token*. Some of today's largest open models use it. They store hundreds of billions of parameters but use only a few tens of billions for each token. We will derive the method and build a working one.

◆ THE ONE IDEA TO HOLD ONTO

A dense layer asks *every* parameter about *every* token. An MoE layer keeps many separate sub-networks (the **experts**), and a tiny **router** sends each token to only a few of them. Total knowledge goes up; work per token stays flat. This lesson is about building that router and stopping it from cheating.

This is the capstone. It uses the transformer block from Section 18 and the next-token training setup from Section 19. If those are fresh in your mind, this lesson will be easy to follow. An MoE changes only *one* sub-layer of the block you already know.

1 Conditional computation: pay for what you use

◆ INTUITION FIRST

Here is the tension. Bigger models usually perform better. This is one of the most reliable findings in machine learning. But in a normal (*dense*) network, “bigger” means every input runs through every weight. Doubling the parameters roughly doubles the compute (the arithmetic work) for *every* token, and that gets expensive fast. **Conditional computation** is the way out. Instead of running the whole network on each token, run only the *part* of the network that is relevant to that token. Most weights stay unused for most inputs.

☂ EVERYDAY PARALLEL

You walk into a large hospital with a broken wrist. You are *not* examined by all three hundred doctors on staff. The front desk (the **router**) looks at your problem and sends you to an orthopedist. The hospital's total expertise is huge: cardiology, neurology, pediatrics, and more. But *your* visit uses the time of only one or two specialists. The hospital can hire more specialists without making each visit longer. That is conditional computation: a lot of available skill, but each case uses only the relevant part.

◆ NOW THE MATH

Let one expert sub-network cost C units of compute to run on one token. Put E of these experts side by side. Let the router send each token to k of them, with $k \ll E$ (read “ k is much smaller than E ”). Then:

$$\text{Total parameters} \propto EC, \quad \text{Compute per token} \propto kC$$

In words: the knowledge you store grows with the number of experts E . The work you do per token grows only with how many experts you use, k . With $E = 64$ experts and $k = 2$, you store $64\times$ the parameters of one expert but do only $2\times$ its work. In a dense model these two numbers are tied together. Here they are separate: E sets the storage and k sets the compute.

DEFINITION

A model's **active parameters** are the weights that take part in processing a given token. Its **total parameters** are all the weights it stores. In a dense model these two numbers are equal. The promise of MoE is to make **total parameters** large while keeping **active parameters** small.

◆ WHY THIS MATTERS NOW

Several large models with published details use this design. Switch-C (2021) has about 1.6 trillion parameters. Mixtral 8x7B (2023) stores about 47 billion parameters but uses about 13 billion per token. DeepSeek-V3 (2024) stores 671 billion and uses 37 billion per token. In each case, the equation above is the reason. You are about to build the same mechanism at a small scale.

CHECK YOURSELF 1

An MoE layer has $E = 8$ experts, each the same size as a dense feed-forward block, and routes each token to $k = 2$ of them. Predict how many times more parameters it stores than one dense block, and how much more compute it does per token.

2 Where the experts go: one swapped sub-layer

◆ INTUITION FIRST

Recall the transformer block from Section 18. Each block has two sub-layers. The **self-attention** sub-layer is where tokens look at each other and mix information. The **feed-forward network** (FFN) is a small two-layer network applied to each token on its own. The FFN holds most of a transformer's parameters and does most of the per-token computation. MoE makes one change: **replace that single FFN with a set of FFNs plus a router**. Attention does not change.

So the block goes from this:

$$\mathbf{x} \rightarrow \mathbf{x} + \text{Attention}(\mathbf{x}) \rightarrow \mathbf{x} + \text{FFN}(\mathbf{x})$$

to this, with only the second step changed:

$$\mathbf{x} \rightarrow \mathbf{x} + \text{Attention}(\mathbf{x}) \rightarrow \mathbf{x} + \text{MoE}(\mathbf{x})$$

In words: the $+$ is the residual connection (add the sub-layer's output back onto its input, from Section 18). The only difference is that the fixed FFN is now an MoE block. Inside, it picks a few experts per token instead of running one shared network. From the outside, the block still takes a vector of size d_{model} in and returns one of the same size.

DEFINITION

An **expert** is an ordinary FFN, the same two-layer network the dense block used. Nothing inside an expert is special. The specialization comes from the **router**: it learns to send different *kinds* of tokens to different experts, so each expert can specialize. We build the expert first because you already know it.

◆ THE SAME THING IN PYTORCH

>_ AN EXPERT IS A PLAIN FEED-FORWARD BLOCK

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Expert(nn.Module):
    """One expert = the standard transformer FFN: expand, nonlinearity, project"""
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.up = nn.Linear(d_model, d_ff)    # widen d_model → d_ff
        self.down = nn.Linear(d_ff, d_model)  # narrow d_ff → d_model

    def forward(self, x):
        # x: (num_tokens, d_model) → (num_tokens, d_model)
        return self.down(F.gelu(self.up(x)))
```

In words: this is the FFN from a normal transformer. It has a wide hidden layer of size d_{ff} (usually $4 d_{\text{model}}$). Then comes a GELU nonlinearity (a smooth activation function; see Section 9). Last is a projection back to d_{model} . An MoE layer will hold E of these in a list.

CHECK YOURSELF 2

What would go wrong if we routed tokens to separate experts inside the attention sub-layer instead of the FFN?

3 The router: top-k gating, derived

◆ INTUITION FIRST

The **router** (also called the **gating network**) is the front desk. For each token it answers two questions. *Which* experts should handle this token? And *how much* should each of them count? It answers with a small learned matrix, a softmax, and a “keep only the top few” step. The router is one linear layer followed by a selection rule. Let's build it step by step.

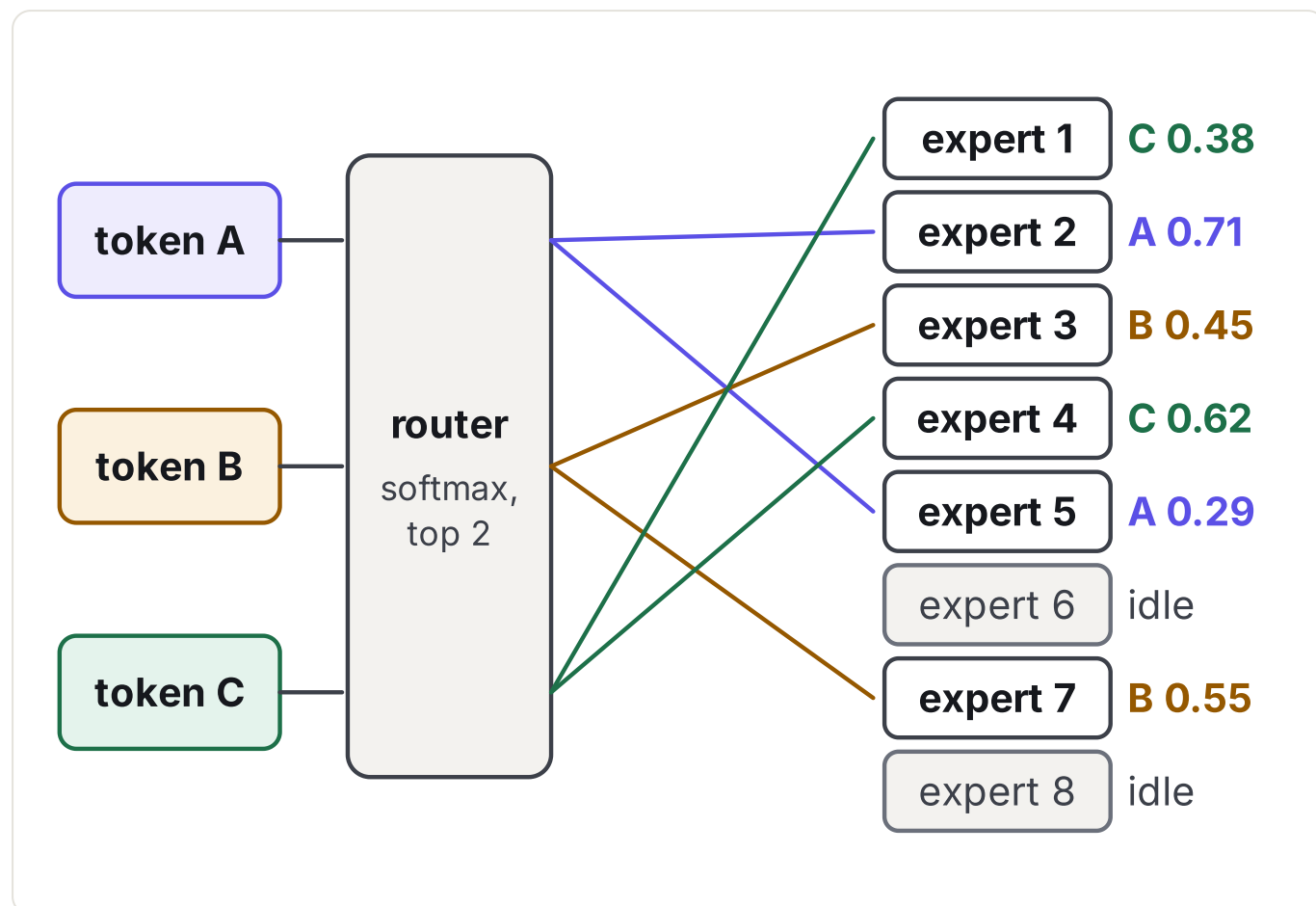


Figure 22.1 Top-2 routing with $E = 8$ experts. The router sends each token to its two highest-scoring experts, with weights that sum to 1 (0.71 + 0.29 for token A). Experts 6 and 8 get no token from this group, so they do no work for it. The labels on the right show which token each expert serves, and with what weight.

◆ NOW THE MATH

Take one token's vector $\mathbf{x}_t \in \mathbb{R}^d$. Here $d = d_{\text{model}}$, and t indexes the token. The router owns one weight matrix $\mathbf{W}_g \in \mathbb{R}^{E \times d}$, where E is the number of experts. **Step 1: gate logits.** Multiply to get one raw score per expert:

$$\mathbf{g}_t = \mathbf{W}_g \mathbf{x}_t \in \mathbb{R}^E$$

In words: $\mathbf{g}_t$ is a list of E numbers, one per expert. Each is the raw affinity of this token for that expert. These are called **logits**: raw scores, not yet probabilities. They can be negative and do not sum to anything in particular. The router matrix $\mathbf{W}_g$ is tiny. For $d = 512$ and $E = 8$ it has about four thousand numbers, which is very small next to one expert.

Step 2: routing probabilities. Turn the raw scores into probabilities with a *softmax*. (A softmax turns a list of numbers into positive values that sum to 1; see Section 4.)

$$p_{t,i} = \text{softmax}(\mathbf{g}_t)_i = \frac{\exp(g_{t,i})}{\sum_{j=1}^E \exp(g_{t,j})}$$

In words: $p_{t,i}$ is the probability the router gives to expert i for token t . The exponential makes every score positive. Dividing by the sum makes the scores add up to 1. Now we have a clean distribution over experts for this token, for example $[0.6, 0.25, 0.1, 0.05]$.

Step 3: top-k selection. Running *all* E experts would defeat the purpose. So we keep only the k experts with the highest probabilities. We call that chosen set $\mathcal{S}_t$:

$$\mathcal{S}_t = \underset{i}{\text{top-}k} p_{t,i} \quad (\text{the } k \text{ experts with the largest } p_{t,i})$$

In words: $\mathcal{S}_t$ is the short list. For $k = 2$ it holds the two experts this token scored highest. Every expert outside $\mathcal{S}_t$ is switched off for this token and does no work. This selection step is where the "sparse" in "sparse MoE" comes from.

Step 4: renormalize the survivors. The kept probabilities no longer sum to 1, because we threw the rest away. If they summed to 0.85, we would scale this token's output down by 15% without meaning to. So we divide by their own sum:

$$w_{t,i} = \frac{p_{t,i}}{\sum_{j \in \mathcal{S}_t} p_{t,j}} \quad \text{for } i \in \mathcal{S}_t$$

In words: $w_{t,i}$ is the final weight for expert i on token t . After this step the chosen weights sum to 1, so the experts' outputs are *blended*, not shrunk. If $k = 2$ and the top two probabilities are 0.6 and 0.25, the weights are $0.6/0.85 \approx 0.706$ and $0.25/0.85 \approx 0.294$.

Step 5: combine. Run the token through each chosen expert and add up the outputs, weighted by $w_{t,i}$:

$$\mathbf{y}_t = \sum_{i \in \mathcal{S}_t} w_{t,i} \text{FFN}_i(\mathbf{x}_t)$$

In words: FFN_i is expert i . The output $\mathbf{y}_t$ is a weighted average of the k chosen experts' answers. Experts the router trusts more get a bigger say. This one line is the whole forward pass of an MoE layer. The rest of the code does it efficiently for a whole batch at once.

◆ WHY THE ROUTER CAN LEARN AT ALL

The weights $w_{t,i}$ multiply the experts' outputs. So gradients (from backprop) flow back through them into $\mathbf{W}_g$. If giving more weight to expert 3 would lower the loss, the router is pushed to raise $p_{t,3}$. The hard “pick the top k ” step has no gradient. But the kept experts are weighted by their *soft* probabilities, so the router still gets a gradient through them. That is why top-k gating trains end to end with plain gradient descent.

▲ WATCH OUT

With $k = 1$, Step 4 always gives $w_{t,i} = p/p = 1$. The weight is then a constant, so the task loss sends *no* gradient to the router. For this reason Switch Transformer (which uses $k = 1$) skips the renormalization and uses the raw probability $p_{t,i}$ as the weight. Renormalize only when $k \geq 2$.

◆ THE SAME THING IN PYTORCH

>_ ROUTER: LINEAR, SOFTMAX, TOP-K, RENORMALIZE (THE AUX LOSS IS DERIVED NEXT)

```

class Router(nn.Module):
    """Scores each token over E experts, keeps the top-k, and renormalizes
    def __init__(self, d_model, num_experts, k, aux_weight=0.01):
        super().__init__()
        self.gate = nn.Linear(d_model, num_experts, bias=False) # W_g: s
        self.num_experts = num_experts
        self.k = k
        self.aux_weight = aux_weight # lambda

    def forward(self, x):
        # x: (num_tokens, d_model)
        logits = self.gate(x) # (num_tokens, E) gate
        probs = logits.softmax(dim=-1) # (num_tokens, E) rou

        # top-k experts per token
        topk_probs, topk_idx = probs.topk(self.k, dim=-1) # each (num_t
        # renormalize the survivors so their weights sum to 1
        topk_weights = topk_probs / topk_probs.sum(dim=-1, keepdim=True)

        aux_loss = self._load_balancing_loss(probs, topk_idx) # derived
        return topk_idx, topk_weights, aux_loss

```

In words: the four math steps map onto four lines. `topk` returns both the kept probabilities and the indices of their experts. Dividing by `topk_probs.sum(...)` is the renormalize step. The `_load_balancing_loss` method comes in the next section, where we derive what it computes.

CHECK YOURSELF 3

A token's softmax over 4 experts is $[0.5, 0.3, 0.15, 0.05]$ and $k = 2$. Predict which experts are chosen and their final weights.

CHECK YOURSELF 4

Find the bug: someone sets `k=1` in the `Router` above to get a Switch-style layer. The task loss falls a little, but the routing never seems to improve. Why?

4 Load balancing: deriving the auxiliary loss

◆ INTUITION FIRST

Left alone, the router cheats. Early in training, one expert happens to be slightly better. So the router sends it slightly more tokens. So it trains faster and gets even better. So the router sends it *even more* tokens. This is a **rich-get-richer** feedback loop, also called *routing collapse*. A few experts get almost all the tokens, and the rest never get enough tokens to learn. You paid for E experts and got 2. We need a gentle force that keeps the traffic spread out.

☂ EVERYDAY PARALLEL

Back at the hospital, one doctor gets a great reputation. Everyone asks for that doctor, so the queue grows to three hours while four other good doctors sit in empty rooms. The hospital is *underused*, and the famous doctor is overloaded. The fix is a scheduling rule that nudges patients toward the idle doctors. It does not force anyone; it tilts the odds so the whole staff stays busy. The **auxiliary loss** is that scheduling rule, written as math and added to training.

DEFINITION

An **auxiliary loss** is an extra term added to the training loss. It does not measure task performance. Instead, it pushes some internal quantity toward a shape we want. Here it pushes the token traffic toward an even spread across experts. The model minimizes *task loss + auxiliary loss* together.

◆ NOW THE MATH

We want to measure imbalance and then penalize it. Let T be the number of tokens in the batch (all sequences flattened into one list). Each token fills k routing slots, so there are kT slots in total. For each expert i we define two quantities. The first is a **hard** count of what was routed:

$$f_i = \frac{1}{kT} \sum_{t=1}^T 1[i \in \mathcal{S}_t]$$

In words: f_i is the *share of routing slots that went to expert i* . The symbol $1[\cdot]$ is the indicator: it is 1 when the condition inside is true and 0 otherwise. We count how many tokens have expert i in their top k and divide by the number of slots, kT . Because every slot goes to some expert, $\sum_i f_i = 1$. A token can use an expert at most once, so no f_i can be larger than $1/k$.

The second is a **soft** average: how much probability the router *wanted* to give each expert, before the hard cutoff:

$$P_i = \frac{1}{T} \sum_{t=1}^T p_{t,i}$$

In words: P_i is the *mean router probability* for expert i , averaged over all tokens. It uses the full softmax $p_{t,i}$, not only the chosen experts. Each token's probabilities sum to 1, so $\sum_i P_i = 1$ too. P_i is smooth and differentiable, which will matter soon. Think of f_i as "who was sent" and P_i as "who the router leaned toward, on average".

The **Switch Transformer auxiliary loss** multiplies these two, sums over the experts, and scales the result:

$$L_{\text{aux}} = \lambda_{\text{aux}} E \sum_{i=1}^E f_i P_i$$

In words: λ_{aux} is a small weight, typically 0.01. E is the number of experts. It is there so the loss keeps the same size when you add experts. The core is $\sum_i f_i P_i$: a dot product between the hard load and the soft preference. At perfect balance, $f_i = P_i = 1/E$, so the sum is $E \cdot (1/E)^2 = 1/E$ and $L_{\text{aux}} = \lambda_{\text{aux}}$. (Switch Transformer uses $k = 1$, where our f_i is the same as theirs. Dividing by k keeps the balanced value at λ_{aux} for any k .)

Why this loss spreads the load

Start with the gradient. The count f_i comes from a hard top- k choice, so it has no gradient. Training treats it as a fixed number. Only P_i carries a gradient:

$$\frac{\partial L_{\text{aux}}}{\partial P_i} = \lambda_{\text{aux}} E f_i$$

In words: the push on expert i 's probability is proportional to its current load f_i . Gradient descent lowers every P_i , but it lowers the overloaded ones (large f_i) the most. Since probabilities must sum to 1, the idle experts gain probability. Top- k picks the experts with the highest probabilities, so the next batch sends more tokens to the idle experts.

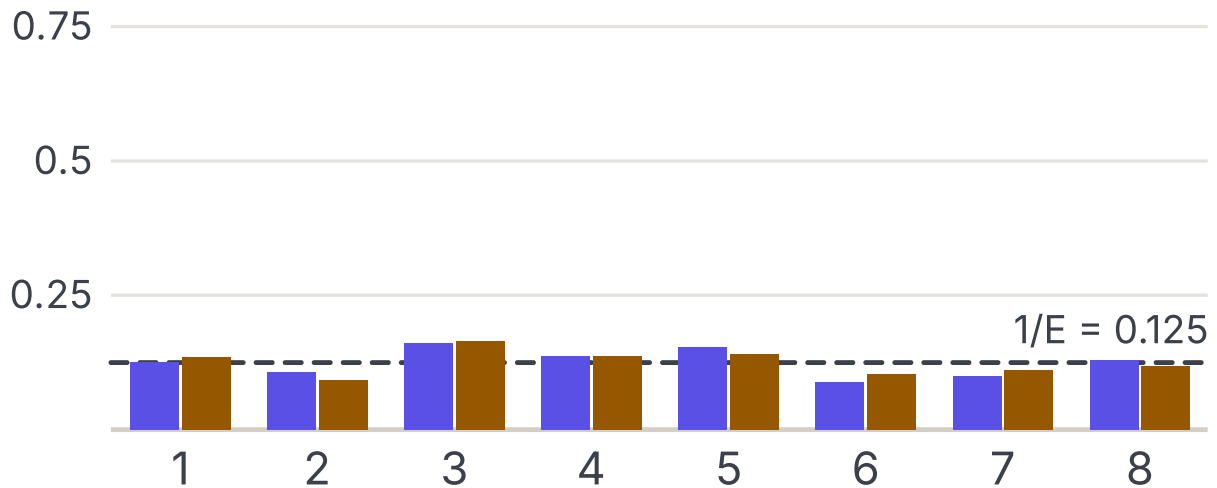
Where does this stop? Top- k sends tokens where the probabilities are high, so over training f_i and P_i rise and fall together. Look at the case $f_i = P_i$ for all i . The loss then depends on $\sum_i P_i^2$, and the Cauchy-Schwarz inequality gives its smallest value:

$$\left(\sum_{i=1}^E P_i \cdot 1 \right)^2 \leq \left(\sum_{i=1}^E P_i^2 \right) \left(\sum_{i=1}^E 1^2 \right) \implies 1 \leq E \sum_{i=1}^E P_i^2$$

$$\sum_{i=1}^E P_i^2 \geq \frac{1}{E}, \quad \text{with equality only if } P_i = \frac{1}{E} \text{ for all } i$$

In words: when load and preference agree, the sum of squares is smallest when every expert has the same share, $1/E$. A lopsided distribution (one expert at 0.9) has a big sum of squares. A flat one has the smallest. So, together with the gradient above, the loss pushes routing toward the even $1/E$ split. Note that this is not a proof that $\sum_i f_i P_i$ is smallest at balance for *every* pair f, P . When f and P disagree, the sum can be lower. But top-k keeps them close, and the gradient always pushes toward balance.

trained router: aux loss = 0.0103



pushed toward expert 1: aux loss = 0.0338

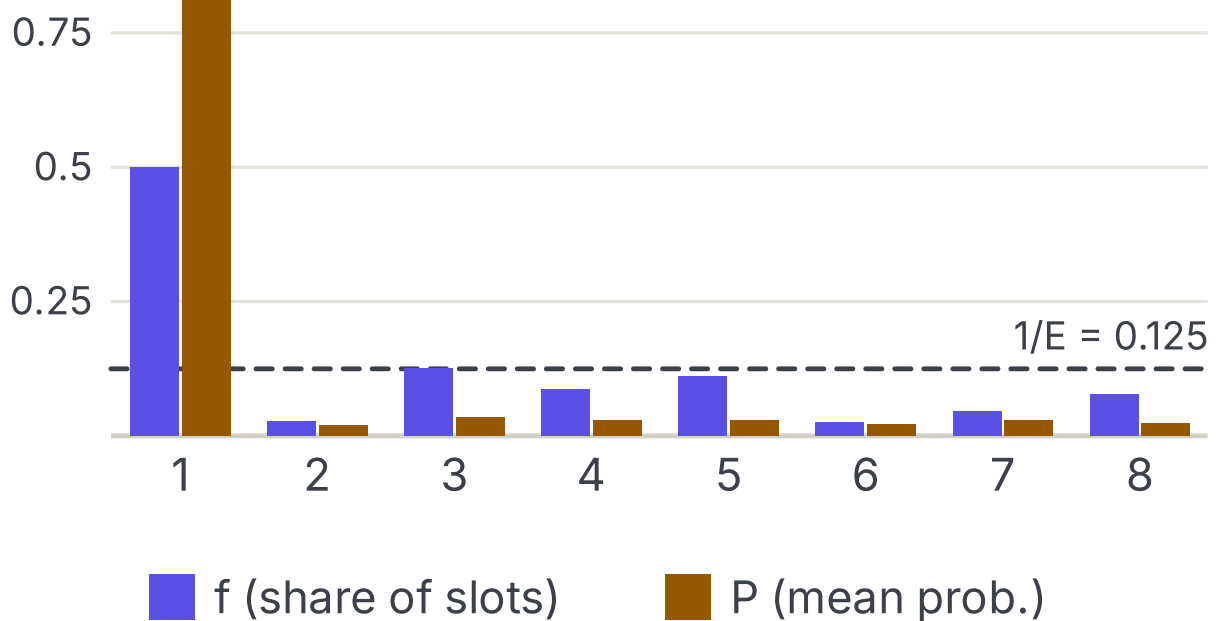


Figure 22.2 Load share f_i (violet) and mean probability P_i (amber) for the 8 experts, measured in this lesson's run. Top: the trained router; all bars sit near the even share $1/8$ (dashed line) and the aux loss is 0.0103. Bottom: the same router with 4 added to expert 1's logit. Expert 1 takes $P_1 = 0.82$ and the maximum possible load $f_1 = 1/k = 0.5$, and the aux loss more than triples to 0.0338.

The weight λ_{aux} is a real trade-off. If it is too small, the balancing force is too weak to stop collapse, and training drifts back to a few favorite experts. If it is too large, it overpowers the task loss. The router is then

forced to spread tokens evenly even when that hurts the prediction. The common value $\lambda_{\text{aux}} = 0.01$ keeps balance without drowning out the task.

◆ THE SAME THING IN PYTORCH

>_ THE LOAD-BALANCING LOSS (A METHOD OF ROUTER)

```
def _load_balancing_loss(self, probs, topk_idx):
    # probs: (num_tokens, E) full softmax routing probabilities
    # topk_idx: (num_tokens, k) indices of the experts each token selected
    mask = torch.zeros_like(probs)
    mask.scatter_(1, topk_idx, 1.0)          # mask[t, i] = 1 if token t selected expert i
    f = mask.mean(dim=0) / self.k            # (E,) share of routing for each expert
    P = probs.mean(dim=0)                   # (E,) mean router probabilities
    self.last_load = f.detach()              # kept only so we can plot
    return self.aux_weight * self.num_experts * (f * P).sum()
```

In words: `scatter_` writes a 1 into the mask at each chosen (token, expert) position. Averaging that mask over tokens and dividing by k gives f_i . Averaging the probabilities gives P_i . The last line is $\lambda_{\text{aux}} E \sum_i f_i P_i$. The result is one number that we add to the task loss. This method belongs inside the `Router` class, which is why it is indented.

▲ WATCH OUT

Do not detach the `P = probs.mean(0)` line or wrap it in `torch.no_grad()`. The gradient must flow through P_i back into the router. It is fine that f_i (built from the hard `topk_idx`) has no gradient; that is by design. If you block the gradient on P_i , the auxiliary loss becomes a constant. It then does nothing, and your experts can collapse without any warning.

CHECK YOURSELF 5

A router with $E = 4$ experts and $k = 1$ collapses to $f = [1, 0, 0, 0]$ and $P = [0.97, 0.01, 0.01, 0.01]$. Predict L_{aux} as a multiple of λ_{aux} , and compare it with a balanced router.

CHECK YOURSELF 6

In Figure 22.2 (bottom), expert 1 has $P_1 = 0.82$ but f_1 stops at exactly 0.5. Why can f_1 not go higher with $k = 2$?

5 Capacity factor and token dropping

◆ INTUITION FIRST

On real hardware, experts run in parallel, and accelerators work best on fixed-size blocks of work. But routing changes from batch to batch. Sometimes one expert gets far more than its fair share. To keep the work a fixed size, each expert gets a **capacity**: the largest number of tokens it will accept in this batch. Tokens beyond that limit are **dropped**. The expert skips them, and they pass through the layer unchanged on the residual connection. They are not retried later.

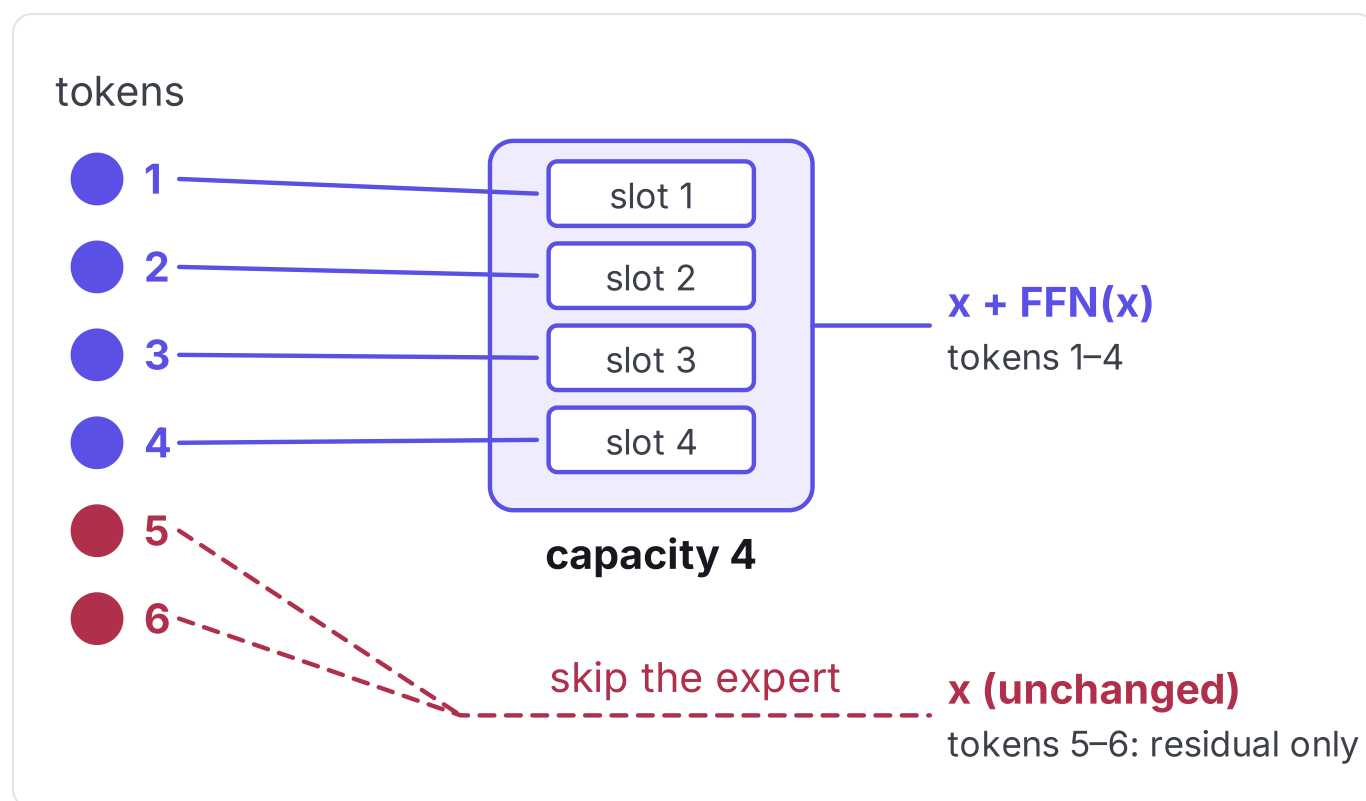


Figure 22.3 Capacity overflow. Six tokens chose this expert, but it has only 4 slots. Tokens 1–4 are processed. Tokens 5 and 6 are dropped: they skip the expert and leave the layer unchanged, carried by the residual connection. They are not retried.

◆ NOW THE MATH

A *capacity factor* c sets the per-expert budget. It is a small multiplier, usually between 1.0 and 2.0:

$$\text{capacity} = c \cdot \frac{T \cdot k}{E}$$

In words: Tk/E is the number of tokens each expert would get if the T tokens (each sent to k experts) were split perfectly evenly. The factor c adds slack: $c = 1.25$ means “accept up to 25% more than the fair share before dropping”. A bigger c drops fewer tokens but wastes more compute on half-empty expert buffers. A smaller c is cheaper but drops more. The load-balancing loss keeps the traffic even enough that few tokens get dropped.

To keep the code in this lesson clear, our implementation does *not* impose a capacity limit. It processes every token, so you can see the routing logic without the bookkeeping. In production systems, capacity matters a lot for speed. A well-balanced router rarely overflows, so the number of dropped tokens stays small.

▲ WATCH OUT

Dropped tokens reduce quality without any error message. Suppose your capacity factor is too low *and* your router is imbalanced. Then many tokens skip the expert layer and get only the residual. For them, the FFN is switched off. If MoE quality is poor for no clear reason, measure the drop rate before you blame the experts.

CHECK YOURSELF 7

A batch has $T = 1024$ tokens, $E = 8$ experts, $k = 2$, and $c = 1.25$. Predict each expert's capacity. If a collapsed router sends half of all slots to one expert, how many of that expert's tokens are dropped?

6 The full MoE layer, in a transformer, training

◆ INTUITION FIRST

We have the `Expert` and the `Router` with its auxiliary loss. Now we assemble the `MoELayer`. It sends each token to its chosen experts and blends the results. Then we put it into a transformer block in place of the FFN. Finally, a tiny training loop lets us watch *task loss + auxiliary loss* fall. The one tricky part is the dispatch: collecting the right tokens for each expert and adding the weighted outputs back. We do it with a clear loop over experts. Read it slowly.

◆ THE SAME THING IN PYTORCH

THE MOE LAYER: LOOP OVER EXPERTS, GATHER EACH EXPERT'S TOKENS, ADD THE WEIGHTED
>- OUTPUTS BACK

```
class MoELayer(nn.Module):
    """Drop-in replacement for a transformer FFN: E experts + a top-k router"""
    def __init__(self, d_model, d_ff, num_experts, k, aux_weight=0.01):
        super().__init__()
        self.experts = nn.ModuleList([Expert(d_model, d_ff) for _ in range(num_experts)])
        self.router = Router(d_model, num_experts, k, aux_weight)

    def forward(self, x):
        # x: (batch, seq_len, d_model). Flatten to one list of tokens.
        batch, seq_len, d_model = x.shape
        x_flat = x.reshape(-1, d_model)  # (num_tokens, d_model)

        topk_idx, topk_weights, aux_loss = self.router(x_flat)
        # topk_idx: (num_tokens, k) which experts each token chose
        # topk_weights: (num_tokens, k) their renormalized weights (sum to 1)

        out = torch.zeros_like(x_flat)  # (num_tokens, d_model)

        for e, expert in enumerate(self.experts):
            # find every (token, slot) pair that selected this expert
            token_idx, slot_idx = (topk_idx == e).nonzero(as_tuple=True)
            if token_idx.numel() == 0:
                continue  # no tokens for this expert
            weights = topk_weights[token_idx, slot_idx].unsqueeze(1)  # (num_tokens, 1, k)
            expert_in = x_flat[token_idx]  # (num_tokens, d_model)
            expert_out = expert(expert_in)  # (num_tokens, d_model)
            # add the weighted outputs back into their tokens' rows; a token
            # that chose two experts gets both contributions summed here.
            out.index_add_(0, token_idx, weights * expert_out)

        return out.reshape(batch, seq_len, d_model), aux_loss
```

In words: for each expert we ask “which tokens picked you?” with `(topk_idx == e).nonzero(...)`. It returns the matching (token, slot) positions. We take those tokens, run them through the expert *once* as a batch, and multiply by their weights. Then `index_add_` adds the results into the right rows. A token that chose two experts appears in two loop

iterations, and `index_add_` adds both weighted outputs. That is the sum $\sum_{i \in \mathcal{S}_t} w_{t,i} \text{FFN}_i(\mathbf{x}_t)$ from the router math.

Do not trust that claim; test it. The slow version below follows the formula one token at a time. It continues the snippets above.

>_ CHECK THE LOOP OVER EXPERTS AGAINST A SLOW PER-TOKEN VERSION

```
def moe_reference(layer, x):
    """Slow per-token version of the MoE formula, for checking."""
    x_flat = x.reshape(-1, x.shape[-1])
    topk_idx, topk_weights, _ = layer.router(x_flat)
    rows = []
    for t in range(x_flat.shape[0]):          # one token at a time
        y = torch.zeros(x_flat.shape[1])
        for slot in range(topk_idx.shape[1]):  # its k chosen experts
            e = topk_idx[t, slot].item()
            y = y + topk_weights[t, slot] * layer.experts[e](x_flat[t])
        rows.append(y)
    return torch.stack(rows).reshape(x.shape)

torch.manual_seed(0)
layer = MoELayer(d_model=16, d_ff=64, num_experts=8, k=2)
x = torch.randn(2, 5, 16)
with torch.no_grad():
    fast, _ = layer(x)
    slow = moe_reference(layer, x)
print(torch.allclose(fast, slow, atol=1e-6))    # True
print(f"{{(fast - slow).abs().max().item():.1e}}")  # largest gap: float r
```

>_ OUTPUT

```
True
6.7e-08
```

In words: the two versions agree to about 10^{-7} . The gap is not zero because the batched version adds the same numbers in a different order, and floating-point addition rounds differently. A gap this size means the logic is the same. A real bug (a wrong weight, a missed token) would give a gap near the size of the outputs themselves.

◆ WHY LOOP OVER EXPERTS, NOT OVER TOKENS

The slow version loops over tokens and runs each token's experts one at a time. That is thousands of tiny matrix multiplies, which is very slow. Looping over *experts* lets us batch: every token sent to expert e goes through it in one matrix multiply. There are only E iterations, however many tokens you have. This is a standard way to write a readable MoE forward pass. Production kernels go further, but the logic is the same.

Next, we put the layer into a transformer block in place of the FFN.

>_ THE BLOCK FROM SECTION 18, WITH MOELAYER WHERE THE FFN USED TO BE

```
class MoETransformerBlock(nn.Module):
    """Standard pre-norm transformer block; the FFN sub-layer is now an MoE"""
    def __init__(self, d_model, n_heads, d_ff, num_experts, k, aux_weight):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.norm2 = nn.LayerNorm(d_model)
        self.moe = MoELayer(d_model, d_ff, num_experts, k, aux_weight)

    def forward(self, x):
        # 1) attention sub-layer, unchanged from a dense transformer
        h = self.norm1(x)
        seq_len = h.size(1)
        causal = torch.triu(torch.full((seq_len, seq_len), float("-inf")),
                             diagonal=-1)
        attn_out, _ = self.attn(h, h, h, attn_mask=causal, need_weights=False)
        x = x + attn_out

        # 2) MoE sub-layer in place of the FFN; also hands back the aux loss
        moe_out, aux_loss = self.moe(self.norm2(x))
        x = x + moe_out
        return x, aux_loss
```

In words: this is the pre-norm block from Section 18: normalize, attend, add; normalize, feed-forward, add. There is one substitution. The feed-forward call is now `self.moe(...)`, and it returns an extra `aux_loss` that we pass up so the training loop can add it. The attention is PyTorch's built-in `nn.MultiheadAttention` with a causal mask. It behaves like the masked self-attention from Section 19: a token can only look at itself and earlier tokens.

Finally, a tiny model and a training run. The toy task is easy on purpose, so the loss falls quickly. The target at each position is the input token *plus one* (mod the vocabulary size). The model can solve it without looking at other tokens, so we can check that the plumbing works.

›_ A TINY MOE TRANSFORMER TRAINED ON A TOY TASK

```

class TinyMoETransformer(nn.Module):
    def __init__(self, vocab, d_model=64, n_heads=4, d_ff=256,
                  num_experts=8, k=2, aux_weight=0.01):
        super().__init__()
        self.embed = nn.Embedding(vocab, d_model)
        self.block = MoETransformerBlock(d_model, n_heads, d_ff, num_experts)
        self.head = nn.Linear(d_model, vocab)

    def forward(self, tokens):
        x = self.embed(tokens)  # (batch, seq_len, d_model)
        x, aux_loss = self.block(x)  # (batch, seq_len, d_model)
        logits = self.head(x)  # (batch, seq_len, vocab)
        return logits, aux_loss

torch.manual_seed(0)
vocab, batch, seq_len = 32, 16, 24
model = TinyMoETransformer(vocab)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

# fixed toy dataset: target[t] = (input[t] + 1) mod vocab
inputs = torch.randint(0, vocab, (batch, seq_len))
targets = (inputs + 1) % vocab

for step in range(61):
    logits, aux_loss = model(inputs)
    # cross-entropy over all positions; flatten (batch, seq_len) into one
    task_loss = F.cross_entropy(logits.reshape(-1, vocab), targets.reshape(-1))
    total_loss = task_loss + aux_loss  # the two terms, added together

    opt.zero_grad()
    total_loss.backward()
    opt.step()

    if step % 10 == 0:
        print(f"step {step:3d} | task {task_loss.item():.3f} | "
              f"| aux {aux_loss.item():.4f} | total {total_loss.item():.3f}")

print("load per expert:", [round(v, 3) for v in model.block.moe.router.loads])

```

In words: we embed the tokens, run the one MoE block, project to vocabulary scores, and compute cross-entropy against the toy targets. This stands in for the next-token objective of Section 19. The line that makes this an MoE run is `total_loss = task_loss + aux_loss`: we optimize the task and the load balance *together*. The whole script (all snippets in this lesson, run in order) takes well under a minute on a laptop CPU. With this seed it prints:

>_ OUTPUT OF THE WHOLE SCRIPT

```
True
6.7e-08
step  0 | task 3.751 | aux 0.0101 | total 3.762
step 10 | task 1.571 | aux 0.0102 | total 1.581
step 20 | task 0.172 | aux 0.0104 | total 0.182
step 30 | task 0.009 | aux 0.0105 | total 0.019
step 40 | task 0.002 | aux 0.0104 | total 0.012
step 50 | task 0.001 | aux 0.0103 | total 0.011
step 60 | task 0.001 | aux 0.0103 | total 0.011
load per expert: [0.126, 0.105, 0.16, 0.137, 0.154, 0.089, 0.1, 0.129]
```

In words: the task loss starts at 3.75. Uniform guessing over 32 tokens would give $\ln 32 \approx 3.47$, and a fresh model is a little worse than uniform. The task loss then falls close to zero, so the MoE block learns the task. The auxiliary loss starts near its balanced value $\lambda_{\text{aux}} = 0.01$, because a freshly initialized router spreads tokens roughly evenly. It rises a little while the task is learned, and the balancing force holds it near 0.0103. The last line shows why: every expert gets between 9% and 16% of the slots, close to the even share $1/8 = 0.125$. No expert is idle.

CHECK YOURSELF 8

In the `MoELayer` forward pass, a token routed to experts 3 and 5 with weights 0.7 and 0.3 is processed in two different loop iterations. What would go wrong if the code used `out[token_idx] = weights * expert_out` instead of `index_add_`?

7 Dense vs MoE: what you gain

◆ INTUITION FIRST

Let's make the trade concrete. Take one transformer layer with $d_{\text{model}} = 512$ and $d_{\text{ff}} = 2048$. A dense FFN has about $2 \cdot d_{\text{model}} \cdot d_{\text{ff}} \approx 2.1$ million parameters. Now swap in an MoE with $E = 8$ experts and $k = 2$. You store eight copies (about 16.8M parameters). But each token runs only two of them (about 4.2M parameters of work), plus a router of $512 \times 8 \approx 4\text{k}$ parameters.

Layer	Total parameters (stored)	Active parameters (per token)	Knowledge : cost
Dense FFN	~2.1M	~2.1M	1× : 1×
MoE, $E=8$, $k=2$	~16.8M (8 experts + router)	~4.2M (2 experts + router)	8× : 2×
MoE, $E=64$, $k=2$	~134M	~4.2M	64× : 2×

In words: the general rule is total $\approx E \times$ dense and active $\approx k \times$ dense, plus a tiny router. Adding experts grows what the model stores without changing what each token costs. The last row stores 64× the parameters at the same 2× cost per token as the second row. This separation is why MoE scales.

The costs

MoE is not free. An engineer should know where the problems are:

- **Routing instability.** The router and the experts adapt to each other, and early training is fragile. Without the auxiliary loss (and often some noise on the gate logits), routing can collapse onto a few experts. Balancing is still an active research and tuning problem.
- **Memory cost.** You still have to *store* all E experts and load them onto accelerators, even though each token uses k . MoE saves compute (FLOPs, floating-point operations), not memory. A 134M-parameter MoE layer needs memory for 134M parameters.
- **Communication cost.** At scale, experts live on different devices. Routing then means sending each token's vector across the network to its expert and sending the result back (an *all-to-all* exchange). This communication can take most of the wall-clock time. It is the main reason MoE is harder to run than its FLOP count suggests.
- **When it helps.** MoE pays off when you want more stored knowledge, have the memory and network to hold many experts, but cannot afford more compute per token. For a small model that fits easily as a dense model, the routing overhead and instability are usually not worth it. MoE is a tool for large scale.

▲ WATCH OUT

Do not read “8× the parameters” as “8× smarter for free”. The gains from MoE are real but smaller than the parameter count suggests. Eight experts are not equal to an eight-times-bigger dense model. Each token sees only two of them, and the router can send a token to the wrong expert. MoE gives a better cost curve, not a miracle. Compare quality per active FLOP, not per stored parameter.

◆ SINCE SWITCH: WHAT CHANGED

The layer you built is the 2021-era design. Newer MoE models change a few parts:

- **Router z-loss** (ST-MoE, Zoph et al., 2022). A second small auxiliary loss penalizes large gate logits. This keeps the softmax numerically stable and made training much less likely to diverge.
- **Fine-grained and shared experts** (DeepSeekMoE, 2024). Split each expert into several smaller ones and route each token to more of them, so experts can specialize more narrowly. Add one or more *shared* experts that every token uses, to hold common knowledge. DeepSeek-V3 has 1 shared and 256 routed experts per MoE layer, with 8 routed experts active per token.
- **Balancing without an auxiliary loss** (DeepSeek-V3, 2024). Each expert gets a bias that is added to its score only when choosing the top k . After each step, the bias goes up for underloaded experts and down for overloaded ones. The task loss is then not disturbed by a balancing gradient. (DeepSeek-V3 still keeps a very small auxiliary loss as a safety net.)

For scale: Switch-C has about 1.6 trillion parameters and 2048 experts per MoE layer with $k = 1$. Mixtral 8x7B has 8 experts with $k = 2$, about 47B total and 13B active parameters. DeepSeek-V3 has 671B total and 37B active parameters.

CHECK YOURSELF 9

A colleague says: “Our MoE model has 10× the parameters of the dense one, so it needs 10× the GPU memory but only about 2× the compute per token.” Judge each half of that claim.

8 What to carry forward

- **Conditional computation** separates knowledge from cost: total parameters grow with the number of experts E , compute per token only with how many you use, k .

- An MoE layer **replaces the FFN sub-layer** of a transformer block with E expert FFNs and a router. Attention does not change.
- The **router** is one small matrix: gate logits, softmax, top-k, renormalize. The output is $\mathbf{y}_t = \sum_{i \in \mathcal{S}_t} w_{t,i} \text{FFN}_i(\mathbf{x}_t)$.
- The **auxiliary loss** $\lambda_{\text{aux}} E \sum_i f_i P_i$ fights routing collapse. Its value at perfect balance is λ_{aux} . Its gradient $\lambda_{\text{aux}} E f_i$ flows through the soft P_i and pushes hardest on overloaded experts.
- The MoE forward pass **loops over experts**, gathers each expert's tokens, and adds the weighted outputs back with `index_add_`. We checked it against a slow per-token version: they agree to float rounding.
- MoE trades **memory and communication for compute**. It is a tool for large scale, not a free gain for small models.

You have now built, from first principles, the mechanism used inside some of today's largest models. It fit into the same transformer block you already understood. That is the arc of this course in one lesson: intuition, then the math, then working code, one swapped sub-layer at a time.

That is the end of the build. Where to go from here (the papers, the projects, and the habits that build fluency) is in Section 23.

Answers to “Check yourself”

1. An MoE layer has $E = 8$ experts, each the same size as a dense feed-forward block, and routes each token to $k = 2$ of them. Predict how many times more parameters it stores than one dense block, and how much more compute it does per token.

It stores $8\times$ the parameters (eight full copies of the block), plus a tiny router. But it runs only 2 of those blocks on any token, so it does about $2\times$ the compute per token, not $8\times$. That gap between $8\times$ the knowledge and $2\times$ the cost is the whole point. The router’s own cost is very small: it is one small matrix, which we size next.

2. What would go wrong if we routed tokens to separate experts inside the attention sub-layer instead of the FFN?

Attention exists so that tokens can mix with each other. If tokens were split among separate experts there, a token could no longer see tokens that went elsewhere, and those connections would break. The FFN treats each token *independently*: it maps one vector to one vector. So each token can go to its own expert without breaking anything. The FFN also holds most of the parameters and per-token compute, so it is the sub-layer worth making sparse.

3. A token’s softmax over 4 experts is $[0.5, 0.3, 0.15, 0.05]$ and $k = 2$. Predict which experts are chosen and their final weights.

The top two are 0.5 (expert 0) and 0.3 (expert 1), so $\mathcal{S} = \{0, 1\}$. Their sum is 0.8, so the weights are $0.5/0.8 = 0.625$ and $0.3/0.8 = 0.375$. Experts 2 and 3 are switched off for this token. The weights sum to 1, so the output is a real blend, $0.625 \text{ FFN}_0 + 0.375 \text{ FFN}_1$, not a faded version.

4. Find the bug: someone sets `k=1` in the `Router` above to get a Switch-style layer. The task loss falls a little, but the routing never seems to improve. Why?

With `k=1`, `topk_probs / topk_probs.sum(...)` is always 1. The weight no longer depends on the router, so the task loss gives the router no gradient. Only the auxiliary loss still moves it. The fix is to use the raw `topk_probs` as the weights when `k = 1`.

5. A router with $E = 4$ experts and $k = 1$ collapses to $f = [1, 0, 0, 0]$ and $P = [0.97, 0.01, 0.01, 0.01]$. Predict L_{aux} as a multiple of λ_{aux} , and compare it with a balanced router.

Collapsed: $\sum_i f_i P_i = 1 \cdot 0.97 = 0.97$, so $L_{\text{aux}} = \lambda_{\text{aux}} \cdot 4 \cdot 0.97 \approx 3.9 \lambda_{\text{aux}}$. Balanced, with $f = P = [0.25, 0.25, 0.25, 0.25]$: the sum is $4 \cdot 0.25^2 = 0.25$, so $L_{\text{aux}} = \lambda_{\text{aux}}$. The collapsed router pays almost four times more. The gradient $\lambda_{\text{aux}} E f_i$ is also largest for expert 0 ($f_0 = 1$), so gradient descent pushes its probability down hardest.

6. In Figure 22.2 (bottom), expert 1 has $P_1 = 0.82$ but f_1 stops at exactly 0.5. Why can f_1 not go higher with $k = 2$?

Each token fills two slots and can use a given expert only once. Even if every token picks expert 1, it gets one slot out of two, so $f_1 = T/(2T) = 0.5 = 1/k$. The other slot must go to some other expert.

7. A batch has $T = 1024$ tokens, $E = 8$ experts, $k = 2$, and $c = 1.25$. Predict each expert's capacity. If a collapsed router sends half of all slots to one expert, how many of that expert's tokens are dropped?

Capacity is $1.25 \cdot 1024 \cdot 2/8 = 320$ tokens. Half of the $2 \cdot 1024 = 2048$ slots is 1024 tokens for that expert, so $1024 - 320 = 704$ of them are dropped. Those tokens get no FFN at all in this layer. This is why balance matters even more once there is a capacity limit.

8. In the `MoELayer` forward pass, a token routed to experts 3 and 5 with weights 0.7 and 0.3 is processed in two different loop iterations. What would go wrong if the code used `out[token_idx] = weights * expert_out` instead of `index_add_`?

Plain assignment overwrites. When the loop reaches expert 3, the row gets $0.7 \text{ FFN}_3(\mathbf{x})$. Later the loop reaches expert 5. Assignment then *replaces* the row with $0.3 \text{ FFN}_5(\mathbf{x})$. Expert 3's part is lost. `index_add_` adds instead, so the row ends up holding $0.7 \text{ FFN}_3 + 0.3 \text{ FFN}_5$, the weighted blend from the router math. The check against `moe_reference` would catch this bug: the gap would be large.

9. A colleague says: “Our MoE model has 10× the parameters of the dense one, so it needs 10× the GPU memory but only about 2× the compute per token.” Judge each half of that claim.

Both halves can be right, and that is the MoE bargain. Memory grows with *total* parameters, because you must store all the experts. So about 10× the parameters means about 10× the memory for the weights. Compute per token grows with *active* parameters, set by k , so with $k = 2$ it stays near 2×. MoE trades memory for compute. When you are limited by compute and have memory to spare, that trade is a win. When memory is the tight resource, it is not.

Where to Go Next

You have gone from a vector to a working Mixture of Experts. This last section gives you a map for what comes next: what to read, what to build, and how to keep learning without getting lost in hype.

18 min read

YOU'LL LEARN

- a reading list in order
- project ideas by level
- how to stay current without the hype

Before you start: Section 22: Mixture of Experts from Scratch.

There is no final exam for machine learning. The field moves every week, and nobody reads everything, not even the people whose names are on the papers. What you need instead is three things. First, a short list of the papers that changed how the field builds models. Second, a habit of building small projects that force real understanding. Third, a way to filter the constant noise. This section gives you all three, and then says goodbye.

◆ THE ONE IDEA TO HOLD ONTO

You do not need to read everything. You need a **path** (papers that build on each other, in order) and a **habit** (read a little, build a little, repeat). Both last longer than any single paper you learn from.

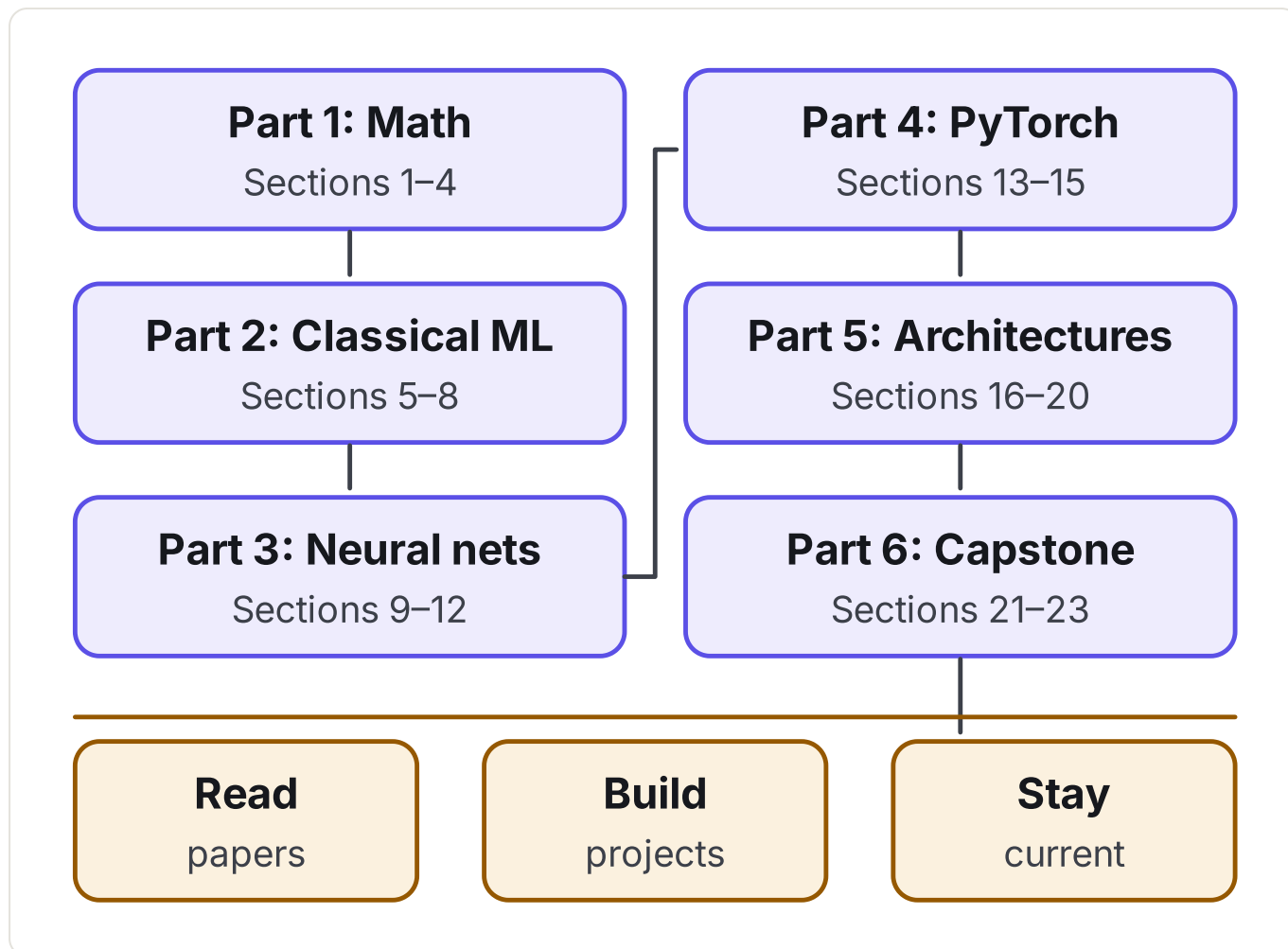


Figure 23.1 The path so far and what comes next. Each part of the course built on the one before (violet). This section covers the three habits (amber) that continue it: reading, building, and filtering the news.

1 A reading list, in order

◆ INTUITION FIRST

Below are papers that shaped the field, grouped by theme. Inside each group they are in date order, so each one can use the ideas before it. Read them after you finish the course, one every week or two, not all at once. Use the three-pass method from Section 21 on each of them.

☂ EVERYDAY PARALLEL

Nobody learns the piano by opening a book of Chopin on day one. You learn scales, then simple songs, then harder ones, and later you improvise. Reading order matters as much as reading itself. The Transformer paper is your Chopin. It will make far more sense once you have done the scales below.

Foundations

1. Rumelhart, Hinton & Williams, 1986. *“Learning representations by back-propagating errors.”* The paper that made training multi-layer networks practical. You built this by hand in Section 10.
2. Krizhevsky, Sutskever & Hinton, 2012. *“ImageNet Classification with Deep Convolutional Neural Networks”* (AlexNet). Showed that deep nets, GPUs, and enough data beat hand-engineered computer vision by a wide margin.
3. Kingma & Ba, 2014. *“Adam: A Method for Stochastic Optimization.”* The default optimizer for most models since. You met it in Section 12.
4. Ioffe & Szegedy, 2015. *“Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.”* Normalizing activations between layers made much deeper networks trainable.
5. He, Zhang, Ren & Sun, 2015. *“Deep Residual Learning for Image Recognition”* (ResNet). The skip connection: a layer can learn nothing and still pass its input through unchanged. This is why networks hundreds of layers deep can train.
6. Ba, Kiros & Hinton, 2016. *“Layer Normalization.”* The normalization that transformers use. It is short, and it helps before you read the Transformer and GPT-2 papers closely.

Sequence to attention

1. Vaswani et al., 2017. *“Attention Is All You Need.”* Replaces recurrence (Section 17) with attention alone. This is the architecture behind modern language models, covered in Section 18.
2. Su et al., 2021. *“RoFormer: Enhanced Transformer with Rotary Position Embedding”* (RoPE). Encodes position by rotating query and key vectors. Most recent open language models use it instead of the original position encoding.

Large language models

1. Radford et al., 2019. *“Language Models are Unsupervised Multitask Learners”* (GPT-2). Shows that one next-token objective, scaled up, does far more than predict text. You built the core of this model in Section 19.
2. Brown et al., 2020. *“Language Models are Few-Shot Learners”* (GPT-3). Scales GPT-2's recipe from 1.5 billion to 175 billion parameters (about 117×). The model can then learn a new task from a few examples in the prompt, with no weight updates.
3. Hoffmann et al., 2022. *“Training Compute-Optimal Large Language Models”* (Chinchilla). Shows that many large models of that time were undertrained for their size. For a fixed compute budget, a smaller model on more data beats a bigger model on less. Read it right after GPT-3 to see the field correct itself.

Efficiency and Mixture of Experts

1. Shazeer et al., 2017. *“Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.”* The modern MoE layer, with noisy top-k gating and a load-balancing loss.

2. Lepikhin et al., 2020. “*GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*.” Top-2 routing with expert capacity, scaled across thousands of accelerators.
3. Fedus, Zoph & Shazeer, 2021. “*Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*.” Simplifies routing to top-1. Its auxiliary loss is the one you derived in Section 22. Section 22 used top-2 with renormalized weights and no capacity limit; this paper shows the top-1 version with capacity.
4. Dao et al., 2022. “*FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*.” Computes the same attention as Section 18, but in tiles that fit in fast GPU memory. A good lesson in how the memory hierarchy, not only FLOPs, sets speed.
5. Zhou et al., 2022. “*Mixture-of-Experts with Expert Choice Routing*.” Turns routing around: each expert picks its top tokens, so load is balanced by construction.
6. Jiang et al., 2024. “*Mixtral of Experts*.” An openly released MoE model with 8 experts and top-2 routing in every layer. It is close to the layer you built, at a scale people run.
7. DeepSeek-AI, 2024. “*DeepSeek-V3 Technical Report*.” A 671B-parameter MoE with 37B active parameters per token. It uses fine-grained and shared experts and balances load without relying on an auxiliary loss.

Fine-tuning and alignment

1. Hu et al., 2021. “*LoRA: Low-Rank Adaptation of Large Language Models*.” Adapt a large pretrained model to a new task by training a small number of extra parameters instead of the whole network.
2. Ouyang et al., 2022. “*Training language models to follow instructions with human feedback*” (InstructGPT / RLHF). How a raw next-token predictor like GPT-3 becomes an assistant that follows instructions.
3. Rafailov et al., 2023. “*Direct Preference Optimization: Your Language Model is Secretly a Reward Model*” (DPO). Gets a similar result to RLHF with one simple loss on pairs of preferred and rejected answers, and no separate reward model.

CHECK YOURSELF 1

You want to understand why Mixtral runs only 2 of its 8 experts per token. Which two papers from the list should you read first, and in what order?

2 Project ideas, by level

◆ INTUITION FIRST

Reading tells you what is possible. Building is what makes it stick. Pick one project below that is slightly above your comfort level, and finish it before you start the next. Half-finished projects teach much less than one completed project.

EVERYDAY PARALLEL

A trainer does not give a beginner the heaviest weights on day one. The trainer also does not keep a strong lifter on the lightest weights forever. Match the project to where you are now, and move up one step only when it is done.

Level	Project	Builds on
Beginner	Reimplement <i>micrograd</i> : a tiny autograd engine, a few hundred lines, no dependencies.	Section 10 (backprop by hand)
Beginner	Train a character-level GPT on a small text file (a few hundred KB is enough). Watch it go from random noise to almost-plausible words.	Section 19
Intermediate	Fine-tune a small open model on a narrow task with LoRA. Compare it with full fine-tuning on the same data and budget. LoRA is new here, so read the LoRA paper first.	Section 14 (modules) and the LoRA paper
Intermediate	Implement an optimizer from scratch (SGD with momentum, then Adam). Check that it matches PyTorch's built-in version on a toy problem.	Section 12
Advanced	Extend the MoE model from Section 22. Add a capacity limit and measure the drop rate. Or replace the auxiliary loss with expert-choice routing or DeepSeek-V3's bias balancing, and measure what changes.	Section 22
Advanced	Pick one paper from the reading list you have not built from yet. Reproduce its smallest core result end to end. Use the three-pass method to read it and the smallest-unit habit to implement it.	Section 21

CHECK YOURSELF 2

Judge this claim: “I reproduced the LoRA result: my fine-tuned model's loss went down.” What is missing before you can call it a reproduction?

3 How to stay current without the hype

◆ INTUITION FIRST

New papers appear faster than anyone can read them. Many of the loudest announcements do not hold up under careful testing. The goal is not to read everything. The goal is a short, reliable set of places to look, and a fast filter that separates a real result from a demo video.

Where to look:

- **arXiv**, the preprint server. The cs.LG and cs.CL listings are where most work appears first, often months before formal publication.
- **Top conferences**: NeurIPS, ICML, and ICLR. Peer review there filters the flood of preprints down to work that passed independent review.
- **Hugging Face Papers**. A daily, community-voted list of new papers, often linked to their code and released models. (Papers With Code used to fill this role; it shut down in 2025 and now redirects here.)
- A few consistently careful **newsletters and blogs**. Prefer the ones that link the paper and the code, not only a demo clip. A short list is worth more than a dozen noisy ones, so remove sources that waste your time.

How to check a claim with healthy doubt, in about two minutes:

- Is there an **ablation table**, or only one headline number? (Section 21 explained why this matters.)
- Is the **baseline** fair (same data, same compute budget, same amount of tuning), or is it a weak opponent chosen to lose?
- Is there **released code**, and has anyone outside the original team reproduced the result?
- Is the size of the claim in proportion to the evidence? “Improves accuracy by 2 points on this benchmark” is a claim. “Solves reasoning” is marketing.

▲ WATCH OUT

The most common hype pattern is a striking demo with no baseline, no ablation, and no released code, often timed with a funding announcement. This does not mean the work is fake. It means you do not yet have enough to judge it. Treating it as settled fact at that point is the trap. Wait for the paper, or move on.

CHECK YOURSELF 3

Judge this claim: “Our new MoE model has 1 trillion parameters and beats a 70-billion-parameter dense model on 5 benchmarks, so sparse models are 14× more efficient.” Which checks from the list does it fail, and what would you ask for?

4 What to carry forward

- Read the list above **in order**: foundations, then attention, then language models, then efficiency and MoE, then fine-tuning. Each group builds on the one before.
- Pick projects **one step above** where you are, and finish each one before you start the next.
- Stay current with a short, reliable source list (arXiv, top conferences, Hugging Face Papers, a few careful newsletters). Check claims fast: ablations, fair baselines, released code, claims in proportion to evidence.
- There is no finish line, only a better practice habit, as with any instrument worth learning.

You started this course with a vector: an arrow, a list of numbers. You end it having built, from scratch, a router that decides which expert network should handle each token inside a transformer. You worked for every step in between. You derived backprop by hand, wrote an MLP in raw NumPy, debugged real shape mismatches, and read a research paper the way an engineer does. Now go build something with it.

Answers to “Check yourself”

1. You want to understand why Mixtral runs only 2 of its 8 experts per token. Which two papers from the list should you read first, and in what order?

Read Shazeer et al. (2017) first. It defines the sparsely-gated MoE layer and top-k routing, so you learn the words (router, gate, expert, top-k) before a later paper assumes them. Then read Switch Transformer (2021). It argues that k can go down to 1 and scales the idea inside a full transformer. Mixtral's “top-2 of 8” is a choice between those points. If you read it without either paper, “why 2 and not 1 or 4” has no motivation.

2. Judge this claim: “I reproduced the LoRA result: my fine-tuned model's loss went down.” What is missing before you can call it a reproduction?

Almost any training run makes the loss go down, so this shows only that the code runs. A reproduction needs a comparison. The paper's claim is that LoRA gets close to full fine-tuning while training far fewer parameters. So you need both runs on the same data and budget, the same evaluation metric, and the number of trained parameters for each. Then you can say whether your gap matches the paper's.

3. Judge this claim: “Our new MoE model has 1 trillion parameters and beats a 70-billion-parameter dense model on 5 benchmarks, so sparse models are 14× more efficient.” Which checks from the list does it fail, and what would you ask for?

The “14×” compares *total* parameters, but compute per token depends on *active* parameters (Section 22). Ask for the active parameter count and the training compute of both models; that is the fair baseline. Ask whether both saw the same data. Five benchmarks with no ablation cannot show which change caused the gain. The claim is also larger than the evidence: “beats on 5 benchmarks” does not support a general efficiency ratio.